

Contents

Язык формул Power Query M

Краткий обзор языка формул Power Query M

Спецификация языка Power Query M

Введение

Лексическая структура

Основные понятия

Значения

Типы

Операторы

Let

Условия

Функции

Обработка ошибок

Разделы

Консолидированная грамматика

Система типов Power Query M

Выражения, значения и выражение let

Комментарии

Модель оценки

Операторы

Преобразование типов

Метаданные

ошибки

Функции Power Query M

Общие сведения о функциях Power Query M

Основные сведения о функциях Power Query M

Функции доступа к данным

Общие сведения о доступе к функциям данных

AccessControlEntry.ConditionToIdentities

AccessControlKind.Allow
AccessControlKind.Deny
Access.Database
ActiveDirectory.Domains
AdobeAnalytics.Cubes
AdoDotNet.DataSource
AdoDotNet.Query
AnalysisServices.Database
AnalysisServices.Databases
AzureStorage.BlobContents
AzureStorage.Blobs
AzureStorage.DataLake
AzureStorage.DataLakeContents
AzureStorage.Tables
Cdm.Contents
Csv.Document
CsvStyle.QuoteAfterDelimiter
CsvStyle.QuoteAlways
Cube.AddAndExpandDimensionColumn
Cube.AddMeasureColumn
Cube.ApplyParameter
Cube.AttributeMemberId
Cube.AttributeMemberProperty
Cube.CollapseAndRemoveColumns
Cube.Dimensions
Cube.DisplayFolders
Cube.MeasureProperties
Cube.MeasureProperty
Cube.Measures
Cube.Parameters
Cube.Properties
Cube.PropertyKey

Cube.ReplaceDimensions

Cube.Transform

DB2.Database

Essbase.Cubes

Excel.CurrentWorkbook

Excel.Workbook

Exchange.Contents

File.Contents

Folder.Contents

Folder.Files

GoogleAnalytics.Accounts

Hdfs.Contents

Hdfs.Files

HdInsight.Containers

HdInsight.Contents

HdInsight.Files

Html.Table

Identity.From

Identity.IsMemberOf

IdentityProvider.Default

Informix.Database

Json.Document

Json.FromValue

MySQL.Database

OData.Feed

ODataOmitValues.Nulls

Odbc.DataSource

Odbc.InferOptions

Odbc.Query

OleDb.DataSource

OleDb.Query

Oracle.Database

Parquet.Document
Pdf.Tables
PostgreSQL.Database
RData.FromBinary
Salesforce.Data
Salesforce.Reports
SapBusinessWarehouse.Cubes
SapBusinessWarehouseExecutionMode.DataStream
SapBusinessWarehouseExecutionMode.BasXml
SapBusinessWarehouseExecutionMode.BasXmlGzip
SapHana.Database
SapHanaDistribution.All
SapHanaDistribution.Connection
SapHanaDistribution.Off
SapHanaDistribution.Statement
SapHanaRangeOperator.Equals
SapHanaRangeOperator.GreaterThan
SapHanaRangeOperator.GreaterThanOrEquals
SapHanaRangeOperator.LessThan
SapHanaRangeOperator.LessThanOrEquals
SapHanaRangeOperator.NotEquals
SharePoint.Contents
SharePoint.Files
SharePoint.Tables
Soda.Feed
Sql.Database
Sql.Databases
Sybase.Database
Teradata.Database
WebAction.Request
Web.BrowserContents
Web.Contents

[Web.Page](#)

[WebMethod.Delete](#)

[WebMethod.Get](#)

[WebMethod.Head](#)

[WebMethod.Patch](#)

[WebMethod.Post](#)

[WebMethod.Put](#)

[Xml.Document](#)

[Xml.Tables](#)

[Двоичные функции](#)

[Общие сведения о двоичных функциях](#)

[Binary.Buffer](#)

[Binary.Combine](#)

[Binary.Compress](#)

[Binary.Decompress](#)

[Binary.From](#)

[Binary.FromList](#)

[Binary.FromText](#)

[Binary.InferContentType](#)

[Binary.Length](#)

[Binary.Range](#)

[Binary.ToList](#)

[Binary.ToText](#)

[BinaryEncoding.Base64](#)

[BinaryEncoding.Hex](#)

[BinaryFormat.7BitEncodedSignedInteger](#)

[BinaryFormat.7BitEncodedUnsignedInteger](#)

[BinaryFormat.Binary](#)

[BinaryFormat.Byte](#)

[BinaryFormat.ByteOrder](#)

[BinaryFormat.Choice](#)

[BinaryFormat.Decimal](#)

BinaryFormat.Double
BinaryFormat.Group
BinaryFormat.Length
BinaryFormat.List
BinaryFormat.Null
BinaryFormat.Record
BinaryFormat.SignedInteger16
BinaryFormat.SignedInteger32
BinaryFormat.SignedInteger64
BinaryFormat.Single
BinaryFormat.Text
BinaryFormat.Transform
BinaryFormat.UnsignedInteger16
BinaryFormat.UnsignedInteger32
BinaryFormat.UnsignedInteger64
BinaryOccurrence.Optional
BinaryOccurrence.Repeating
BinaryOccurrence.Required
ByteOrder.BigEndian
ByteOrder.LittleEndian
Compression.Brotli
Compression.Deflate
Compression.GZip
Compression.LZ4
Compression.None
Compression.Snappy
Compression.Zstandard
Occurrence.Optional
Occurrence.Repeating
Occurrence.Required
#binary

Функции объединения

Общие сведения о функциях объединения

Combiner.CombineTextByDelimiter

Combiner.CombineTextByEachDelimiter

Combiner.CombineTextByLengths

Combiner.CombineTextByPositions

Combiner.CombineTextByRanges

Функции сравнения

Общие сведения о функциях сравнения

Comparer.Equals

Comparer.FromCulture

Comparer.Ordinal

Comparer.OrdinalIgnoreCase

Culture.Current

Функции данных

Общие сведения о функциях для работы с датами

Date.AddDays

Date.AddMonths

Date.AddQuarters

Date.AddWeeks

Date.AddYears

Date.Day

Date.DayOfWeek

Date.DayOfWeekName

Date.DayOfYear

Date.DaysInMonth

Date.EndOfDay

Date.EndOfMonth

Date.EndOfQuarter

Date.EndOfWeek

Date.EndOfYear

Date.From

Date.FromText

Date.IsInCurrentDay
Date.IsInCurrentMonth
Date.IsInCurrentQuarter
Date.IsInCurrentWeek
Date.IsInCurrentYear
Date.IsInNextDay
Date.IsInNextMonth
Date.IsInNextNDays
Date.IsInNextNMonths
Date.IsInNextNQuarters
Date.IsInNextNWeeks
Date.IsInNextNYears
Date.IsInNextQuarter
Date.IsInNextWeek
Date.IsInNextYear
Date.IsInPreviousDay
Date.IsInPreviousMonth
Date.IsInPreviousNDays
Date.IsInPreviousNMonths
Date.IsInPreviousNQuarters
Date.IsInPreviousNWeeks
Date.IsInPreviousNYears
Date.IsInPreviousQuarter
Date.IsInPreviousWeek
Date.IsInPreviousYear
Date.IsInYearToDate
Date.IsLeapYear
Date.Month
Date.MonthName
Date.QuarterOfYear
Date.StartOfDay
Date.StartOfMonth

Date.StartOfQuarter

Date.StartOfWeek

Date.StartOfYear

Date.ToRecord

Date.ToText

Date.WeekOfMonth

Date.WeekOfYear

Date.Year

Day.Friday

Day.Monday

Day.Saturday

Day.Sunday

Day.Thursday

Day.Tuesday

Day.Wednesday

#Дата

Функции для работы с датами и временем

Общие сведения о функциях для работы с датами и временем

DateTime.AddZone

DateTime.Date

DateTime.FixedLocalNow

DateTime.From

DateTime.FromFileTime

DateTime.FromText

DateTime.IsInCurrentHour

DateTime.IsInCurrentMinute

DateTime.IsInCurrentSecond

DateTime.IsInNextHour

DateTime.IsInNextMinute

DateTime.IsInNextNHours

DateTime.IsInNextNMinutes

DateTime.IsInNextNSeconds

[DateTime.IsInNextSecond](#)
[DateTime.IsInPreviousHour](#)
[DateTime.IsInPreviousMinute](#)
[DateTime.IsInPreviousNHours](#)
[DateTime.IsInPreviousNMinutes](#)
[DateTime.IsInPreviousNSeconds](#)
[DateTime.IsInPreviousSecond](#)
[DateTime.LocalNow](#)
[DateTime.Time](#)
[DateTime.ToRecord](#)
[DateTime.ToText](#)
[#DATETIME](#)

Функции для работы с датами, временем и часовыми поясами

[Общие сведения о функциях для работы с датами и часовыми поясами](#)

[DateTimeZone.FixedLocalNow](#)
[DateTimeZone.FixedUtcNow](#)
[DateTimeZone.From](#)
[DateTimeZone.FromFileTime](#)
[DateTimeZone.FromText](#)
[DateTimeZone.LocalNow](#)
[DateTimeZone.RemoveZone](#)
[DateTimeZone.SwitchZone](#)
[DateTimeZone.ToLocal](#)
[DateTimeZone.ToRecord](#)
[DateTimeZone.ToText](#)
[DateTimeZone.ToUtc](#)
[DateTimeZone.UtcNow](#)
[DateTimeZone.ZoneHours](#)
[DateTimeZone.ZoneMinutes](#)
[#datetimezone](#)

Функции для работы с длительностью

[Общие сведения о функциях для работы с длительностью](#)

Duration.Days

Duration.From

Duration.FromText

Duration.Hours

Duration.Minutes

Duration.Seconds

Duration.ToRecord

Duration.TotalDays

Duration.TotalHours

Duration.TotalMinutes

Duration.TotalSeconds

Duration.ToText

#длительность

Обработка ошибок

Общие сведения об обработке ошибок

Diagnostics.ActivityId

Diagnostics.Trace

Error.Record

TraceLevel.Critical

TraceLevel.Error

TraceLevel.Information

TraceLevel.Verbose

TraceLevel.Warning

Функции выражений

Общие сведения о функциях выражения

Expression.Constant

Expression.Evaluate

Expression.Identifier

Значения функции

Общие сведения о значениях функций

Function.From

Function.Invoke

Function.InvokeAfter

Function.IsDataSource

Function.ScalarVector

Функции для работы со строками

Общие сведения о функциях для работы с линиями

Lines.FromBinary

Lines.FromText

Lines.ToBinary

Lines.ToText

Функции для работы со списками

Общие сведения о функциях для работы со списками

List.Accumulate

List.AllTrue

List.Alternate

List.AnyTrue

List.Average

List.Buffer

List.Combine

List.ConformToPageReader

List.Contains

List.ContainsAll

List.ContainsAny

List.Count

List.Covariance

List.Dates

List.DateTimes

List.DateTimeZones

List.Difference

List.Distinct

List.Durations

List.FindText

List.First

List.FirstN
List.Generate
List.InsertRange
List.Intersect
List.IsDistinct
List.IsEmpty
List.Last
List.LastN
List.MatchesAll
List.MatchesAny
List.Max
List.MaxN
List.Median
List.Min
List.MinN
List.Mode
List.Modes
List.NonNullCount
List.Numbers
List.Percentile
List.PositionOf
List.PositionOfAny
List.Positions
List.Product
List.Random
List.Range
List.RemoveFirstN
List.RemoveItems
List.RemoveLastN
List.RemoveMatchingItems
List.RemoveNulls
List.RemoveRange

List.Repeat
List.ReplaceMatchingItems
List.ReplaceRange
List.ReplaceValue
List.Reverse
List.Select
List.Single
List.SingleOrDefault
List.Skip
List.Sort
List.Split
List.StandardDeviation
List.Sum
List.Times
List.Transform
List.TransformMany
List.Union
List.Zip
PercentileMode.ExcelExc
PercentileMode.ExcelInc
PercentileMode.SqlCont
PercentileMode.SqlDisc

Логические функции

Общие сведения о логических функциях

Logical.From
Logical.FromText
Logical.ToText

Функции для работы с числами

Общие сведения о функциях для работы с числами

Byte.From
Currency.From
Decimal.From

Double.From
Int8.From
Int16.From
Int32.From
Int64.From
Number.Abs
Number.Acos
Number.Asin
Number.Atan
Number.Atan2
Number.BitwiseAnd
Number.BitwiseNot
Number.BitwiseOr
Number.BitwiseShiftLeft
Number.BitwiseShiftRight
Number.BitwiseXor
Number.Combinations
Number.Cos
Number.Cosh
Number.E
Number.Epsilon
Number.Exp
Number.Factorial
Number.From
Number.FromText
Number.IntegerDivide
Number.IsEven
Number.IsNaN
Number.IsOdd
Number.Ln
Number.Log
Number.Log10

Number.Mod
Number.NaN
Number.NegativeInfinity
Number.Permutations
Number.PI
Number.PositiveInfinity
Number.Power
Number.Random
Number.RandomBetween
Number.Round
Number.RoundAwayFromZero
Number.RoundDown
Number.RoundTowardZero
Number.RoundUp
Number.Sign
Number.Sin
Number.Sinh
Number.Sqrt
Number.Tan
Number.Tanh
Number.ToText
Percentage.From
RoundingMode.AwayFromZero
RoundingMode.Down
RoundingMode.ToEven
RoundingMode.TowardZero
RoundingMode.Up
Single.From

Функции для работы с записями

Общие сведения о функциях для работы с записями

Geography.FromWellKnownText
Geography.ToWellKnownText

GeographyPoint.From
Geometry.FromWellKnownText
Geometry.ToWellKnownText
GeometryPoint.From
MissingField.Error
MissingField.Ignore
MissingField.UseNull
Record.AddField
Record.Combine
Record.Field
Record.FieldCount
Record.FieldNames
Record.FieldOrDefault
Record.FieldValues
Record.FromList
Record.FromTable
Record.HasFields
Record.RemoveFields
Record.RenameFields
Record.ReorderFields
Record.SelectFields
Record.ToList
Record.ToTable
Record.TransformFields

Функции замены

Общие сведения о функциях замены
Replacer.ReplaceText
Replacer.ReplaceValue

Функции разделения

Общие сведения о функциях разделения
QuoteStyle.Csv
QuoteStyle.None

Splitter.SplitByNothing
Splitter.SplitTextByAnyDelimiter
Splitter.SplitTextByCharacterTransition
Splitter.SplitTextByDelimiter
Splitter.SplitTextByEachDelimiter
Splitter.SplitTextByLengths
Splitter.SplitTextByPositions
Splitter.SplitTextByRanges
Splitter.SplitTextByRepeatedLengths
Splitter.SplitTextByWhitespace

Табличные функции

Общие сведения о табличных функциях

ExtraValues.Error
ExtraValues.Ignore
ExtraValues.List
GroupKind.Global
GroupKind.Local
ItemExpression.From
ItemExpression.Item
JoinAlgorithm.Dynamic
JoinAlgorithm.LeftHash
JoinAlgorithm.LeftIndex
JoinAlgorithm.PairwiseHash
JoinAlgorithm.RightHash
JoinAlgorithm.RightIndex
JoinAlgorithm.SortMerge
JoinKind.FullOuter
JoinKind.Inner
JoinKind.LeftAnti
JoinKind.LeftOuter
JoinKind.RightAnti
JoinKind.RightOuter

JoinSide.Left
JoinSide.Right
Occurrence.All
Occurrence.First
Occurrence.Last
Order.Ascending
Order.Descending
RowExpression.Column
RowExpression.From
RowExpression.Row
Table.AddColumn
Table.AddFuzzyClusterColumn
Table.AddIndexColumn
Table.AddJoinColumn
Table.AddKey
Table.AggregateTableColumn
Table.AlternateRows
Table.Buffer
Table.Column
Table.ColumnCount
Table.ColumnNames
Table.ColumnsOfType
Table.Combine
Table.CombineColumns
Table.CombineColumnsToRecord
Table.ConformToPageReader
Table.Contains
Table.ContainsAll
Table.ContainsAny
Table.DemoteHeaders
Table.Distinct
Table.DuplicateColumn

Table.ExpandListColumn
Table.ExpandRecordColumn
Table.ExpandTableColumn
Table.FillDown
Table.FillUp
Table.FilterWithDataTable
Table.FindText
Table.First
Table.FirstN
Table.FirstValue
Table.FromColumns
Table.FromList
Table.FromPartitions
Table.FromRecords
Table.FromRows
Table.FromValue
Table.FuzzyGroup
Table.FuzzyJoin
Table.FuzzyNestedJoin
Table.Group
Table.HasColumns
Table.InsertRows
Table.IsDistinct
Table.IsEmpty
Table.Join
Table.Keys
Table.Last
Table.LastN
Table.MatchesAllRows
Table.MatchesAnyRows
Table.Max
Table.MaxN

Table.Min
Table.MinN
Table.NestedJoin
Table.Partition
Table.PartitionValues
Table.Pivot
Table.PositionOf
Table.PositionOfAny
Table.PrefixColumns
Table.Profile
Table.PromoteHeaders
Table.Range
Table.RemoveColumns
Table.RemoveFirstN
Table.RemoveLastN
Table.RemoveMatchingRows
Table.RemoveRows
Table.RemoveRowsWithErrors
Table.RenameColumns
Table.ReorderColumns
Table.Repeat
Table.ReplaceErrorValues
Table.ReplaceKeys
Table.ReplaceMatchingRows
Table.ReplaceRelationshipIdentity
Table.ReplaceRows
Table.ReplaceValue
Table.Reverse
Table.ReverseRows
Table.RowCount
Table.Schema
Table.SelectColumns

Table.SelectRows

Table.SelectRowsWithErrors

Table.SingleRow

Table.Skip

Table.Sort

Table.Split

Table.SplitColumn

Table.ToColumns

Table.ToList

Table.ToRecords

Table.ToRows

Table.TransformColumnNames

Table.TransformColumns

Table.TransformColumnTypes

Table.TransformRows

Table.Transpose

Table.Unpivot

Table.UnpivotOtherColumns

Table.View

Table.ViewFunction

Tables.GetRelationships

#table

Текстовые функции

Общие сведения о текстовых функциях

Character.FromNumber

Character.ToNumber

Guid.From

Json.FromValue

RelativePosition.FromEnd

RelativePosition.FromStart

Text.AfterDelimiter

Text.At

Text.BeforeDelimiter
Text.BetweenDelimiters
Text.Clean
Text.Combine
Text.Contains
Text.End
Text.EndsWith
Text.Format
Text.From
Text.FromBinary
Text.InferNumberType
Text.Insert
Text.Length
Text.Lower
Text.Middle
Text.NewGuid
Text.PadEnd
Text.PadStart
Text.PositionOf
Text.PositionOfAny
Text.Proper
Text.Range
Text.Remove
Text.RemoveRange
Text.Repeat
Text.Replace
Text.ReplaceRange
Text.Reverse
Text.Select
Text.Split
Text.SplitAny
Text.Start

[Text.StartsWith](#)

[Text.ToBinary](#)

[Text.ToList](#)

[Text.Trim](#)

[Text.TrimEnd](#)

[Text.TrimStart](#)

[Text.Upper](#)

[TextEncoding.Ascii](#)

[TextEncoding.BigEndianUnicode](#)

[TextEncoding.Unicode](#)

[TextEncoding.Utf8](#)

[TextEncoding.Utf16](#)

[TextEncoding.Windows](#)

[Функции для работы со временем](#)

[Общие сведения о функциях для работы со временем](#)

[Time.EndOfHour](#)

[Time.From](#)

[Time.FromText](#)

[Time.Hour](#)

[Time.Minute](#)

[Time.Second](#)

[Time.StartOfHour](#)

[Time.ToRecord](#)

[Time.ToText](#)

[#time](#)

[Функции типов](#)

[Общие сведения о функциях типов](#)

[Type.AddTableKey](#)

[Type.ClosedRecord](#)

[Type.Facets](#)

[Type.ForFunction](#)

[Type.ForRecord](#)

Type.FunctionParameters
Type.FunctionRequiredParameters
Type.FunctionReturn
Type.Is
Type.IsNullable
Type.IsOpenRecord
Type.ListItem
Type.NonNullable
Type.OpenRecord
Type.RecordFields
Type.ReplaceFacets
Type.ReplaceTableKeys
Type.TableColumn
Type.TableKeys
Type.TableRow
Type.TableSchema
Type.Union

Функции для работы с URI

Общие сведения о функциях для работы с URI
Uri.BuildQueryString
Uri.Combine
Uri.EscapeDataString
Uri.Parts

Функции для работы со значениями

Общие сведения о функциях для работы со значениями
DirectQueryCapabilities.From
Embedded.Value
Graph.Nodes
Precision.Decimal
Precision.Double
SqlExpression.SchemaFrom
SqlExpression.ToExpression

Value.Add
Value.Alternates
Value.As
Value.Compare
Value.Divide
Value.Equals
Value.Expression
Value.Firewall
Value.FromText
Value.Is
Value.Lineage
Value.Metadata
Value.Multiply
Value.NativeQuery
Value.NullableEquals
Value.Optimize
Value.RemoveMetadata
Value.ReplaceMetadata
Value.ReplaceType
Value.Subtract
Value.Traits
Value.Type
Variable.Value

Краткий обзор языка формул Power Query M

24.03.2021 • 3 minutes to read

В этом кратком обзоре описывается создание запросов на языке формул Power Query M.

NOTE

M — это язык с учетом регистра.

Создание запроса с помощью редактора запросов

Для создания расширенного запроса используется **редактор запросов**. Комбинированный запрос состоит из переменных, выражений и значений, инкапсулированных в выражении `let`. Переменная может содержать пробелы. Для этого нужно указать идентификатор `#` и имя в кавычках, например `#"Имя переменной"`.

Выражение `let` имеет следующую структуру:

```
let
    Variablename = expression,
    #"Variable name" = expression2
in
    Variablename
```

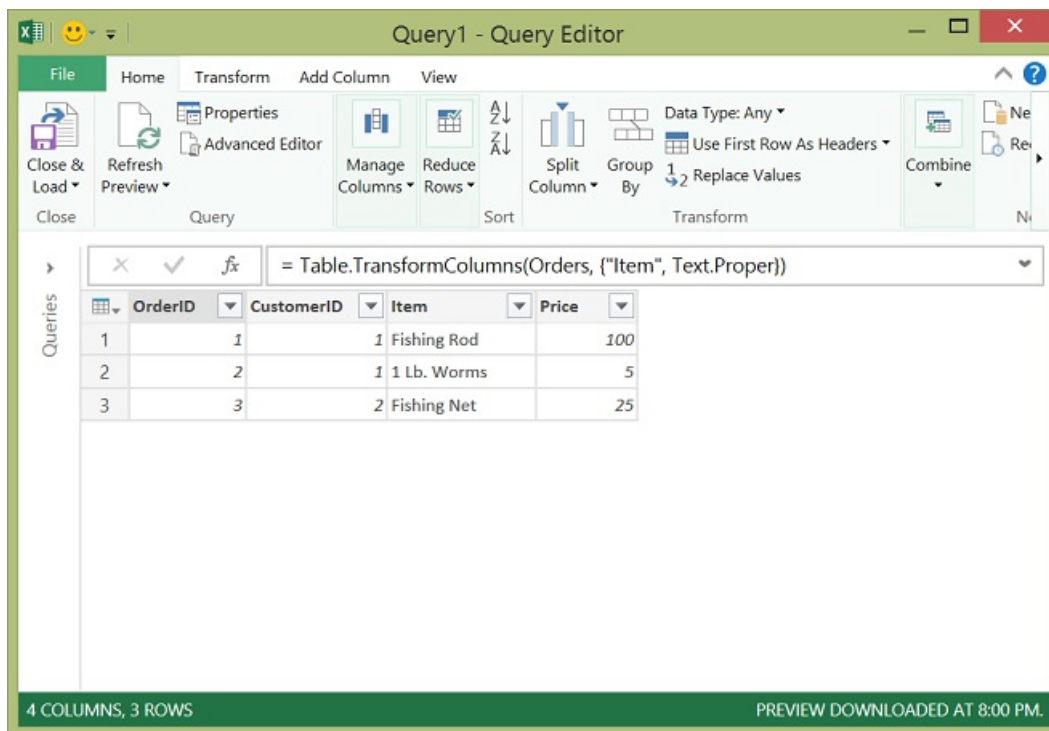
Чтобы создать запрос M в **редакторе запросов**, необходимо выполнить указанную ниже последовательность действий.

- Создайте последовательность шагов для формулы запроса, которая начинается с инструкции `let`. Каждый шаг определяется именем переменной шага. **Переменная** в M может содержать пробелы, но для этого необходимо использовать символ `#`, например как в `#"Имя шага"`. Шаг формулы может быть пользовательской формулой. Обратите внимание, что в языке формул Power Query учитывается регистр.
- Каждый шаг формулы запроса основан на предыдущем, на который он ссылается по имени переменной.
- Выведите шаг формулы запроса с помощью инструкции `in`. Как правило, в качестве результирующего набора данных используется последний шаг запроса.

Дополнительные сведения о выражениях и значениях см. в статье [Выражения, значения и выражение let](#).

Простые шаги формулы Power Query M

Предположим, что вы создали приведенное ниже преобразование в **редакторе запросов** для преобразования названий продуктов в надлежащий регистр.



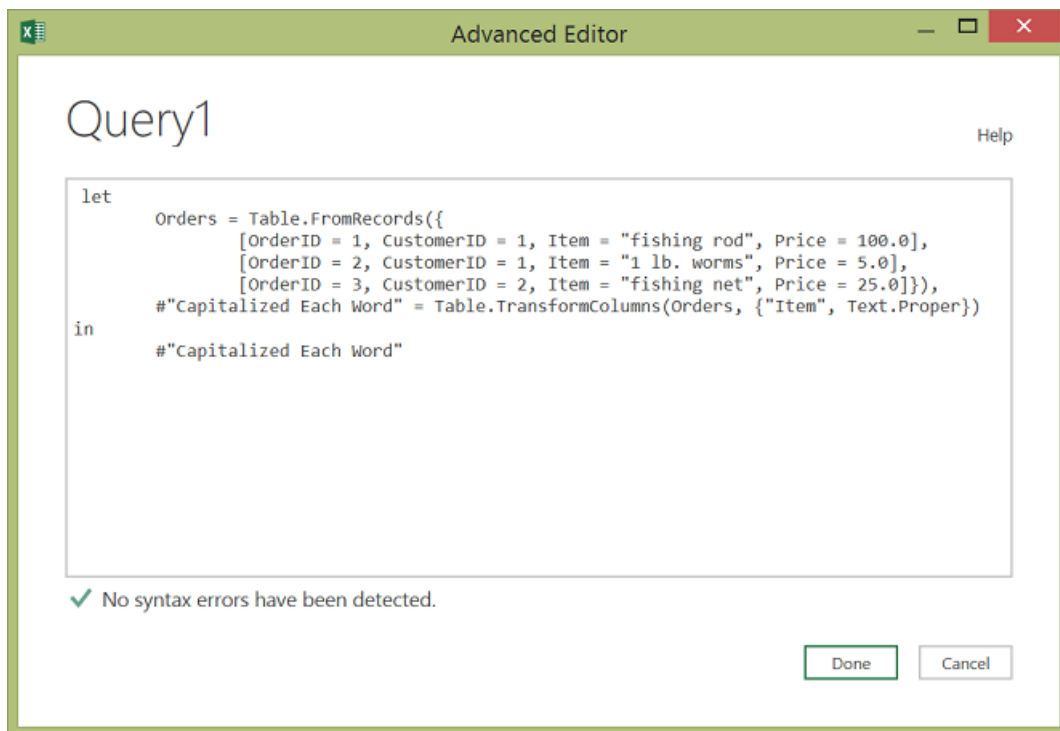
Вы получили таблицу, которая выглядит так:

ORDERID	CUSTOMERID	ЭЛЕМЕНТ	ЦЕНА
1	1	удочка	100
2	1	Черви, 1 фунт	5
3	2	рыболовная сеть	25

Вы хотите изменить первую букву в каждом слове в столбце Item на прописную, чтобы получить следующую таблицу:

ORDERID	CUSTOMERID	ЭЛЕМЕНТ	ЦЕНА
1	1	Удочка	100
2	1	Черви, Вирусы-черви	5
3	2	Рыболовная сеть	25

Шаги формулы M для преобразования исходной таблицы в результирующую выглядят следующим образом:



Вот код, который можно вставить в **редактор запросов**:

```
let Orders = Table.FromRecords({
    [OrderID = 1, CustomerID = 1, Item = "fishing rod", Price = 100.0],
    [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
    [OrderID = 3, CustomerID = 2, Item = "fishing net", Price = 25.0]}),
#"Capitalized Each Word" = Table.TransformColumns(Orders, {"Item", Text.Proper})
in
    #"Capitalized Each Word"
```

Давайте рассмотрим каждый шаг формулы.

1. **Orders** — создает таблицу [Table](#_Table_value) с данными из таблицы Orders.
2. **#"Capitalized Each Word"** — для изменения первой буквы в каждом слове на прописную используется Table.TransformColumns().
3. **in #"Capitalized Each Word"** — выводит таблицу, в которой каждое слово начинается с прописной буквы.

См. также

[Выражения, значения и выражение let](#)

[Операторы](#)

[Преобразование типов](#)

Спецификация языка Power Query M

16.03.2021 • 2 minutes to read

Спецификация описывает значения, выражения, среды и переменные, идентификаторы и модель оценки, которые образуют основные понятия языка Power Query M.

Спецификация содержится в следующих разделах.

- [Введение](#)
- [Лексическая структура](#)
- [Основные понятия](#)
- [Значения](#)
- [Типы](#)
- [Операторы](#)
- [Let](#)
- [Условия](#)
- [Функции](#)
- [Обработка ошибок](#)
- [Разделы](#)
- [Консолидированная грамматика](#)

Введение

16.12.2020 • 19 minutes to read

Обзор

Microsoft Power Query предоставляет эффективный интерфейс получения данных, обладающий множеством возможностей. К ключевым возможностям Power Query относятся фильтрация и объединение данных, позволяющие комбинировать данные из одного или нескольких поддерживаемых источников данных из обширной коллекции. Для выражения всех подобных комбинаций данных используется язык формул Power Query (формально известный как язык M). Power Query внедряет документы M в книги Excel и Power BI, чтобы обеспечить повторяемое объединение данных.

В этом документе содержится спецификация для M. После краткого введения, цель которого предоставить ознакомительные сведения о языке, в документе подробно рассматривается язык в нескольких последовательных шагах:

1. *Лексическая структура* определяет набор текстов, которые являются лексически допустимыми.
2. Значения, выражения, среды и переменные, идентификаторы и модель оценки, которые образуют *основные понятия* языка.
3. Подробное описание *значений*, как примитивов, так и структурированных данных, определяет целевое назначение языка.
4. Значения имеют *типы*, которые сами по себе являются особым видом значений. Оба они характеризуют фундаментальные типы значений и содержат дополнительные метаданные, относящиеся к формам структурированных значений.
5. Набор *операторов* в языке M определяет, какие типы выражений могут быть сформированы.
6. *Функции*, другая разновидность специальных значений, предоставляют основу для обширной стандартной библиотеки для языка M и позволяют добавлять новые абстракции.
7. *Ошибки* могут возникать при применении операторов или функций во время вычисления выражения. Несмотря на то что ошибки не являются значениями, существуют способы *обработки ошибок*, с помощью которых ошибки можно сопоставить обратно со значениями.
8. *Выражения let* позволяют создавать дополнительные определения, используемые для построения сложных выражений в несколько небольших шагов.
9. *Выражения if* поддерживают условную оценку.
10. *Разделы* обеспечивают простой механизм модульности. (Разделы пока еще не используются Power Query.)
11. Наконец, *консолидированная грамматика* объединяет в себе фрагменты грамматики из других разделов этого документа в одно полное определение.

Для теоретиков компьютерных языков: язык формул, описываемый в этом документе, является по большей части чистым, динамически типизированным, частично функционально упрощенным языком высшего порядка.

Выражения и значения

Основной компонент в языке M — это *выражение*. Выражение можно вычислить, возвратив одно *значение*.

Хотя многие значения могут быть записаны буквально в виде выражения, значение не является выражением. Например, вычисление выражения `1` дает значение `1`, а вычисление выражений `1+1` дает значение `2`. Это неочевидное, но важное различие. Выражения являются наборами правил для вычисления, а значения — результатами вычисления.

В следующих примерах показаны различные виды значений, доступные в M. По соглашению значение записывается в литеральной форме, в которой они будут отображаться в выражении, результатом которого является только это значение. (Обратите внимание, что `//` указывает начало комментария, который продолжается до конца строки.)

- *Примитив* — это однокомпонентное значение, например число, логическое значение, текст или значение NULL. Значение NULL может использоваться для указания отсутствия каких-либо данных.

```
123           // A number
true          // A logical
"abc"         // A text
null          // null value
```

- Значение *списка* является упорядоченной последовательностью значений. M поддерживает бесконечные списки, но в случае написания в виде литерала списки имеют фиксированную длину. Символы фигурных скобок (`{` и `}`) обозначают начало и конец списка.

```
{123, true, "A"} // list containing a number, a logical, and
                  //      a text
{1, 2, 3}        // list of three numbers
```

- *Запись* представляет собой набор *полей*. Поле — это пара "имя-значение", где имя является текстовым значением, уникальным в пределах записи поля. Литеральный синтаксис значений записи позволяет записывать имена без кавычек, в такой форме они также называются *идентификаторами*. Ниже показана запись, содержащая три поля с именами `"A"`, `"B"` и `"C"`, которые имеют значения `1`, `2` и `3`.

```
[
  A = 1,
  B = 2,
  C = 3
]
```

- *Таблица* — это набор значений, упорядоченных по столбцам (идентифицируемые по имени) и строкам. Для создания таблицы не предусмотрен литеральный синтаксис, однако существует несколько стандартных функций, которые можно использовать для создания таблиц из списков или записей.

Пример:

```
#table( {"A", "B"}, { {1, 2}, {3, 4} } )
```

Данный код создает таблицу следующей формы:

A	B
1	2
3	4

- **Функция** — это значение, которое при вызове с аргументами создает новое значение. Функции записываются путем перечисления *параметров* функции в круглых скобках, за которыми следует символ перехода [=>], за которым следует определяющее функцию выражение. Это выражение обычно ссылается на параметры (по имени).

```
(x, y) => (x + y) / 2`
```

Оценка

Модель оценки языка M базируется на модели оценки, часто применяемой в электронных таблицах, где порядок вычислений можно определить на основе зависимостей между формулами в ячейках.

Если вы ввели формулы в электронных таблицах, например в Excel, можно определить, что при вычислении указанные слева формулы дадут значения, указанные справа:

	A		A
1	=A2 * 2	1	4
2	=A3 + 1	2	2
3	1	3	1

В языке M части выражения могут ссылаться на другие части выражения по имени, и процесс оценки автоматически определит порядок вычисления выражений, на которые имеются ссылки.

Можно использовать запись для создания выражения, эквивалентного приведенному выше примеру электронной таблицы. При инициализации значения поля мы можем сослаться на другие поля в записи по имени поля, как показано ниже:

```
[
  A1 = A2 * 2,
  A2 = A3 + 1,
  A3 = 1
]
```

Приведенное выше выражение эквивалентно следующему (в той части, что оба вычисляют равные значения):

```
[
  A1 = 4,
  A2 = 2,
  A3 = 1
]
```

Записи могут содержаться в других записях или *вкладываться* в них. Для обращения к полям записи по имени можно использовать *оператор подстановки* ([]). Например, следующая запись содержит поле `Sales`, содержащее запись, и поле `Total`, которое обращается к полям `FirstHalf` и `SecondHalf` записи

Sales :

```
[
  Sales = [ FirstHalf = 1000, SecondHalf = 1100 ],
  Total = Sales[FirstHalf] + Sales[SecondHalf]
]
```

Приведенное выше выражение эквивалентно следующему при вычислении:

```
[
  Sales = [ FirstHalf = 1000, SecondHalf = 1100 ],
  Total = 2100
]
```

Записи также могут содержаться в списках. Чтобы обратиться к элементу списка по его числовому индексу, используйте *позиционный оператор индекса* (`{}`). Для ссылки на значения в списке используется начинающийся с нуля индекс, отсчитываемый от начала списка. Например, индексы `0` и `1` используются для ссылки на первый и второй элементы в списке ниже:

```
[
  Sales =
  {
    [
      Year = 2007,
      FirstHalf = 1000,
      SecondHalf = 1100,
      Total = FirstHalf + SecondHalf // 2100
    ],
    [
      Year = 2008,
      FirstHalf = 1200,
      SecondHalf = 1300,
      Total = FirstHalf + SecondHalf // 2500
    ]
  },
  TotalSales = Sales{0}[Total] + Sales{1}[Total] // 4600
]
```

Выражения элементов списка и записи (а также выражения `let`, представленные ниже) оцениваются с помощью *отложенных вычислений*, что означает, что они оцениваются только по мере необходимости. Все остальные выражения оцениваются с помощью *безотложного вычисления*, то есть сразу же при их обнаружении в процессе оценки. При этом следует помнить, что оценка выражения записи или списка возвратит значение записи или списка, которое помнит соответствующе поля записи или элементы списка, которые требуется вычислить при необходимости (с помощью операторов индекса или подстановки).

Функции

В языке M *функция* — это сопоставление набора входных значений с одним выходным значением. Функция записывается путем именования требуемого набора входных значений (параметров функции) и последующего предоставления выражения, которое будет вычислять результат функции с использованием этих входных значений (основная часть функции) после символа перехода (`=>`). Пример:

```
(x) => x + 1           // function that adds one to a value
(x, y) => x + y       // function that adds two values
```

Функция — это значение, аналогичное числу или текстовому значению. В следующем примере показана функция, являющаяся значением поля "Add", которая затем *вызывается* или выполняется из нескольких других полей. При вызове функции указывается набор значений, которые логически подставляются для требуемого набора входных значений в выражении тела функции.

```
[
  Add = (x, y) => x + y,
  OnePlusOne = Add(1, 1),      // 2
  OnePlusTwo = Add(1, 2)      // 3
]
```

Библиотека

Язык M включает общий набор определений, доступных для использования в выражении, который именуется *стандартной библиотекой* или просто библиотекой для краткости. Эти определения состоят из набора именованных значений. Имена значений, предоставленных в библиотеке, можно использовать в выражении без необходимости определять их явным образом в выражении. Пример:

```
Number.E                // Euler's number e (2.7182...)
Text.PositionOf("Hello", "ll") // 2
```

Операторы

Язык M включает набор операторов, которые можно использовать в выражениях. *Операторы* применяются к *операндам* для формирования символьных выражений. Например, в выражении `1 + 2` числа `1` и `2` являются операндами, а оператор представлен оператором сложения `(+)`.

Значение оператора может меняться в зависимости от типа значений операндов. Например, оператор сложения можно использовать с другими видами значений, отличными от чисел:

```
1 + 2                // numeric addition: 3
#time(12,23,0) + #duration(0,0,2,0)
                    // time arithmetic: #time(12,25,0)
```

Еще один пример оператора с операндом в зависимости от значения — оператор комбинирования `(&)`:

```
"A" & "BC"           // text concatenation: "ABC"
{1} & {2, 3}          // list concatenation: {1, 2, 3}
[ a = 1 ] & [ b = 2 ] // record merge: [ a = 1, b = 2 ]
```

Обратите внимание, что оператор может поддерживать не все сочетания значений. Пример:

```
1 + "2" // error: adding number and text is not supported
```

Выражения, при вычислении которых возникают неопределенные условия оператора, обрабатываются как ошибки. Дополнительные сведения об ошибках в языке M представлены ниже.

Метаданные

Метаданные — это сведения о значении, которое связано со значением. Метаданные представлены в

виде значения записи, которое называется *записью метаданных*. Поля записи метаданных можно использовать для хранения метаданных значения.

Каждое значение имеет запись метаданных. Если значение записи метаданных не указано, то запись метаданных пуста (не содержит поля).

Записи метаданных предоставляют способ связать дополнительную информацию с любым видом значения ненавязчивым способом. Связывание записи метаданных со значением не приводит к изменению значения или его поведения.

Значение записи метаданных `y` связывается с существующим значением `x` с помощью синтаксиса `x meta y`. Например, следующая связывает запись метаданных с полями `Rating` и `Tags` с текстовым значением `"Mozart"`:

```
"Mozart" meta [ Rating = 5, Tags = {"Classical"} ]
```

Для значений, которые уже содержат непустую запись метаданных, результат применения мета заключается в вычислении слияния записей существующей и новой записей метаданных. Например, следующие два выражения эквивалентны друг другу и предыдущему выражению:

```
("Mozart" meta [ Rating = 5 ]) meta [ Tags = {"Classical"} ]  
"Mozart" meta ([ Rating = 5 ] & [ Tags = {"Classical"} ])
```

Обратиться к записи метаданных для указанного значения можно с помощью функции *Value.Metadata*. В следующем примере выражение в поле `ComposerRating` обращается к записи метаданных значения в поле `Composer`, а затем обращается к полю `Rating` записи метаданных.

```
[  
  Composer = "Mozart" meta [ Rating = 5, Tags = {"Classical"} ],  
  ComposerRating = Value.Metadata(Composer)[Rating] // 5  
]
```

Выражение let

Многие из приведенных выше примеров содержат в результате выражения все литеральные значения выражения. Выражение *let* позволяет вычислить, снабдить именами набор значений, а затем использовать их в последующем выражении, которое следует за оператором *in*. Например, в нашем примере данных о продажах можно сделать следующее:

```
let  
  Sales2007 =  
    [  
      Year = 2007,  
      FirstHalf = 1000,  
      SecondHalf = 1100,  
      Total = FirstHalf + SecondHalf // 2100  
    ],  
  Sales2008 =  
    [  
      Year = 2008,  
      FirstHalf = 1200,  
      SecondHalf = 1300,  
      Total = FirstHalf + SecondHalf // 2500  
    ]  
in Sales2007[Total] + Sales2008[Total] // 4600
```

Результатом приведенного выше выражения является числовое значение (4600), вычисленное на основе значений, привязанных к именам Sales2007 и Sales2008 .

Выражение if

Выражение if осуществляет выбор между двумя выражениями на основе логического условия. Пример:

```
if 2 > 1 then
    2 + 2
else
    1 + 1
```

Первое выражение (2 + 2) выбирается, если логическое выражение (2 > 1) истинно, а второе выражение (1 + 1) выбирается, если оно ложно. Выбранное выражение (в данном случае 2 + 2) вычисляется и становится результатом выражения if (4).

ошибки

Ошибка в языке M означает, что при вычислении выражения не удалось получить значение.

Ошибки выдаются операторами и функциями, сталкивающимися с условиями ошибки, либо с помощью выражения error. Ошибки обрабатываются с помощью выражения try. При возникновении ошибки указывается значение, которое можно использовать для указания причины возникновения ошибки.

```
let Sales =
[
    Revenue = 2000,
    Units = 1000,
    UnitPrice = if Units = 0 then error "No Units"
                else Revenue / Units
],
UnitPrice = try Number.ToText(Sales[UnitPrice])
in "Unit Price: " &
    (if UnitPrice[HasError] then UnitPrice[Error][Message]
    else UnitPrice[Value])
```

Приведенный выше пример обращается к полю Sales[UnitPrice] и форматирует значение, формирующее результат:

```
"Unit Price: 2"
```

Если бы поле Units было равно нулю, то поле UnitPrice привело бы к возникновению ошибки, которая была бы обработана выражением try . В этом случае итоговое значение имело бы следующий вид:

```
"No Units"
```

Выражение try преобразует надлежащие значения и ошибки в значение записи, которое указывает, обрабатывало ли выражение try ошибку или нет, и правильное значение либо запись об ошибке, извлеченные при обработке ошибки. Например, рассмотрим следующее выражение, которое вызывает ошибку и затем сразу же обрабатывает ее:

```
try error "negative unit count"
```

Результатом вычисления этого выражения является следующее вложенное значение записи, поясняющее подстановки полей `[HasError]`, `[Error]` и `[Message]` в приведенном ранее примере с ценой на единицу.

```
[
  HasError = true,
  Error =
    [
      Reason = "Expression.Error",
      Message = "negative unit count",
      Detail = null
    ]
]
```

Распространена замена ошибок значениями по умолчанию. Чтобы добиться того же, но в компактной форме, можно использовать выражение `try` с необязательным предложением `otherwise`:

```
try error "negative unit count" otherwise 42
// 42
```

Лексическая структура

16.12.2020 • 18 minutes to read

Документы

Документ *M* представляет собой упорядоченную последовательность символов Юникода. *M* допускает использование различных классов символов Юникода в разных частях документа *M*. Сведения о классах символов Юникода см. в разделе *Стандарт Юникод, версия 3.0*, подраздел 4.5.

Документ состоит либо только из одного *выражения*, либо из групп *определений*, упорядоченных в *разделы*. Разделы подробно описаны в Главе 10. По существу, для считывания выражения из документа используются следующие шаги.

1. Документ декодируется в соответствии со схемой кодировки символов в последовательность символов Юникода.
2. Затем выполняется лексический анализ, в результате которого поток символов Юникода преобразуется в поток токенов. В оставшихся подразделах этого раздела рассматривается лексический анализ.
3. После выполняется синтаксический анализ, целью которого является преобразование потока токенов в форму, которую можно вычислить. Этот процесс рассматривается в последующих разделах.

Грамматические соглашения

Лексическая и синтаксическая грамматика представлены с помощью *грамматических правил*. Каждое грамматическое правило определяет неконечный символ и возможные расширения этого неконечного символа в последовательности неконечных или конечных символов. В грамматических правилах *неконечные* символы обозначаются курсивом, а *конечные* символы — шрифтом с фиксированной шириной.

Первая строка грамматического правила — это имя определяемого неконечного символа, за которым следует двоеточие. Каждая последующая строка с отступом содержит возможное расширение неконечного символа, представленная в виде последовательности неконечных или конечных символов. Например, следующее правило:

выражение-if:

```
if условие-if then выражение-true else выражение-false
```

определяет, что *выражение-if* состоит из токена `if`, за которым следуют *условие-if*, токен `then`, *выражение-true*, токен `else`, *выражение-false*.

При наличии более одного возможного расширения неконечного символа альтернативные варианты перечисляются в разных строках. Например, следующее правило:

список-переменных:

```
переменная  
список-переменных , переменная
```

определяет, что *список-переменных* состоит либо из *переменной*, либо из *списка-переменных*, за которым следует *переменная*. Иными словами, определение является рекурсивным и указывает, что список переменных состоит из одной или нескольких переменных, разделенных запятыми.

Суффикс с индексом "необязательно" используется для указания необязательного символа. Правило:

спецификация-поля:

`optional` *опционально* `ИМЯ-ПОЛЯ` `=` `ТИП-ПОЛЯ`

является сокращенным вариантом следующего:

спецификация-поля:

`ИМЯ-ПОЛЯ` `=` `ТИП-ПОЛЯ`

`optional` `ИМЯ-ПОЛЯ` `=` `ТИП-ПОЛЯ`

Это правило определяет, что *спецификация-поля* при необходимости может начинаться с конечного символа `optional`, за которым следуют *ИМЯ-ПОЛЯ*, конечный символ `=` и *ТИП-ПОЛЯ*.

Альтернативные варианты обычно перечисляются в отдельных строках, но в тех случаях, когда существует много альтернатив, списку расширений, заданных в одной строке, может предшествовать фраза "один из". Это простое сокращение для перечисления всех альтернативных вариантов в отдельной строке. Например, следующее правило:

десятичный-знак: один из

`0 1 2 3 4 5 6 7 8 9`

является сокращенным вариантом следующего:

десятичный-знак:

`0`
`1`
`2`
`3`
`4`
`5`
`6`
`7`
`8`
`9`

Лексический анализ

Правило *лексическая-единица* в производстве определяет лексическую грамматику для документа М. Каждый допустимый документ М следует этой грамматике.

лексическая-единица:

лексические-элементы *необязательно*

лексические-элементы:

лексический-элемент

лексический-элемент

лексические-элементы

лексический-элемент:

пробел

комментарий к токenu

На лексическом уровне документ М состоит из потока элементов *пробел*, *комментарий* и *токен*. Каждое из этих правил описывается в следующих разделах. В синтаксической грамматике учитываются только элементы *токен*.

Пробел

Пробелы используются для разделения комментариев и токенов в документе M. Пробелы включают символ пробела (который является частью класса Zs Юникода), а также горизонтальную и вертикальную табуляции, перевод страницы и последовательности символов новой строки. Последовательности символов новой строки включают возврат каретки, перевод строки, возврат каретки, за которым следует символ перевода строки, следующей строки и разделителя абзацев.

пробел.

Любой символ с классом Zs Юникода

Символ горизонтальной табуляции (U+0009)

Символ вертикальной табуляции (U+000B)

Символ перевода страницы (U+000C)

Символ возврата каретки (U+000D), за которым следует символ перевода строки (U+000A)

символ-новой-строки

символ-новой-строки.

Символ возврата каретки (U+000D)

Символ перевода строки (U+000A)

Символ следующей строки (U+0085)

Символ разделителя строк (U+2028)

Символ разделителя абзацев (U+2029)

Для обеспечения совместимости со средствами редактирования исходного кода, которые добавляют маркеры конца файла и позволяют просматривать документ в виде последовательности правильно завершенных строк, к документу M применяются следующие преобразования (по порядку).

- Если последний символ документа является символом сочетания клавиш CTRL+Z (U+001A), этот символ удаляется.
- Если этот документ не является пустым и если последний символ документа не является символом возврата каретки (U+000D), символом перевода строки (U+000A), разделителем строк (U+2028) или разделителем абзацев (U+2029), то в конец документа добавляется символ возврата каретки (U+000D).

Комментарии

Поддерживаются две формы комментариев: однострочные комментарии и комментарии с разделителями. *Однострочные комментарии* начинаются с символов `//` и дополняются до конца исходной строки. *Комментарии с разделителями* начинаются с символов `/*` и заканчиваются символами `*/`.

Комментарии с разделителями могут включать несколько строк.

комментарий:

однострочный-комментарий

комментарий-с-разделителями

однострочный-комментарий:

`//` *символы-однострочного-комментария* необязательно

символы-однострочного-комментария:

символ-однострочного-комментария *символы-однострочного-комментария* необязательно

символ-однострочного-комментария:

Любой символ Юникода, кроме *символа-новой-строки*

с комментарием-с-разделителями:

`/*` *текст-комментария-с-разделителями* необязательно *звездочки* `/`

текст-комментария-с-разделителями:

раздел-комментария-с-разделителями текст-комментария-с-разделителями необязательно

раздел-комментария-с-разделителями:

/

звездочки необязательно не-косая-черта-или-звездочка

звездочки:

* звездочки необязательно

не-косая-черта-или-звездочка:

Любой символ Юникода, кроме * или /

Комментарии не предусматривают вложения. Последовательности символов /* и */ не имеют специального значения в однострочном комментарии, а последовательности символов // и /* не имеют специального значения в комментариях с разделителями.

Комментарии не обрабатываются в текстовых литералах. Пример

```
/* Hello, world
*/
    "Hello, world"
```

содержит комментарий с разделителями.

Пример

```
// Hello, world
//
"Hello, world" // This is an example of a text literal
```

содержит несколько однострочных комментариев.

Токены

Токен — это идентификатор, ключевое слово, литерал, оператор или знак препинания. Пробелы и комментарии используются для разделения токенов, но не считаются токенами.

токен:

идентификатор

ключевое-слово

литерал

оператор-или-знак-препинания

Escape-последовательности

Текстовые значения М могут содержать произвольные символы Юникода. Однако текстовые литералы ограничены графическими символами и требуют использования *escape-последовательностей* для неграфических символов. Например, чтобы включить в текстовый литерал символ возврата каретки, перевода строки или табуляции, можно использовать escape-последовательности `#{cr}`, `#{lf}` и `#{tab}` соответственно. Чтобы внедрить символы начала escape-последовательности `#{` в текстовый литерал, необходимо экранировать сам символ `#`.

```
#{#}(
```

Escape-последовательности также могут содержать короткие (четыре шестнадцатеричные цифры) или длинные (восемь шестнадцатеричных цифр) значения кодовых точек Юникода. Таким образом, следующие три escape-последовательности эквивалентны:

```
#(000D)      // short Unicode hexadecimal value
#(0000000D) // long Unicode hexadecimal value
#(cr)        // compact escape shorthand for carriage return
```

```
#(cr,lf)
#(cr)#(lf)
```

```
#( список-escape-последовательностей )
```

одна-escape-последовательность

#

Литерал является представлением исходного кода значения

логический-литерал

буквальный-литерал

Литерал NULL используется для записи значения `null`. Значение `null` представляет отсутствующее

null

Логические литералы

Логический литерал используется для записи значений `true` и `false` и создает логическое значение.

логический-литерал:

`true`

`false`

Числовые литералы

Числовой литерал используется для записи числового значения и дает числовое значение.

числовой-литерал:

десятичный-числовой-литерал

шестнадцатеричный-числовой-литерал

десятичный-числовой-литерал:

десятичные-знаки `.` *экспоненциальная-часть-десятичных-знаков* *необязательно*

`.` *экспоненциальная-часть-десятичных-знаков* *необязательно*

экспоненциальная-часть-десятичных-знаков *необязательно*

десятичные-знаки:

десятичный-знак *десятичные-знаки* *необязательно*

десятичный-знак: один из

`0 1 2 3 4 5 6 7 8 9`

экспоненциальная-часть:

`e` *знак* *необязательно* *десятичные-знаки*

`E` *знак* *необязательно* *десятичные-знаки*

знак: один из следующего

`+ -`

шестнадцатеричный-числовой-литерал:

`0x` *шестнадцатеричные-знаки*

`0X` *шестнадцатеричные-знаки*

шестнадцатеричные-знаки:

шестнадцатеричный-знак *шестнадцатеричные-знаки* *необязательно*

шестнадцатеричный-знак: один из

`0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f`

Число может быть представлено в шестнадцатеричном формате (*шестнадцатеричные-знаки* перед символами `0x`). Например:

```
0xff // 255
```

Обратите внимание, что если в числовой литерал включена десятичная запятая, то после нее должна содержаться хотя бы одна цифра. Например, `1.3` является числовым литералом, но `1.` и `1.e3` не являются таковыми.

Текстовые литералы

Текстовый литерал используется для записи последовательности символов Юникода и создает текстовое значение.

текстовый-литерал:

`"` *символы-текстового-литерала* *необязательно* `"`

символы-текстового-литерала:

символ-текстового-литерала *необязательно*

символ-текстового-литерала:

один-текстовый-символ

escape-последовательность-символов

escape-последовательность-двойных-кавычек

один-текстовый-символ:

Любой символ, кроме `"` (`U+0022`) или `#` (`U+0023`), за которым следует `(` (`U+0028`)

escape-последовательность-двойных-кавычек:

`" "` (`U+0022` , `U+0022`)

Чтобы включить кавычки в текстовое значение, знак кавычек повторяется следующим образом:

```
"The ""quoted"" text" // The "quoted" text
```

Правило [символ-escape-последовательности](#) можно использовать для записи символов в текстовые значения без необходимости непосредственно кодировать их как символы Юникода в документе. Например, символ возврата каретки и перевода строки можно записать в текстовое значение следующим образом:

```
"Hello world#(cr,lf)"
```

Буквальные литералы

Буквальный литерал используется для хранения последовательности символов Юникода, которые введены пользователем как код, но не могут быть обработаны должным образом в виде кода. Во время выполнения он выдает значение ошибки.

буквальный-литерал:

`#!"` символы-текстового-литерала *необязательно* `"`

Идентификаторы

Идентификатор — это имя, используемое для ссылки на значение. Идентификаторы могут быть либо обычными идентификаторами, либо нестандартными идентификаторами.

идентификатор:

обычный-идентификатор

нестандартный-идентификатор

обычный-идентификатор:

доступный-идентификатор

доступный-идентификатор символ-точки обычный-идентификатор

доступный-идентификатор:

ключевое-слово-или-идентификатор, не являющийся ключевым-словом

ключевое-слово-или-идентификатор:

начальный-символ-идентификатора символы-идентификатора *необязательно*

начальный-символ-идентификатора:

буква

символ-подчеркивания

символы-идентификатора:

символ-идентификатора *необязательно*

символ-идентификатора:

буква

десятичный-знак

символ-подчеркивания

символ-соединения

символ-объединения

символ-форматирования

символ-точки:

`.` (U+002E)

символ-подчеркивания:

`_` (U+005F)

буква:

Символ Юникода класса Lu, Ll, Lt, Lm, Lo или NI

символ-объединения:

Символ Юникода класса Mn или Mc

десятичный-знак:

Символ Юникода класса Nd

символ-соединения:

Символ Юникода класса Pc

символ-форматирования:

Символ Юникода класса Cf

Нестандартный идентификатор можно использовать, чтобы разрешить использование любой последовательности из нулей или других символов Юникода в качестве идентификатора, включая ключевые слова, пробелы, комментарии, операторы и знаки препинания.

нестандартный-идентификатор:

`#"` *символы-текстового-литерала* необязательно `"`

Обратите внимание, что в *нестандартном идентификаторе* можно использовать escape-последовательности и двойные кавычки для экранирования так же, как в *текстовых литералах*.

В следующем примере используется идентификатор, указывающий на имя, содержащее символ пробела:

```
[
  #"1998 Sales" = 1000,
  #"1999 Sales" = 1100,
  #"Total Sales" = #"1998 Sales" + #"1999 Sales"
]
```

В следующем примере используется идентификатор, указывающий на включение оператора `+` в идентификатор:

```
[
  #"A + B" = A + B,
  A = 1,
  B = 2
]
```

Обобщенные идентификаторы

В М есть два места, где в идентификаторах, содержащих пробелы, ключевые слова или числовые литералы, нет неоднозначностей. Это имена полей записей в литерале записи и в операторе доступа к полю (`[ ]`). М позволяет использовать такие идентификаторы без необходимости использования нестандартных идентификаторов.

```
[
  Data = [ Base Line = 100, Rate = 1.8 ],
  Progression = Data[Base Line] * Data[Rate]
]
```

Идентификаторы, используемые для именования полей и доступа к ним, называются *обобщенными*

идентификаторами и определяются следующим образом:

обобщенный-идентификатор:

часть-обобщенного-идентификатора

обобщенный-идентификатор, разделенный только пробелами (`U+0020`)

часть-обобщенного-идентификатора

часть-обобщенного-идентификатора:

сегмент-обобщенного-идентификатора

десятичный-знак сегмент-обобщенного-идентификатора

сегмент-обобщенного-идентификатора:

ключевое-слово-или-идентификатор

ключевое-слово-или-идентификатор символ-точки ключевое-слово-или-идентификатор

Ключевые слова

Ключевое слово является последовательностью символов, аналогичных идентификатору, которая зарезервирована и не может использоваться в качестве идентификатора, кроме случаев, когда используется [нестандартный идентификатор](#) или [разрешено использование обобщенного идентификатора](#).

ключевое-слово: одно из

```
and as each else error false if in is let meta not null or otherwise
```

```
section shared then true try type #binary #date #datetime
```

```
#datetimezone #duration #infinity #nan #sections #shared #table #time
```

Операторы и знаки препинания

Существует несколько видов операторов и знаков препинания. Операторы используются в выражениях для описания операций, включающих один или несколько операндов. Например, выражение `a + b` использует оператор `+` для добавления двух операндов `a` и `b`. Знаки препинания предназначены для группирования и разделения.

оператор-или-знак-препинания: один из следующего

```
, ; = < <= > >= <> + - * / & ( ) [ ] { } @ ! ? => .. ...
```

Основные понятия

16.12.2020 • 14 minutes to read

Этот раздел описывает основные понятия, встречающиеся в последующих разделах.

Значения

Отдельный элемент данных называется *значением*. Если говорить в целом, то существуют две общие категории значений: *примитивные значения*, являющиеся атомарными, и *структурированные значения*, создаваемые из примитивных значений и других структурированных значений. Например, значения

```
1
true
3.14159
"abc"
```

являются примитивами, так как не состоят из других значений. С другой стороны, значения

```
{1, 2, 3}
[ A = {1}, B = {2}, C = {3} ]
```

создаются с использованием примитивных значений и — в случае записи — других структурированных значений.

Выражения

Выражение — это формула, используемая для создания значений. Выражение может быть сформировано с помощью различных синтаксических конструкций. Рассмотрим несколько примеров выражений.

Каждая строка является отдельным выражением.

```
"Hello World"      // a text value
123                // a number
1 + 2              // sum of two numbers
{1, 2, 3}          // a list of three numbers
[ x = 1, y = 2 + 3 ] // a record containing two fields:
                    //      x and y
(x, y) => x + y      // a function that computes a sum
if 2 > 1 then 2 else 1 // a conditional expression
let x = 1 + 1 in x * 2 // a let expression
error "A"          // error with message "A"
```

Как показано выше, простейшей формой выражения является литерал, представляющий значение.

Более сложные выражения создаются из других выражений, которые называются *частями выражения*.

Пример:

```
1 + 2
```

Приведенное выше выражение фактически состоит из трех выражений. Литералы `1` и `2` являются частями родительского выражения `1 + 2`.

Исполнение алгоритма, определенного синтаксическими конструкциями, используемыми в выражении, называется *вычислением* выражения. Для каждого вида выражений имеются правила его вычисления. Например, литеральное выражение, такое как `1`, даст постоянное значение, а выражение `a + b` будет принимать результирующие значения, полученные путем вычисления двух других выражений (`a` и `b`), и складывать их в соответствии с определенным набором правил.

Среды и переменные

Выражения вычисляются в заданной среде. *Среда* — это набор именованных значений, называемый *переменными*. Каждая переменная в среде имеет в ней уникальное имя, называемое *идентификатором*.

Выражение верхнего уровня (или *корневое*) вычисляется в *глобальной среде*. Глобальная среда предоставляется вычислителем выражений, а не определяется содержимым вычисляемого выражения. Содержимое глобальной среды включает определения стандартных библиотек и может зависеть от экспорта из разделов из некоторого набора документов. (Для простоты примеры в этом разделе подразумевают пустую глобальную среду. То есть предполагается, что стандартная библиотека отсутствует и не существует других определений на основе разделов.)

Среда, используемая для вычисления части выражения, определяется родительским выражением. Большинство типов родительских выражений вычисляют часть выражения в той же среде, в которой были вычислены они сами, но некоторые используют другую среду. Глобальная среда — это *родительская среда*, в которой вычисляется глобальное выражение.

Например, *выражение-инициализатора-записи* вычисляет часть выражения для каждого поля в измененной среде. Измененная среда включает переменную для каждого поля записи, за исключением того, которое инициализируется. Включение других полей записи позволяет полям зависеть от значений полей. Пример:

```
[
  x = 1,          // environment: y, z
  y = 2,          // environment: x, z
  z = x + y       // environment: x, y
]
```

Аналогичным образом *выражение-let* вычисляет часть выражения для каждой переменной в среде, содержащей каждую из переменных для "let", за исключением той, которая инициализируется. *Выражение-let* вычисляет выражение, следующее за "in", в среде, содержащей все переменные:

```
let
  x = 1,          // environment: y, z
  y = 2,          // environment: x, z
  z = x + y       // environment: x, y
in
  x + y + z       // environment: x, y, z
```

(Фактически *выражение-инициализатора-записи* и *выражение-let* определяют две среды, одна из которых включает в себя инициализируемую переменную. Это удобно для расширенных рекурсивных определений и описано в разделе [ссылки на идентификаторы](#).

Чтобы формировать среды для частей выражений, новые переменные объединяются с переменными в родительской среде. В следующем примере показаны среды для вложенных записей:

```
[
  a =
  [
    x = 1,      // environment: b, y, z
    y = 2,      // environment: b, x, z
    z = x + y   // environment: b, x, y
  ],
  b = 3         // environment: a
]
```

В следующем примере показаны среды для записи, вложенной в "let":

```
Let
  a =
  [
    x = 1,      // environment: b, y, z
    y = 2,      // environment: b, x, z
    z = x + y   // environment: b, x, y
  ],
  b = 3         // environment: a
in
  a[z] + b     // environment: a, b
```

Объединение переменных в среде может привести к конфликту между переменными (так как каждая переменная в среде должна иметь уникальное имя). Конфликт разрешается следующим образом: если имя новой объединяемой переменной совпадает с существующей переменной в родительской среде, новая переменная будет иметь приоритет в новой среде. В следующем примере внутренняя (более глубоко вложенная) переменная `x` будет иметь приоритет над внешней переменной `x`.

```
[
  a =
  [
    x = 1,      // environment: b, x (outer), y, z
    y = 2,      // environment: b, x (inner), z
    z = x + y   // environment: b, x (inner), y
  ],
  b = 3,        // environment: a, x (outer)
  x = 4         // environment: a, b
]
```

Ссылки на идентификаторы

Ссылка-на-идентификатор используется для ссылки на переменную в среде.

выражение-идентификатора:

ссылка-на-идентификатор

ссылка-на-идентификатор:

эксклюзивная-ссылка-на-идентификатор

инклюзивная-ссылка-на-идентификатор

Самая простая форма ссылки на идентификатор — это *эксклюзивная-ссылка-на-идентификатор*.

эксклюзивная-ссылка-на-идентификатор:

идентификатор

Ошибкой является то, когда *эксклюзивная-ссылка-на-идентификатор* ссылается на переменную, которая не является частью среды выражения, в которой отображается идентификатор, либо на идентификатор,

который в данный момент инициализируется.

Инклюзивную-ссылку-на-идентификатор можно использовать для получения доступа к среде, включающей в себя инициализируемый идентификатор. Если она использовалась в контексте, где инициализируемый идентификатор отсутствует, то она эквивалентна *экслюзивной-ссылке-на-идентификатор*.

Инклюзивная-ссылка-на-идентификатор:

@ идентификатор

Это удобно при определении рекурсивных функций, так как имя функции обычно не входит в область действия.

```
[
  Factorial = (n) =>
    if n <= 1 then
      1
    else
      n * @Factorial(n - 1), // @ is scoping operator

  x = Factorial(5)
]
```

Как и в случае с *выражением-инициализатора-записи*, *инклюзивную-ссылку-на-идентификатор* можно использовать в *выражении-let* для получения доступа к среде, включающей в себя инициализируемый идентификатор.

Порядок оценки

Рассмотрим следующее выражение, инициализирующее запись:

```
[
  C = A + B,
  A = 1 + 1,
  B = 2 + 2
]
```

При вычислении это выражение дает следующее значение записи:

```
[
  C = 6,
  A = 2,
  B = 4
]
```

Выражение указывает, что для выполнения вычисления `A + B` для поля `C` должны быть известны значения полей `A` и `B`. Это пример *упорядочения зависимостей* вычислений, предоставляемого выражением. Средство вычисления `M` придерживается упорядочения зависимостей, предоставляемого выражениями, но может выполнять оставшиеся вычисления в любом удобном для него порядке. Например, порядок вычисления может быть следующим:

```
A = 1 + 1
B = 2 + 2
C = A + B
```

Или он может быть следующим:

```
B = 2 + 2
A = 1 + 1
C = A + B
```

Либо, поскольку `A` и `B` не зависят друг от друга, их можно вычислить параллельно:

```
B = 2 + 2 параллельно с A = 1 + 1
C = A + B
```

Побочные эффекты

Предоставление средству вычисления выражений возможности автоматического расчета порядка вычислений в случаях, когда нет явных зависимостей, заданных выражением, является простой и эффективной вычислительной моделью.

Однако она полагается на возможность изменения порядка вычислений. Так как выражения могут вызывать функции и эти функции могут наблюдать за состоянием, внешним по отношению к выражению, выдавая внешние запросы, можно создать сценарий, в котором порядок вычисления имеет значение, но не фиксируется в частичном порядке выражения. Например, функция может считывать содержимое файла. Если эта функция вызывается многократно, то можно наблюдать за внешними изменениями в этом файле, поэтому изменение порядка может привести к наблюдаемым различиям в работе программы. В зависимости от такого наблюдаемого упорядочения оценки правильность выражения `M` вызывает зависимость от определенных вариантов реализации, которые могут отличаться между средствами вычисления или даже меняться для одного средства вычисления в зависимости от различных обстоятельств.

Неизменяемость

После вычисления значения оно является *неизменяемым*, то есть больше не может быть изменено. Это упрощает модель для вычисления выражения, а также позволяет легче определить результат, так как значение уже невозможно изменить после того, как оно использовалось для вычисления последующей части выражения. Например, поле записи вычисляется только при необходимости. Однако после вычисления оно остается фиксированным в течение времени существования записи. Даже если попытка вычисления поля вызвала ошибку, эта же ошибка будет возникать снова при каждой попытке обращения к этому полю записи.

Важное исключение из правила "неизменяемости после вычисления" применяется к значениям списка и таблицы. Оба имеют *потокową семантику*. То есть повторяющееся перечисление элементов в списке или строк в таблице может привести к различным результатам. Потокová семантика позволяет создавать выражения `M`, преобразующие наборы данных, которые не помещаются в память целиком.

Кроме того, обратите внимание, что приложение-функция *не* то же самое, что и конструкция значения. Функции библиотеки могут предоставлять внешнее состояние (например, текущее время или результаты запроса к базе данных, которая изменяется со временем), что делает их *недетерминированными*. Несмотря на то что функции, определенные в `M`, не будут обладать таким недетерминированным поведением, они могут проявлять его, будучи определенными для вызова других функций, являющихся недетерминированными.

Окончательным источником недетерминизма в `M` являются *ошибки*. При возникновении ошибки прекращают вычисление (вплоть до уровня, на котором они обрабатываются выражением `try`). Обычно невозможно узнать, вызвал ли `a + b` вычисление `a` перед `b` или `b` перед `a` (для простоты здесь игнорируется параллелизм). Однако если часть выражения, вычисленная первой, вызывает ошибку, то можно определить, какое из двух выражений было вычислено первым.

Значения

16.12.2020 • 29 minutes to read

Значение — это данные, получаемые при вычислении выражения. Этот раздел описывает виды значений в языке M. Каждый тип значения связан с синтаксисом литерала, набором значений этого вида, набором операторов, определенных для этого набора значений, и внутренним типом, приписываемым к вновь сформированным значениям.

ВИД	ЛИТЕРАЛ
<i>Null</i>	<code>null</code>
<i>Логическое</i>	<code>true</code> <code>false</code>
<i>Число</i>	<code>0</code> <code>1</code> <code>-1</code> <code>1.5</code> <code>2.3e-5</code>
<i>Time</i>	<code>#time(09,15,00)</code>
<i>Date</i>	<code>#date(2013,02,26)</code>
<i>DateTime</i>	<code>#datetime(2013,02,26, 09,15,00)</code>
<i>DateTimeZone</i>	<code>#datetimezone(2013,02,26, 09,15,00, 09,00)</code>
<i>Длительность</i>	<code>#duration(0,1,30,0)</code>
<i>Текстовые</i>	<code>"hello"</code>
<i>Двоичный</i>	<code>#binary("AQID")</code>
<i>Список</i>	<code>{1, 2, 3}</code>
<i>Запись</i>	<code>[A = 1, B = 2]</code>
<i>Таблица</i>	<code>#table({"X","Y"},{{0,1},{1,0}})</code>
<i>Компонент</i>	<code>(x) => x + 1</code>
<i>Тип</i>	<code>type { number }</code> <code>type table [A = any, B = text]</code>

В следующих разделах подробно рассматривается каждый вид значений. Типы и приписывание типов формально определены в разделе [Типы](#). Значения функций определены в разделе [Функции](#). В следующих разделах перечислены операторы, определенные для каждого вида значений, и приведены примеры. Полное определение семантики оператора приведено в разделе [Операторы](#).

NULL

Значение *NULL* используется для обозначения отсутствия значения либо значения в неопределенном или неизвестном состоянии. Значение *NULL* записывается с помощью литерала `null`. Для значений *NULL* определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x > y</code>	Больше чем
<code>x >= y</code>	Больше или равно
<code>x < y</code>	Меньше чем
<code>x <= y</code>	Меньше или равно
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно

Собственный тип значения `null` — это встроенный тип `null`.

Логические

Логическое значение используется для логических операций и имеет значение `true` или `false`. Логическое значение записывается с помощью литералов `true` и `false`. Для логических значений определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x > y</code>	Больше чем
<code>x >= y</code>	Больше или равно
<code>x < y</code>	Меньше чем
<code>x <= y</code>	Меньше или равно
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно
<code>x or y</code>	Условное логическое ИЛИ
<code>x and y</code>	Условное логическое И
<code>not x</code>	Логическое НЕ

Собственный тип обоих логических значений (`true` и `false`) — это внутренний тип `logical`.

Число;

Числовое значение используется для числовых и арифметических операций. Ниже приведены примеры числовых литералов:

```
3.14 // Fractional number
-1.5 // Fractional number
1.0e3 // Fractional number with exponent
123 // Whole number
1e3 // Whole number with exponent
0xff // Whole number in hex (255)
```

Число выводится с точностью не ниже *Double* (но может иметь и более высокую точность). Представление *Double* соответствует стандарту IEEE на 64-разрядные значения двойной точности для арифметических операций с плавающей запятой [IEEE 754-2008]. (Представление *Double* имеет приблизительный динамический диапазон от $5,0 \times 10^{-324}$ до $1,7 \times 10^{308}$ с точностью до 15–16 цифр.)

Следующие специальные значения также считаются *числовыми*.

- Положительный ноль и отрицательный ноль. В большинстве случаев положительный ноль и отрицательный ноль ведут себя одинаково — как просто нулевое значение, но **определенные операции для них различаются**.
- Положительная бесконечность (`#infinity`) и отрицательная бесконечность (`-#infinity`). Бесконечности создаются такими операциями, как деление ненулевого числа на ноль. Например, `1.0 / 0.0` дает положительную бесконечность, а `-1.0 / 0.0` — отрицательную.
- *Нечисловое* значение (`#nan`), часто обозначаемое как "не число" или "NaN". Нечисловые значения создаются недопустимыми операциями с плавающей запятой, например делением нуля на ноль.

Двоичные математические операции выполняются с использованием *точности*. Точность определяет область значений, в которой округляются операнды, и область значений, в которой выполняется операция. При отсутствии явно заданной точности такие операции выполняются с помощью использования *двойной точности*.

- Если результат математической операции слишком мал для целевого формата, он превращается в положительный или отрицательный ноль.
- Если результат математической операции слишком велик для целевого формата, он превращается в положительную или отрицательную бесконечность.
- Если математическая операция является недопустимой, результат операции становится нечисловым.
- Если один или оба операнда операции с плавающей запятой не являются числовыми, результат операции становится нечисловым.

Для числовых значений определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x > y</code>	Больше чем
<code>x >= y</code>	Больше или равно
<code>x < y</code>	Меньше чем
<code>x <= y</code>	Меньше или равно

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно
<code>x + y</code>	Сумма
<code>x - y</code>	Разность
<code>x * y</code>	Продукт
<code>x / y</code>	Частное
<code>+x</code>	Унарный плюс
<code>-x</code>	Отрицание

Собственный тип числовых значений — это встроенный тип `number`.

Время

Значение времени хранит непрозрачное представление времени суток. Время кодируется в виде числа *тактов*, начиная с *полночи*, то есть подсчитываются 100-наносекундные такты в 24-часовом формате. Максимальное число *тактов* начиная с *полночи* соответствует 23:59:59,9999999 часам.

Значения времени могут быть созданы с помощью встроенной функции `#time`.

<code>#time(hour, minute, second)</code>
--

Должно соблюдаться следующее, или возникает ошибка с кодом причины `Expression.Error`.

$0 \leq \text{hour (час)} \leq 24$

$0 \leq \text{minute (минута)} \leq 59$

$0 \leq \text{second (секунда)} \leq 59$

Кроме того, если `hour = 24`, то минуты и секунды должны быть равны нулю.

Для значений времени определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно
<code>x >= y</code>	Больше или равно
<code>x > y</code>	Больше чем
<code>x < y</code>	Меньше чем

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x <= y</code>	Меньше или равно

Следующие операторы допускают, чтобы один или оба их операнда были датой.

ОПЕРАТОР	ЛЕВЫЙ ОПЕРАНД	ПРАВЫЙ ОПЕРАНД	ЗНАЧЕНИЕ
<code>x + y</code>	<code>time</code>	<code>duration</code>	Смещение даты по длительности
<code>x + y</code>	<code>duration</code>	<code>time</code>	Смещение даты по длительности
<code>x - y</code>	<code>time</code>	<code>duration</code>	Смещение даты по длительности, к которой применено отрицание
<code>x - y</code>	<code>time</code>	<code>time</code>	Длительность между датами
<code>x & y</code>	<code>date</code>	<code>time</code>	Объединенное значение datetime

Собственный тип значений времени — это встроенный тип `time`.

Дата

Значение даты хранит непрозрачное представление определенного дня. Дата кодируется как число *дней с начала эпохи* начиная с 1 января 0001 года нашей эры по григорианскому календарю. Максимальное число дней с начала эпохи равно 3652058, что соответствует 31 декабря 9999 года.

Значения даты могут быть созданы с помощью встроенной функции `#date`.

<code>#date(year, month, day)</code>

Должно соблюдаться следующее, или возникает ошибка с кодом причины `Expression.Error`.

- $1 \leq \text{year (год)} \leq 9999$
- $1 \leq \text{month (месяц)} \leq 12$
- $1 \leq \text{day (день)} \leq 31$

Кроме того, день должен быть допустимым для выбранного месяца и года.

Для значений даты определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x >= y</code>	Больше или равно
<code>x > y</code>	Больше чем
<code>x < y</code>	Меньше чем
<code>x <= y</code>	Меньше или равно

Следующие операторы допускают, чтобы один или оба их операнда были датой.

ОПЕРАТОР	ЛЕВЫЙ ОПЕРАНД	ПРАВЫЙ ОПЕРАНД	ЗНАЧЕНИЕ
<code>x + y</code>	<code>date</code>	<code>duration</code>	Смещение даты по длительности
<code>x + y</code>	<code>duration</code>	<code>date</code>	Смещение даты по длительности
<code>x - y</code>	<code>date</code>	<code>duration</code>	Смещение даты по длительности, к которой применено отрицание
<code>x - y</code>	<code>date</code>	<code>date</code>	Длительность между датами
<code>x & y</code>	<code>date</code>	<code>time</code>	Объединенное значение datetime

Собственный тип значений даты — это встроенный тип `date`.

DateTime

Значение *datetime* содержит дату и время.

Значения DateTime могут быть созданы с помощью встроенной функции `#datetime`.

```
#datetime(year, month, day, hour, minute, second)
```

Должно соблюдаться следующее, или возникает ошибка с кодом причины Expression.Error. $1 \leq \text{year (год)} \leq 9999$

$1 \leq \text{month (месяц)} \leq 12$

$1 \leq \text{day (день)} \leq 31$

$0 \leq \text{hour (час)} \leq 23$

$0 \leq \text{minute (минута)} \leq 59$

$0 \leq \text{second (секунда)} \leq 59$

Кроме того, день должен быть допустимым для выбранного месяца и года.

Для значений datetime определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно
<code>x >= y</code>	Больше или равно
<code>x > y</code>	Больше чем
<code>x < y</code>	Меньше чем
<code>x <= y</code>	Меньше или равно

Следующие операторы допускают, чтобы один или оба их операнда были значением `datetime`.

ОПЕРАТОР	ЛЕВЫЙ ОПЕРАНД	ПРАВЫЙ ОПЕРАНД	ЗНАЧЕНИЕ
<code>x + y</code>	<code>datetime</code>	<code>duration</code>	Смещение значения <code>datetime</code> по длительности
<code>x + y</code>	<code>duration</code>	<code>datetime</code>	Смещение значения <code>datetime</code> по длительности
<code>x - y</code>	<code>datetime</code>	<code>duration</code>	Смещение значения <code>datetime</code> по длительности, к которой применено отрицание
<code>x - y</code>	<code>datetime</code>	<code>datetime</code>	Длительность между значениями <code>datetime</code>

Собственный тип значений `datetime` — это встроенный тип `datetime`.

DateTimeZone

Значение `datetimezone` содержит значение `datetime` и часовой пояс. *Часовой пояс* кодируется как число минут, составляющих смещение от *UTC*, что учитывает количество минут, на которое часть времени в значении `datetime` должна быть смещена относительно времени в формате *UTC*. Минимальное число минут смещения от *UTC* равно 840, что соответствует смещению от *UTC* -14:00 или времени на четырнадцать часов раньше времени в формате *UTC*. Максимальное число минут смещения от *UTC* равно 840, что соответствует смещению от *UTC* 14:00.

Значения `DateTimeZone` могут быть созданы с помощью встроенной функции `#datetimezone`.

```
#datetimezone(  
    year, month, day,  
    hour, minute, second,  
    offset-hours, offset-minutes)
```

Должно соблюдаться следующее, или возникает ошибка с кодом причины `Expression.Error`.

- 1 ≤ year (год) ≤ 9999
- 1 ≤ month (месяц) ≤ 12
- 1 ≤ day (день) ≤ 31
- 0 ≤ hour (час) ≤ 23
- 0 ≤ minute (минута) ≤ 59
- 0 ≤ second (секунда) ≤ 59
- 14 ≤ offset-hours (часы смещения) ≤ 14
- 59 ≤ offset-minutes (минуты смещения) ≤ 59

Кроме того, день должен быть допустимым для выбранного месяца и года, а если значение offset-hours = 14, то offset-minutes ≤ 0, и если offset-hours = -14, то offset-minutes ≥ 0.

Для значений `datetimezone` определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно
<code>x >= y</code>	Больше или равно
<code>x > y</code>	Больше чем
<code>x < y</code>	Меньше чем
<code>x <= y</code>	Меньше или равно

Следующие операторы допускают, чтобы один или оба их операнда были значением `datetimezone`.

ОПЕРАТОР	ЛЕВЫЙ ОПЕРАНД	ПРАВЫЙ ОПЕРАНД	ЗНАЧЕНИЕ
<code>x + y</code>	<code>datetimezone</code>	<code>duration</code>	Смещение значения <code>datetimezone</code> по длительности
<code>x + y</code>	<code>duration</code>	<code>datetimezone</code>	Смещение значения <code>datetimezone</code> по длительности
<code>x - y</code>	<code>datetimezone</code>	<code>duration</code>	Смещение значения <code>datetimezone</code> по длительности, к которой применено отрицание
<code>x - y</code>	<code>datetimezone</code>	<code>datetimezone</code>	Длительность между значениями <code>datetimezone</code>

Собственный тип значений `datetimezone` — это встроенный тип `datetimezone`.

Длительность

Значение *длительности* хранит непрозрачное представление расстояния между двумя точками на

временной шкале, измеряемое 100-наносекундными тактами. Величина *длительности* может быть положительной или отрицательной, при этом положительные значения обозначают движение по времени в прямом направлении, а отрицательные — в обратном. Минимальное значение, которое может храниться для *длительности*, составляет −9 223 372 036 854 775 808 тактов или 10 675 199 дней 2 часа 48 минут 05,4775808 секунды при движении по времени в обратном направлении. Максимальное значение, которое может храниться для *длительности* составляет −9 223 372 036 854 775 807 тактов или 10 675 199 дней 2 часа 48 минут 05,4775807 секунды при движении по времени в прямом направлении.

Значения длительности могут быть созданы с помощью встроенной функции `#duration`.

```
#duration(0, 0, 0, 5.5)      // 5.5 seconds
#duration(0, 0, 0, -5.5)     // -5.5 seconds
#duration(0, 0, 5, 30)       // 5.5 minutes
#duration(0, 0, 5, -30)      // 4.5 minutes
#duration(0, 24, 0, 0)       // 1 day
#duration(1, 0, 0, 0)        // 1 day
```

Для значений длительности определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно
<code>x >= y</code>	Больше или равно
<code>x > y</code>	Больше чем
<code>x < y</code>	Меньше чем
<code>x <= y</code>	Меньше или равно

Кроме того, следующие операторы допускают, чтобы один или оба их операнда были значением длительности.

ОПЕРАТОР	ЛЕВЫЙ ОПЕРАНД	ПРАВЫЙ ОПЕРАНД	ЗНАЧЕНИЕ
<code>x + y</code>	<code>datetime</code>	<code>duration</code>	Смещение значения <code>datetime</code> по длительности
<code>x + y</code>	<code>duration</code>	<code>datetime</code>	Смещение значения <code>datetime</code> по длительности
<code>x + y</code>	<code>duration</code>	<code>duration</code>	Сумма длительностей
<code>x - y</code>	<code>datetime</code>	<code>duration</code>	Смещение значения <code>datetime</code> по длительности, к которой применено отрицание
<code>x - y</code>	<code>datetime</code>	<code>datetime</code>	Длительность между значениями <code>datetime</code>

ОПЕРАТОР	ЛЕВЫЙ ОПЕРАНД	ПРАВЫЙ ОПЕРАНД	ЗНАЧЕНИЕ
<code>x - y</code>	<code>duration</code>	<code>duration</code>	Разница длительностей
<code>x * y</code>	<code>duration</code>	<code>number</code>	Длительность, увеличенная в N раз
<code>x * y</code>	<code>number</code>	<code>duration</code>	Длительность, увеличенная в N раз
<code>x / y</code>	<code>duration</code>	<code>number</code>	Частное длительности

Собственный тип значений длительности — это встроенный тип `duration`.

Текст

Текстовое значение представляет последовательность символов Юникода. Текстовые значения имеют форму литерала со следующей грамматикой:

_текстовый-литерал:

`"` символы-текстового-литерала необязательно `"`

символы-текстового-литерала:

символ-текстового-литерала необязательно

символ-текстового-литерала:

один-текстовый-символ

escape-последовательность-символов

escape-последовательность-двойных-кавычек

один-текстовый-символ:

Любой символ, кроме `"` (`U+0022`) или `#` (`U+0023`), за которым следует `(` (`U+0028`)

escape-последовательность-двойных-кавычек:

`" "` (`U+0022`, `U+0022`)

Ниже приведен пример *текстового* значения:

```
"ABC" // the text value ABC
```

Для *текстовых* значений определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно
<code>x >= y</code>	Больше или равно
<code>x > y</code>	Больше чем
<code>x < y</code>	Меньше чем
<code>x <= y</code>	Меньше или равно

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x & y</code>	Объединение

Собственный тип текстовых значений — это встроенный тип `text`.

Двоичный

Двоичное значение представляет последовательность байтов. Литеральный формат отсутствует. Для создания двоичных значений предусмотрено несколько функций стандартной библиотеки. Например, `#binary` можно использовать для создания двоичного значения из списка байтов:

```
#binary( {0x00, 0x01, 0x02, 0x03} )
```

Для *двоичных* значений определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно
<code>x >= y</code>	Больше или равно
<code>x > y</code>	Больше чем
<code>x < y</code>	Меньше чем
<code>x <= y</code>	Меньше или равно

Собственный тип двоичных значений — это встроенный тип *binary*.

Список

Значение списка — это значение, которое при перечислении формирует последовательность значений. Значение, создаваемое списком, может содержать любой тип значения, включая список. Списки можно создавать с использованием синтаксиса инициализации, как показано ниже.

выражение-списка:

`{ элемент-списка необязательно }`

элемент-списка:

элемент

`элемент` `,` *элемент-списка*

элемент:

выражение

выражение `..` *выражение*

Ниже приведен пример *выражения-списка*, который определяет список с тремя текстовыми значениями:

`"A"` `,` `"B"` и `"C"`.

```
{"A", "B", "C"}
```

Значение `"A"` является первым элементом в списке, а значение `"C"` — последним.

- Элементы списка не оцениваются до тех пор, пока к ним не обратятся.
- Несмотря на то что значения списка, созданные с помощью синтаксиса списка, будут создавать элементы в том порядке, в котором они отображаются в *списке-элементов*, в общем случае списки, возвращаемые функциями библиотеки, могут создавать разные наборы или различное количество значений при каждом перечислении.

Чтобы включить в список последовательность целого числа, можно использовать форму `a..b`:

```
{ 1, 5..9, 11 } // { 1, 5, 6, 7, 8, 9, 11 }
```

Число элементов в списке, называемое *размером списка*, можно определить с помощью функции

`List.Count`.

```
List.Count({true, false}) // 2
List.Count({})           // 0
```

Фактически список может содержать бесконечное количество элементов. Размер `List.Count` для таких списков не определен, что может вызвать ошибку или препятствовать завершению операции.

Если список не содержит элементов, он называется *пустым*. Пустой список записывается следующим образом:

```
{ } // empty list
```

Для списков определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно
<code>x & y</code>	Concatenate

Например:

```
{1, 2} & {3, 4, 5} // {1, 2, 3, 4, 5}
{1, 2} = {1, 2}    // true
{2, 1} <> {1, 2}   // true
```

Собственный тип значений списка — это встроенный тип `list`, который указывает тип элемента `any`.

Запись

Значение записи является упорядоченной последовательностью полей. *Поле* состоит из *имени поля*, которое является текстовым значением, однозначно определяющим поле в записи, и *значения поля*.

Значение поля может быть значением любого вида, включая запись. Записи можно создавать с использованием синтаксиса инициализации, как показано ниже:

выражение-записи:

[*список-полей* необязательно]

список-полей:

поле

поле , *список-полей*

поле:

имя-поля = *выражение*

имя-поля:

обобщенный-идентификатор

нестандартный-идентификатор

Следующий пример создает запись с полем `x` со значением `1` и полем `y` со значением `2`.

```
[ x = 1, y = 2 ]
```

Следующий пример создает запись с полем `a` со значением вложенной записи. Вложенная запись содержит поле `b` со значением `2`.

```
[ a = [ b = 2 ] ]
```

При вычислении выражения записи происходит следующее.

- Выражение, присвоенное каждому имени поля, используется для определения значения связанного поля.
- Если выражение, присвоенное имени поля, при вычислении дает значение, то оно становится значением поля итоговой записи.
- Если выражение, присвоенное имени поля, при вычислении вызывает ошибку, то регистрируется факт возникновения ошибки и ее значение. Последующее обращение к этому полю приведет к повторному возникновению ошибки с записанным значением ошибки.
- Выражение вычисляется в среде, подобной родительской среде, только с объединенными переменными, что соответствует значению каждого поля записи, за исключением инициализируемого.
- Значение в записи не вычисляется до тех пор, пока не будет совершено обращение к соответствующему полю.
- Значение в записи вычисляется не более одного раза.
- Результатом выражения является значение записи с пустой записью метаданных.
- Порядок полей в записи определяется порядком, в котором они стоят в *выражении-инициализатора-записи*.
- Каждое указанное имя поля должно быть уникальным в пределах записи, либо возникает ошибка. Имена сравниваются по порядковому номеру.

```
[ x = 1, x = 2 ] // error: field names must be unique
[ X = 1, x = 2 ] // OK
```

Запись без полей называется *пустой* и записывается следующим образом:

```
[ ] // empty record
```

Хотя порядок полей в записи не имеет значения при обращении к полю или при сравнении двух записей, он важен в других контекстах, например при перечислении полей записи.

Одни и те же две записи дают разные результаты при получении полей:

```
Record.FieldNames([ x = 1, y = 2 ]) // [ "x", "y" ]  
Record.FieldNames([ y = 1, x = 2 ]) // [ "y", "x" ]
```

Количество полей в записи можно определить с помощью функции `Record.FieldCount`. Например:

```
Record.FieldCount([ x = 1, y = 2 ]) // 2  
Record.FieldCount([ ]) // 0
```

Кроме использования синтаксиса инициализации записей `[ ]`, записи можно создавать из списка значений, списка имен полей или типа записи. Например:

```
Record.FromList({1, 2}, {"a", "b"})
```

Приведенный выше пример эквивалентен следующему:

```
[ a = 1, b = 2 ]
```

Для значений записей определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно
<code>x & y</code>	Объединить

Следующие примеры иллюстрируют использование приведенных выше операторов. Обратите внимание, что объединение записей использует поля правого операнда для переопределения полей левого операнда, если имена полей перекрываются друг с другом.

```
[ a = 1, b = 2 ] & [ c = 3 ] // [ a = 1, b = 2, c = 3 ]  
[ a = 1, b = 2 ] & [ a = 3 ] // [ a = 3, b = 2 ]  
[ a = 1, b = 2 ] = [ b = 2, a = 1 ] // true  
[ a = 1, b = 2, c = 3 ] <> [ a = 1, b = 2 ] // true
```

Собственный тип значений записей — это встроенный тип `record`, который указывает открытый пустой список полей.

Таблица

Табличное значение является упорядоченной последовательностью строк. *Строка* является

упорядоченной последовательностью значений. Тип таблицы определяет длину всех строк в таблице, имена столбцов таблицы, типы столбцов таблицы, а также структуру ключей таблицы (если они есть).

Для таблиц не существует литерального синтаксиса. Для создания двоичных значений предусмотрено несколько функций стандартной библиотеки. Например, `#table` можно использовать для создания таблицы из списка списков строк и списка имен заголовков:

```
#table({"x", "x^2"}, {{1,1}, {2,4}, {3,9}})
```

Приведенный выше пример создает таблицу с двумя столбцами, которые имеют тип `type any`.

`#table` также можно использовать для указания полного типа таблицы:

```
#table(
  type table [Digit = number, Name = text],
  {{1,"one"}, {2,"two"}, {3,"three"}}
)
```

В этом случае новое табличное значение имеет тип таблицы, указывающий имена столбцов и типы столбцов.

Для значений таблиц определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно
<code>x & y</code>	Объединение

При объединении таблиц группируются столбцы с похожими именами и указываются значения `null` для столбцов, присутствующих только в одной из таблиц операндов. В следующем примере показано объединение таблиц.

```
#table({"A","B"}, {{1,2}})
& #table({"B","C"}, {{3,4}})
```

A	B	C
1	2	null
null	3	4

Собственный тип табличных значений — это пользовательский тип таблиц (производный от встроенного типа `table`), который перечисляет имена столбцов, задает все типы столбцов (при наличии) и не имеет ключей. (Дополнительные сведения о типах таблиц см. в разделе [Типы таблиц](#).)

Функция

Значение функции — это значение, которое сопоставляет набор аргументов с одним значением.

Сведения о значениях *функции* описаны в разделе [Функции](#).

Тип

Значение типа — это значение, которое классифицирует другие значения. Сведения о значениях *типа* описаны в разделе [Типы](#).

Типы

16.12.2020 • 21 minutes to read

Значение типа — это значение, которое *классифицирует* другие значения. Значение, классифицированное типом, *соответствует* ему. Система типов M состоит из следующих видов типов.

- Примитивные типы, которые классифицируют примитивные значения (`binary` , `date` , `datetime` , `datetimezone` , `duration` , `list` , `logical` , `null` , `number` , `record` , `text` , `time` , `type`), а также включают несколько абстрактных типов (`function` , `table` , `any` и `none`).
- Типы записей, которые классифицируют значения записей на основе имен полей и типов значений.
- Типы списков, которые классифицируют списки, используя базовый тип отдельного элемента.
- Типы функций, которые классифицируют значения функций на основе типов их параметров и возвращаемых значений.
- Типы таблиц, которые классифицируют табличные значения на основе имен столбцов, типов столбцов и ключей.
- Типы, допускающие значение NULL, которые классифицируют значение NULL в дополнение ко всем значениям, классифицируемым базовым типом.
- Типы типов, которые классифицируют значения, являющиеся типами.

Набор *примитивных типов* включает типы примитивных значений, несколько *абстрактных типов*, типы, которые не могут однозначно классифицировать какие-либо значения: `function` , `table` , `any` и `none` . Все значения функций соответствуют абстрактному типу `function` , все табличные значения — абстрактному типу `table` , все значения — абстрактному типу `any` и никакие значения — абстрактному типу `none` . Выражение типа `none` должно вызвать ошибку или завершиться сбоем, так как невозможно создать значение, соответствующее типу `none` . Обратите внимание, что примитивные типы `function` и `table` являются абстрактными, так как ни одна функция или таблица, соответственно, не относятся непосредственно к этим типам. Примитивные типы `record` и `list` не являются абстрактными, так как они представляют открытую запись без определенных полей и список типа "any" соответственно.

Все типы, не являющиеся членами закрытого набора примитивных типов, в совокупности называются *пользовательскими типами*. Пользовательские типы можно написать с помощью `type-expression` :

выражение-типа:

основное-выражение

`type` *основной-тип*

тип:

выражение-в-круглых-скобках

основной-тип

основной-тип:

тип-примитива

тип-записи

тип-списка

тип-функции

тип-таблицы

тип-допускающий-значение-NULL

тип-примитива: один из следующего

```
any binary date datetime datetimezone duration function list logical
```

```
none null number record table text time type
```

Имена *примитивных-типов* являются *контекстными ключевыми словами*, распознаваемыми только в контексте *типа*. Использование круглых скобок в контексте *типа* переводит грамматику обратно в контекст регулярного выражения, что требует использования ключевого слова "type" для перемещения обратно в контекст типа. Например, чтобы вызвать функцию в контексте *типа*, можно использовать круглые скобки:

```
type nullable ( Type.ForList({type number}) )  
// type nullable {number}
```

Круглые скобки также можно использовать для доступа к переменной, имя которой конфликтует с именем *примитивного типа*.

```
let record = type [ A = any ] in type {(record)}  
// type {[ A = any ]}
```

В следующем примере определяется тип, классифицирующий список чисел:

```
type { number }
```

Аналогичным образом в следующем примере определяется пользовательский тип, классифицирующий записи с обязательными полями `x` и `y`, значения которых являются числами:

```
type [ X = number, Y = number ]
```

Приписываемый тип значения получается с помощью функции стандартной библиотеки `Value.Type`, как показано в следующих примерах:

```
Value.Type( 2 )           // type number  
Value.Type( {2} )         // type list  
Value.Type( [ X = 1, Y = 2 ] ) // type record
```

Оператор `is` используется для определения того, совместим ли тип значения с заданным типом, как показано в следующих примерах:

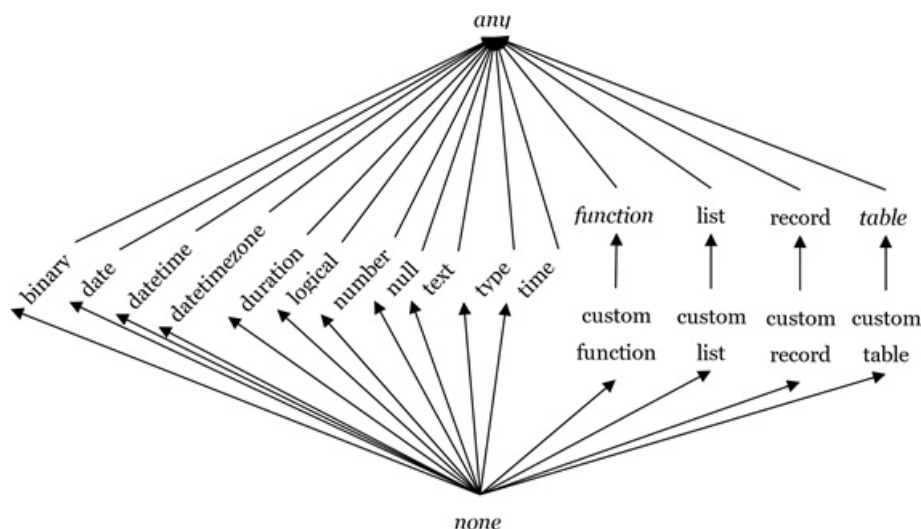
```
1 is number      // true  
1 is text        // false  
{2} is list      // true
```

Оператор `as` проверяет, совместимо ли значение с заданным типом, и вызывает ошибку, если это не так. В противном случае он возвращает исходное значение.

```
Value.Type( 1 as number ) // type number  
{2} as text               // error, type mismatch
```

Обратите внимание, что операторы `is` и `as` принимают в качестве правого операнда только примитивные типы. Язык M не предоставляет средства для проверки значений на соответствие пользовательским типам.

Тип `x` *совместим* с типом `y` только в том случае, если все значения, соответствующие `x`, также соответствуют `y`. Все типы совместимы с типом `any`, и никакие типы (кроме самого `none`) не совместимы с типом `none`. На следующем графе показана связь совместимости. (Совместимость типов является рефлексивной и транзитивной. Она формирует решетку с типом `any` в качестве верхнего значения и типом `none` в качестве нижнего значения.) Имена абстрактных типов выделены *курсивом*.



Для значений типов определены следующие операторы.

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно

Собственный тип значений типа — это встроенный тип `type`.

Примитивные типы

Типы в языке М формируют несвязанную иерархию, корнем которой является тип `any`, классифицирующий все значения. Любое значение М соответствует только одному примитивному подтипу `any`. Закрытый набор примитивных типов, производных от типа `any`, выглядит следующим образом.

- `type null`, который классифицирует значение NULL.
- `type logical`, который классифицирует значения true и false.
- `type number`, который классифицирует числовые значения.
- `type time`, который классифицирует значения времени.
- `type date`, который классифицирует значения даты.
- `type datetime`, который классифицирует значения datetime.
- `type datetimezone`, который классифицирует значения datetimezone.
- `type duration`, который классифицирует значения длительности.
- `type text`, который классифицирует текстовые значения.
- `type binary`, который классифицирует двоичные значения.
- `type type`, который классифицирует значения типов.
- `type list`, который классифицирует значения списков.
- `type record`, который классифицирует значения записей.

- `type table`, который классифицирует табличные значения.
- `type function`, который классифицирует значения функций.
- `type anynonnull`, который классифицирует все значения, кроме NULL. Внутренний тип `none` не классифицирует никакие значения.

Тип any

Тип `any` является абстрактным, классифицирует все значения в M, и все типы в M совместимы с `any`. Переменные типа `any` могут быть привязаны ко всем возможным значениям. Так как `any` является абстрактным, он не может быть приписан значениям, — т. е. никакие значение не может напрямую иметь тип `any`.

Типы списков

Любое значение, которое является списком, соответствует встроенному типу `list`, который не налагает никаких ограничений на элементы в значении списка.

тип-списка:

`{ тип-элемента }`

тип-элемента:

`тип`

Результатом вычисления *типа-списка* является *значение типа списка* с базовым типом `list`.

В следующих примерах показан синтаксис объявления однородных типов списков:

```
type { number }      // list of numbers type
    { record }       // list of records type
    {{ text }}       // list of lists of text values
```

Значение соответствует типу списка, если оно является списком, а каждый элемент в этом значении списка соответствует типу элемента типа списка.

Тип элемента типа списка указывает привязку: все элементы соответствующего списка соответствуют типу элемента.

Типы записей

Любое значение, которое является записью, соответствует встроенному типу "record", который не налагает никаких ограничений на имена полей или значения в значении записи. *Значение типа-записи* используется для ограничения набора допустимых имен, а также типов значений, которые могут быть связаны с этими именами.

тип-записи:

`[ маркер-открытой-записи ]`
`[ список-спецификаций-поля необязательно ]`
`[ список-спецификаций-поля, маркер-открытой-записи ]`

список-спецификаций-поля:

спецификация-поля
 спецификация-поля `,` список-спецификаций-поля

спецификация-поля:

`optional` необязательно *имя-поля спецификация-типа-поля* необязательно

спецификация-типа-поля:

`=` *тип-поля*

тип-поля:

тип

маркер-открытой-записи:

...

Результатом вычисления *типа-записи* является значение типа с базовым типом `record`.

В следующих примерах показан синтаксис объявления типов записей:

```
type [ X = number, Y = number ]
type [ Name = text, Age = number ]
type [ Title = text, optional Description = text ]
type [ Name = text, ... ]
```

Типы записей *закрываются* по умолчанию, что означает, что дополнительные поля, отсутствующие в *списке-спецификаций-поля*, не могут присутствовать в соответствующих значениях. Включение *маркера-открытой-записи* в тип записи объявляет тип как *открытый*, что разрешает поля, отсутствующие в списке спецификаций поля. Следующие два выражения эквивалентны:

```
type record // primitive type classifying all records
type [ ... ] // custom type classifying all records
```

Значение соответствует типу записи, если оно является записью и все спецификации полей в типе записи удовлетворены. Спецификация поля считается удовлетворенной, если выполняется одно из следующих условий.

- В записи существует имя поля, совпадающее с идентификатором спецификации, а связанное значение соответствует типу спецификации.
- Спецификация помечена как необязательная, и в записи не найдено соответствующее имя поля.

Соответствующее значение может содержать имена полей, не указанные в списке спецификаций поля, и только в том случае, если тип записи является открытым.

Типы функций

Любое значение функции соответствует примитивному типу `function`, который не налагает никаких ограничений на типы формальных параметров функции или возвращаемое значение функции. Пользовательское *значение типа функции* используется для наложения ограничений на типы в сигнатурах соответствующих значений функций.

тип-функций:

`function (` *список-спецификаций-параметра* *необязательно* `)` *тип-возвращаемого-значения-функции*

список-спецификаций-параметра:

список-спецификаций-обязательного-параметра

список-спецификаций-обязательного-параметра `,` *список-спецификаций-необязательного-параметра*

список-спецификаций-необязательного-параметра

список-спецификаций-обязательного-параметра:

список-спецификаций-обязательного-параметра

список-спецификаций-обязательного-параметра `,` *список-спецификаций-обязательного-параметра*

список-спецификаций-обязательного-параметра:

спецификация-параметра

список-спецификаций-необязательного-параметра:

спецификация-необязательного-параметра

спецификация-необязательного-параметра `,` *список-спецификаций-необязательного-параметра*
спецификация-необязательного-параметра:
`optional` *спецификация-параметра*
спецификация-параметра:
имя-параметра *тип-параметра*
тип-возвращаемого-значения-функции:
утверждение
утверждение:
`as` *тип-примитива-допускающий-значение-NULL*

Результатом вычисления *типа-функции* является значение типа с базовым типом `function`.

В следующих примерах показан синтаксис объявления типов функций:

```
type function (x as text) as number
type function (y as number, optional z as text) as any
```

Значение функции соответствует типу функции, если тип возвращаемого значения для значения функции совместим с типом возвращаемого значения для типа функции, а каждая спецификация параметра типа функции совместима с формальным параметром функции, стоящим на аналогичной позиции. Спецификация параметра совместима с формальным параметром, если указанный *тип-параметра* совместим с типом формального параметра, а спецификация параметра является необязательной, если формальный параметр является необязательным.

Имена формальных параметров игнорируются в целях определения соответствия типов функций.

Типы таблиц

Значение-типа-таблицы используется для определения структуры табличного значения.

тип-таблицы:

`table` *тип-строки*

тип-строки:

`[` *список-спецификаций-поля* `]`

Результатом вычисления *типа-таблицы* является значение типа с базовым типом `table`.

Тип строки таблицы указывает имена столбцов и типы столбцов таблицы в виде закрытого типа записи. Чтобы все табличные значения соответствовали типу `table`, тип строки имеет тип `record` (пустой открытый тип записи). Таким образом, тип таблицы является абстрактным, так как никакое табличное значение не может иметь тип строк типа `table` (но все табличные значения имеют тип строки, совместимый с типом строки типа `table`). В следующем примере показана конструкция типа таблицы:

```
type table [A = text, B = number, C = binary]
// a table type with three columns named A, B, and C
// of column types text, number, and binary, respectively
```

Значение типа таблицы также содержит определение *ключей* табличного значения. Ключ — это набор имен столбцов. В качестве *первичного ключа* можно назначить не более одного ключа. (В языке M ключи таблиц не имеют семантического значения. Однако обычно для внешних источников данных, таких как базы данных или веб-каналы OData, определяются ключи в таблицах. Power Query использует сведения о ключе для повышения производительности расширенных функций, таких как операции объединения между источниками.)

Функции стандартной библиотеки `Type.TableKeys`, `Type.AddTableKey` и `Type.ReplaceTableKeys` можно

использовать для получения ключей типа таблицы, добавления ключа в тип таблицы и замены всех ключей типа таблицы соответственно.

```
Type.AddTableKey(tableType, {"A", "B"}, false)
// add a non-primary key that combines values from columns A and B
Type.ReplaceTableKeys(tableType, {})
// returns type value with all keys removed
```

Типы, допускающие значение NULL

Для любого `type T` можно получить вариант, допускающий значение NULL, с помощью *типа-допускающего-значение-NULL*:

тип-допускающий-значение-NULL:

`nullable` *ТИП*

Результатом является абстрактный тип, допускающий значения типа *T* или значение `null`.

```
42 is nullable number           // true null is
nullable number                 // true
```

Приписка `type nullable` *T* сокращается до приписки `type null` или `type T`. (Помните, что типы, допускающие значение NULL, являются абстрактными и никакое значение не может относиться непосредственно к абстрактному типу.)

```
Value.Type(42 as nullable number) // type number
Value.Type(null as nullable number) // type null
```

Функции стандартной библиотеки `Type.IsNullable` и `Type.NonNullable` можно использовать для проверки типа на допустимость значений NULL и для удаления допустимости значений NULL из типа.

Происходит следующее (для любого `type T`):

- `type T` СОВМЕСТИМ С `type nullable T`
- `Type.NonNullable(type T)` СОВМЕСТИМ С `type T`

Ниже приведены попарные эквиваленты (для любого `type T`):

`type nullable any`
`any`

`Type.NonNullable(type any)`
`type anynonnull`

`type nullable none`
`type null`

`Type.NonNullable(type null)`
`type none`

`type nullable nullable T`
`type nullable T`

`Type.NonNullable(Type.NonNullable(type T))`
`Type.NonNullable(type T)`

```
Type.NonNullable(type nullable T)
```

```
Type.NonNullable(type T)
```

```
type nullable (Type.NonNullable(type T))
```

```
type nullable T
```

Приписываемый тип значения

Приписываемый тип значения — это тип, которому, согласно *объявлению*, соответствует значение. Если значение приписывается типу, то выполняется только ограниченная проверка соответствия. *Язык M не выполняет проверку соответствия за пределами примитивного типа, допускающего значение NULL.* Авторы программы на языке M, решающие приписать определения типов значения, более сложные, чем примитивный тип, допускающий значение NULL, должны обеспечить соответствие таких значений этим типам.

Значение можно приписать типу с помощью функции библиотеки `Value.ReplaceType`. Эта функция возвращает либо новое значение приписываемого типа, либо ошибку, если новый тип несовместим с собственным примитивным типом значения. В частности, эта функция возвращает ошибку при попытке приписать абстрактный тип, такой как `any`.

Функции библиотеки могут вычислять и приписывать сложные типы результатам на основе приписываемых типов входных значений.

Приписываемый тип значения можно получить с помощью функции библиотеки `Value.Type`. Например:

```
Value.Type( Value.ReplaceType( {1}, type {number} )  
// type {number}
```

Эквивалентность типов и совместимость

Эквивалентность типов в языке M не определена. Значения двух любых типов, сравниваемые на предмет равенства, могут возвращать или не возвращать `true`. Однако связь между этими двумя типами (независимо от того, получено ли значение `true` или `false`) всегда будет одинаковой.

Совместимость между заданным типом и примитивным типом, допускающим значение NULL, можно определить с помощью функции библиотеки `Type.Is`, которая принимает произвольное значение типа в качестве первого аргумента и значение примитивного типа, допускающего значение NULL, в качестве второго аргумента:

```
Type.Is(type text, type nullable text) // true  
Type.Is(type nullable text, type text) // false  
Type.Is(type number, type text)       // false  
Type.Is(type [a=any], type record)    // true  
Type.Is(type [a=any], type list)       // false
```

В M отсутствует поддержка для определения совместимости заданного типа с пользовательским типом.

Стандартная библиотека включает коллекцию функций для извлечения определяющих характеристик из пользовательского типа, поэтому определенные тесты совместимости можно реализовать как выражения M. Ниже приведены некоторые примеры, полные сведения см. в спецификации библиотеки M.

```
Type.ListItem( type {number} )
    // type number
Type.NonNullable( type nullable text )
    // type text
Type.RecordFields( type [A=text, B=time] )
    // [ A = [Type = type text, Optional = false],
    //   B = [Type = type time, Optional = false] ]
Type.TableRow( type table [X=number, Y=date] )
    // type [X = number, Y = date]
Type.FunctionParameters(
    type function (x as number, optional y as text) as number)
    // [ x = type number, y = type nullable text ]
Type.FunctionRequiredParameters(
    type function (x as number, optional y as text) as number)
    // 1
Type.FunctionReturn(
    type function (x as number, optional y as text) as number)
    // type number
```

Поведение оператора

16.03.2021 • 63 minutes to read

В этом разделе описано поведение различных операторов М.

Приоритет операторов

Если выражение содержит несколько операторов, их *приоритет* определяет порядок, в котором они оцениваются. Например, выражение `x + y * z` вычисляется как `x + (y * z)`, так как оператор `*` имеет более высокий приоритет, чем бинарный оператор `+`. Приоритет оператора устанавливается определением связанного с ним правила грамматики. Например, *выражение сложения* состоит из последовательности *выражений умножения*, разделенных операторами `+` или `-`, таким образом операторы `+` и `-` имеют более низкий приоритет, чем операторы `*` и `/`.

Для изменения порядка очередности по умолчанию можно использовать правило *выражения в круглых скобках*.

выражение-в-круглых-скобках:

`( выражение )`

Например:

```
1 + 2 * 3      // 7
(1 + 2) * 3    // 9
```

В следующей таблице перечислены операторы М по категории в порядке приоритета от самого высокого до самого низкого. Операторы в одной категории имеют одинаковый приоритет.

КАТЕГОРИЯ	ВЫРАЖЕНИЕ	ОПИСАНИЕ
Первичная	<code>i</code> <code>@i</code>	Выражение идентификатора
	<code>(x)</code>	Выражение в круглых скобках
	<code>x[]</code>	Уточняющий запрос
	<code>x{y}</code>	Доступ к элементам
	<code>x(...)</code>	Вызов функций
	<code>{x, y, ...}</code>	Инициализация списка
	<code>[i = x, ...]</code>	Инициализация записи
	<code>...</code>	Не реализовано
Унарный	<code>+x</code>	Идентификатор
	<code>-x</code>	Отрицание

	<code>not</code> x	Логическое отрицание
Метаданные	x <code>meta</code> y	Связывание метаданных
Мультипликативные	$x \setminus * y$	Умножение
	x / y	Деление
Аддитивная	$x + y$	Сложение
	$x - y$	Вычитание
Отношение	$x < > y$	Меньше чем
	$x > y$	Больше чем
	$x < = > y$	Меньше или равно
	$x > = y$	Больше или равно
Равенство	$x = y$	Равно
	$x < > y$	Не равно
Утверждение типа	x <code>as</code> y	Совместимый примитивный тип, допускающий значение NULL, или ошибка
Согласованность типа	x <code>is</code> y	Проверка, является ли совместимым примитивным типом, допускающим значение NULL
Логическое И	x <code>and</code> y	Сокращенное объединение
Логическое ИЛИ	x <code>or</code> y	Сокращенное разъединение

Операторы и метаданные

Каждое значение имеет связанное значение записи, которое может содержать дополнительные сведения о значении. Эта запись называется *записью метаданных* для значения. Запись метаданных может быть связана с любым типом значения, даже `null`. Результатом такой привязки является новое значение с заданными метаданными.

Запись метаданных — это просто обычная запись, которая может содержать любые поля и значения, как и обычная запись, и сама имеет запись метаданных. Связывание записи метаданных со значением не прерывает работу. Оно не влияет на поведение значения в вычислениях, за исключением тех, которые явно проверяют записи метаданных.

Каждое значение имеет запись метаданных по умолчанию, даже если она не была указана. Запись метаданных по умолчанию пуста. В следующих примерах показан доступ к записи метаданных текстового значения с помощью функции стандартной библиотеки `Value.Metadata`:

```
Value.Metadata( "Mozart" ) // []
```

Записи метаданных обычно *не сохраняются*, если значение используется с оператором или функцией, создающими новое значение. Например, если два текстовых значения сцеплены с помощью оператора `&`, метаданные итогового текстового значения будут представлены пустой записью `[]`. Следующие выражения эквивалентны:

```
"Amadeus " & ("Mozart" meta [ Rating = 5 ])
"Amadeus " & "Mozart"
```

Функции стандартной библиотеки `Value.RemoveMetadata` и `Value.ReplaceMetadata` можно использовать для удаления всех метаданных из значения и для замены метаданных значения (вместо объединения метаданных с потенциально существующими метаданными).

Единственный оператор, возвращающий результаты, которые содержат метаданные, — это [оператор метаданных](#).

Структурные рекурсивные операторы

Значения могут быть *циклическими*. Например:

```
let l = {0, @l} in l
// {0, {0, {0, ... }}}
[A={B}, B={A}]
// [A = {{ ... }}, B = {{ ... }}]
```

M обрабатывает циклические значения с отложенным созданием записей, списков и таблиц. При попытке создать циклическое значение, которое не использует преимущества вставленных значений с отложенным созданием, возникает ошибка:

```
[A=B, B=A]
// [A = Error.Record("Expression.Error",
//      "A cyclic reference was encountered during evaluation"),
//   B = Error.Record("Expression.Error",
//      "A cyclic reference was encountered during evaluation"),
// ]
```

Некоторые операторы в M определяются структурной рекурсией. Например, равенство записей и списков определяется с помощью объединения равенства соответствующих полей записей и списков элементов соответственно.

Для нециклических значений применение структурной рекурсии приводит к *конечному расширению* значения: общие вложенные значения проходят многократный обход, но процесс рекурсии всегда завершается.

Циклическое значение имеет *бесконечное расширение* при применении структурной рекурсии. Семантика M не предусматривает специальных операций для такого бесконечного расширения — попытка сравнить циклические значения на равенство, например, обычно расходует все ресурсы и завершается с исключением.

Операторы выбора и проекции

Операторы выбора и проекции позволяют извлекать данные из значений списка и записи.

Доступ к элементам

Значение может быть выбрано из списка или таблицы на основе позиции с отсчетом от нуля в этом списке или таблице с использованием *выражения доступа к элементам*.

Выражение-доступа-к-элементу:

выбор-элемента

выбор-необязательного-элемента

выбор-элемента:

основное-выражение { *селектор-элементов* }

выбор-необязательного-элемента:

основное-выражение { *селектор-элементов* } ?

селектор-элементов:

выражение

Выражение-доступа-к-элементам `x{y}` возвращает:

- Для списка `x` и числа `y` — элемент списка `x` в позиции `y`. Первый элемент списка считается нулевым порядковым индексом. Если запрошенной позиции нет в списке, возникает ошибка.
- Для таблицы `x` и числа `y` — строку таблицы `x` в позиции `y`. Первая строка таблицы считается нулевым порядковым индексом. Если запрошенной позиции нет в таблице, возникает ошибка.
- Для таблицы `x` и записи `y` — строку таблицы `x`, которая соответствует значениям полей записи `y` для полей с именами, совпадающими с соответствующими именами столбцов таблицы. Если в таблице нет уникальной совпадающей строки, возникает ошибка.

Например:

```
{ "a", "b", "c" } { 0 } // "a"
{ 1, [A=2], 3 } { 1 } // [A=2]
{ true, false } { 2 } // error
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { 0 } // [A=0, B=1]
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [A=2] } // [A=2, B=1]
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [B=3] } // error
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [B=1] } // error
```

Выражение доступа к элементам также поддерживает форму `x{y}?`, которая возвращает `null`, если позиция (или совпадение) `y` не существует в списке или таблице `x`. Если для `y` существует несколько соответствий, возникает ошибка.

Например:

```
{ "a", "b", "c" } { 0 } ? // "a"
{ 1, [A=2], 3 } { 1 } ? // [A=2]
{ true, false } { 2 } ? // null
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { 0 } // [A=0, B=1]
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [A=2] } // [A=2, B=1]
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [B=3] } // null
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [B=1] } // error
```

Доступ к элементам не требует вычисления элементов списка или таблицы, кроме тех, к которым осуществляется доступ. Например:

```
{ error "a", 1, error "c" } { 1 } // 1
{ error "a", error "b" } { 1 } // error "b"
```

При вычислении оператора доступа к элементу `x{y}` происходит следующее.

- Ошибки, возникающие при вычислении выражений `x` или `y`, распространяются.
- Выражение `x` выдает значение списка или таблицы.
- Выражение `y` выдает числовое значение или, если `x` создает табличное значение, значение записи.
- Если `y` создает числовое значение и значение `y` является отрицательным, возникает ошибка с кодом причины `"Expression.Error"`.
- Если `y` выдает числовое значение, а значение `y` больше или равно числу `x`, возникает ошибка с кодом причины `"Expression.Error"`, если не используется форма необязательного оператора `x{y}?`, и в этом случае возвращается значение `null`.
- Если `x` выдает табличное значение и `y` выдает значение записи, а для `y` в `x` нет совпадений, вызывается ошибка с кодом причины `"Expression.Error"`, если не используется форма необязательного оператора `x{y}?`, и в этом случае возвращается значение `null`.
- Если `x` выдает табличное значение, `y` выдает значение записи и существует несколько соответствий для `y` в `x`, возникает ошибка с кодом причины `"Expression.Error"`.

В процессе выбора элементов в `x` вычисляются только элементы в позиции `y`. (Для таблиц или списков потоковой передачи элементы или строки, предшествующие позиции `y`, пропускаются, что может привести к их оценке в зависимости от источника списка или таблицы.)

Доступ к полю

Выражение-доступа-к-полю используется для *выбора* значения из записи или *проецирования* записи или таблицы с меньшим числом полей или столбцов соответственно.

выражение-доступа-к-полю:

выбор-поля

неявный-выбор-целевого-поля

проекция

неявная-проекция-целевого-поля

выбор-поля:

селектор-поля-основного-выражения

селектор-поля:

селектор-обязательного-поля

селектор-необязательного-поля

селектор-обязательного-поля:

`[ имя-поля ]`

селектор-необязательного-поля:

`[ имя-поля ] ?`

имя-поля:

обобщенный-идентификатор

нестандартный-идентификатор

неявный-выбор-целевого-поля:

селектор-поля

проекция:

основное-выражение обязательная-проекция

основное-выражение необязательная-проекция

обязательная-проекция:

`[ список-обязательных-селекторов ]`

необязательная-проекция:

[список-обязательных-селекторов] ?

список-обязательных-селекторов:

селектор-обязательного-поля

список-обязательных-селекторов , селектор-обязательного-поля

неявная-целевая-проекция:

обязательная-проекция

необязательная-проекция

Самая простая форма доступа к полю — *выбор обязательного поля*. Он использует оператор `x[y]` для поиска поля в записи по имени поля. Если поле `y` не существует в `x`, возникает ошибка. Форма `x[y]?` используется для выбора *необязательного* поля и возвращает `null`, если запрошенное поле не существует в записи.

Например:

```
[A=1,B=2][B]      // 2
[A=1,B=2][C]      // error
[A=1,B=2][C]?     // null
```

Общий доступ к нескольким полям поддерживается операторами для *проекции обязательной записи* и *проекции необязательной записи*. Оператор `x[[y1],[y2],...]` проецирует запись в новую запись с меньшим числом полей (выбрано `y1`, `y2`, ...). Если выбранное поле не существует, возникает ошибка. Оператор `x[[y1],[y2],...]?` проецирует запись в новую запись с полями, выбранными `y1`, `y2`, Если поле отсутствует, вместо него используется `null`. Например:

```
[A=1,B=2][[B]]      // [B=2]
[A=1,B=2][[C]]      // error
[A=1,B=2][[B],[C]]? // [B=2,C=null]
```

Формы `[y]` и `[y]?` поддерживаются в качестве *краткой ссылки* на идентификатор `_` (подчеркивание).

Следующие два выражения эквивалентны:

```
[A]
_[A]
```

В следующем примере показана краткая форма доступа к полю:

```
let _ = [A=1,B=2] in [A] //1
```

Формы `[[y1],[y2],...]` и `[[y1],[y2],...]?` также поддерживаются в качестве сокращений, а следующие два выражения аналогичны:

```
[[A],[B]]
_[[A],[B]]
```

Краткая форма особенно полезна в сочетании с кратким `each`. Это способ представления функции с одним параметром `_` (дополнительные сведения см. в разделе [Упрощенные объявления](#)). Вместе эти два сокращения упрощают стандартные функциональные выражения более высокого порядка:

```
List.Select( {[a=1, b=1], [a=2, b=4]}, each [a] = [b])  
// {[a=1, b=1]}
```

Приведенное выше выражение эквивалентно следующей длинной и менее понятной записи:

```
List.Select( {[a=1, b=1], [a=2, b=4]}, (_) => _[a] = _[b])  
// {[a=1, b=1]}
```

Доступ к полям не требует вычисления других полей, кроме тех, к которым осуществляется доступ.
Например:

```
[A=error "a", B=1, C=error "c"] [B] // 1  
[A=error "a", B=error "b"] [B] // error "b"
```

При вычислении оператора доступа к полям `x[y]`, `x[y]?`, `x[[y]]` или `x[[y]]?` происходит следующее.

- Ошибки, возникающие при вычислении выражения `x`, распространяются.
- Ошибки, возникающие при вычислении поля `y`, постоянно связываются с полем `y`, а затем распространяются. Любой будущий доступ к полю `y` вызовет идентичную ошибку.
- Выражение `x` создает запись или значение таблицы, или возникает ошибка.
- Если идентификатор `y` обозначает поле, которое не существует в `x`, возникает ошибка с кодом причины `"Expression.Error"`, если не используется необязательный оператор `...?`, и в этом случае возвращается значение `null`.

В процессе доступа к полям не вычисляются поля `x`, кроме поля, указанного `y`.

Оператор метаданных

Запись метаданных для значения дополняется с помощью *оператора метаданных* (`x meta y`).

выражение-метаданных:

унарное-выражение

унарное-выражение `meta` *унарное-выражение*

В следующем примере создается текстовое значение с записью метаданных с помощью оператора `meta`, а затем осуществляется доступ к записи метаданных результирующего значения с помощью

`Value.Metadata` :

```
Value.Metadata( "Mozart" meta [ Rating = 5 ] )  
// [Rating = 5 ]  
Value.Metadata( "Mozart" meta [ Rating = 5 ] )[Rating]  
// 5
```

При применении оператора объединения метаданных `x meta y` происходит следующее.

- Ошибки, возникающие при вычислении выражений `x` или `y`, распространяются.
- Выражение `y` должно быть записью, или возникает ошибка с кодом причины `"Expression.Error"`.
- Результирующая запись метаданных — это запись метаданных `x`, объединенная с `y`. (Семантика слияния записей описана в разделе [Объединение записей](#).)

- Полученное значение является значением из выражения `x` без метаданных с присоединенной вычисленной записью метаданных.

Функции стандартной библиотеки `Value.RemoveMetadata` и `Value.ReplaceMetadata` можно использовать для удаления всех метаданных из значения и для замены метаданных значения (вместо объединения метаданных с потенциально существующими метаданными). Следующие выражения эквивалентны:

```
x meta y
Value.ReplaceMetadata(x, Value.Metadata(x) & y)
Value.RemoveMetadata(x) meta (Value.Metadata(x) & y)
```

Операторы равенства

Оператор равенства `=` используется для определения равенства двух значений. *Оператор неравенства* `<>` используется для определения неравенства двух значений.

выражение-равенства:

выражение-отношения

выражение-отношения `=` *выражение-равенства*

выражение-отношения `<>` *выражение-равенства*

Например:

```
1 = 1           // true
1 = 2           // false
1 <> 1          // false
1 <> 2          // true
null = true     // false
null = null     // true
```

Метаданные не являются частью сравнения равенства или неравенства. Например:

```
(1 meta [ a = 1 ]) = (1 meta [ a = 2 ]) // true
(1 meta [ a = 1 ]) = 1                  // true
```

При применении операторов равенства `x = y` и `x <> y` происходит следующее.

- Ошибки, возникающие при вычислении выражений `x` или `y`, распространяются.
- Оператор `=` имеет результат `true`, если значения равны, и `false` — в противном случае.
- Оператор `<>` имеет результат `false`, если значения равны, и `true` — в противном случае.
- Записи метаданных не включаются в сравнение.
- Если значения, полученные при вычислении выражений `x` и `y`, имеют разный тип, значения не равны.
- Если значения, полученные при вычислении выражений `x` и `y`, имеют одинаковый тип, существуют определенные правила для определения того, равны ли они, как определено ниже.
- Следующее выражение всегда является истинным:

```
(x = y) = not (x <> y)
```

Операторы равенства определяются для следующих типов:

- Значение `null` равно только самому себе.

```
null = null    // true
null = true    // false
null = false   // false
```

- Логические значения `true` и `false` равны только друг другу. Например:

```
true = true      // true
false = false     // true
true = false     // false
true = 1         // false
```

- Числа сравниваются с указанной точностью:
 - Если одно из чисел `#nan`, числа не совпадают.
 - Если ни одно из чисел не `#nan`, числа сравниваются с помощью побитового сравнения числовых значений.
 - `#nan` — единственное значение, которое не равно самому себе.

Например:

```
1 = 1,          // true
1.0 = 1         // true
2 = 1           // false
#nan = #nan     // false
#nan <> #nan     // true
```

- Два периода равны, если они представляют одно и то же число 100-наносекундных интервалов.
- Два значения времени равны, если величина их частей (час, минута, секунда) равны.
- Две даты равны, если величина их частей (год, месяц, день) равны.
- Два значения `datetime` равны, если величина их частей (год, месяц, день, час, минута, секунда) равны.
- Два значения `datetimezone` равны, если соответствующие значения `datetime` в формате UTC равны. Чтобы получить соответствующее значение `datetime` в формате UTC, смещение часов/минут вычитается из компонента `datetime` значения `datetimezone`.
- Два текстовых значения равны, если при ординарном сравнении с учетом регистра и без учета языка они имеют одинаковую длину и равные символы в соответствующих позициях.
- Два значения списка равны, если выполняются все приведенные ниже условия:
 - Оба списка содержат одинаковое количество элементов.
 - Значения каждого элемента в одинаковых позициях списка равны. Это означает, что списки должны содержать равные элементы и эти элементы должны находиться в том же порядке.

Например:

```
{1, 2} = {1, 2}    // true
{2, 1} = {1, 2}    // false
{1, 2, 3} = {1, 2} // false
```

- Две записи равны, если выполняются все перечисленные ниже условия:
 - Количество полей совпадает.
 - Каждое имя поля одной записи также присутствует в другой записи.
 - Значение каждого поля одной записи равно значению поля с тем же именем в другой записи.

Например:

```
[ A = 1, B = 2 ] = [ A = 1, B = 2 ]      // true
[ B = 2, A = 1 ] = [ A = 1, B = 2 ]      // true
[ A = 1, B = 2, C = 3 ] = [ A = 1, B = 2 ] // false
[ A = 1 ] = [ A = 1, B = 2 ]             // false
```

- Две таблицы равны, если выполняются все приведенные ниже условия:
 - Количество столбцов совпадает.
 - Каждое имя столбца в одной таблице также присутствует в другой таблице.
 - Количество строк совпадает.
 - Каждая строка содержит равные значения в соответствующих ячейках.

Например:

```
#table({"A","B"},{{1,2}}) = #table({"A","B"},{{1,2}}) // true
#table({"A","B"},{{1,2}}) = #table({"X","Y"},{{1,2}}) // false
#table({"A","B"},{{1,2}}) = #table({"B","A"},{{2,1}}) // true
```

- Значение функции равно самому себе, но может быть равно другому значению функции. Если два значения функции считаются равными, они будут работать одинаково при вызове.

Два заданных значения функции всегда имеют одинаковое отношение равенства.

- Значение типа равно самому себе, но может быть равно другому значению типа. Если два значения типа считаются равными, они будут работать одинаково при запросе на соответствие.

Два заданных значения типа всегда имеют одинаковое отношение равенства.

Реляционные операторы

Операторы `<`, `>`, `<=` и `>=` называются *реляционными операторами*.

реляционное-выражение:

выражение-сложения

выражение-сложения `<` *реляционное-выражение*

выражение-сложения `>` *реляционное-выражение*

выражение-сложения `<=` *реляционное-выражение*

выражение-сложения `>=` *реляционное-выражение*

Эти операторы используются для определения относительных отношений порядка между двумя значениями, как показано в следующей таблице:

ОПЕРАЦИЯ	РЕЗУЛЬТАТ
<code>x < y</code>	<code>true</code> , если значение <code>x</code> меньше значения <code>y</code> ; в противном случае — <code>false</code> .
<code>x > y</code>	<code>true</code> , если значение <code>x</code> больше значения <code>y</code> ; в противном случае — <code>false</code> .
<code>x <= y</code>	<code>true</code> , если значение <code>x</code> меньше или равно значению <code>y</code> ; в противном случае — <code>false</code> .
<code>x >= y</code>	<code>true</code> , если значение <code>x</code> больше или равно значению <code>y</code> ; в противном случае — <code>false</code> .

Например:

```
0 <= 1          // true
null < 1         // null
null <= null     // null
"ab" < "abc"     // true
#nan >= #nan     // false
#nan <= #nan     // false
```

При вычислении выражения, содержащего реляционные операторы, происходит следующее.

- Ошибки, возникающие при вычислении выражений операндов `x` или `y`, распространяются.
- Значения, полученные при вычислении выражений `x` и `y`, должны быть числами, значениями даты, `datetime`, `datetimezone`, длительности, логики, `NULL` или значениями времени. В противном случае возникает ошибка с кодом причины `"Expression.Error"`.
- Если один или оба операнда — `null`, результатом будет значение `null`.
- Если оба операнда являются логическими, считается, что значение `true` больше значения `false`.
- Если оба операнда выражают длительность, значения сравниваются в соответствии с общим числом 100-наносекундных интервалов, которые они представляют.
- Два значения времени сравниваются по часам. Если они равны, то по минутам. Если минуты равны, то по секундам.
- Два значения даты сравниваются по годам. Если они равны, то по месяцам. Если месяцы равны, то по дням.
- Два значения `datetime` сравниваются по годам, если они равны, то по месяцам и так далее: по дням, часам, минутам и секундам.
- Два значения `datetimezone` сравниваются с помощью нормализации в формат UTC путем вычитания смещения в часах и минутах, а затем сравнения компонентов `datetime`.
- Два числа `x` и `y` сравниваются в соответствии с правилами стандарта IEEE 754.
 - Если один из операндов — `#nan`, результатом будет `false` для всех реляционных операторов.
 - Если ни один из операндов не является `#nan`, операторы сравнивают значения двух операндов с плавающей запятой по отношению к упорядочивающему

$-\infty < -\max < \dots < -\min < -0.0 = +0.0 < +\min < \dots < +\max < +\infty$, где $\min$ и $\max$ — наименьшее и наибольшее положительное конечное значение, которое можно представить. Имена M для $-^\circ$ и $+^\circ$ — `-#infinity` и `#infinity` .

Важные примечания о таком порядке.

- Отрицательные и положительные нули считаются равными.
- Считается, что значение `-#infinity` меньше, чем все остальные числовые значения, но равно другому `-#infinity` .
- Считается, что значение `#infinity` больше, чем все остальные числовые значения, но равно другому `#infinity` .

Условные логические операторы

Операторы `and` и `or` называются условными логическими.

логическое-выражение-ИЛИ:

логическое-выражение-И

логическое-выражение-И `or` *логическое-выражение-ИЛИ*

логическое-выражение-И:

выражение-is

выражение-is `and` *логическое-выражение-И*

Оператор `or` возвращает `true` , если хотя бы один из его операндов является `true` . Правый операнд вычисляется только в том случае, если левый операнд не является `true` .

Оператор `and` возвращает `false` , если хотя бы один из его операндов является `false` . Правый операнд вычисляется только в том случае, если левый операнд не является `false` .

Таблицы истинности для операторов `or` и `and` показаны ниже. Результат вычисления левого операнда показан по вертикали, а результат вычисления правого операнда — по горизонтали.

<code> AND </code>	<code> TRUE </code>	<code> FALSE </code>	<code> NULL </code>	<code> ERROR </code>
<code> true </code>	<code> true </code>	<code> false </code>	<code> null </code>	<code> error </code>
<code> false </code>	<code> false </code>	<code> false </code>	<code> false </code>	<code> false </code>
<code> null </code>	<code> null </code>	<code> false </code>	<code> null </code>	<code> error </code>
<code> error </code>	<code> error </code>	<code> error </code>	<code> error </code>	<code> error </code>

<code> OR </code>	<code> TRUE </code>	<code> FALSE </code>	<code> NULL </code>	<code> ERROR </code>
<code> or </code>	<code> true </code>	<code> false </code>	<code> null </code>	<code> error </code>
<code> true </code>	<code> true </code>	<code> true </code>	<code> true </code>	<code> true </code>
<code> false </code>	<code> true </code>	<code> false </code>	<code> null </code>	<code> error </code>
<code> null </code>	<code> true </code>	<code> null </code>	<code> null </code>	<code> error </code>

OR	TRUE	FALSE	NULL	ERROR
error	error	error	error	error

При вычислении выражения, содержащего условные логические операторы, происходит следующее.

- Ошибки, возникающие при вычислении выражений `x` или `y`, распространяются.
- Условные логические операторы определены для типов `logical` и `null`. Если значения операнда не относятся к этим типам, возникает ошибка с кодом причины `"Expression.Error"`.
- Результатом является логическое значение.
- В выражении `x` или `y` выражение `y` будет вычисляться только в том случае, если `x` не равно `true`.
- В выражении `x` и `y` выражение `y` будет вычисляться только в том случае, если `x` не равно `false`.

Последние два свойства отвечают за компонент "условности" в условных логических операторах. Свойства также можно считать сокращенными. Эти свойства полезны для написания компактных *защищенных предикатов*. Например, следующие выражения эквивалентны:

```
d <> 0 and n/d > 1 if d <> 0 then n/d > 1 else false
```

Арифметические операторы

Операторы `+`, `-`, `*` и `/` являются *арифметическими операторами*.

выражение-сложения:

выражение-умножения

выражение-сложения `+` *выражение-умножения*

выражение-сложения `-` *выражение-умножения*

выражение-умножения:

выражение-метаданных

выражение-умножения `*` *выражение-метаданных*

выражение-умножения `/` *выражение-метаданных*

Точность

Числа в M хранятся в различных представлениях, позволяющих хранить как можно больше информации о числах, поступающих из различных источников. Числа преобразуются из одного представления в другое в зависимости от применяемых к ним операторов. В M поддерживаются две точности:

ТОЧНОСТЬ	СЕМАНТИКА
<code>Precision.Decimal</code>	128-разрядное десятичное представление с диапазоном от $\pm 1,0 \times 10^{-28}$ до $\pm 7,9 \times 10^{28}$ и 28–29 значащих цифр.
<code>Precision.Double</code>	Научное представление с использованием мантиссы и степени; соответствует 64-разрядному двоичному числу двойной точности по стандарту IEEE 754 IEEE 754-2008 .

Арифметические операции выполняются путем выбора точности, преобразования обоих операндов в эту точность (при необходимости), выполнения фактической операции и, наконец, возврата числа с выбранной точностью.

Во встроенных арифметических операторах (`+`, `-`, `*`, `/`) используется двойная точность. Функции стандартной библиотеки (`Value.Add`, `Value.Subtract`, `Value.Multiply`, `Value.Divide`) можно использовать для запроса этих операций с помощью определенной модели точности.

- Числовое переполнение через верхнюю границу невозможно: `#infinity` или `-#infinity` представляют значения слишком большой величины для представления.
- Числовое переполнение через нижнюю границу невозможно: `0` и `-0` представляют значения слишком малой величины для представления.
- Специальное значение IEEE 754 `#nan` (NaN — не является числом) используется для выявления арифметически недопустимых вариантов, таких как деление нуля на ноль.
- Преобразование из десятичной точности в двойную выполняется с помощью округления десятичных чисел до ближайшего эквивалентного значения двойной точности.
- Преобразование из двойной точности в десятичную выполняется путем округления двойных чисел до ближайшего эквивалентного десятичного значения и, при необходимости, переполнения до значений `#infinity` или `-#infinity`.

Оператор сложения

Интерпретация оператора сложения (`x + y`) зависит от вида значения вычисляемых выражений `x` и `y`, как показано ниже.

x	y	РЕЗУЛЬТАТ	ИНТЕРПРЕТАЦИЯ
<code>type number</code>	<code>type number</code>	<code>type number</code>	Числовая сумма
<code>type number</code>	<code>null</code>	<code>null</code>	
<code>null</code>	<code>type number</code>	<code>null</code>	
<code>type duration</code>	<code>type duration</code>	<code>type duration</code>	Числовая сумма величин
<code>type duration</code>	<code>null</code>	<code>null</code>	
<code>null</code>	<code>type duration</code>	<code>null</code>	
<code>type <i>datetime</i></code>	<code>type duration</code>	<code>type <i>datetime</i></code>	Смещение значения <i>datetime</i> по длительности
<code>type duration</code>	<code>type <i>datetime</i></code>	<code>type <i>datetime</i></code>	
<code>type <i>datetime</i></code>	<code>null</code>	<code>null</code>	
<code>null</code>	<code>type <i>datetime</i></code>	<code>null</code>	

В таблице `type datetime` обозначает `type date`, `type datetime`, `type datetimezone` или `type time`. При добавлении длительности и значения типа `datetime` результирующее значение относится к тому же типу.

Для других сочетаний значений, отличных от указанных в таблице, возникает ошибка с кодом причины `"Expression.Error"`. Все сочетания описываются в следующих разделах.

Ошибки, возникающие при вычислении любого операнда, распространяются.

Числовая сумма

Сумма двух чисел вычисляется с помощью *оператора сложения*, выдающего число.

Например:

```
1 + 1          // 2
#nan + #infinity // #nan
```

Оператор сложения `+` для чисел использует двойную точность. Стандартную функцию библиотеки `Value.Add` можно использовать для указания десятичной точности. При вычислении суммы чисел происходит следующее.

- Сумма с двойной точностью вычисляется в соответствии с правилами 64-разрядного двоичного числа двойной точности по стандарту IEEE 754 [IEEE 754-2008](#). В следующей таблице перечислены результаты всех возможных сочетаний ненулевых конечных значений, нулей, бесконечности и значений, не являющихся числом. В таблице `x` и `y` являются ненулевыми конечными значениями, а `z` является результатом `x + y`. Если `x` и `y` имеют одинаковую величину, но противоположные знаки, `z` является положительным нулем. Если `x + y` слишком велико для представления в целевом типе, `z` является бесконечностью с тем же знаком, что и `x + y`.

<code>+</code>	<code>y</code>	<code>+0</code>	<code>-0</code>	<code>+°</code>	<code>-°</code>	НЕ ЧИСЛО
<code>x</code>	<code>z</code>	<code>x</code>	<code>x</code>	<code>+°</code>	<code>-°</code>	Не число
<code>+0</code>	<code>y</code>	<code>+0</code>	<code>+0</code>	<code>+°</code>	<code>-°</code>	Не число
<code>-0</code>	<code>y</code>	<code>+0</code>	<code>-0</code>	<code>+°</code>	<code>-°</code>	Не число
<code>+°</code>	<code>+°</code>	<code>+°</code>	<code>+°</code>	<code>+°</code>	Не число	Не число
<code>-°</code>	<code>-°</code>	<code>-°</code>	<code>-°</code>	Не число	<code>-°</code>	Не число
Не число	Не число	Не число	Не число	Не число	Не число	Не число

- Сумма с десятичной точностью вычисляется без потери точности. В качестве величины результата выбирается большая из величин двух операндов.

Сумма длительностей

Сумма двух значений длительности — это длительность, представляющая сумму количества 100-наносекундных интервалов обоих значений. Например:

```
#duration(2,1,0,15.1) + #duration(0,1,30,45.3)
// #duration(2, 2, 31, 0.4)
```

Смещение значения `datetime` по длительности

Значение `datetime` `x` и длительность `y` можно сложить с помощью `x + y`, чтобы вычислить новое значение `datetime`, которое находится от `x` на линейной временной шкале на расстоянии `y`. Здесь значением `datetime` может быть `Date`, `DateTime`, `DateTimeZone` или `Time`, и результат, отличный от NULL,

будет иметь тот же тип. Смещение значения datetime по длительности можно вычислить следующим образом:

- Если задано значение "days since epoch" для datetime, создайте новое значение datetime со следующими элементами сведений:
 - Вычислите новый эквивалент "days since epoch": разделите величину у на число 100-наносекундных интервалов за 24-часовой период, усеките десятичную часть результата и добавьте это значение к числу "days since epoch" для х.
 - Вычислите новый эквивалент интервалов от полуночи: добавьте величину у к интервалам х с полуночи с остатком числа 100-наносекундных интервалов за 24 часа. Если х не задает значение для интервалов с полуночи, то предполагается значение 0.
 - Скопируйте значение х для смещения минут в формате UTC без изменений.
- Если значение "days since epoch" для datetime не задано, создайте новое значение datetime со следующими элементами сведений:
 - Вычислите новый эквивалент интервалов от полуночи: добавьте величину у к интервалам х с полуночи с остатком числа 100-наносекундных интервалов за 24 часа. Если х не задает значение для интервалов с полуночи, то предполагается значение 0.
 - Скопируйте значения х для значения "days since epoch" и смещения минут в формате UTC без изменений.

В следующих примерах показано вычисление абсолютной темпоральной суммы, когда для datetime задано значение *days since epoch*:

```
#date(2010,05,20) + #duration(0,8,0,0)
//#datetime( 2010, 5, 20, 8, 0, 0 )
//2010-05-20T08:00:00

#date(2010,01,31) + #duration(30,08,0,0)
//#datetime(2010, 3, 2, 8, 0, 0)
//2010-03-02T08:00:00

#datetime(2010,05,20,12,00,00,-08) + #duration(0,04,30,00)
//#datetime(2010, 5, 20, 16, 30, 0, -8, 0)
//2010-05-20T16:30:00-08:00

#datetime(2010,10,10,0,0,0,0,0) + #duration(1,0,0,0)
//#datetime(2010, 10, 11, 0, 0, 0, 0, 0)
//2010-10-11T00:00:00+00:00
```

В следующем примере показано вычисление смещения datetime по длительности для заданного времени:

```
#time(8,0,0) + #duration(30,5,0,0)
//#time(13, 0, 0)
//13:00:00
```

Оператор вычитания

Интерпретация оператора вычитания ($x - y$) зависит от вида значения вычисляемых выражений x и y , как показано ниже:

x	y	РЕЗУЛЬТАТ	ИНТЕРПРЕТАЦИЯ
type number	type number	type number	Числовая разность

X	Y	РЕЗУЛЬТАТ	ИНТЕРПРЕТАЦИЯ
type number	null	null	
null	type number	null	
type duration	type duration	type duration	Числовая разность величин
type duration	null	null	
null	type duration	null	
type datetime	type datetime	type duration	Длительность между значениями datetime
type datetime	type duration	type datetime	Смещение значения datetime по длительности, к которой применено отрицание
type datetime	null	null	
null	type datetime	null	

В таблице `type datetime` обозначает `type date`, `type datetime`, `type datetimetzoned` или `type time`. При вычитании длительности из значения типа `datetime` результирующее значение относится к тому же типу.

Для других сочетаний значений, отличных от указанных в таблице, возникает ошибка с кодом причины `"Expression.Error"`. Все сочетания описываются в следующих разделах.

Ошибки, возникающие при вычислении любого операнда, распространяются.

Числовая разность

Разность двух чисел вычисляется с помощью *оператора вычитания*, выдающего число. Например:

```
1 - 1 // 0
#nan - #infinity // #nan
```

Оператор вычитания `-` для чисел использует двойную точность. Стандартную функцию библиотеки `Value.Subtract` можно использовать для указания десятичной точности. При вычислении разности чисел происходит следующее.

- Разность с двойной точностью вычисляется в соответствии с правилами 64-разрядного двоичного числа двойной точности по стандарту IEEE 754 [IEEE 754-2008](#). В следующей таблице перечислены результаты всех возможных сочетаний ненулевых конечных значений, нулей, бесконечности и значений, не являющихся числом. В таблице `x` и `y` являются ненулевыми конечными значениями, а `z` является результатом `x - y`. Если `x` и `y` равны, `z` является положительным нулем. Если `x - y` слишком велико для представления в целевом типе, `z` является бесконечностью с тем же знаком, что и `x - y`.

-	y	+0	-0	+°	-°	НЕ ЧИСЛО
x	z	x	x	-°	+°	Не число
+0	-y	+0	+0	-°	+°	Не число
-0	-y	-0	+0	-°	+°	Не число
+°	+°	+°	+°	Не число	+°	Не число
-°	-°	-°	-°	-°	Не число	Не число
Не число	Не число	Не число	Не число	Не число	Не число	Не число

- Разность с десятичной точностью вычисляется без потери точности. В качестве величины результата выбирается большая из величин двух операндов.

Разница длительностей

Разность двух значений длительности — это длительность, представляющая разность между количеством 100-наносекундных интервалов обоих значений. Например:

```
#duration(1,2,30,0) - #duration(0,0,0,30.45)
// #duration(1, 2, 29, 29.55)
```

Смещение значения `datetime` по длительности, к которой применено отрицание

Из значения `datetime` `x` можно вычесть длительность `y` с помощью `x - y` для вычисления нового `datetime`. Здесь `datetime` может быть `date`, `datetime`, `datetimezone` или `time`. Итоговое значение `datetime` находится от `x` на линейной временной шкале на расстоянии `y`, в направлении, противоположном знаку `y`. Вычитание положительных периодов длительности дает результаты в прошлом относительно `x`, а вычитание отрицательных значений дает результаты в будущем.

```
#date(2010,05,20) - #duration(00,08,00,00)
//#datetime(2010, 5, 19, 16, 0, 0)
//2010-05-19T16:00:00
#date(2010,01,31) - #duration( 30,08,00,00)
//#datetime(2009, 12, 31, 16, 0, 0)
//2009-12-31T16:00:00
```

Длительность между двумя значениями `datetime`

Два значения `datetime` `t` и `u` можно вычесть друг из друга с помощью `t - u`, чтобы вычислить длительность между ними. Здесь `datetime` может быть `date`, `datetime`, `datetimezone` или `time`. Длительность, полученная вычитанием `u` из `t`, должна давать `t` при сложении с `u`.

```
#date(2010,01,31) - #date(2010,01,15)
// #duration(16,00,00,00)
// 16.00:00:00

#date(2010,01,15) - #date(2010,01,31)
// #duration(-16,00,00,00)
// -16.00:00:00

#datetime(2010,05,20,16,06,00,-08,00) -
#datetime(2008,12,15,04,19,19,03,00)
// #duration(521,22,46,41)
// 521.22:46:41
```

Вычитание `t - u`, когда `u > t` приводит к отрицательной длительности:

```
#time(01,30,00) - #time(08,00,00)
// #duration(0, -6, -30, 0)
```

При вычитании двух значений *datetime* с помощью `t - u` происходит следующее:

- $u + (t - u) = t$

Оператор умножения

Интерпретация оператора умножения (`x * y`) зависит от вида значения вычисляемых выражений *x* и *y*, как показано ниже.

x	y	РЕЗУЛЬТАТ	ИНТЕРПРЕТАЦИЯ
type number	type number	type number	Числовое умножение
type number	null	null	
null	type number	null	
type duration	type number	type duration	Умножение длительностей
type number	type duration	type duration	Умножение длительностей
type duration	null	null	
null	type duration	null	

Для других сочетаний значений, отличных от указанных в таблице, возникает ошибка с кодом причины `"Expression.Error"`. Все сочетания описываются в следующих разделах.

Ошибки, возникающие при вычислении любого операнда, распространяются.

Числовое умножение

Произведение двух чисел вычисляется с помощью *оператора умножения*, выдающего число. Например:


```
2 * 4           // 8
6 * null        // null
#nan * #infinity // #nan
```

Оператор умножения `*` для чисел использует двойную точность. Стандартную функцию библиотеки `Value.Multiply` можно использовать для указания десятичной точности. При вычислении произведения чисел происходит следующее.

- Произведение с двойной точностью вычисляется в соответствии с правилами 64-разрядного двоичного числа двойной точности по стандарту IEEE 754 [IEEE 754-2008](#). В следующей таблице перечислены результаты всех возможных сочетаний ненулевых конечных значений, нулей, бесконечности и значений, не являющихся числом. В таблице `x` и `y` являются положительными конечными значениями. `z` является результатом `x * y`. Если результат слишком велик для целевого типа, `z` является бесконечностью. Если результат слишком мал для целевого типа, `z` является нулем.

*	+Y	-Y	+0	-0	+°	-°	НЕ ЧИСЛО
+x	+z	-Z	+0	-0	+°	-°	Не число
-x	-Z	+z	-0	+0	-°	+°	Не число
+0	+0	-0	+0	-0	Не число	Не число	Не число
-0	-0	+0	-0	+0	Не число	Не число	Не число
+°	+°	-°	Не число	Не число	+°	-°	Не число
-°	-°	+°	Не число	Не число	-°	+°	Не число
Не число	Не число	Не число	Не число	Не число	Не число	Не число	Не число

- Произведение с десятичной точностью вычисляется без потери точности. В качестве величины результата выбирается большая из величин двух операндов.

Умножение длительности

Произведение длительности и числа — это длительность, представляющая количество 100-наносекундных интервалов, представленная операндом длительности, умноженным на операнд числа. Например:

```
#duration(2,1,0,15.1) * 2
// #duration(4, 2, 0, 30.2)
```

Оператор деления

Интерпретация оператора деления (`x / y`) зависит от вида значения вычисляемых выражений `x` и `y`, как показано ниже.

x	y	РЕЗУЛЬТАТ	ИНТЕРПРЕТАЦИЯ
<code>type number</code>	<code>type number</code>	<code>type number</code>	Числовое частное

X	Y	РЕЗУЛЬТАТ	ИНТЕРПРЕТАЦИЯ
type number	null	null	
null	type number	null	
type duration	type number	type duration	Частное длительности
type duration	type duration	type duration	Числовое частное длительностей
type duration	null	null	
null	type duration	null	

Для других сочетаний значений, отличных от указанных в таблице, возникает ошибка с кодом причины "Expression.Error". Все сочетания описываются в следующих разделах.

Ошибки, возникающие при вычислении любого операнда, распространяются.

Числовое частное

Частное двух чисел вычисляется с помощью *оператора деления*, выдающего число. Например:

```
8 / 2           // 4
8 / 0           // #infinity
0 / 0           // #nan
0 / null        // null
#nan / #infinity // #nan
```

Оператор деления `/` для чисел использует двойную точность. Стандартную функцию библиотеки `Value.Divide` можно использовать для указания десятичной точности. При вычислении частного чисел происходит следующее.

- Частное с двойной точностью вычисляется в соответствии с правилами 64-разрядного двоичного числа двойной точности по стандарту IEEE 754 [IEEE 754-2008](#). В следующей таблице перечислены результаты всех возможных сочетаний ненулевых конечных значений, нулей, бесконечности и значений, не являющихся числом. В таблице `x` и `y` являются положительными конечными значениями. `z` является результатом `x / y`. Если результат слишком велик для целового типа, `z` является бесконечностью. Если результат слишком мал для целового типа, `z` является нулем.

/	+Y	-Y	+0	-0	+°	-°	НЕ ЧИСЛО
+x	+z	-Z	+°	-°	+0	-0	Не число
-x	-Z	+z	-°	+°	-0	+0	Не число
+0	+0	-0	Не число	Не число	+0	-0	Не число
-0	-0	+0	Не число	Не число	-0	+0	Не число
+°	+°	-°	+°	-°	Не число	Не число	Не число

/	+Y	-Y	+0	-0	+°	-°	НЕ ЧИСЛО
-°	-°	+°	-°	+°	Не число	Не число	Не число
Не число	Не число	Не число	Не число	Не число	Не число	Не число	Не число

- Сумма с десятичной точностью вычисляется без потери точности. В качестве величины результата выбирается большая из величин двух операндов.

Частное длительностей

Частное двух значений длительности — это число, представляющее частное количества 100-наносекундных интервалов обоих значений. Например:

```
#duration(2,0,0,0) / #duration(0,1,30,0)
// 32
```

Масштабированные длительности

Коэффициент длительности `x` и числа `y` — это длительность, представляющая коэффициент числа 100-наносекундных интервалов и представленная длительностью `x` и числом `y`. Например:

```
#duration(2,0,0,0) / 32
// #duration(0,1,30,0)
```

Структурные комбинирования

Оператор комбинирования (`x & y`) определен для следующих типов значений.

x	y	РЕЗУЛЬТАТ	ИНТЕРПРЕТАЦИЯ
<code>type text</code>	<code>type text</code>	<code>type text</code>	Объединение
<code>type text</code>	<code>null</code>	<code>null</code>	
<code>null</code>	<code>type text</code>	<code>null</code>	
<code>type date</code>	<code>type time</code>	<code>type datetime</code>	Объединить
<code>type date</code>	<code>null</code>	<code>null</code>	
<code>null</code>	<code>type time</code>	<code>null</code>	
<code>type list</code>	<code>type list</code>	<code>type list</code>	Объединение
<code>type record</code>	<code>type record</code>	<code>type record</code>	Объединить
<code>type table</code>	<code>type table</code>	<code>type table</code>	Объединение

Объединение

С помощью `x & y` можно соединить два текста, два списка или два значения таблицы.

В следующем примере показано соединение текстовых значений:

```
"AB" & "CDE" // "ABCDE"
```

В следующем примере показано соединение списков:

```
{1, 2} & {3} // {1, 2, 3}
```

При соединении двух значений с помощью `x & y` происходит следующее.

- Ошибки, возникающие при вычислении выражений `x` или `y`, распространяются.
- Ошибки не распространяются, если элемент `x` или `y` содержит ошибку.
- Результатом соединения двух текстовых значений является текстовое значение, которое содержит значение `x`, за которым следует значение `y`. Если один из операндов имеет значение `NULL`, а второй — текстовое значение, то результатом будет `NULL`.
- Результатом соединения двух списков является список, содержащий все элементы `x`, за которыми следуют все элементы `y`.
- Результатом соединения двух таблиц является таблица с объединением столбцов таблицы двух операндов. Упорядочение столбцов `x` сохраняется, за ними следуют столбцы, присутствующие только в `y`, с сохранением относительного упорядочения. Для столбцов, отображаемых только в одном из операндов, `null` используется для заполнения значений ячеек для другого операнда.

Объединить

Объединение записей

С помощью `x & y` можно объединить две записи, создав запись, включающую поля из `x` и `y`.

В следующих примерах показано объединение записей:

```
[ x = 1 ] & [ y = 2 ] // [ x = 1, y = 2 ]  
[ x = 1, y = 2 ] & [ x = 3, z = 4 ] // [ x = 3, y = 2, z = 4 ]
```

При объединении двух записей с помощью `x + y` происходит следующее.

- Ошибки, возникающие при вычислении выражений `x` или `y`, распространяются.
- Если поле присутствует как в `x`, так и в `y`, используется значение из `y`.
- В результирующей записи используется порядок полей из `x`, за ними следуют поля из `y`, которые не являются частью `x`, в том же порядке, в котором они отображаются в `y`.
- Объединение записей не приводит к вычислению значений.
- Если поле содержит ошибку, ошибки не возникают.
- Результатом является запись.

Объединение значений даты и времени

Даты `x` можно объединить со временем `y` с помощью `x & y`, создав значение `datetime` со значениями `x` и `y`.

В следующем примере показано объединение даты и времени:

```
#date(2013,02,26) & #time(09,17,00)
// #datetime(2013,02,26,09,17,00)
```

При объединении двух записей с помощью `x + y` происходит следующее.

- Ошибки, возникающие при вычислении выражений `x` или `y`, распространяются.
- Результатом является значение `datetime`.

Унарные операторы

Операторы `+`, `-` и `not` являются унарными.

унарное-выражение:

выражение-типа

`+` унарное выражение

`-` унарное выражение

`not` унарное выражение

Оператор унарного плюса

Оператор унарного плюса (`+x`) определен для следующих типов значений:

x	РЕЗУЛЬТАТ	ИНТЕРПРЕТАЦИЯ
<code>type number</code>	<code>type number</code>	Унарный плюс
<code>type duration</code>	<code>type duration</code>	Унарный плюс
<code>null</code>	<code>`null</code>	

Для других значений возникает ошибка с кодом причины `"Expression.Error"`.

Оператор унарного плюса позволяет применять знак `+` к числу, значению `datetime` или значению `NULL`.
Результатом будет то же значение. Например:

```
+ - 1          // -1
+ + 1          // 1
+ #nan         // #nan
+ #duration(0,1,30,0) // #duration(0,1,30,0)
```

При вычислении оператора унарного плюса `+x` происходит следующее.

- Ошибки, возникающие при вычислении `x`, распространяются.
- Если результат вычисления `x` не является числовым значением, возникает ошибка с кодом причины `"Expression.Error"`.

Оператор унарного минуса

Оператор унарного минуса (`-x`) определен для следующих типов значений.

x	РЕЗУЛЬТАТ	ИНТЕРПРЕТАЦИЯ
<code>type number</code>	<code>type number</code>	Отрицание

x	РЕЗУЛЬТАТ	ИНТЕРПРЕТАЦИЯ
<code>type duration</code>	<code>type duration</code>	Отрицание
<code>null</code>	<code>null</code>	

Для других значений возникает ошибка с кодом причины `"Expression.Error"`.

Оператор унарного минуса используется для изменения знака числа или длительности. Например:

```
- (1 + 1)      // -2
- - 1          // 1
- - - 1        // -1
- #nan         // #nan
- #infinity    // -#infinity
- #duration(1,0,0,0) // #duration(-1,0,0,0)
- #duration(0,1,30,0) // #duration(0,-1,-30,0)
```

При вычислении оператора унарного минуса `-x` происходит следующее.

- Ошибки, возникающие при вычислении `x`, распространяются.
- Если выражение является числом, результатом является числовое значение из выражения `x` с измененным знаком. Если значение не является числом, результат также не будет являться числом.

Оператор логического отрицания

Оператор логического отрицания (`not`) определен для следующих типов значений.

x	РЕЗУЛЬТАТ	ИНТЕРПРЕТАЦИЯ
<code>type logical</code>	<code>type logical</code>	Отрицание
<code>null</code>	<code>null</code>	

Этот оператор рассчитывает логическую операцию `not` для заданного логического значения.

Например:

```
not true      // false
not false     // true
not (true and true) // false
```

При вычислении оператора логического отрицания `not x` происходит следующее.

- Ошибки, возникающие при вычислении `x`, распространяются.
- Результатом вычисления выражения `x` должно быть логическое значение, или возникает ошибка с кодом причины `"Expression.Error"`. Если значение `true`, результатом будет `false`. Если операнд `false`, результатом будет `true`.

Результатом является логическое значение.

Операторы типов

Операторы `is` и `as` называются операторами типов.

Оператор совместимости типов

Оператор совместимости типов `x is y` определяется для следующих типов значений.

x	y	РЕЗУЛЬТАТ
<code>type any</code>	<i>примитивный тип, допускающий значение NULL</i>	<code>type logical</code>

Выражение `x is y` возвращает `true`, если присписываемый тип `x` совместим с `y`, и `false` — если присписываемый тип `x` несовместим с `y`. `y` должен быть *примитивным типом, допускающим значение NULL*.

выражение-is:

выражение-as

выражение-is is примитивный-тип-допускающий-значение-NULL

примитивный-тип-допускающий-значение-NULL:

`nullable` *необязательно* *тип-примитива*

Совместимость типов, поддерживаемая оператором `is`, является подмножеством [совместимости общего типа](#) и определяется с помощью следующих правил.

- Если `x` имеет значение NULL, совместимость присутствует, если `y` допускает значение NULL или имеет тип `any`.
- Если `x` не равно NULL, совместимость присутствует, если примитивный тип `x` совпадает с `y`.

При вычислении выражения `x is y` происходит следующее.

- Ошибка, возникшая при вычислении выражения `x`, распространяется.

Оператор утверждения типа

Оператор утверждения типа `x as y` определяется для следующих типов значений.

x	y	РЕЗУЛЬТАТ
<code>type any</code>	<i>примитивный тип, допускающий значение NULL</i>	<code>type any</code>

Выражение `x as y` утверждает, что значение `x` совместимо с `y` в соответствии с применением оператора `is`. В случае несовместимости возвращается ошибка. `y` должен быть *примитивным-типом-допускающим-значение-NULL*.

выражение-as:

выражение-равенства

выражение-as as тип-примитива-допускающий-значение-NULL

Выражение `x as y` вычисляется следующим образом.

- Выполняется проверка совместимости типов `x is y`, и утверждение возвращает `x` без изменений, если тест пройден.

- Если проверка совместимости завершается ошибкой, возникает ошибка с кодом причины

`"Expression.Error"`.

Примеры:

```
1 as number           // 1
"A" as number         // error
null as nullable number // null
```

При вычислении выражения `x as y` происходит следующее.

- Ошибка, возникшая при вычислении выражения `x`, распространяется.

Let

16.12.2020 • 2 minutes to read

Выражение let

Выражение `let` можно использовать для записи значения из промежуточного вычисления в переменную.

выражение-let:

`let` список-переменных `in` выражение

список-переменных:

переменная

переменная , список-переменных

переменная:

имя-переменной = выражение

имя-переменной:

идентификатор

В следующем примере показаны промежуточные результаты, вычисляемые и хранимые в переменных `x`, `y` и `z`, которые используются в последующих вычислениях `x + y + z`:

```
let    x = 1 + 1,
      y = 2 + 2,
      z = y + 1
in     x + y + z
```

Результат этого выражения:

```
11  // (1 + 1) + (2 + 2) + (2 + 2 + 1)
```

При вычислении выражений в *выражении let* происходит следующее.

- Выражения в списке переменных определяют новую область, содержащую идентификаторы из вывода *списка переменных*, и должны присутствовать при вычислении выражений в выводе *списка переменных*. Выражения в *списке переменных* могут ссылаться друг на друга.
- Выражения в *списке переменных* должны вычисляться до вычисления выражения в *выражении let*.
- Если выражения в *списке переменных* не доступны, они не должны вычисляться.
- Ошибки, возникающие во время вычисления выражений в *выражении let*, распространяются.

Выражение `let` можно рассматривать как синтаксический сахар для неявного выражения записи.

Следующее выражение эквивалентно приведенному выше примеру:

```
[    x = 1 + 1,
  y = 2 + 2,
  z = y + 1,
  result = x + y + z
][result]
```

Условные выражения

16.12.2020 • 2 minutes to read

Выражение *if* выбирает одно из двух выражений на основе значения логического входного значения и вычисляет только выбранное выражение.

Выражение *if*:

```
if условие-if then выражение-true else выражение-false
```

условие-if:

выражение

выражение-true:

выражение

выражение-false:

выражение

Ниже приведены примеры *выражений-if*.

```
if 2 > 1 then 2 else 1          // 2
if 1 = 1 then "yes" else "no"  // "yes"
```

При вычислении *выражения-if* происходит следующее.

- Если значение, полученное путем вычисления *условия-if*, не является логическим, возникает ошибка с кодом причины `"Expression.Error"`.
- *Выражение true* вычисляется только в том случае, если *условие-if* равно значению `true`.
- *Выражение false* вычисляется только в том случае, если *условие-if* равно значению `false`.
- Результатом *выражения-if* является значение *выражения-true*, если *условие-if* — `true`, и значение *выражения-false*, если *условие-if* — `false`.
- Ошибки, возникающие при вычислении *условия-if*, *выражения-true* или *выражения-false*, распространяются.

Функции

16.12.2020 • 10 minutes to read

Функция — это значение, представляющее собой сопоставление набора значений аргументов с одним значением. При вызове функции предоставляется набор входных значений (значений аргументов) и создается одно выходное значение (возвращаемое значение).

Написание функций

Функции записываются с помощью *выражения-функции*.

выражение-функции:

`(` *список-параметров* необязательно `)` *возвращаемый-тип-функции* необязательно `=>` *основная-часть-функции*

основная-часть-функции:

выражение

список-параметров:

список-фиксированных-параметров

список-фиксированных-параметров `,` *список-необязательных-параметров*

список-необязательных-параметров

список-фиксированных-параметров:

параметр

параметр `,` *список-фиксированных-параметров*

параметр:

имя-параметра *имя-параметра* необязательно

имя-параметра:

идентификатор

тип-параметра:

проверочное-утверждение

возвращаемый-тип-функции:

проверочное-утверждение

проверочное-утверждение:

`as` *тип-примитива-допускающий-значение-NULL*

список-необязательных-параметров:

необязательный-параметр

необязательный-параметр `,` *список-необязательных-параметров*

необязательный-параметр:

`optional` *параметр*

тип-примитива-допускающего-значение-NULL

`nullable` необязательно *тип-примитива_*

Ниже приведен пример функции, для которой требуется ровно два значения `x` и `y`, и результат применения оператора `+` к этим значениям. Значения `x` и `y` являются *параметрами*, которые являются частью *списка-формальных-параметров* функции, а `x + y` — это *основная часть функции*.

```
(x, y) => x + y
```

Результатом вычисления *выражения-функции* является получение значения функции (а не вычисление *основной-части-функции*). По соглашению в этом документе значения функций (в отличие от выражений

функций) отображаются со *списком-формальных-значений*, но с многоточием (`...`) вместо *основной-части-функции*. Например, после вычисления приведенного выше выражения функции оно будет отображаться как следующее значение функции:

```
(x, y) => ...
```

Для значений функций определены следующие операторы:

ОПЕРАТОР	РЕЗУЛЬТАТ
<code>x = y</code>	Равно
<code>x <> y</code>	Не равно

Собственный тип значений функций — это настраиваемый тип функции (производный от встроенного типа `function`), в котором содержится список имен параметров и указаны все типы параметров и тип возвращаемого значения (`any`). (Дополнительные сведения о типах функций см. в разделе [Типы функций](#).)

Вызов функций

Основная-часть-функции выполняется путем *вызова* значения функции с помощью *invokeexpression*. Вызов значения функции означает, что вычисляется *основная-часть-функции* и возвращается значение или возникает ошибка.

выражение-вызова:

основное-выражение (`[` *список-аргументов* *необязательно* `]`)

список-аргументов:

список-выражений

При каждом вызове значения функции набор значений задается как *список-аргументов*, называемый *аргументами* функции.

Список-аргументов используется для указания фиксированного числа аргументов непосредственно в виде списка выражений. В следующем примере определяется запись со значением функции в поле, а затем вызывается функция из другого поля записи:

```
[
  MyFunction = (x, y, z) => x + y + z,
  Result1 = MyFunction(1, 2, 3)           // 6
]
```

При вызове функции:

- Среда, используемая для вычисления *основной-части-функции*, включает переменную, которая соответствует каждому параметру с тем же именем, что и у параметра. Значение каждого параметра соответствует значению, созданному на основе *списка-аргументов* *invokeexpression*, как указано в разделе [Параметры](#).
- Все выражения, соответствующие аргументам функции, вычисляются перед вычислением *основной-части-функции*.
- Ошибки, возникающие при вычислении выражений в *списке-выражений* или *functionexpression*, распространяются.

- Количество аргументов, сформированных на основе *списка-аргументов*, должно быть совместимо с формальными параметрами функции, иначе возникает ошибка с кодом причины

"Expression.Error". Процесс определения совместимости указан в разделе [Параметры](#).

Параметры

Существует два типа формальных параметров, которые могут присутствовать в *списке-формальных-параметров*.

- **Обязательный параметр** указывает, что аргумент, соответствующий параметру, должен всегда указываться при вызове функции. Обязательные параметры должны быть указаны первыми в *списке-формальных-параметров*. Функция в следующем примере определяет обязательные параметры `x` и `y`.

```
[
  MyFunction = (x, y) => x + y,

  Result1 = MyFunction(1, 1),      // 2
  Result2 = MyFunction(2, 2)      // 4
]
```

- **Необязательный параметр** указывает, что аргумент, соответствующий параметру, может быть указан при вызове функции, однако это необязательно. Если аргумент, соответствующий необязательному параметру, не указан при вызове функции, вместо него используется значение `null`. Необязательные параметры должны быть указаны в *списке-формальных-параметров* после любых обязательных параметров. Функция в следующем примере определяет фиксированный параметр `x` и необязательный параметр `y`:

```
[
  MyFunction = fn(x, optional y) =>
    if (y = null) x else x + y,
  Result1 = MyFunction(1),          // 1
  Result2 = MyFunction(1, null),    // 1
  Result3 = MyFunction(2, 2),      // 4
]
```

Количество аргументов, указанных при вызове функции, должно быть совместимо со списком формальных параметров. Совместимость набора аргументов `A` для функции `F` вычисляется следующим образом:

- Допустим, что значение N представляет количество аргументов `A`, созданных из списка *argumentlist*. Например:

```
MyFunction()           // N = 0
MyFunction(1)           // N = 1
MyFunction(null)        // N = 1
MyFunction(null, 2)     // N = 2
MyFunction(1, 2, 3)     // N = 3
MyFunction(1, 2, null)  // N = 3
MyFunction(1, 2, {3, 4}) // N = 3
```

- Допустим, что значение *Required* представляет количество фиксированных параметров `F`, а значение *Optional* — количество необязательных параметров `F`. Например:

```
( )           // Required = 0, Optional = 0
(x)           // Required = 1, Optional = 0
(optional x)  // Required = 0, Optional = 1
(x, optional y) // Required = 1, Optional = 1
```

- Аргументы `A` совместимы с функцией `F`, если выполняются следующие условия:
 - ($N \geq$ фиксированные) и ($N \leq$ (фиксированные + необязательные))
 - Типы аргументов совместимы с соответствующими типами параметров `F`
- Если функция имеет объявленный возвращаемый тип, то значение результата вычисления основной части функции `F` совместимо с типом возвращаемого значения `F`, если выполняется следующее:
 - Значение, полученное путем вычисления основной части функции с предоставленными аргументами для параметров функции, имеет тип, совместимый с типом возвращаемого значения.
- Если при вычислении основной части функции возвращается значение, несовместимое с типом возвращаемого значения функции, возникает ошибка с кодом причины `"Expression.Error"`.

Рекурсивные функции

Чтобы записать рекурсивное значение функции, необходимо использовать оператор области (`@`) для ссылки на функцию в ее области. Например, следующая запись содержит поле, определяющее функцию `Factorial`, и другое поле, которое вызывает его:

```
[
  Factorial = (x) =>
    if x = 0 then 1 else x * @Factorial(x - 1),
  Result = Factorial(3) // 6
]
```

Аналогично взаимно рекурсивные функции можно написать в случае, если каждая функция, к которой требуется получить доступ, имеет имя. В следующем примере часть функции `Factorial` была преобразована во вторую функцию `Factorial2`.

```
[
  Factorial = (x) => if x = 0 then 1 else Factorial2(x),
  Factorial2 = (x) => x * Factorial(x - 1),
  Result = Factorial(3) // 6
]
```

Замыкания

Функция может возвращать другую функцию в виде значения. В свою очередь, эта функция может зависеть от одного или нескольких параметров исходной функции. В следующем примере функция, связанная с полем `MyFunction`, возвращает функцию, которая возвращает указанный параметр:

```
[
  MyFunction = (x) => () => x,
  MyFunction1 = MyFunction(1),
  MyFunction2 = MyFunction(2),
  Result = MyFunction1() + MyFunction2() // 3
]
```

При каждом вызове функции будет возвращено новое значение функции, сохраняющее значение параметра, чтобы при вызове было возвращено значение параметра.

Функции и среды

Помимо параметров, *основная-часть-функции выражения-функции* может ссылаться на переменные, которые имеются в среде при инициализации функции. Например, функция, определенная полем

`MyFunction`, обращается к полю `c` включенной записи `A`:

```
[
  A =
    [
      MyFunction = () => c,
      c = 1
    ],
  B = A[MyFunction]()      // 1
]
```

При вызове `MyFunction` она обращается к значению переменной `c`, несмотря на то, что она вызывается из среды (`B`), которая не содержит переменную `c`.

Упрощенные объявления

Выражение-each является синтаксическим сокращением для объявления нетипизированных функций, принимающих один формальный параметр с именем `_` (подчеркивание).

выражение-each:

`each` *основная-часть-выражения-each*

основная-часть-выражения-each:

основная-часть-функции

Упрощенные объявления обычно используются для повышения удобства читаемости вызова функции более высокого порядка.

Например, следующие пары объявлений семантически эквивалентны:

```
each _ + 1
(_) => _ + 1
each [A]
(_) => _[A]

Table.SelectRows( aTable, each [Weight] > 12 )
Table.SelectRows( aTable, (_) => _[Weight] > 12 )
```

Обработка ошибок

16.12.2020 • 7 minutes to read

Результат вычисления выражения `M` дает один из следующих результатов.

- Создается одно значение.
- *Выдается ошибка*, что означает, что при вычислении выражения не удалось получить значение. Ошибка содержит одно значение записи, которое можно использовать для предоставления дополнительных сведений о том, что привело к неполному вычислению.

Ошибки могут возникать в выражении и могут обрабатываться из выражения.

Возникновение ошибок

Синтаксис для вызова ошибки выглядит следующим образом:

выражение-вызывающее-ошибку:

```
error выражение
```

Текстовые значения можно использовать в качестве сокращений для значений ошибок. Например:

```
error "Hello, world" // error with message "Hello, world"
```

Полные значения ошибок — это записи, которые можно создать с помощью функции `Error.Record`:

```
error Error.Record("FileNotFound", "File my.txt not found",  
    "my.txt")
```

Выражение выше эквивалентно следующему:

```
error [  
    Reason = "FileNotFound",  
    Message = "File my.txt not found",  
    Detail = "my.txt"  
]
```

Возникновение ошибки приводит к остановке вычисления текущего выражения, и стек вычисления выражения остается развернутым до тех пор, пока не произойдет одно из следующего.

- Пока не будет достигнуто поле записи, элемент раздела или переменная `let` — в совокупности — *запись*. Запись помечается как содержащая ошибку, значение ошибки сохраняется вместе с этой записью, а затем распространяется. Любой последующий доступ к этой записи приведет к возникновению идентичной ошибки. Другие записи в разделе, записи или выражении `let` не обязательно затрагиваются (за исключением случаев, когда они обращаются к записи, ранее помеченной как содержащую ошибку).
- Пока не будет достигнуто выражение верхнего уровня. В этом случае результатом вычисления выражения верхнего уровня является ошибка, а не значение.
- Пока не будет достигнуто выражение `try`. В этом случае ошибка захватывается и возвращается в виде значения.

Обработка ошибок

Для обработки ошибки используется *выражение-обработки-ошибок*.

*_*выражение-обработки-ошибок:

`try` *защищенное-выражение-предложения-otherwise* *необязательно*

защищенное-выражение:

выражение

предложение-otherwise:

`otherwise` *выражение-по-умолчанию*

выражение-по-умолчанию:

выражение

При вычислении *выражения-обработки-ошибок* без *предложения-otherwise* происходит следующее.

- Если вычисление защищенного-выражения не приводит к ошибке и создает значение *x*, то значение, формируемое выражением-обработки-ошибок, является записью следующего вида:

```
[ HasErrors = false, Value = x ]
```

- Если вычисление защищенного-выражения вызывает ошибку *e*, результатом вычисления выражения-обработки-ошибок является запись следующего вида:

```
[ HasErrors = true, Error = e ]
```

При вычислении *выражения-обработки-ошибок* с *предложением-otherwise* происходит следующее.

- Защищенное-выражение должно вычисляться перед предложением-otherwise.
- Предложение-otherwise должно вычисляться только в том случае, если при вычислении защищенного-выражения возникает ошибка.
- Если вычисление *защищенного-выражения* вызывает ошибку, значением, которое создает *выражение-обработки-ошибок*, будет результат вычисления предложения-otherwise.
- Ошибки, возникающие при вычислении предложения-otherwise, распространяются.

В следующем примере показано *выражение-обработки-ошибок* в случае отсутствия ошибок:

```
let
  x = try "A"
in
  if x[HasError] then x[Error] else x[Value]
// "A"
```

В следующем примере показано возникновение ошибки и ее обработка:

```
let
  x = try error "A"
in
  if x[HasError] then x[Error] else x[Value]
// [ Reason = "Expression.Error", Message = "A", Detail = null ]
```

Предложение *otherwise* можно использовать для замены ошибок, обрабатываемых выражением *try*, на альтернативное значение:

```
try error "A" otherwise 1
// 1
```

Если предложение `otherwise` также вызывает ошибку, то и все выражение `try` также вызывает ошибку:

```
try error "A" otherwise error "B"
// error with message "B"
```

Ошибки в записи и инициализаторы `let`

В следующем примере показан инициализатор записи с полем `A`, которая вызывает ошибку и к которой обращаются два других поля `B` и `C`. Поле `B` не обрабатывает ошибку, вызванную `A`, однако `C` обрабатывает ошибку. Последнее поле `D` не имеет доступа к `A`, поэтому оно не затрагивается ошибкой в `A`.

```
[
  A = error "A",
  B = A + 1,
  C = let x =
    try A in
      if not x[HasError] then x[Value]
      else x[Error],
  D = 1 + 1
]
```

Результат вычисления выражения выше будет следующим:

```
[
  A = // error with message "A"
  B = // error with message "A"
  C = "A",
  D = 2
]
```

Обработка ошибок в М должна выполняться близко к причине ошибок, чтобы учесть влияние отложенной инициализации полей и отложенное вычисление закрытий. В следующем примере показана неудачная попытка обработки ошибки с помощью выражения `try`:

```
let
  f = (x) => [ a = error "bad", b = x ],
  g = try f(42) otherwise 123
in
  g[a] // error "bad"
```

В этом примере определение `g` было предназначено для обработки ошибки, возникшей при вызове `f`. Однако ошибка вызывается инициализатором поля, который выполняется только при необходимости, и поэтому запись была возвращена из `f` и передана с помощью выражения `try`.

Ошибка отсутствия реализации

Во время разработки выражения автор может захотеть оставить реализацию для некоторых частей выражения, однако ему по-прежнему может потребоваться возможность выполнить выражение. Один из способов решения этого сценария — вызвать ошибку для нереализованных частей. Например:

```
(x, y) =>
  if x > y then
    x - y
  else
    error Error.Record("Expression.Error",
      "Not Implemented")
```

Символ многоточия (`...`) можно использовать в качестве ярлыка для `error` .

выражение-отсутствия-реализации:

`...`

Следующий пример эквивалентен предыдущему:

```
(x, y) => if x > y then x - y else ...
```

Разделы

16.12.2020 • 7 minutes to read

Документ-с-разделами — это программа M, которая состоит из нескольких именованных выражений.

документ-с-разделами:

раздел

раздел:

литеральные-атрибуты необязательно `section` *имя-раздела* `;` *элементы-раздела* необязательно

имя-раздела:

идентификатор

элементы-раздела:

элемент-раздела *элементы-раздела* необязательно

элемент-раздела:

литеральные-атрибуты необязательно `shared` opt *имя-элемента-раздела* `=` *выражение* `;`

название-элемента-раздела:

идентификатор

В языке M раздел представляет собой организационный принцип, с помощью которого можно именовать и группировать связанные выражения в документе. Каждый раздел содержит *название-раздела*, которое определяет раздел и уточняет названия *элементов-раздела*, объявленные в разделе. *Элемент-раздела* состоит из *названия-элемента* и *выражения*. Выражения элементов раздела могут ссылаться на другие элементы раздела в пределах одного раздела непосредственно по названию элемента.

В следующем примере показан документ-с-разделами, содержащий один раздел:

```
section Section1;

A = 1;                //1
B = 2;                //2
C = A + B;            //3
```

Выражения элементов раздела могут ссылаться на элементы раздела, расположенные в других разделах, с помощью *выражения-доступа-к-разделу*, который уточняет название элемента раздела с использованием названия содержащего его раздела.

выражение-доступа-к-разделу:

идентификатор `!` *идентификатор*

В следующем примере показан документ, содержащий два раздела, которые ссылаются друг на друга.

```
section Section1;
A = "Hello";                //"Hello"
B = 1 + Section2!A;         //3

section Section2;
A = 2;                      //2
B = Section1!A & " world!";  //"Hello, world"
```

Элементы раздела при необходимости можно объявить как `shared`, что исключает необходимость использовать *выражение-доступа-к-разделу* при ссылке на общие элементы за пределами содержащего его раздела. Для ссылки на общие элементы во внешних разделах можно использовать их неполные

названия разделов, если ни один из элементов с таким же именем не объявлен в ссылающемся разделе и ни в одном из других разделов нет общего элемента с подобным названием.

В следующем примере показано поведение общих элементов при использовании в разных разделах одного документа:

```
section Section1;
shared A = 1;      // 1

section Section2;
B = A + 2;         // 3 (refers to shared A from Section1)

section Section3;
A = "Hello";       // "Hello"
B = A + " world";  // "Hello world" (refers to local A)
C = Section1!A + 2; // 3
```

Определение общего элемента с одинаковым названием в разных разделах приведет к созданию действительной глобальной среды, однако при доступе к общему элементу произойдет ошибка.

```
section Section1;
shared A = 1;

section Section2;
shared A = "Hello";

section Section3;
B = A;    //Error: shared member A has multiple definitions
```

При вычислении документа-с-разделами происходит следующее.

- Каждое *название-раздела* должно быть уникальным в глобальной среде.
- В пределах раздела каждый *элемент-раздела* должен иметь уникальное *название-элемента-раздела*.
- Общие элементы раздела с несколькими определениями вызывают ошибку при доступе к общему элементу.
- Компонент выражения *элемента-раздела* не должен оцениваться перед обращением к элементу раздела.
- Ошибки, возникающие при вычислении компонента выражения *элемента-раздела*, связываются с этим элементом раздела перед распространением наружу, а затем повторно создаются при каждом обращении к элементу раздела.

Связывание документов

Набор документов с разделами *M* можно связать со значением непрозрачной записи, которое содержит одно поле для общего элемента документа с разделами. Если общие элементы имеют неоднозначные имена, возникает ошибка.

Результирующее значение записи полностью закрывается в глобальной среде, в которой был выполнен процесс связывания. Таким образом, такие записи являются подходящими компонентами для создания документов *M* из других (связанных) наборов документов *M*. Возможности для возникновения конфликтов именования отсутствуют.

Функции стандартной библиотеки `Embedded.Value` можно использовать для получения таких "встроенных" значений записей, которые соответствуют повторно используемым компонентам *M*.

Самоанализ документов

Язык М предоставляет программный доступ к глобальной среде с помощью ключевых слов `#sections` и `#shared`.

`#sections`

Встроенная переменная `#sections` возвращает все разделы в глобальной среде в виде записи. В качестве ключа для этой записи используется название раздела, а каждое значение — это представление соответствующего раздела, в качестве индекса которого используется название элемента раздела.

В следующем примере показан документ, состоящий из двух разделов и записи, полученной путем вычисления внутренней переменной `#sections` в контексте этого документа:

```
section Section1;
A = 1;
B = 2;

section Section2;
C = "Hello";
D = "world";

#sections
//[
// Section1 = [ A = 1, B = 2],
// Section2 = [ C = "Hello", D = "world" ]
//]
```

При вычислении `#sections` происходит следующее.

- Встроенная переменная `#sections` сохраняет состояние вычисления всех выражений элементов раздела в документе.
- Встроенная переменная `#sections` не выполняет принудительное вычисление каких бы то ни было невычисленных элементов раздела.

`#shared`

Встроенная переменная `#shared` возвращает запись, содержащую названия и значения всех общих элементов раздела, находящихся в области.

В следующем примере показан документ с двумя общими элементами и соответствующей записью, созданной путем вычисления внутренней переменной `#shared` в контексте этого документа:

```
section Section1;
shared A = 1;
B = 2;

Section Section2;
C = "Hello";
shared D = "world";

//[
// A = 1,
// D = "world"
//]
```

При вычислении `#shared` происходит следующее.

- Встроенная переменная `#shared` сохраняет состояние вычисления всех выражений элементов общего раздела в документе.

- Встроенная переменная `#shared` не выполняет принудительное вычисление каких бы то ни было невычисленных элементов раздела.

Консолидированная грамматика

16.12.2020 • 10 minutes to read

Лексическая грамматика

лексическая-единица:

лексические-элементы необязательно

лексические-элементы:

лексический-элемент лексический-элементы необязательно

лексический-элемент:

пробел

комментарий к токену

Пробел

пробел:

Любой символ с классом Zs Юникода

Символ горизонтальной табуляции (U+0009)

Символ вертикальной табуляции (U+000B)

Символ перевода страницы (U+000C)

Символ возврата каретки (U+000D), за которым следует символ перевода строки (U+000A) СИМВОЛ-

новой-строки

символ-новой-строки:

Символ возврата каретки (U+000D)

Символ перевода строки (U+000A)

Символ следующей строки (U+0085)

Символ разделителя строк (U+2028)

Символ разделителя абзацев (U+2029)

Комментарий

комментарий:

однострочный-комментарий

комментарий-с-разделителями

однострочный-комментарий:

// символы-однострочного-комментария необязательно

символы-однострочного-комментария:

символ-однострочного-комментария символы-однострочного-комментария необязательно

символ-однострочного-комментария:

Любой символ Юникода, кроме символа-новой-строки

с комментарием-с-разделителями:

/* текст-комментария-с-разделителями необязательно звездочки /

текст-комментария-с-разделителями:

раздел-комментария-с-разделителями текст-комментария-с-разделителями необязательно

раздел-комментария-с-разделителями:

/

звездочки необязательно не-косая-черта-или-звездочка

звездочки:

* звездочки необязательно

не-косая-черта-или-звездочка:

/

десятичный знак: один из

0 1 2 3 4 5 6 7 8 9

шестнадцатеричный-числовой-литерал:

0x шестнадцатеричные-знаки

0X шестнадцатеричные-знаки

шестнадцатеричные-знаки:

шестнадцатеричный-знак шестнадцатеричные-знаки необязательно

шестнадцатеричный-знак: один из

0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f

десятичный-числовой-литерал:

десятичные-знаки . экспоненциальная-часть-десятичных-знаков необязательно

. экспоненциальная-часть-десятичных-знаков необязательно

экспоненциальная-часть-десятичных-знаков десятичные-знаки необязательно

экспоненциальная-часть:

e знак необязательно десятичные-знаки

E знак необязательно десятичные-знаки

знак: один из следующего

+ -

текстовый-литерал:

" символы-текстового-литерала необязательно "

символы-текстового-литерала:

символ-текстового-литерала символы-текстового-литерала необязательно

символ-текстового-литерала:

один-текстовый-символ

escape-последовательность-символов

escape-последовательность-двойных-кавычек

один-текстовый-символ:

Любой символ, кроме " (U+0022) или # (U+0023), за которым следует ((U+0028)

escape-последовательность-двойных-кавычек:

" " (U+0022 , U+0022)

литерал-NULL:

null

буквальный-литерал:

#!" символы-текстового-литерала необязательно "

Идентификаторы

идентификатор:

обычный-идентификатор

нестандартный-идентификатор

обычный-идентификатор:

доступный-идентификатор

доступный-идентификатор символ-точки обычный-идентификатор

доступный-идентификатор:

ключевое-слово-или-идентификатор, не являющийся ключевым-словом

ключевое-слово-или-идентификатор:

буква

символ-подчеркивания

начальный-символ-идентификатора символы-идентификатора

начальный-символ-идентификатора:

буква

символ-подчеркивания

символы-идентификатора:

символы-идентификатора символ-идентификатора необязательно

символ-идентификатора:

буква

десятичный-знак

символ-подчеркивания

символ-соединения

символ-объединения

символ-форматирования

обобщенный-идентификатор:

часть-обобщенного-идентификатора

обобщенный-идентификатор только через пробелы (`U+0020`) часть-обобщенного-идентификатора

часть-обобщенного-идентификатора:

сегмент-обобщенного-идентификатора

десятичный-знак сегмент-обобщенного-идентификатора

сегмент-обобщенного-идентификатора:

ключевое-слово-или-идентификатор

ключевое-слово-или-идентификатор символ-точки ключевое-слово-или-идентификатор

символ-точки:

`.` (`U+002E`)

символ-подчеркивания:

`_` (`U+005F`)

буквенный-символ:

Символ Юникода класса Lu, Ll, Lt, Lm, Lo или Nl

символ-объединения:

Символ Юникода класса Mn или Mc

десятичный-знак:

Символ Юникода класса Nd

символ-соединения:

Символ Юникода класса Pc

символ-форматирования:

Символ Юникода класса Cf

нестандартный-идентификатор:

`#"` символы-текстового-литерала *необязательно* `"`

Ключевые слова и предварительно заданные идентификаторы

Предварительно заданные идентификаторы и ключевые слова нельзя переопределить. Нестандартный идентификатор можно использовать для управления идентификаторами, которые в ином случае конфликтуют с предварительно заданными идентификаторами или ключевыми словами.

ключевое-слово: одно из следующего

`and as each else error false if in is let meta not null or otherwise`

`section shared then true try type #binary #date #datetime`

`#datetimezone #duration #infinity #nan #sections #shared #table #time`

Операторы и знаки препинания

оператор-или-знак-препинания: один из следующего

`, ; = < <= > >= <> + - * / & ( ) [ ] { } @ ? => .. ...`

Синтаксическая грамматика

Документы

документ:

документ-с-разделами

документ-с-выражениями

Документы с разделами

документ-с-разделами:

раздел

раздел:

литеральные-атрибуты необязательно `section` имя-раздела `;` элементы-раздела необязательно

имя-раздела:

идентификатор

элементы-раздела:

элемент-раздела элементы-раздела необязательно

элемент-раздела:

литеральные-атрибуты необязательно `общее` необязательно имя-элемента-раздела `=` выражение `;`

название-элемента-раздела:

идентификатор

Документы с выражениями

Выражения

документ-с-выражениями:

выражение

выражение:

логическое-выражение-or

выражение-each

выражение-function

выражение-let

выражение-if

выражение-вызывающее-ошибку

выражение-обработки-ошибок

Логические выражения

логическое-выражение-or:

логическое-выражение-and

логическое-выражение-and `or` логическое-выражение-or

логическое-выражение-and:

выражение-is

логическое-выражение-and `and` выражение-is

Выражение is

выражение-is:

выражение-as

выражение-is `is` тип-примитива-допускающий-значение-NULL

тип-примитива-допускающий-значение-NULL:

`nullable` необязательно тип-примитива

Выражение as

выражение-as:

выражение-равенства

выражение-as `as` тип-примитива-допускающий-значение-NULL

Выражение равенства

выражение-равенства:

выражение-отношения

выражение-отношения `=` выражение-равенства

выражение-отношения `<>` выражение-равенства

Выражение отношения

выражение-отношения:

выражение-сложения

выражение-сложения выражение-отношения

выражение-сложения выражение-отношения

выражение-сложения выражение-отношения

выражение-сложения выражение-отношения

Арифметические выражения

выражение-сложения:

выражение-умножения

выражение-умножения выражение-сложения

выражение-умножения выражение-сложения

выражение-умножения _выражение-сложения

выражение-умножения:

выражение-метаданных

выражение-метаданных выражение-умножения

выражение-метаданных выражение-умножения

Выражение метаданных

выражение-метаданных:

унарное-выражение

унарное-выражение унарное-выражение

Унарное выражение

унарное-выражение:

выражение-типа

унарное-выражение

унарное-выражение

унарное-выражение

Основное выражение

основное-выражение:

выражение-литерала

выражение-списка

выражение-записи

выражение-идентификатора

выражение-доступа-к-разделу

выражение-в-круглых-скобках

выражение-доступа-к-полю

выражение-доступа-к-элементу

выражение-вызова

выражение-отсутствия-реализации

Выражение литерала

выражение-литерала:

литерал

Выражение идентификатора

выражение-идентификатора:

ссылка-на-идентификатор

ссылка-на-идентификатор:

экслюзивная-ссылка-на-идентификатор

инклюзивная-ссылка-на-идентификатор

экслюзивная-ссылка-на-идентификатор:

идентификатор

инклюзивная-ссылка-на-идентификатор:

@ идентификатор

Выражение доступа к разделу

выражение-доступа-к-разделу:

идентификатор ! идентификатор

Выражение в круглых скобках

выражение-в-круглых-скобках:

(выражение)

Выражение отсутствия реализации

выражение-отсутствия-реализации:

...

Выражение вызова

выражение-вызова:

основное-выражение (список-аргументов необязательно)

список-аргументов:

выражение

выражение , список-аргументов

Выражение списка

выражение-списка:

{ элемент-списка необязательно }

элемент-списка:

элемент

элемент , элемент-списка

элемент:

выражение

выражение .. выражение

Выражение записи

выражение-записи:

[список-полей необязательно]

список-полей:

поле

поле , список-полей

поле:

имя-поля = выражение

имя-поля:

обобщенный-идентификатор

нестандартный-идентификатор

Выражение доступа к элементу

выражение-доступа-к-элементу:

выбор-элемента

выбор-необязательного-элемента

выбор-элемента:

основное-выражение { селектор-элементов }

выбор-необязательного-элемента:

основное-выражение { селектор-элементов } ?

селектор-элементов:

выражение

Выражения доступа к полю

выражение-доступа-к-полю:

выбор-поля

неявный-выбор-целевого-поля

проекция

неявная-проекция-целевого-поля

выбор-поля:

селектор-поля-основного выражения

селектор-поля:

селектор-обязательного-поля

селектор-необязательного-поля

селектор-обязательного-поля:

[имя-поля]

селектор-необязательного-поля:

[имя-поля] ?

имя-поля:

обобщенный-идентификатор

нестандартный-идентификатор

неявный-выбор-целевого-поля:

селектор-поля

проекция:

основное-выражение обязательная-проекция

основное-выражение необязательная-проекция

обязательная-проекция:

[список-обязательных-селекторов]

необязательная-проекция:

[список-обязательных-селекторов] ?

список-обязательных-селекторов:

селектор-обязательного-поля

селектор-обязательного-поля , список-обязательных-селекторов

неявная-целевая-проекция:

обязательная-проекция

необязательная-проекция

Выражение функции

выражение-функции:

(список-параметров необязательно) возвращаемый-тип необязательно => основная-часть-функции

основная-часть-функции:

выражение

список-параметров:

список-фиксированных-параметров

список-фиксированных-параметров , список-необязательных-параметров

список-необязательных-параметров

список-фиксированных-параметров:

параметр

параметр , список-фиксированных-параметров

параметр:

имя-параметра тип-параметра необязательно

имя-параметра:

идентификатор

тип-параметра:

проверочное-утверждение

возвращаемый-тип:

проверочное-утверждение

проверочное-утверждение:

`as` тип-примитива-допускающий-значение-NULL

список-необязательных-параметров:

необязательный-параметр

необязательный-параметр `,` список-необязательных-параметров

необязательный-параметр:

`optional` параметр

Выражение each

выражение-each:

`each` основная-часть-выражения-each

основная-часть-выражения-each:

основная-часть-функции

Выражение let

выражение-let:

`let` список-переменных `in` выражение

список-переменных:

переменная

переменная `,` список-переменных

переменная:

имя-переменной `=` выражение

имя-переменной:

идентификатор

Выражение if

выражение-if:

`if` условие-if `then` выражение-true `else` выражение-false

условие-if:

выражение

выражение-true:

выражение

выражение-false:

выражение

Выражение типа

выражение-типа:

основное-выражение

`type` основной-тип

тип:

выражение-в-круглых-скобках

основной-тип

основной-тип:

тип-примитива

тип-записи

тип-списка

тип-функции

тип-таблицы

тип-допускающий-значение-NULL

тип-примитива: один из следующего

`any` `anynonnull` `binary` `date` `datetime` `datetimezone` `duration` `function`

`list` `logical` `none` `null` `number` `record` `table` `text` `type`

тип-записи:

[маркер-открытой-записи]

[список-спецификаций-поля необязательно]

[список-спецификаций-поля , маркер-открытой-записи]

список-спецификаций-поля:

спецификация-поля

спецификация-поля , список-спецификаций-поля

спецификация-поля:

optional необязательно имя-поля спецификация-типа-поля необязательно

спецификация-типа-поля:

= тип-поля

тип-поля:

тип

маркер-открытой-записи:

...

тип-списка:

{ тип-элемента }

тип-элемента:

тип

тип-функции:

function (список-спецификаций-параметра необязательно) возвращаемый-тип

список-спецификаций-параметра:

список-спецификаций-обязательного-параметра

список-спецификаций-обязательного-параметра , список-спецификаций-необязательного-

параметра

список-спецификаций-необязательного-параметра

список-спецификаций-обязательного-параметра:

спецификация-обязательного-параметра

спецификация-обязательного-параметра , список-спецификаций-обязательного-параметра

спецификация-обязательного-параметра:

спецификация-параметра

список-спецификаций-необязательного-параметра:

спецификация-необязательного-параметра

спецификация-необязательного-параметра , список-спецификаций-необязательного-параметра

спецификация-необязательного-параметра:

optional спецификация-параметра

спецификация-параметра:

имя-параметра тип-параметра

тип-таблицы:

table тип-строки

тип-строки:

[список-спецификаций-поля]

тип-допускающий-значение-NULL:

nullable тип

Выражение, вызывающее ошибку

выражение-вызывающее-ошибку:

error выражение_

Выражение обработки ошибок

выражение-обработки-ошибок:

try защищенное-выражение-предложения-otherwise необязательно

защищенное-выражение:

выражение

предложение-otherwise:

`otherwise` выражение-по-умолчанию

выражение-по-умолчанию:

выражение

Литеральные атрибуты

литеральные-атрибуты:

литерал-записи

литерал-записи:

`[` список-литеральных-полей необязательно `]`

список-литеральных-полей:

литеральное-поле

литеральное-поле `,` список-литеральных-полей

литеральное-поле:

имя-поля `=` любой-литерал

литерал-списка:

`{` список-литеральных-элементов необязательно `}`

список-литеральных-элементов:

любой-литерал

любой-литерал `,` список-литеральных-элементов

любой-литерал:

литерал-записи

литерал-списка

логический-литерал

числовой-литерал

текстовый-литерал

литерал-NULL

Типы в языке формул Power Query M

16.03.2021 • 15 minutes to read

Язык формул Power Query M — это полезный и выразительный язык агрегирования данных. Но у него есть некоторые ограничения. Например, не существует строгой принудительной реализации системы типов. В некоторых случаях требуется более строгая проверка. К счастью, M предоставляет встроенную библиотеку с поддержкой типов, чтобы обеспечить более надежную проверку.

Разработчики должны иметь полное представление о системе типов, чтобы применять этот метод во всех случаях. Кроме того, хотя спецификация языка Power Query M хорошо объясняет систему типов, встречаются некоторые неожиданности. Например, для проверки экземпляров функций требуется способ сравнения типов для обеспечения совместимости.

Если внимательно изучить систему типов M, можно увидеть, что многие из этих проблем прояснены и разработчики смогут создавать необходимые решения.

Чтобы понять используемые обозначения, достаточно иметь представление об исчислении предикатов и наивной теории множеств.

ПРЕДВАРИТЕЛЬНЫЕ ТРЕБОВАНИЯ

(1) $B := \{ true, false \}$

B — это типичный набор логических значений

(2) $N := \{ \text{допустимые идентификаторы M} \}$

N — это набор всех допустимых имен в M. Он определяется в другом месте.

(3) $P := \square B, \top$

P — это набор параметров функции. Каждый из них может быть необязательным и имеет тип. Имена параметров несущественны.

(4) $P^n := \bigcup_{0 \leq i \leq n} \square i, P^i$

P^n — это набор всех упорядоченных последовательностей параметров функции n .

(5) $P^* := \bigcup_{0 \leq i \leq \infty} P^i$

P^* — это набор всех возможных последовательностей параметров функции, от 0 и более.

(6) $F := \square B, N, \top$

F — это набор всех полей записей. Каждое поле может быть необязательным, имеет имя и тип.

(7) $F^n := \prod_{0 \leq i \leq n} F$

F^n — это набор всех наборов из n полей записи.

(8) $F^* := \left(\bigcup_{0 \leq i \leq \infty} F^i \right) \setminus \{ F \mid \square b_1, n_1, t_1 \square, \square b_2, n_2, t_2 \square \in F \wedge n_1 = n_2 \}$

F^* — это набор всех наборов (любой длины) полей записей, за исключением наборов, в которых несколько полей имеют одно и то же имя.

(9) $C := \square N, \top$

C — это набор типов столбцов для таблиц. У каждого столбца есть имя и тип.

(10) $C^n \subset \bigcup_{0 \leq i \leq n} \square i, C^i$

C^n — это набор всех упорядоченных последовательностей типов столбцов n .

$$(11) C^* := \left(\bigcup_{0 \leq i \leq m} C^i \right) \setminus \{ C^m \mid \Box a, \Box n_1, t_1 \Box \Box, \Box b, \Box n_2, t_2 \Box \Box \in C^m \wedge n_1 = n_2 \}$$

C^* — это набор всех комбинаций (любой длины) типов столбцов, за исключением тех, где несколько столбцов имеют одно и то же имя.

ТИПЫ М

$$(12) T_F := \Box P, P^* \Box$$

Тип функции состоит из типа возвращаемого значения и упорядоченного списка, состоящего из нуля или более параметров функции.

$$(13) T_L := \llbracket T \rrbracket$$

Тип списка обозначается заданным типом (называемым "типом элемента"), заключенным в фигурные скобки. Так как фигурные скобки используются в метаязыке, в этом документе используются скобки $\llbracket \rrbracket$.

$$(14) T_R := \Box B, F^* \Box$$

Тип записи имеет флаг, указывающий, является ли он "открытым", и ноль или более неупорядоченных полей записи.

$$(15) T_R^o := \Box true, F \Box$$

$$(16) T_R^* := \Box false, F \Box$$

T_R^o и T_R^* — это сокращенные нотации для открытых и закрытых типов записей соответственно.

$$(17) T_T := C^*$$

Табличный тип является упорядоченной последовательностью нуля и более типов столбцов, где нет конфликтов имен.

$$(18) T_P := \{ \text{any; none; null; logical; number; time; date; datetime; datetimetype; duration; text; binary; type; list; record; table; function; anynonnull} \}$$

Примитивный тип берется из списка ключевых слов M .

$$(19) T_N := \{ t_n, u \in T \mid t_n = u + \text{null} \} = \text{nullable } t$$

Любой тип можно дополнительно пометить как допускающий значение $NULL$, используя ключевое слово "nullable".

$$(20) T := T_F \cup T_L \cup T_R \cup T_T \cup T_P \cup T_N$$

Набор всех типов M является объединением этих шести наборов типов: типы функций, типы списков, типы записей, типы таблиц, примитивные типы и типы, допускающие значение $NULL$.

ФУНКЦИИ

Необходимо определить одну функцию: $Nonnullable: T \leftarrow T$

Эта функция принимает тип и возвращает эквивалентный тип, которые не соответствуют значению $NULL$.

УДОСТОВЕРЕНИЯ

Некоторые удостоверения необходимы для определения особых случаев, а также могут помочь прояснить указанные выше элементы.

$$(21) \text{nullable any} = \text{any}$$

$$(22) \text{nullable anynonnull} = \text{any}$$

$$(23) \text{nullable null} = \text{null}$$

$$(24) \text{nullable none} = \text{null}$$

$$(25) \text{nullable nullable } t \in T = \text{nullable } t$$

$$(26) NonNullable(\text{nullable } t \in T) = NonNullable(t)$$

(27) $Nonnullable(any) = anynonnull$

СОВМЕСТИМОСТЬ ТИПОВ

Как определено в других случаях, тип M совместим с другим типом M , только если все значения, соответствующие первому типу, также соответствуют второму типу.

Здесь определяется отношение совместимости, которое не зависит от значений согласованности и основано на свойствах самих типов. Ожидается, что это отношение, как определено в этом документе, полностью эквивалентно исходному семантическому определению.

Отношение "совместимо с": $\leq : B \leftarrow T \times T$

В приведенном ниже разделе t в нижнем регистре всегда будет представлять тип M , элемент T .

Φ будет представлять подмножество F^* или C^* .

(28) $t \leq t$

Это отношение является рефлексивным.

(29) $t_a \leq t_b \wedge t_b \leq t_c \rightarrow t_a \leq t_c$

Это отношение является транзитивным.

(30) $none \leq t \leq any$

Типы M формируют структуру для этого отношения, где $none$ на находится сверху, а any — снизу.

(31) $t_a, t_b \in T_N \wedge t_a \leq t_b \rightarrow NonNullable(t_a) \leq NonNullable(t_b)$

Если два типа совместимы, то эквиваленты $Nonnullable$ также совместимы.

(32) $null \leq t \in T_N$

Примитивный тип $NULL$ совместим со всеми типами, допускающими значение $NULL$.

(33) $t \notin T_N \leq anynonnull$

Все типы, не допускающие значения $NULL$, совместимы с $anynonnull$.

(34) $Nonnullable(t) \leq t$

Тип $Nonnullable$ совместим с эквивалентом, допускающим значение $NULL$.

(35) $t \in T_F \rightarrow t \leq function$

Все типы функций совместимы с $function$.

(36) $t \in T_L \rightarrow t \leq list$

Все типы списков совместимы с $list$.

(37) $t \in T_R \rightarrow t \leq record$

Все типы записей совместимы с $record$.

(38) $t \in T_T \rightarrow t \leq table$

Все типы таблиц совместимы с $table$.

(39) $t_a \leq t_b \boxplus \llbracket t_a \rrbracket \leq \llbracket t_b \rrbracket$

Тип списка совместим с другим типом списка, если типы элементов совместимы, и наоборот.

(40) $t_a \in T_F = \square p_a, p^* \square, t_b \in T_F = \square p_b, p^* \square \wedge p_a \leq p_b \rightarrow t_a \leq t_b$

Тип функции совместим с другим типом функции, если типы возвращаемых значений совместимы, а списки параметров идентичны.

(41) $t_a \in T_R^o, t_b \in T_R^* \rightarrow t_a \not\leq t_b$

Тип открытой записи не совместим с типом закрытой записи.

(42) $t_a \in T_R^* = \square false, \Phi \square, t_b \in T_R^o = \square true, \Phi \square \rightarrow t_a \leq t_b$

Тип закрытой записи совместим с типом открытой записи, который в других обстоятельствах был бы идентичным.

$$(43) t_a \in T_R^o = \langle \Phi, \langle \text{true}, n, \text{any} \rangle \rangle, t_b \in T_R^o = \langle \Phi, \langle \text{true}, n, \text{any} \rangle \rangle \rightarrow t_a \leq t_b \wedge t_b \leq t_a$$

Необязательное поле с типом any может быть пропущено при сравнении двух открытых типов записей.

$$(44) t_a \in T_R = \langle \Phi, \langle \beta, n, u_a \rangle \rangle, t_b \in T_R = \langle \Phi, \langle \beta, n, u_b \rangle \rangle \wedge u_a \leq u_b \rightarrow t_a \leq t_b$$

Два типа записей, отличающиеся только одним полем, являются совместимыми, если имя и необязательность поля идентичны, а типы этих полей совместимы.

$$(45) t_a \in T_R = \langle \Phi, \langle \text{false}, n, u \rangle \rangle, t_b \in T_R = \langle \Phi, \langle \text{true}, n, u \rangle \rangle \rightarrow t_a \leq t_b$$

Тип записи с обязательным полем совместим с типом записи, идентичным во всем, кроме того, что это поле является необязательным.

$$(46) t_a \in T_R^o = \langle \Phi, \langle \text{true}, n, u \rangle \rangle, t_b \in T_R^o = \langle \Phi, \langle \text{true}, n, u \rangle \rangle \rightarrow t_a \leq t_b$$

Тип открытой записи совместим с другим типом открытой записи, где на одно поле меньше.

$$(47) t_a \in T_T = \langle \Phi, \langle i, n, u_a \rangle \rangle, t_b \in T_T = \langle \Phi, \langle i, n, u_b \rangle \rangle \wedge u_a \leq u_b \rightarrow t_a \leq t_b$$

Тип таблицы совместим с типом второй таблицы, который идентичен во всем, кроме того, что один столбец имеет другой тип, если типы этого столбца совместимы.

REFERENCES

Корпорация Майкрософт (август 2015 года)

Спецификация языка формул Microsoft Power Query для Excel [PDF]

Получено со страницы <https://msdn.microsoft.com/library/mt807488.aspx>

Корпорация Майкрософт (дата не указана)

Справочник по функциям Power Query M [веб-страница]

Получено со страницы <https://msdn.microsoft.com/library/mt779182.aspx>

Выражения, значения и выражение let

19.04.2021 • 9 minutes to read

Запрос на языке формул Power Query M состоит из шагов **выражения** формулы, составляющих гибридный запрос. Выражение формулы можно вычислить (оценить), возвратив значение. Выражение **Let** инкапсулирует набор значений, которые требуется вычислить, снабдить именами, а затем использовать в последующем выражении, которое следует за оператором **in**. Например, выражение **let** может содержать переменную **Source**, которая равна значению **Text.Proper()** и возвращает текстовое значение в правильном регистре.

Выражение let

```
let
    Source = Text.Proper("hello world")
in
    Source
```

В приведенном выше примере **ext.Proper("hello world")** вычисляется как **"Hello World"**.

В следующих разделах описаны типы значений в языке.

Значение примитива

Примитив — это однокомпонентное значение, например число, логическое значение, текст или значение **NULL**. Значение **NULL** может использоваться для указания отсутствия каких-либо данных.

TYPE	ПРИМЕР ЗНАЧЕНИЯ
Двоичные данные	00 00 00 02 // число точек (2)
Date	5/23/2015
Дата и время	5/23/2015 12:00:00 AM
DateTimeZone	5/23/2015 12:00:00 AM -08:00
Duration	15:35:00
Логические	True и False
NULL	null
Число	0, 1, -1, 1.5 и 2.3e-5
Текст	"abc"
Время	12:34:12 PM

Значение функции

Функция — это значение, которое при вызове с аргументами создает новое значение. Функции записываются путем перечисления **параметров** функции в круглых скобках, за которыми следует символ перехода `=>`, за которым следует определяющее функцию выражение. Например, чтобы создать функцию `MyFunction`, которая имеет два параметра, `parameter1` и `parameter2`, и выполняет вычисление по ним:

```
let
  MyFunction = (parameter1, parameter2) => (parameter1 + parameter2) / 2
in
  MyFunction

Calling the MyFunction() returns the result:

let
  Source = MyFunction(2, 4)
in
  Source
```

Этот код выдает значение 3.

Значения структурированных данных

Язык M поддерживает следующие значения структурированных данных:

- [Список](#)
- [Запись](#)
- [Таблица](#)
- [Дополнительные примеры структурированных данных](#)

NOTE

Структурированные данные могут содержать любое значение M. Несколько примеров см. в разделе [Дополнительные примеры структурированных данных](#).

Список

Список — это отсчитываемая от нуля упорядоченная последовательность значений, заключенная в фигурные скобки `{ }`. Фигурные скобки `{ }` также используются для получения элемента из списка по позиции индекса. См. раздел `[Значение списка](#_List_value)`.

NOTE

Power Query M поддерживает неограниченный размер списка, но если список записывается как литерал, он имеет фиксированную длину. Например, `{1, 2, 3}` имеет фиксированную длину, равную 3.

Ниже приведены некоторые примеры **списков**.

ЗНАЧЕНИЕ	TYPE
{123, true, "A"}	Список, содержащий число, логическое значение и текст.
{1, 2, 3}	Список чисел.

ЗНАЧЕНИЕ	TYPE
{ {1, 2, 3}, {4, 5, 6} }	Список из списка чисел.
{ [CustomerID = 1, Name = "Bob", Phone = "123-4567"], [CustomerID = 2, Name = "Jim", Phone = "987-6543"] }	Список записей.
{123, true, "A"}{0}	Возвращение значения первого элемента в списке. Это выражение возвращает значение 123.
{ {1, 2, 3}, {4, 5, 6} }{0}{1}	Возвращение значения второго элемента из первого элемента списка. Это выражение возвращает значение 2.

Запись

Запись представляет собой набор полей. **Поле** — это пара "имя-значение", где имя является текстовым значением, уникальным в пределах записи поля. Синтаксис значений записи позволяет записывать имена без кавычек, в такой форме они также называются **идентификаторами**. Идентификатор может принимать две следующие формы:

- Имя_идентификатора, например OrderID.
- #"имя идентификатора", например #"Сегодняшняя дата: ".

Ниже приведена запись, содержащая поля "OrderID", "CustomerID", "Item" и "Price" со значениями 1, 1, "Fishing rod" и 100,00. Квадратные скобки [] обозначают начало и конец выражения записи и используются для получения значения поля из записи. В следующих примерах показана запись и получение значения поля "Item".

Ниже приведен пример записи:

```
let Source =
    [
        OrderID = 1,
        CustomerID = 1,
        Item = "Fishing rod",
        Price = 100.00
    ]
in Source
```

Чтобы получить значение поля "Item", используйте квадратные скобки для получения выражения Source[Item]:

```
let Source =
    [
        OrderID = 1,
        CustomerID = 1,
        Item = "Fishing rod",
        Price = 100.00
    ]
in Source[Item] //equals "Fishing rod"
```

Таблица

Таблица — это набор значений, упорядоченных по именованным столбцам и строкам. Тип столбца может быть неявным или явным. `#table` можно использовать для создания списка имен столбцов и списка строк. **Таблица** значений представляет собой список в **списке**. Фигурные скобки `{ }` также используются для извлечения строки из **таблицы** по позиции индекса (см. раздел [Пример 3. Получение строки из таблицы по позиции индекса](#)).

Пример 1. Создание таблицы с неявными типами столбцов

```
let
  Source = #table(
    {"OrderID", "CustomerID", "Item", "Price"},
    {
      {1, 1, "Fishing rod", 100.00},
      {2, 1, "1 lb. worms", 5.00}
    })
in
  Source
```

Пример 2. Создание таблицы с явными типами столбцов

```
let
  Source = #table(
    type table [OrderID = number, CustomerID = number, Item = text, Price = number],
    {
      {1, 1, "Fishing rod", 100.00},
      {2, 1, "1 lb. worms", 5.00}
    }
  )
in
  Source
```

Оба приведенных выше примера создают таблицу следующего вида:

ORDERID	CUSTOMERID	ЭЛЕМЕНТ	ЦЕНА
1	1	Удочка	100,00
2	1	Черви, 1 фунт	5.00

Пример 3. Получение строки из таблицы по позиции индекса

```
let
  Source = #table(
    type table [OrderID = number, CustomerID = number, Item = text, Price = number],
    {
      {1, 1, "Fishing rod", 100.00},
      {2, 1, "1 lb. worms", 5.00}
    }
  )
in
  Source{1}
```

Это выражение возвращает следующую запись:

ПОЛЕ	ЗНАЧЕНИЕ
OrderID	2

ПОЛЕ	ЗНАЧЕНИЕ
CustomerID	1
Элемент	Черви, 1 фунт
Цена	5

Дополнительные примеры структурированных данных

Структурированные данные могут содержать любое значение M. Ниже приводится несколько примеров.

Пример 1. Список со значениями [Primitive](#_Primitive_value_1), [Function](#_Function_value) и [Record](#_Record_value)

```
let
    Source =
    {
        1,
        "Bob",
        DateTime.ToText(DateTime.LocalNow(), "yyyy-MM-dd"),
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0]
    }
in
    Source
```

Наглядно вычисление этого выражения можно представить следующим образом:

A List containing a Record	
1	
"Bob"	
2015-05-22	
OrderID	1
CustomerID	1
Item	"Fishing rod"
Price	100.0

Пример 2. Запись, содержащая значения примитивов и вложенные записи

```
let
    Source = [CustomerID = 1, Name = "Bob", Phone = "123-4567", Orders =
        {
            [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],
            [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0]
        }]
in
    Source
```

Наглядно вычисление этого выражения можно представить следующим образом:

A record containing a List of Records		
CustomerID	1	
Name	"Bob"	
Phone	"123-4567"	
Orders	OrderID	1
	CustomerID	1
	Item	"Fishing rod"
	Price	100.0
	OrderID	2
	CustomerID	1
	Item	"1 lb. worms"
	Price	5.0

NOTE

Хотя многие значения могут быть записаны буквально в виде выражения, значение не является выражением. Например, вычисление выражения 1 дает значение 1, а вычисление выражения 1 + 1 дает значение 2. Это неочевидное, но важное различие. Выражения являются наборами правил для вычисления, а значения — результатами вычисления.

Выражение if

Выражение if осуществляет выбор между двумя выражениями на основе логического условия. Пример:

```
if 2 > 1 then
  2 + 2
else
  1 + 1
```

Первое выражение (2 + 2) выбирается, если логическое выражение (2 > 1) истинно, а второе выражение (1 + 1) выбирается, если оно ложно. Выбранное выражение (в данном случае 2 + 2) вычисляется и становится результатом выражения if (4).

Комментарии

16.03.2021 • 2 minutes to read

В код можно добавлять однострочные комментарии (`//`) или многострочные комментарии, начинающиеся с `/*` и заканчивающиеся на `*/`.

Пример однострочного комментария

```
let
    //Convert to proper case.
    Source = Text.Proper("hello world")
in
    Source
```

Пример многострочного комментария

```
/* Capitalize each word in the Item column in the Orders table. Text.Proper
is evaluated for each Item in each table row. */
let
    Orders = Table.FromRecords({
        [OrderID = 1, CustomerID = 1, Item = "fishing rod", Price = 100.0],
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
        [OrderID = 3, CustomerID = 2, Item = "fishing net", Price = 25.0]}),
    #"Capitalized Each Word" = Table.TransformColumns(Orders, {"Item", Text.Proper})
in
    #"Capitalized Each Word"
```

Модель оценки

16.03.2021 • 3 minutes to read

Модель оценки языка формул Power Query M базируется на модели оценки, часто применяемой в электронных таблицах, где порядок вычислений можно определить на основе зависимостей между формулами в ячейках.

Если вы ввели формулы в электронных таблицах, например в Excel, можно определить, что при вычислении указанные слева формулы дадут значения, указанные справа:

	A
1	=A2 * 2
2	=A3 + 1
3	1

	A
1	4
2	2
3	1

В языке M выражение может ссылаться на предыдущие выражения по имени, и процесс оценки автоматически определит порядок вычисления выражений, на которые имеются ссылки.

Давайте используем запись для создания выражения, эквивалентное приведенному выше примеру электронной таблицы. При инициализации значения поля вы ссылаетесь на другие поля в записи по имени поля, как показано ниже:

```
[  
    A1 = A2 * 2,  
    A2 = A3 + 1,  
    A3 = 1  
]
```

При оценке приведенное выше выражение дает следующую запись:

```
[  
    A1 = 4,  
    A2 = 2,  
    A3 = 1  
]
```

Записи могут содержаться в других записях или **вкладываться** в них. Для обращения к полям записи по имени можно использовать **оператор подстановки** ([]). Например, следующая запись содержит поле Sales, содержащее запись, и поле Total, которое обращается к полям FirstHalf и SecondHalf записи Sales:

```
[  
    Sales = [ FirstHalf = 1000, SecondHalf = 1100 ],  
    Total = Sales[FirstHalf] + Sales[SecondHalf]  
]
```

При оценке приведенное выше выражение дает следующую запись:

```
[
  Sales = [ FirstHalf = 1000, SecondHalf = 1100 ],
  Total = 2100
]
```

Чтобы обратиться к элементу списка по его числовому индексу, используйте **позиционный оператор индекса** (`{}`). Для ссылки на значения в списке используется начинающийся с нуля индекс, отсчитываемый от начала списка. Например, индексы 0 и 1 используются для ссылки на первый и второй элементы в списке ниже:

```
[
  Sales =
  {
    [
      Year = 2007,
      FirstHalf = 1000,
      SecondHalf = 1100,
      Total = FirstHalf + SecondHalf // equals 2100
    ],
    [
      Year = 2008,
      FirstHalf = 1200,
      SecondHalf = 1300,
      Total = FirstHalf + SecondHalf // equals 2500
    ]
  },
  #"Total Sales" = Sales{0}[Total] + Sales{1}[Total] // equals 4600
]
```

Отложенная и безотложная оценка

Выражения членов `List`, `Record` и `Table`, как и выражения `let` (см. раздел [Выражения, значения и выражение let](#)), оцениваются с помощью **Отложенной оценки**, то есть по мере необходимости. Все остальные выражения оцениваются с помощью **безотложной оценки**, то есть сразу же при их обнаружении в процессе оценки. При этом следует помнить, что оценка выражения записи или списка возвратит значение записи или списка, которому известны соответствующие поля записи или элементы списка, которые требуется вычислить при необходимости (с помощью операторов индекса или подстановки).

Операторы

16.03.2021 • 5 minutes to read

Язык формул Power Query M включает набор операторов, которые можно использовать в выражении.

Операторы применяются к **операндам** для формирования символьных выражений. Например, в выражении `1 + 2` числа 1 и 2 являются операндами, а оператор представлен оператором сложения (+).

Значение оператора может меняться в зависимости от типа значений операндов. В этом языке есть следующие операторы:

Оператор сложения (+)

ВЫРАЖЕНИЕ	РАВНО
<code>1 + 2</code>	Суммирование числа: 3
<code>#time (12, 23, 0) + #duration (0, 0, 2, 0)</code>	Арифметические операции со временем: <code>#time(12,25,0)</code>

Оператор комбинирования (&)

ФУНКЦИЯ	РАВНО
<code>"A" & "BC"</code>	Объединение текста: "ABC"
<code>{1} & {2, 3}</code>	Объединение списков: {1, 2, 3}
<code>[a = 1] & [b = 2]</code>	Слияние записей: [a = 1, b = 2]

Список операторов M

Общие операторы, которые применяются к значению NULL, логическому значению, числу, времени, дате, значению `datetime`, значению `datetimezone`, длительности, тексту, двоичному значению)

ОПЕРАТОР	ОПИСАНИЕ
<code>></code>	Больше чем
<code>>=</code>	Больше или равно
<code><</code>	Меньше чем
<code><=</code>	Меньше или равно
<code>=</code>	Равно
<code><></code>	Не равно

Логические операторы (в дополнение к **общим операторам**)

ОПЕРАТОР	ОПИСАНИЕ
or	Условное логическое ИЛИ
и	Условное логическое И
not	Логическое НЕ

Числовые операторы (в дополнение к **общим операторам**)

ОПЕРАТОР	ОПИСАНИЕ
+	Сумма
-	Разность
*	Продукт
/	Знаменатель
+x	Унарный плюс
-x	Отрицание

Текстовые операторы (в дополнение к **общим операторам**)

ОПЕРАТОР	ОПИСАНИЕ
&	Объединение

Операторы для списков, записей, таблиц

ОПЕРАТОР	ОПИСАНИЕ
=	Равно
<>	Не равно
&	Объединение

Оператор поиска записей

ОПЕРАТОР	ОПИСАНИЕ
[]	Обращение к полям записи по имени.

Оператор индекса списка

ОПЕРАТОР	ОПИСАНИЕ
{}	Обращение к элементу списка по его числовому индексу, отсчитываемому от нуля.

Операторы утверждения и совместимости типов

ОПЕРАТОР	ОПИСАНИЕ
is	Выражение "x is y" возвращает true, если тип значения x совместим с y, и false, если они несовместимы.
as	Выражение "x as y" утверждает, что значение x совместимо с y в соответствии с применением оператора is.

Операторы даты

ОПЕРАТОР	ЛЕВЫЙ ОПЕРАНД	ПРАВЫЙ ОПЕРАНД	ЗНАЧЕНИЕ
x + y	time	длительность	Смещение даты по длительности
x + y	длительность	time	Смещение даты по длительности
x − y	time	длительность	Смещение даты по длительности, к которой применено отрицание
x − y	time	time	Длительность между датами
x & y	Дата	time	Объединенное значение datetime

Операторы для значений datetime

ОПЕРАТОР	ЛЕВЫЙ ОПЕРАНД	ПРАВЫЙ ОПЕРАНД	ЗНАЧЕНИЕ
x + y	DATETIME	длительность	Смещение значения datetime по длительности
x + y	длительность	DATETIME	Смещение значения datetime по длительности
x − y	DATETIME	длительность	Смещение значения datetime по длительности, к которой применено отрицание
x − y	DATETIME	DATETIME	Длительность между значениями datetime

Операторы для значений datetimezone

ОПЕРАТОР	ЛЕВЫЙ ОПЕРАНД	ПРАВЫЙ ОПЕРАНД	ЗНАЧЕНИЕ
x + y	datetimezone	длительность	Смещение значения datetimezone по длительности

ОПЕРАТОР	ЛЕВЫЙ ОПЕРАНД	ПРАВЫЙ ОПЕРАНД	ЗНАЧЕНИЕ
$x + y$	длительность	datetimezone	Смещение значения datetimezone по длительности
$x - y$	datetimezone	длительность	Смещение значения datetimezone по длительности, к которой применено отрицание
$x - y$	datetimezone	datetimezone	Длительность между значениями datetimezone

Операторы длительности

ОПЕРАТОР	ЛЕВЫЙ ОПЕРАНД	ПРАВЫЙ ОПЕРАНД	ЗНАЧЕНИЕ
$x + y$	DATETIME	длительность	Смещение значения datetime по длительности
$x + y$	длительность	DATETIME	Смещение значения datetime по длительности
$x + y$	длительность	длительность	Сумма длительностей
$x - y$	DATETIME	длительность	Смещение значения datetime по длительности, к которой применено отрицание
$x - y$	DATETIME	DATETIME	Длительность между значениями datetime
$x - y$	длительность	длительность	Разница длительностей
$x * y$	длительность	number	Длительность, увеличенная в N раз
$x * y$	number	длительность	Длительность, увеличенная в N раз
x / y	длительность	number	Частное длительности

NOTE

Оператор может поддерживать не все сочетания значений. Выражения, при вычислении которых возникают неопределенные условия оператора, обрабатываются как ошибки. Дополнительные сведения об ошибках в М см. в разделе [Ошибки](#).

Пример ошибки:

ФУНКЦИЯ	РАВНО
1 + "2"	Ошибка: добавление числа и текста не поддерживается

Преобразование типов

16.03.2021 • 3 minutes to read

В языке формул Power Query M имеются формулы для преобразования типов. Ниже приведена сводка по формулам преобразований в M.

Число

ПРЕОБРАЗОВАНИЕ ТИПОВ	ОПИСАНИЕ
Number.FromText(текст в формате текста) в виде числа	Возвращает числовое значение для текстового значения.
Number.ToText(число в формате числа) в виде текста	Возвращает текстовое значение для числового значения.
Number.From(значение в любом формате) в виде числа	Возвращает числовое значение для заданного значения.
Int32.From(значение в любом формате) в виде числа	Создает 32-разрядное целое число по заданному значению.
Int64.From(значение в любом формате) в виде числа	Создает 64-разрядное целое число по заданному значению.
Single.From(значение в любом формате) в виде числа	Возвращает значение типа Single для заданного значения.
Double.From(значение в любом формате) в виде числа	Возвращает значение типа Double для заданного значения.
Decimal.From(значение в любом формате) в виде числа	Возвращает значение типа Decimal для заданного значения.
Currency.From(значение в любом формате) в виде числа	Возвращает значение типа Currency для заданного значения.

Текст

ПРЕОБРАЗОВАНИЕ ТИПОВ	ОПИСАНИЕ
Text.From(значение в любом формате) в виде числа	Возвращает текстовое представление числа, даты, времени, значения datetime, значения datetimezone, логического значения, длительности или двоичного значения.

Логические

ПРЕОБРАЗОВАНИЕ ТИПОВ	ОПИСАНИЕ
Logical.FromText(текст в формате текста) в виде логического значения	Возвращает логическое значение true или false для текстового значения.
Logical.ToText(логическое значение в формате логического значения) в виде текста	Возвращает текстовое значение для логического значения.
Logical.From(значение в любом формате) в виде логического значения	Возвращает логическое значение для значения.

Значение даты, времени, DateTime и DateTimeZone

ПРЕОБРАЗОВАНИЕ ТИПОВ	ОПИСАНИЕ
.FromText(текст в формате текста) в виде значения даты, времени, datetime или datetimezone	Возвращает значение даты, времени, datetime или datetimezone из набора форматов даты и значения языка и региональных параметров.
.ToText(значение даты, времени, dateTime или dateTimeZone в формате значения даты, времени, datetime или datetimezone) в виде текста	Возвращает текстовое значение из значения даты, времени, datetime или datetimezone.
.From(значение в любом формате)	Возвращает значение даты, времени, datetime или datetimezone из значения.
.ToRecord(значение даты, времени, dateTime или dateTimeZone в формате значения даты, времени, datetime или datetimezone)	Возвращает запись, содержащую части значения даты, времени, datetime или datetimezone.

Метаданные

16.03.2021 • 2 minutes to read

Метаданные — это сведения о значении, которое связано со значением. **Метаданные** представлены в виде значения записи, которое называется записью метаданных. Поля **записи метаданных** можно использовать для хранения метаданных значения. Каждое значение имеет запись метаданных. Если значение записи метаданных не указано, то запись метаданных пуста (не содержит поля). Связывание записи метаданных со значением не влияет на поведение значения в вычислениях, за исключением тех, которые явно проверяют записи метаданных.

Значение записи метаданных связывается со значением *x* с помощью синтаксиса "значение meta [запись]". Например, следующая связывает запись метаданных с полями Rating и Tags с текстовым значением "Mozart":

```
"Mozart" meta [ Rating = 5,  
Tags = {"Classical"} ]
```

Обратиться к записи метаданных для значения можно с помощью функции `Value.Metadata`. В следующем примере выражение в поле `ComposerRating` обращается к записи метаданных значения в поле `Composer`, а затем обращается к полю `Rating` записи метаданных.

```
[  
  Composer = "Mozart" meta [ Rating = 5, Tags = {"Classical"} ],  
  ComposerRating = Value.Metadata(Composer)[Rating] // 5  
]
```

Записи метаданных не сохраняются, если значение используется с оператором или функцией, создающими новое значение. Например, если два текстовых значения сцеплены с помощью оператора `&`, метаданные итогового текстового значения будут представлены пустой записью `[]`.

Функции стандартной библиотеки `Value.RemoveMetadata` и `Value.ReplaceMetadata` можно использовать для удаления всех метаданных из значения и для замены метаданных значения.

Ошибки

16.03.2021 • 2 minutes to read

Ошибка в языке формул Power Query M означает, что при вычислении выражения не удалось получить значение. Ошибки выдаются операторами и функциями, сталкивающимися с условиями **ошибки**, либо с помощью выражения `error`. Ошибки обрабатываются с помощью выражения `try`. При возникновении ошибки указывается значение, которое можно использовать для указания причины возникновения ошибки.

Выражение try

Выражение `try` преобразует значения и ошибки в значение записи, которое указывает, обрабатывало ли выражение `try` ошибку или нет, и правильное значение либо запись об ошибке, извлеченные при обработке ошибки. Например, рассмотрим следующее выражение, которое вызывает ошибку и затем сразу же обрабатывает ее:

```
try error "negative unit count"
```

Результатом вычисления этого выражения является следующее вложенное значение записи, поясняющее подстановки полей `[HasError]`, `[Error]` и `[Message]` в приведенном ранее примере с ценой на единицу.

Запись ошибки

```
[
  HasError = true,
  Error =
    [
      Reason = "Expression.Error",
      Message = "negative unit count",
      Detail = null
    ]
]
```

Распространена замена ошибок значениями по умолчанию. Чтобы добиться того же, но в компактной форме, можно использовать выражение `try` с необязательным предложением `otherwise`:

```
try error "negative unit count" otherwise 42
// equals 42
```

Пример ошибки


```

let Sales =
    [
        ProductName = "Fishing rod",
        Revenue = 2000,
        Units = 1000,
        UnitPrice = if Units = 0 then error "No Units"
                    else Revenue / Units
    ],

    //Get UnitPrice from Sales record
    textUnitPrice = try Number.ToText(Sales[UnitPrice]),
    Label = "Unit Price: " &
        (if textUnitPrice[HasError] then textUnitPrice[Error][Message]
        //Continue expression flow
        else textUnitPrice[Value])
in
    Label

```

Приведенный выше пример обращается к полю `Sales[UnitPrice]` и форматирует значение, формирующее результат:

```
"Unit Price: 2"
```

Если бы поле Units было равно нулю, то поле `UnitPrice` привело бы к возникновению ошибки, которая была бы обработана выражением `try`. В этом случае итоговое значение имело бы следующий вид:

```
"No Units"
```

Справочник по функциям Power Query M

16.03.2021 • 2 minutes to read

Справочник по функциям Power Query M содержит статьи по каждой из более чем 700 функций. Справочные статьи на сайте docs.microsoft.com создаются автоматически из встроенной в продукт справки. Дополнительные сведения о функциях и их использовании в выражениях см. в статье [Основные сведения о функциях Power Query M](#).

Функции по категориям

- [Функции доступа к данным](#)
- [Двоичные функции](#)
- [Функции объединения](#)
- [Функции сравнения](#)
- [Функции данных](#)
- [Функции для работы с датами и временем](#)
- [Функции для работы с датами, временем и часовыми поясами](#)
- [Функции для работы с длительностью](#)
- [Обработка ошибок](#)
- [Функции выражений](#)
- [Значения функции](#)
- [Функции для работы со списками](#)
- [Функции для работы со строками](#)
- [Логические функции](#)
- [Функции для работы с числами](#)
- [Функции для работы с записями](#)
- [Функции замены](#)
- [Функции разделения](#)
- [Табличные функции](#)
- [Текстовые функции](#)
- [Функции для работы со временем](#)
- [Функции типов](#)
- [Функции для работы с URI](#)
- [Функции для работы со значениями](#)

Основные сведения о функциях Power Query M

16.03.2021 • 3 minutes to read

В языке формул Power Query M **функция** — это сопоставление набора входных значений с одним выходным значением. Для создания функции нужно указать ее параметры, а затем выражение для вычисления ее результата. Тело функции следует за символом перехода (`=>`). При необходимости сведения о типе можно включить в параметры и возвращаемое значение функции. Функция определяется и вызывается в теле оператора `let`. Параметры и (или) возвращаемое значение могут быть неявными или явными. Неявные параметры и (или) возвращаемое значение имеют тип `any`. Тип `any` похож на тип `object` в других языках. Все типы в M являются производными от типа `any`.

Функция — это значение, аналогичное числу или текстовому значению, которое можно включить в строку так же, как и любое другое выражение. В следующем примере показана функция, являющаяся значением переменной `Add`, которая затем вызывается или выполняется из нескольких других переменных. При вызове функции указывается набор значений, которые логически подставляются для требуемого набора входных значений в выражении тела функции.

Пример: явные параметры и возвращаемое значение

```
let
    AddOne = (x as number) as number => x + 1,
    //additional expression steps
    CalcAddOne = AddOne(5)
in
    CalcAddOne
```

Пример: неявные параметры и возвращаемое значение

```
let
    Add = (x, y) => x + y,
    AddResults =
        [
            OnePlusOne = Add(1, 1),    // equals 2
            OnePlusTwo = Add(1, 2)    // equals 3
        ]
in
    AddResults
```

Поиск первого элемента больше 5 в списке или вывод значения NULL

```
let
    FirstGreaterThan5 = (list) =>
        let
            GreaterThan5 = List.Select(list, (n) => n > 5),
            First = List.First(GreaterThan5)
        in
            First,
    Results =
        [
            Found = FirstGreaterThan5({3,7,9}), // equals 7
            NotFound = FirstGreaterThan5({1,3,4}) // equals null
        ]
in
    Results
```

Функции можно использовать рекурсивно. Чтобы рекурсивно сослаться на функцию, добавьте в идентификатор префикс @.

```
let
  fact = (num) => if num = 0 then 1 else num * @fact (num-1)
in
  fact(5) // equals 120
```

Ключевое слово each

Ключевое слово **each** используется для удобного создания простых функций. "each ..." — это синтаксический сахар для сигнатуры функции, которая принимает параметр "(_) => ...".

Ключевое слово **each** удобно использовать совместно с оператором подстановки, который по умолчанию применяется к `_`.

Например, запись `each [CustomerID]` аналогичная записи `each _[CustomerID]`, которая аналогичная записи `(_) => _[CustomerID]`.

Пример: использование each в фильтре строк таблицы

```
Table.SelectRows(
  Table.FromRecords({
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"] ,
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"] ,
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
  }),
  each [CustomerID] = 2
)[Name]

// equals "Jim"
```

Функции доступа к данным

16.03.2021 • 14 minutes to read

Доступ к данным

Эти функции обращаются к данным и возвращают табличные значения. Большинство этих функций возвращают табличное значение, которое называется **таблицей навигации**. Таблицы навигации основном используются в пользовательском интерфейсе Power Query для обеспечения навигации по потенциально большим возвращаемым иерархическим наборам данных.

ФУНКЦИЯ	ОПИСАНИЕ
AccessControlEntry.ConditionToIdentities	Возвращает список удостоверений, которые будут приняты условием.
AccessControlKind.Allow	Доступ разрешен.
AccessControlKind.Deny	Доступ отклонен.
Access.Database	Возвращает структурное представление базы данных Microsoft Access.
ActiveDirectory.Domains	Возвращает список доменов Active Directory в том же лесу, что и указанный домен, или из домена текущего компьютера, если значение не указано.
AdobeAnalytics.Cubes	Возвращает наборы отчетов в Adobe Analytics.
AdoDotNet.DataSource	Возвращает коллекцию схем для источника данных ADO.NET.
AdoDotNet.Query	Возвращает коллекцию схем для источника данных ADO.NET.
AnalysisServices.Database	Возвращает таблицу многомерных кубов или табличных моделей из базы данных служб Analysis Services.
AnalysisServices.Databases	Возвращает базы данных Analysis Services на определенном узле.
AzureStorage.BlobContents	Возвращает содержимое указанного BLOB-объекта из хранилища Azure.
AzureStorage.Blobs	Возвращает навигационную таблицу, содержащую все контейнеры, найденные в учетной записи службы хранилища Azure. Каждая строка содержит имя контейнера и ссылку на BLOB-объекты контейнера.
AzureStorage.DataLake	Возвращает таблицу навигации, содержащую документы, обнаруженные в указанном контейнере в Azure Data Lake Storage и его подпапках.

ФУНКЦИЯ	ОПИСАНИЕ
AzureStorage.DataLakeContents	Возвращает содержимое указанного файла из файловой системы Azure Data Lake Storage.
AzureStorage.Tables	Возвращает навигационную таблицу, содержащую одну строку для каждой таблицы, найденной по URL-адресу учетной записи в хранилище Azure. Каждая строка содержит ссылку на таблицу Azure.
Cdm.Contents	Эта функция недоступна, так как для нее требуется .NET 4.5.
Csv.Document	Возвращает содержимое CSV-документа в виде таблицы, используя указанную кодировку.
CsvStyle.QuoteAfterDelimiter	Кавычки в поле учитываются только сразу после разделителя.
CsvStyle.QuoteAlways	Кавычки в поле всегда учитываются независимо от того, где они находятся.
Cube.AddAndExpandDimensionColumn	Объединяет указанную таблицу измерения dimensionSelector в контексте фильтра куба cube и изменяет степень детализации измерения, расширяя указанный набор атрибутов измерения attributeNames.
Cube.AddMeasureColumn	Добавляет столбец с именем column в куб, который содержит результаты меры measureSelector, примененной в контексте каждой строки.
Cube.ApplyParameter	Возвращает куб после применения параметра с аргументами к кубу.
Cube.AttributeMemberId	Возвращает уникальный идентификатор члена из значения свойства члена.
Cube.AttributeMemberProperty	Возвращает свойство <code>propertyName</code> атрибута измерения <code>attribute</code> .
Cube.CollapseAndRemoveColumns	Изменяет степень детализации измерения контекста фильтра куба, сворачивая атрибуты, сопоставленные с указанными столбцами columnNames.
Cube.Dimensions	Возвращает таблицу, содержащую набор доступных измерений в кубе.
Cube.DisplayFolders	Возвращает вложенное дерево таблиц, представляющих отображаемую иерархию папок объектов (т. е. измерений и мер), доступных для использования в кубе.
Cube.MeasureProperties	Возвращает таблицу с набором доступных свойств мер, которые расширяются в кубе.
Cube.MeasureProperty	Возвращает свойство меры.

ФУНКЦИЯ	ОПИСАНИЕ
Cube.Measures	Возвращает таблицу, содержащую набор доступных мер, в кубе.
Cube.Parameters	Возвращает таблицу, содержащую набор параметров, которые можно применить к кубу.
Cube.Properties	Возвращает таблицу с набором доступных свойств измерений, которые расширяются в кубе.
Cube.PropertyKey	Возвращает ключ свойства <code>property</code> .
Cube.ReplaceDimensions	
Cube.Transform	Применяет список функций куба transforms к кубу.
DB2.Database	Возвращает таблицу таблиц и представлений SQL, доступных в базе данных DB2.
Essbase.Cubes	Возвращает кубы в экземпляре Essbase, сгруппированные сервером Essbase.
Excel.CurrentWorkbook	Возвращает таблицы в текущей книге Excel.
Excel.Workbook	Возвращает таблицу, представляющую листы в заданной книге Excel.
Exchange.Contents	Возвращает содержание из учетной записи Microsoft Exchange.
File.Contents	Возвращает двоичное содержимое файла, расположенного по пути.
Folder.Contents	Возвращает таблицу, содержащую свойства и содержимое файлов и папок, найденных по пути.
Folder.Files	Возвращает таблицу, содержащую строку для каждого файла, найденного по пути, и вложенные папки. Каждая строка содержит свойства папки или файла и ссылку на их содержимое.
GoogleAnalytics.Accounts	Возвращает учетные записи Google Analytics для текущих учетных данных.
Hdfs.Contents	Возвращает таблицу, содержащую строку для каждой папки и файла, найденных по URL-адресу папки {0}, из файловой системы Hadoop. Каждая строка содержит свойства папки или файла и ссылку на их содержимое.
Hdfs.Files	Возвращает таблицу, содержащую строку для каждого файла, найденного по URL-адресу папки {0}, и вложенные папки из файловой системы Hadoop. Каждая строка содержит свойства файла и ссылку на его содержимое.

ФУНКЦИЯ	ОПИСАНИЕ
HdInsight.Containers	Возвращает навигационную таблицу, содержащую все контейнеры, найденные в учетной записи HDInsight. Каждая строка содержит имя контейнера и таблицу файлов.
HdInsight.Contents	Возвращает навигационную таблицу, содержащую все контейнеры, найденные в учетной записи HDInsight. Каждая строка содержит имя контейнера и таблицу файлов.
HdInsight.Files	Возвращает таблицу, содержащую строку для каждой папки и файла, найденных по URL-адресу контейнера, и вложенные папки из учетной записи HDInsight. Каждая строка содержит свойства файла/папки и ссылку на их содержимое.
Html.Table	Возвращает таблицу, содержащую результаты выполнения указанных селекторов CSS для предоставленного HTML-кода.
Identity.From	Создает удостоверение.
Identity.IsMemberOf	Определяет, входит ли удостоверение в коллекцию удостоверений.
IdentityProvider.Default	Поставщик удостоверений по умолчанию для текущего узла.
Informix.Database	Возвращает список таблиц и представлений SQL, доступных в базе данных Informix на сервере <code>server</code> в экземпляре базы данных <code>database</code> .
Json.Document	Возвращает содержимое документа JSON. Оно может быть передано непосредственно в функцию в виде текста либо может быть двоичным значением, возвращаемым такой функцией, как <code>File.Contents</code> .
Json.FromValue	Создает представление JSON указанного значения <code>value</code> с кодировкой текста, заданной в <code>encoding</code> .
MySQL.Database	Возвращает таблицу с данными, относящимися к таблицам в указанной базе данных MySQL.
OData.Feed	Возвращает таблицу веб-каналов OData, предлагаемых <code>serviceUri</code> OData.
ODataOmitValues.Nulls	позволяет службе OData пропускать значения NULL.
Odbc.DataSource	Возвращает таблицу таблиц и представлений SQL из источника данных ODBC, заданного в строке подключения <code>connectionString</code> .
Odbc.InferOptions	Возвращает результат попытки вывести возможности SQL для драйвера ODBC.

ФУНКЦИЯ	ОПИСАНИЕ
Odbc.Query	Подключается к универсальному поставщику с указанной строкой подключения и возвращает результат вычисления запроса.
OleDb.DataSource	Возвращает таблицу таблиц и представлений SQL из источника данных OLE DB, заданного в строке подключения.
OleDb.Query	Возвращает результат запуска собственного запроса в источнике данных OLE DB.
Oracle.Database	Возвращает таблицу с данными, относящимися к таблицам в указанной базе данных Oracle Database.
Parquet.Document	Возвращает содержимое документа Parquet в виде таблицы.
Pdf.Tables	Возвращает все таблицы, найденные в PDF.
PostgreSQL.Database	Возвращает таблицу с данными, относящимися к таблицам в указанной базе данных PostgreSQL.
RData.FromBinary	Возвращает запись кадров данных из файла RData.
Salesforce.Data	Подключается к API объектов Salesforce и возвращает набор доступных объектов (например, учетных записей).
Salesforce.Reports	Подключается к API отчетов Salesforce и возвращает набор доступных отчетов.
SapBusinessWarehouse.Cubes	Возвращает инфокубы и запросы в системе SAP Business Warehouse, сгруппированные по инфообласти.
SapBusinessWarehouseExecutionMode.DataStream	Параметр "Режим выравнивания DataStream" для выполнения MDX в SAP Business Warehouse.
SapBusinessWarehouseExecutionMode.BasXml	Параметр "bXML flattening mode" (Режим выравнивания bXML) для выполнения многомерного выражения в SAP Business Warehouse.
SapBusinessWarehouseExecutionMode.BasXmlGzip	Параметр "Gzip compressed bXML flattening mode" (Режим выравнивания сжатого bXML Gzip) для выполнения многомерного выражения в SAP Business Warehouse. Рекомендуется при низкой задержке или большом количестве запросов.
SapHana.Database	Возвращает пакеты в базе данных SAP HANA.
SapHanaDistribution.All	Возвращает пакеты в базе данных SAP HANA.
SapHanaDistribution.Connection	Параметр распределения "Connection" для SAP HANA.
SapHanaDistribution.Off	Параметр распределения "Off" для SAP HANA.

ФУНКЦИЯ	ОПИСАНИЕ
SapHanaDistribution.Statement	Параметр распределения "Statement" для SAP HANA.
SapHanaRangeOperator.Equals	Оператор диапазона "Equals" (равно) для входных параметров SAP HANA.
SapHanaRangeOperator.GreaterThan	Оператор диапазона "Greater than" (больше чем) для входных параметров SAP HANA.
SapHanaRangeOperator.GreaterThanOrEquals	Оператор диапазона "Greater than or equals" (больше или равно) для входных параметров SAP HANA.
SapHanaRangeOperator.LessThan	Оператор диапазона "Less than" (меньше чем) для входных параметров SAP HANA.
SapHanaRangeOperator.LessThanOrEquals	Оператор диапазона "Less than or equals" (меньше или равно) для входных параметров SAP HANA.
SapHanaRangeOperator.NotEquals	Оператор диапазона "Not equals" (не равно) для входных параметров SAP HANA.
SharePoint.Contents	Возвращает таблицу, содержащую строку для каждой папки и документа, найденных по URL-адресу сайта SharePoint. Каждая строка содержит свойства папки или файла и ссылку на их содержимое.
SharePoint.Files	Возвращает таблицу, содержащую строку для каждого документа, найденного по URL-адресу сайта SharePoint, и вложенные папки. Каждая строка содержит свойства папки или файла и ссылку на их содержимое.
SharePoint.Tables	Возвращает таблицу с результатом списка SharePoint в виде веб-канала OData.
Soda.Feed	Возвращает результирующую таблицу CSV-файла, доступ к которому можно получить с помощью API SODA 2.0. URL-адрес должен указывать на допустимый источник, совместимый с SODA, который заканчивается расширением CSV.
Sql.Database	Возвращает таблицу, содержащую таблицы SQL, расположенные в базе данных экземпляра SQL Server.
Sql.Databases	Возвращает таблицу со ссылками на базы данных, расположенные на экземпляре SQL Server. Возвращает таблицу навигации.
Sybase.Database	Возвращает таблицу с данными, относящимися к таблицам в указанной базе данных Sybase.
Teradata.Database	Возвращает таблицу с данными, относящимися к таблицам в указанной базе данных Teradata.

ФУНКЦИЯ	ОПИСАНИЕ
WebAction.Request	Создает действие, которое при выполнении возвратит результаты выполнения запроса метода, выполняемого по URL-адресу с использованием HTTP, в виде двоичного значения.
Web.BrowserContents	Возвращает HTML для указанного URL-адреса в веб-браузере.
Web.Contents	Возвращает содержимое, скачанное с URL-адреса в виде двоичного значения.
Web.Page	Возвращает содержимое веб-страницы HTML в виде таблицы.
WebMethod.Delete	Указывает метод DELETE для HTTP.
WebMethod.Get	Указывает метод GET для HTTP.
WebMethod.Head	Указывает метод HEAD для HTTP.
WebMethod.Patch	Указывает метод PATCH для HTTP.
WebMethod.Post	Указывает метод POST для HTTP.
WebMethod.Put	Указывает метод PUT для HTTP.
Xml.Document	Возвращает содержимое XML-документа в виде иерархической таблицы (списка записей).
Xml.Tables	Возвращает содержимое XML-документа в виде вложенной коллекции преобразованных в плоскую структуру таблиц.

AccessControlEntry.ConditionToIdentities

16.03.2021 • 2 minutes to read

Синтаксис

```
AccessControlEntry.ConditionToIdentities(identityProvider as function, condition as function) as list
```

Сведения

Используя указанный `identityProvider`, преобразует `condition` в список идентификаторов, для которых `condition` будет возвращать `true` во всех контекстах авторизации с `identityProvider` в качестве поставщика удостоверений. Если не удастся преобразовать `condition` в список удостоверений, возникает ошибка, например, если `condition` обращается к атрибутам, отличным от удостоверений пользователей или групп, для принятия решения.

Обратите внимание, что список удостоверений представляет удостоверения в том виде, в котором они отображаются в `condition`, и для них не выполняется нормализация (например, расширение группы).

AccessControlKind.Allow

16.03.2021 • 2 minutes to read

Сведения

Доступ разрешен.

AccessControlKind.Deny

16.03.2021 • 2 minutes to read

Сведения

Отказано в доступе.

Access.Database

16.03.2021 • 2 minutes to read

Синтаксис

```
Access.Database(database as binary, optional options as nullable record) as table
```

Сведения

Возвращает структурное представление базы данных Access `database`. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `CreateNavigationProperties`: логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — false.
- `NavigationPropertyNameGenerator`: функция, которая используется для создания имен свойств навигации.

Параметр записи указывается, например, как [параметр1 = значение1, параметр2 = значение2...].

ActiveDirectory.Domains

16.03.2021 • 2 minutes to read

Синтаксис

```
ActiveDirectory.Domains(optional forestRootDomainName as nullable text) as table
```

Сведения

Возвращает список доменов Active Directory в том же лесу, что и указанный домен, или из домена текущего компьютера, если значение не указано.

AdobeAnalytics.Cubes

16.03.2021 • 2 minutes to read

Синтаксис

```
AdobeAnalytics.Cubes(optional options as nullable record) as table
```

Сведения

Возвращает таблицу многомерных пакетов из Adobe Analytics. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `HierarchicalNavigation`: логическое значение (true или false), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — false.
- `MaxRetryCount`: число повторных попыток при опросе результатов запроса. Значение по умолчанию — 120.
- `RetryInterval`: длительность интервала между повторными попытками. Значение по умолчанию — 1 секунда.

AdoDotNet.DataSource

16.03.2021 • 2 minutes to read

Синтаксис

```
AdoDotNet.DataSource(providerName as text, connectionString as any, optional options as nullable record) as table
```

Сведения

Возвращает коллекцию схем для источника данных ADO.NET с именем поставщика `providerName` и строкой подключения `connectionString`. `connectionString` может быть текстом или записью пар "свойство-значение". Значения свойств могут быть текстовыми или числовыми. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля: - `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут. - `SqlCompatibleWindowsAuth`: логическое значение (true или false), которое определяет, следует ли создавать параметры строки подключения, совместимые с SQL Server, для проверки подлинности Windows. Значение по умолчанию — true.

- `TypeMap`

AdoDotNet.Query

16.03.2021 • 2 minutes to read

Синтаксис

```
AdoDotNet.Query(providerName as text, connectionString as any, query as text, optional options  
as nullable record) as table
```

Сведения

Возвращает результат запуска `query` со строкой подключения `connectionString` с помощью поставщика ADO.NET `providerName`. `connectionString` может быть текстом или записью пар "свойство-значение". Значения свойств могут быть текстовыми или числовыми. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `SqlCompatibleWindowsAuth`: логическое значение (true или false), которое определяет, следует ли создавать параметры строки подключения, совместимые с SQL Server, для проверки подлинности Windows. Значение по умолчанию — true.

AnalysisServices.Database

16.03.2021 • 2 minutes to read

Синтаксис

```
AnalysisServices.Database(server as text, database as text, optional options as nullable record)
as table
```

Сведения

Возвращает таблицу многомерных кубов или табличных моделей из базы данных служб Analysis Services `database` на сервере `server`. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `Query`: собственный запрос многомерных выражений для извлечения данных.
- `TypedMeasureColumns`: логическое значение, указывающее, будут ли типы, указанные в многомерной или табличной модели, использоваться для типов добавленных столбцов мер. Если задано значение `false`, для всех столбцов мер будет использоваться тип `"number"`. По умолчанию этот параметр имеет значение `False`.
- `Culture`: имя языка и региональных параметров для данных. Соответствует свойству строки подключения `"Код языка"`.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию зависит от драйвера.
- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `SubQueries`: число (0, 1 или 2), которое задает значение свойства `"SubQueries"` в строке подключения. Управляет поведением вычисляемых элементов во вложенных выбранных элементах или вложенных кубах. (По умолчанию используется значение 2.)
- `Implementation`

AnalysisServices.Databases

16.03.2021 • 2 minutes to read

Синтаксис

```
AnalysisServices.Databases(server as text, optional options as nullable record) as table
```

Сведения

Возвращает базы данных в экземпляре Analysis Services `server`. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `TypedMeasureColumns`: логическое значение, указывающее, будут ли типы, указанные в многомерной или табличной модели, использоваться для типов добавленных столбцов мер. Если задано значение false, для всех столбцов мер будет использоваться тип "number". По умолчанию этот параметр имеет значение False.
- `Culture`: имя языка и региональных параметров для данных. Соответствует свойству строки подключения "Код языка".
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию зависит от драйвера.
- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `SubQueries`: число (0, 1 или 2), которое задает значение свойства "SubQueries" в строке подключения. Управляет поведением вычисляемых элементов во вложенных выбранных элементах или вложенных кубах. (По умолчанию используется значение 2.)
- `Implementation`

AzureStorage.BlobContents

16.03.2021 • 2 minutes to read

Синтаксис

```
AzureStorage.BlobContents(url as text, optional options as nullable record) as binary
```

Сведения

Возвращает содержимое BLOB-объекта по URL-адресу `url` из хранилища Azure. Можно указать `options` для управления следующими параметрами:

- `BlockSize`: число байтов, считываемых перед ожиданием потребителя данных. Значение по умолчанию — 4 МБ.
- `RequestSize`: число байтов, которое пытается считать один HTTP-запрос к серверу. Значение по умолчанию — 4 МБ.
- `ConcurrentRequests`: параметр `ConcurrentRequests` поддерживает ускоренное скачивание данных, позволяя указать число параллельно выполняемых запросов за счет использования памяти. Требуемый объем памяти: `ConcurrentRequest (Число параллельных запросов) * RequestSize (Размер запроса)`. Значение по умолчанию — 16.

AzureStorage.Blobs

16.03.2021 • 2 minutes to read

Синтаксис

```
AzureStorage.Blobs(account as text, optional options as nullable record) as table
```

Сведения

Возвращает навигационную таблицу, содержащую одну строку для каждого контейнера, найденного по URL-адресу учетной записи `account` в хранилище Azure. Каждая строка содержит ссылку на большие двоичные объекты контейнера. Можно указать `options` для управления следующими параметрами:

- `BlockSize`: число байтов, считываемых перед ожиданием потребителя данных. Значение по умолчанию — 4 МБ.
- `RequestSize`: число байтов, которое пытается считать один HTTP-запрос к серверу. Значение по умолчанию — 4 МБ.
- `ConcurrentRequests`: параметр `ConcurrentRequests` поддерживает ускоренное скачивание данных, позволяя указать число параллельно выполняемых запросов за счет использования памяти. Требуемый объем памяти: `ConcurrentRequest (Число параллельных запросов) * RequestSize (Размер запроса)`. Значение по умолчанию — 16.

AzureStorage.DataLake

16.03.2021 • 2 minutes to read

Синтаксис

```
AzureStorage.DataLake(endpoint as text, optional options as nullable record) as table
```

Сведения

Возвращает таблицу навигации, содержащую документы, обнаруженные в указанном контейнере и его подпапках по URL-адресу учетной записи `endpoint` из файловой системы Azure Data Lake Storage. Можно указать `options` для управления следующими параметрами:

- `BlockSize`: число байтов, считываемых перед ожиданием потребителя данных. Значение по умолчанию — 4 МБ.
- `RequestSize`: число байтов, которое пытается считать один HTTP-запрос к серверу. Значение по умолчанию — 4 МБ.
- `ConcurrentRequests`: параметр `ConcurrentRequests` поддерживает ускоренное скачивание данных, позволяя указать число параллельно выполняемых запросов за счет использования памяти. Требуемый объем памяти: `ConcurrentRequest (Число параллельных запросов) * RequestSize (Размер запроса)`. Значение по умолчанию — 16.
- `HierarchicalNavigation`: логическое значение (`true` или `false`), определяющее вид возвращаемых файлов: древовидное представление каталогов или плоский список. Значением по умолчанию является `false`.

AzureStorage.DataLakeContents

16.03.2021 • 2 minutes to read

Синтаксис

```
AzureStorage.DataLakeContents(url as text, optional options as nullable record) as binary
```

Сведения

Возвращает содержимое файла по URL-адресу `url` из файловой системы Azure Data Lake Storage. Можно указать `options` для управления следующими параметрами:

- `BlockSize`: число байтов, считываемых перед ожиданием потребителя данных. Значение по умолчанию — 4 МБ.
- `RequestSize`: число байтов, которое пытается считать один HTTP-запрос к серверу. Значение по умолчанию — 4 МБ.
- `ConcurrentRequests`: параметр `ConcurrentRequests` поддерживает ускоренное скачивание данных, позволяя указать число параллельно выполняемых запросов за счет использования памяти. Требуемый объем памяти: `ConcurrentRequest (Число параллельных запросов) * RequestSize (Размер запроса)`. Значение по умолчанию — 16.

AzureStorage.Tables

16.03.2021 • 2 minutes to read

Синтаксис

```
AzureStorage.Tables(account as text) as table
```

Сведения

Возвращает навигационную таблицу, содержащую одну строку для каждой таблицы, найденной по URL-адресу `account` учетной записи в хранилище Azure. Каждая строка содержит ссылку на таблицу Azure.

Cdm.Contents

16.03.2021 • 2 minutes to read

Синтаксис

```
Cdm.Contents(table as table) as table
```

Сведения

Эта функция недоступна, так как для нее требуется .NET 4.5.

Csv.Document

16.03.2021 • 2 minutes to read

Синтаксис

```
Csv.Document(source as any, optional columns as any, optional delimiter as any, optional extraValues as nullable number, optional encoding as nullable number) as table
```

Сведения

Возвращает содержимое документа CSV в виде таблицы.

- `columns` может иметь следующие значения: NULL, число столбцов, список имен столбцов, тип таблицы или запись параметров. (Дополнительные сведения о записи параметров см. ниже.)
- `delimiter` может быть одним символом или списком символов. Значение по умолчанию: `" , "`.
- Список поддерживаемых значений `extraValues` см. в `ExtraValues.Type`.
- `encoding` определяет тип кодировки текста.

Если указана запись для `columns` (а `delimiter`, `extraValues` и `encoding` имеют значение NULL), можно указать следующие поля записи:

- `Delimiter`: разделитель столбцов. Значение по умолчанию: `" , "`.
- `Columns`: может иметь следующие значения: NULL, число столбцов, список имен столбцов или тип таблицы. Если число столбцов меньше их числа во входных данных, дополнительные столбцы будут игнорироваться. Если число столбцов превышает их число во входных данных, значения дополнительных столбцов будут равны NULL. Если значение не указано, число столбцов будет определяться входными данными.
- `Encoding`: кодировка текста файла. Значение по умолчанию — 65001 (UTF-8).
- `CsvStyle`: определяет обработку кавычек. `CsvStyle.QuoteAfterDelimiter` (по умолчанию): кавычки в поле учитываются только сразу после разделителя. `CsvStyle.QuoteAlways`: кавычки в поле всегда учитываются независимо от того, где они находятся.
- `QuoteStyle`: определяет обработку разрывов строк в кавычках. `QuoteStyle.None` (по умолчанию): все разрывы строк рассматриваются как конец текущей строки, даже если они находятся внутри значения в кавычках. `QuoteStyle.Csv`: разрывы строк в кавычках рассматриваются как часть данных, а не как конец текущей строки.

Пример 1

Обработка текста в формате CSV с заголовками столбцов.

```
let
    csv = Text.Combine({"OrderID,Item", "1,Fishing rod", "2,1 lb. worms"}, "#(cr)#(lf)")
in
    Table.PromoteHeaders(Csv.Document(csv))
```

ORDERID	ЭЛЕМЕНТ
1	Удочка
2	Черви, 1 фунт

CsvStyle QuoteAfterDelimiter

16.03.2021 • 2 minutes to read

Синтаксис

```
CsvStyle QuoteAfterDelimiter
```

Сведения

Кавычки в поле учитываются только сразу после разделителя.

CsvStyle QuoteAlways

16.03.2021 • 2 minutes to read

Синтаксис

```
CsvStyle.QuoteAlways
```

Сведения

Кавычки в поле всегда учитываются независимо от того, где они находятся.

Cube.AddAndExpandDimensionColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.AddAndExpandDimensionColumn(**cube** as table, **dimensionSelector** as any,  
**attributeNames** as list, optional **newColumnNames** as any) as table
```

Сведения

Объединяет указанную таблицу измерения `dimensionSelector` в контексте фильтра куба `cube` и изменяет степень детализации измерения, расширяя указанный набор атрибутов измерения `attributeNames`. Атрибуты измерения добавляются в табличное представление со столбцами `newColumnNames` или `attributeNames`, если это значение не задано.

Cube.AddMeasureColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.AddMeasureColumn(**cube** as table, **column** as text, **measureSelector** as any) as table
```

Сведения

Добавляет столбец с именем `column` в куб `cube`, который содержит результаты меры `measureSelector`, примененной в контексте каждой строки. На применение меры оказывают влияние изменения в степени детализации измерения и параметры срезов. Значения меры будут скорректированы после выполнения некоторых операций с кубом.

Cube.ApplyParameter

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.ApplyParameter(cube as table, parameter as any, optional arguments as nullable list) as table
```

Сведения

Возвращает куб после применения "parameter" с "arguments" к "cube".

Cube.AttributeMemberId

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.AttributeMemberId(attribute as any) as any
```

Сведения

Возвращает уникальный идентификатор члена из значения свойства члена. `attribute`. Возвращает значение NULL для любых других значений.

Cube.AttributeMemberProperty

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.AttributeMemberProperty(attribute as any, propertyName as text) as any
```

Сведения

Возвращает свойство `propertyName` атрибута измерения `attribute`.

Cube.CollapseAndRemoveColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.CollapseAndRemoveColumns(**cube** as table, **columnNames** as list) as table
```

Сведения

Изменяет степень детализации измерения контекста фильтра для `cube`, сворачивая атрибуты, сопоставленные с указанными столбцами `columnNames`. Столбцы также удаляются из табличного представления куба.

Cube.Dimensions

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.Dimensions(**cube** as table) as table
```

Сведения

Возвращает таблицу, содержащую набор доступных измерений в `cube`. Каждое измерение — это таблица с набором атрибутов измерения, а каждый атрибут измерения представлен в этой таблице измерения как столбец. Измерения в кубе можно развернуть с помощью `Cube.AddAndExpandDimensionColumn`.

Cube.DisplayFolders

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.DisplayFolders(**cube** as table) as table
```

Сведения

Возвращает вложенное дерево таблиц, представляющих отображаемую иерархию папок объектов (т. е. измерений и мер), доступных для использования в `cube`.

Cube.MeasureProperties

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.MeasureProperties(cube as table) as table
```

Сведения

Возвращает таблицу с набором доступных свойств мер, которые расширяются в кубе.

Cube.MeasureProperty

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.MeasureProperty(measure as any, propertyName as text) as any
```

Сведения

Возвращает свойство `propertyName` меры `measure`.

Cube.Measures

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.Measures(**cube** as any) as table
```

Сведения

Возвращает таблицу, содержащую набор доступных мер, в `cube`. Каждая мера представлена в виде функции. Меры можно применить к кубу с помощью `Cube.AddMeasureColumn`.

Cube.Parameters

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.Parameters(cube as table) as table
```

Сведения

Возвращает таблицу, содержащую набор параметров, которые можно применить к `cube`. Каждый параметр представляет собой функцию, которую можно вызвать, чтобы получить `cube` с примененными параметром и аргументами.

Cube.Properties

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.Properties(cube as table) as table
```

Сведения

Возвращает таблицу с набором доступных свойств измерений, которые расширяются в кубе.

Cube.PropertyKey

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.PropertyKey(property as any) as any
```

Сведения

Возвращает ключ свойства `property`.

Cube.ReplaceDimensions

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.ReplaceDimensions(cube as table, dimensions as table) as table
```

Сведения

Cube.ReplaceDimensions

Cube.Transform

16.03.2021 • 2 minutes to read

Синтаксис

```
Cube.Transform(cube as table, transforms as list) as table
```

Сведения

Применяет список функций куба `transforms` к `cube`.

DB2.Database

16.03.2021 • 2 minutes to read

Синтаксис

```
DB2.Database(server as text, database as text, optional options as nullable record) as table
```

Сведения

Возвращает список таблиц и представлений SQL, доступных в базе данных DB2 на сервере `server` в экземпляре базы данных `database`. Порт может дополнительно указываться сервером после двоеточия. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `CreateNavigationProperties`: логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — true.
- `NavigationPropertyNameGenerator`: функция, которая используется для создания имен свойств навигации.
- `Query`: собственный запрос SQL для извлечения данных. Если такой запрос генерирует несколько результирующих наборов, возвращается только первый из них.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `HierarchicalNavigation`: логическое значение (true или false), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — false.
- `Implementation`: указывает используемую реализацию поставщика внутренней базы данных. Допустимые значения: "IBM" и "Microsoft".
- `BinaryCodePage`: число для CCSID (идентификатор набора кодированных знаков) для декодирования двоичных данных Db2 FOR BIT в символьные строки. Применяется к Implementation = "Microsoft". Задайте значение 0, чтобы отключить преобразование (по умолчанию). Задайте значение 1 для преобразования на основе кодировки базы данных. Задайте другое число CCSID для преобразования в кодировку приложения.
- `PackageCollection`: указывает строковое значение для коллекции пакетов (по умолчанию "NULLID"), чтобы использовать общие пакеты, необходимые для обработки инструкций SQL. Применяется к Implementation = "Microsoft".
- `UseDb2ConnectGateway`: указывает, выполняется ли подключение через шлюз DB2 Connect. Применяется к Implementation = "Microsoft".

Параметр записи указывается, например, как [option1 = value1, option2 = value2...] или [Query = "select ..."].

Essbase.Cubes

16.03.2021 • 2 minutes to read

Синтаксис

```
Essbase.Cubes(url as text, optional options as nullable record) as table
```

Сведения

Возвращает таблицу кубов, сгруппированных сервером Essbase, из экземпляра Essbase на сервере APS

`url`. Для управления следующими параметрами можно указать необязательный параметр записи

`options` :

- `CommandTimeout` : длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.

Excel.CurrentWorkbook

16.03.2021 • 2 minutes to read

Синтаксис

```
Excel.CurrentWorkbook() as table
```

Сведения

Возвращает содержимое текущей книги Excel.

Синтаксис

```
Excel.Workbook(workbook as binary, optional useHeaders as any, optional delayTypes as nullable logical) as table
```

Сведения

Возвращает содержимое книги Excel.

- `useHeaders` может принимать значение null, логическое значение (true или false), указывающее, должна ли первая строка каждой возвращаемой таблицы обрабатываться как заголовок, или может представлять собой запись параметров. (Дополнительные сведения о записи параметров см. ниже.) Значение по умолчанию — false.
- `delayTypes` может принимать значение null или логическое значение (true или false), указывающее, следует ли оставлять столбцы каждой из возвращаемых таблиц нетипизированными. По умолчанию: false.

Если указана запись для `useHeaders` (и `delayTypes` имеет значение null), можно указать следующие поля записи:

- `UseHeaders` — может принимать значение null или логическое значение (true или false), указывающее, должна ли первая строка каждой возвращаемой таблицы обрабатываться как заголовок. По умолчанию: false.
- `DelayTypes` — может принимать значение null или логическое значение (true или false), указывающее, следует ли оставлять столбцы каждой из возвращаемых таблиц нетипизированными. По умолчанию: false.
- `InferSheetDimensions` — может принимать значение null или логическое значение (true или false), указывающее, должна ли область листа, содержащая данные, формироваться путем чтения самого листа, а не путем считывания метаданных измерений из файла. Это может быть полезно в случаях, когда метаданные измерений неверны. Обратите внимание, что этот параметр поддерживается только для файлов Excel Open XML, а не для устаревших файлов Excel. По умолчанию: false.

Exchange.Contents

16.03.2021 • 2 minutes to read

Синтаксис

```
Exchange.Contents (optional mailboxAddress as nullable text) as table
```

Сведения

Возвращает содержание из учетной записи `mailboxAddress` Microsoft Exchange. Если "`mailboxAddress`" не указана, будет использоваться учетная запись по умолчанию для учетных данных.

File.Contents

16.03.2021 • 2 minutes to read

Синтаксис

```
File.Contents(path as text, optional options as nullable record) as binary
```

Сведения

Возвращает содержимое файла `path` в двоичном виде.

Folder.Contents

16.03.2021 • 2 minutes to read

Синтаксис

```
Folder.Contents(path as text, optional options as nullable record) as table
```

Сведения

Возвращает таблицу, содержащую строку для каждой папки и файла, найденного по пути к папке `path`. Каждая строка содержит свойства папки или файла и ссылку на их содержимое.

Folder.Files

16.03.2021 • 2 minutes to read

Синтаксис

```
Folder.Files(path as text, optional options as nullable record) as table
```

Сведения

Возвращает таблицу, содержащую строку для каждого файла, найденного по пути `path`, и вложенные папки. Каждая строка содержит свойства файла и ссылку на его содержимое.

GoogleAnalytics.Accounts

16.03.2021 • 2 minutes to read

Синтаксис

```
GoogleAnalytics.Accounts() as table
```

Сведения

Возвращает учетные данные Google Analytics, доступные с текущими учетными данными.

Hdfs.Contents

16.03.2021 • 2 minutes to read

Синтаксис

```
Hdfs.Contents(url as text) as table
```

Сведения

Возвращает таблицу, содержащую строку для каждой папки и файла, найденных по URL-адресу папки `url`, из файловой системы Hadoop. Каждая строка содержит свойства папки или файла и ссылку на их содержимое.

Hdfs.Files

16.03.2021 • 2 minutes to read

Синтаксис

```
Hdfs.Files(url as text) as table
```

Сведения

Возвращает таблицу, содержащую строку для каждого файла, найденного по URL-адресу папки `url`, и вложенные папки из файловой системы Hadoop. Каждая строка содержит свойства файла и ссылку на его содержимое.

HdInsight.Containers

16.03.2021 • 2 minutes to read

Синтаксис

```
HdInsight.Containers(account as text) as table
```

Сведения

Возвращает навигационную таблицу, содержащую одну строку для каждого контейнера, найденного по URL-адресу учетной записи `account` в хранилище Azure. Каждая строка содержит ссылку на большие двоичные объекты контейнера.

HdInsight.Contents

16.03.2021 • 2 minutes to read

Синтаксис

```
HdInsight.Contents(account as text) as table
```

Сведения

Возвращает навигационную таблицу, содержащую одну строку для каждого контейнера, найденного по URL-адресу учетной записи `account` в хранилище Azure. Каждая строка содержит ссылку на большие двоичные объекты контейнера.

HdInsight.Files

16.03.2021 • 2 minutes to read

Синтаксис

```
HdInsight.Files(account as text, containerName as text) as table
```

Сведения

Возвращает таблицу, содержащую одну строку для каждого файла большого двоичного объекта, найденного по URL-адресу контейнера `account` в хранилище Azure. Каждая строка содержит свойства файла и ссылку на его содержимое.

Html.Table

16.03.2021 • 2 minutes to read

Синтаксис

```
Html.Table(html as any, columnNameSelectorPairs as list, optional options as nullable record) as table
```

Сведения

Возвращает таблицу, содержащую результаты выполнения указанных селекторов CSS для предоставленного `html`. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `RowSelector`

Пример 1

Возвращает таблицу из примера текстового значения HTML.

```
Html.Table("<div class='name'>Jo</div><span>Manager</span>", [{"Name", ".name"}, {"Title", "span"}], [RowSelector=".name"]\
```

```
#table({"Name", "Title"}, {"Jo", "Manager"})
```

Пример 2

Извлекает все ссылки href из примера текстового значения HTML.

```
Html.Table("<a href='/test.html'>Test</a>", [{"Link", "a", each [Attributes][href]}])
```

```
#table({"Link"}, {"/test.html"})
```

Identity.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Identity.From(identityProvider as function, value as any) as record
```

Сведения

Создает удостоверение.

Identity.IsMemberOf

16.03.2021 • 2 minutes to read

Синтаксис

```
Identity.IsMemberOf(identity as record, collection as record) as logical
```

Сведения

Определяет, входит ли удостоверение в коллекцию удостоверений.

IdentityProvider.Default

16.03.2021 • 2 minutes to read

Синтаксис

```
IdentityProvider.Default() as any
```

Сведения

Поставщик удостоверений по умолчанию для текущего узла.

Informix.Database

16.03.2021 • 2 minutes to read

Синтаксис

```
Informix.Database(server as text, database as text, optional options as nullable record) as table
```

Сведения

Возвращает список таблиц и представлений SQL, доступных в базе данных Informix на сервере `server` в экземпляре базы данных `database`. Порт может дополнительно указываться сервером после двоеточия. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `CreateNavigationProperties`: логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — true.
- `NavigationPropertyNameGenerator`: функция, которая используется для создания имен свойств навигации.
- `Query`: собственный запрос SQL для извлечения данных. Если такой запрос генерирует несколько результирующих наборов, возвращается только первый из них.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `HierarchicalNavigation`: логическое значение (true или false), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — false.

Параметр записи указывается, например, как `[option1 = value1, option2 = value2...]` или `[Query = "select ..."]`.

Json.Document

16.03.2021 • 2 minutes to read

Синтаксис

```
Json.Document(jsonText as any, optional encoding as nullable number) as any
```

Сведения

Возвращает содержимое документа JSON.

Json.FromValue

16.03.2021 • 2 minutes to read

Синтаксис

```
Json.FromValue(value as any, optional encoding as nullable number) as binary
```

Сведения

Создает представление JSON указанного значения `value` с кодировкой текста, определенной `encoding`. Если `encoding` опускается, используется UTF8. Значения представляются следующим образом:

- значения NULL, текстовые и логические значения — в виде соответствующих типов JSON;
- числа — как числа в формате JSON, за исключением того, что `#infinity`, `-#infinity` и `#nan` преобразуются в значения NULL;
- списки — в виде массивов JSON;
- записи — в виде объектов JSON;
- таблицы — в виде массивов объектов;
- значения даты, времени, зон и длительности — в виде текста ISO-8601;
- двоичные значения — в виде текста в кодировке Base-64.
- Типы и функции вызывают ошибку.

Пример 1

Преобразование сложного значения в JSON.

```
Text.FromBinary(Json.FromValue([A = {1, true, "3"}, B = #date(2012, 3, 25)]))
```

```
"{"A": [1, true, "3"], "B": "2012-03-25"}"
```

MySQL.Database

16.03.2021 • 2 minutes to read

Синтаксис

```
MySQL.Database(server as text, database as text, optional options as nullable record) as table
```

Сведения

Возвращает таблицу таблиц, представлений и хранимых скалярных функций SQL, доступных в базе данных MySQL на сервере `server` в экземпляре базы данных `database`. Порт может дополнительно указываться сервером после двоеточия. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `Encoding`: значение `TextEncoding`, задающее кодировку для всех запросов, отправляемых на сервер (значение по умолчанию — `NULL`).
- `CreateNavigationProperties`: логическое значение (`true` или `false`), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — `true`.
- `NavigationPropertyNameGenerator`: функция, которая используется для создания имен свойств навигации.
- `Query`: собственный запрос SQL для извлечения данных. Если такой запрос генерирует несколько результирующих наборов, возвращается только первый из них.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `TreatTinyAsBoolean`: логическое значение (`true` или `false`), которое указывает, задавать ли для столбцов `tinyint` на сервере логические значения. Значение по умолчанию — `true`.
- `OldGuids`: логическое значение (`true` или `false`), которое указывает, будут ли столбцы `char(36)` (если `false`) или столбцы `binary(16)` (если `true`), учитываться как GUID. Значением по умолчанию является `false`.
- `ReturnSingleDatabase`: логическое значение (`true` или `false`), которое указывает, следует ли возвращать все таблицы в базах данных (`false`) или возвращать таблицы и представления указанной базы данных (`true`). Значением по умолчанию является `false`.
- `HierarchicalNavigation`: логическое значение (`true` или `false`), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — `false`.

Параметр записи указывается, например, как `[option1 = value1, option2 = value2...]` или `[Query = "select ..."]`.

OData.Feed

16.03.2021 • 4 minutes to read

Синтаксис

```
OData.Feed(serviceUri as text, optional headers as nullable record, optional options as any) as any
```

Сведения

Возвращает таблицу веб-каналов OData, предлагаемых службой OData, с использованием URI `serviceUri` и заголовков `headers`. Можно указать логическое значение, указывающее, следует ли использовать одновременные подключения, или необязательный параметр записи `options` для управления следующими параметрами:

- `Query` — позволяет программно добавить параметры запроса в URL-адрес, не беспокоясь об экранировании.
- `Headers` : указание этого значения в виде записи позволяет предоставить дополнительные заголовки для HTTP-запроса.
- `ExcludedFromCacheKey` — если указать это значение в виде списка, эти ключи заголовков HTTP будут исключены из вычисления для кэширования данных.
- `ApiKeyName` : если целевой сайт имеет понятие ключа API, этот параметр можно применить для указания имени (не значения) параметра ключа, который нужно использовать в URL-адресе. Фактическое значение ключа указывается в учетных данных.
- `Timeout` : указание этого значения в качестве длительности приведет к изменению времени ожидания для HTTP-запроса. Значение по умолчанию — 600 секунд.
- `EnableBatch` : логическое значение (true или false), определяющее, разрешено ли создание запроса \$batch OData при превышении MaxUriLength (по умолчанию — false).
- `MaxUriLength` : число, указывающее максимальную длину допустимого URI, отправляемого в службу OData. Если это значение превышено, а параметр EnableBatch имеет значение true, то запрос будет выполнен к конечной точке \$batch OData, в противном случае он завершится сбоем (значение по умолчанию — 2048).
- `Concurrent` : логическое значение (true или false). Если задано значение true, запросы к службе будут выполняться параллельно. Если задано значение false, запросы будут выполняться последовательно. Если значение не указано, оно определяется аннотацией AsynchronousRequestsSupported службы. Если поддержка AsynchronousRequestsSupported в службе не определена, запросы будут выполняться последовательно.
- `ODataVersion` : Число (3 или 4), указывающее версию протокола OData для этой службы OData. Если ничего не указано, запрашиваются все поддерживаемые версии. Версия службы определяется заголовком версии OData, возвращаемым службой.
- `FunctionOverloads` : Логическое значение (True или False); если задано значение True, перегрузки импорта функции будут перечислены в отдельных записях в навигаторе; если задано значение False, перегрузки импорта функции будут перечислены в виде одной функции соединения в навигаторе. Значение по умолчанию для версии 3 — False, для версии 4 — True.
- `MoreColumns` : логическое значение (true или false). Если задано значение true, добавляется столбец "More Columns" для каждого веб-канала сущности, содержащего открытые и полиморфные типы. Столбец будет содержать поля, не объявленные в базовом типе. Если задано значение False, это поле

будет отсутствовать. Значение по умолчанию — false.

- `IncludeAnnotations` : список названий терминов или шаблонов с указанием пространств имен, разделенный запятыми, включаемый при указании подстановочного знака "*". По умолчанию ни одна из аннотаций не включена.
- `IncludeMetadataAnnotations` : список названий терминов или шаблонов с указанием пространств имен, разделенный запятыми, включаемый в запросы документов метаданных при указании подстановочного знака "*". По умолчанию включены те же аннотации, что и в `IncludeAnnotations`.
- `OmitValues` : позволяет службе OData не указывать определенные значения в ответах. При подтверждении мы будем выводить эти значения из пропущенных полей. Возможны следующие значения.
- `ODataOmitValues.Nulls` : позволяет службе OData пропускать значения NULL.
- `Implementation` : Указывает реализацию соединителя OData, которую следует использовать. Допустимые значения — "2.0" или NULL.

ODataOmitValues.Nulls

16.03.2021 • 2 minutes to read

Сведения

позволяет службе OData пропускать значения NULL.

Odbc.DataSource

16.03.2021 • 2 minutes to read

Синтаксис

```
Odbc.DataSource(connectionString as any, optional options as nullable record) as table
```

Сведения

Возвращает таблицу таблиц и представлений SQL из источника данных ODBC, заданного в строке подключения `connectionString`. `connectionString` может быть текстом или записью пар "свойство-значение". Значения свойств могут быть текстовыми или числовыми. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `CreateNavigationProperties`: логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — true.
- `HierarchicalNavigation`: логическое значение (true или false), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — false.
- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию - 15 секунды.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `SqlCompatibleWindowsAuth`: логическое значение (true или false), которое определяет, следует ли создавать параметры строки подключения, совместимые с SQL Server, для проверки подлинности Windows. Значение по умолчанию — true.

Odbc.InferOptions

16.03.2021 • 2 minutes to read

Синтаксис

```
Odbc.InferOptions(connectionString as any) as record
```

Сведения

Возвращает результат попытки вывести возможности SQL со строкой подключения `connectionString` с помощью ODBC. `connectionString` может быть текстом или записью пар "свойство-значение". Значения свойств могут быть текстовыми или числовыми.

Odbc.Query

16.03.2021 • 2 minutes to read

Синтаксис

```
Odbc.Query(connectionString as any, query as text, optional options as nullable record) as table
```

Сведения

Возвращает результат запуска `query` со строкой подключения `connectionString` с помощью ODBC.

`connectionString` может быть текстом или записью пар "свойство-значение". Значения свойств могут быть текстовыми или числовыми. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию - 15 секунд.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `SqlCompatibleWindowsAuth`: логическое значение (true или false), которое определяет, следует ли создавать параметры строки подключения, совместимые с SQL Server, для проверки подлинности Windows. Значение по умолчанию — true.

OleDb.DataSource

16.03.2021 • 2 minutes to read

Синтаксис

```
OleDb.DataSource(connectionString as any, optional options as nullable record) as table
```

Сведения

Возвращает таблицу таблиц и представлений SQL из источника данных OLE DB, заданного в строке подключения `connectionString`. `connectionString` может быть текстом или записью пар "свойство-значение". Значения свойств могут быть текстовыми или числовыми. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `CreateNavigationProperties`: логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — true.
- `NavigationPropertyNameGenerator`: функция, которая используется для создания имен свойств навигации.
- `Query`: собственный запрос SQL для извлечения данных. Если такой запрос генерирует несколько результирующих наборов, возвращается только первый из них.
- `HierarchicalNavigation`: логическое значение (true или false), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — true.
- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `SqlCompatibleWindowsAuth`: логическое значение (true или false), которое определяет, следует ли создавать параметры строки подключения, совместимые с SQL Server, для проверки подлинности Windows. Значение по умолчанию — true.

Параметр записи указывается, например, как [option1 = value1, option2 = value2...] или [Query = "select ..."].

OleDb.Query

16.03.2021 • 2 minutes to read

Синтаксис

```
OleDb.Query(connectionString as any, query as text, optional options as nullable record) as table
```

Сведения

Возвращает результат запуска `query` со строкой подключения `connectionString` с помощью OLE DB.

`connectionString` может быть текстом или записью пар "свойство-значение". Значения свойств могут быть текстовыми или числовыми. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `SqlCompatibleWindowsAuth`: логическое значение (true или false), которое определяет, следует ли создавать параметры строки подключения, совместимые с SQL Server, для проверки подлинности Windows. Значение по умолчанию — true.

Oracle.Database

16.03.2021 • 2 minutes to read

Синтаксис

```
Oracle.Database(server as text, optional options as nullable record) as table
```

Сведения

Возвращает таблицу таблиц и представлений SQL из базы данных Oracle на сервере `server`. Порт может дополнительно указываться сервером после двоеточия. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `CreateNavigationProperties`: логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — true.
- `NavigationPropertyNameGenerator`: функция, которая используется для создания имен свойств навигации.
- `query`: собственный запрос SQL для извлечения данных. Если такой запрос генерирует несколько результирующих наборов, возвращается только первый из них.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `HierarchicalNavigation`: логическое значение (true или false), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — false.

Параметр записи указывается, например, как `[option1 = value1, option2 = value2...]` или `[Query = "select ..."]`.

Parquet.Document

16.03.2021 • 2 minutes to read

Синтаксис

```
Parquet.Document(binary as binary, optional options as nullable record) as any
```

Сведения

Возвращает содержимое документа Parquet в виде таблицы.

Pdf.Tables

16.03.2021 • 2 minutes to read

Синтаксис

```
Pdf.Tables(pdf as binary, optional options as nullable record) as table
```

Сведения

Возвращает все таблицы, найденные в `pdf`. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `Implementation` — Версия алгоритма, используемая при идентификации таблиц. Старые версии доступны только для обеспечения обратной совместимости, чтобы не нарушать работу старых запросов при обновлении алгоритмов. Новейшая версия практически всегда дает оптимальный результат. Допустимые значения: "1.2", "1.1" или NULL.
- `StartPage` — задает первую страницу диапазона страниц для проверки. Значение по умолчанию: 1.
- `EndPage` — задает последнюю страницу диапазона страниц для проверки. Значение по умолчанию — последняя страница документа.
- `MultiPageTables` — определяет, будут ли похожие таблицы на идущих одна за другой страницах автоматически объединяться в одну таблицу. Значение по умолчанию — true.
- `EnforceBorderLines` — определяет, будут ли для определения границ ячеек использоваться только линии границ (при значении true) или они будут использоваться просто как одно из множества указаний (при значении false). По умолчанию: false.

Пример 1

Получение таблиц, содержащихся в файле sample.pdf.

```
Pdf.Tables(File.Contents("c:\sample.pdf"))
```

```
#table({"Name", "Kind", "Data"}, ...)
```


PostgreSQL.Database

16.03.2021 • 2 minutes to read

Синтаксис

```
PostgreSQL.Database(server as text, database as text, optional options as nullable record) as table
```

Сведения

Возвращает список таблиц и представлений SQL, доступных в базе данных PostgreSQL на сервере `server` в экземпляре базы данных `database`. Порт может дополнительно указываться сервером после двоеточия. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `CreateNavigationProperties`: логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — true.
- `NavigationPropertyNameGenerator`: функция, которая используется для создания имен свойств навигации.
- `Query`: собственный запрос SQL для извлечения данных. Если такой запрос генерирует несколько результирующих наборов, возвращается только первый из них.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `HierarchicalNavigation`: логическое значение (true или false), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — false.

Параметр записи указывается, например, как `[option1 = value1, option2 = value2...]` или `[Query = "select ..."]`.

RData.FromBinary

16.03.2021 • 2 minutes to read

Синтаксис

```
RData.FromBinary(stream as binary) as any
```

Сведения

Возвращает запись кадров данных из файла RData.

Salesforce.Data

16.03.2021 • 2 minutes to read

Синтаксис

```
Salesforce.Data(optional loginUrl as any, optional options as nullable record) as table
```

Сведения

Возвращает объекты из указанной в учетных данных учетной записи Salesforce. Учетная запись будет подключена с помощью заданного окружения `loginUrl`. Если оно не указано, учетная запись подключится к рабочей среде (<https://login.salesforce.com>). Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `CreateNavigationProperties`: логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — false.
- `ApiVersion`: Версия API Salesforce для этого запроса. По умолчанию используется версия API 29.0.
- `Timeout`: Интервал, определяющий время ожидания, по истечении которого запрос к серверу будет отменен. Значение по умолчанию зависит от источника.

Salesforce.Reports

16.03.2021 • 2 minutes to read

Синтаксис

```
Salesforce.Reports(optional loginUrl as nullable text, optional options as nullable record) as table
```

Сведения

Возвращает отчеты из указанной в учетных данных учетной записи Salesforce. Учетная запись будет подключена с помощью заданного окружения `loginUrl`. Если оно не указано, учетная запись подключится к рабочей среде (<https://login.salesforce.com>). Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `ApiVersion`: Версия API Salesforce для этого запроса. По умолчанию используется версия API 29.0.
- `Timeout`: Интервал, определяющий время ожидания, по истечении которого запрос к серверу будет отменен. Значение по умолчанию зависит от источника.

SapBusinessWarehouse.Cubes

16.03.2021 • 2 minutes to read

Синтаксис

```
SapBusinessWarehouse.Cubes(server as text, systemNumberOrSystemId as text, clientId as text,  
optional optionsOrLogonGroup as any, optional options as nullable record) as table
```

Сведения

Возвращает таблицу инфокубов и запросов, сгруппированных по инфообласти, из экземпляра SAP Business Warehouse на сервере `server` с номером системы `systemNumberOrSystemId` и идентификатором клиента `clientId`. Необязательный параметр записи `optionsOrLogonGroup` может быть указан для управления параметрами.

SapBusinessWarehouseExecutionMode.DataStream

16.03.2021 • 2 minutes to read

Сведения

Параметр "Режим выравнивания DataStream" для выполнения MDX в SAP Business Warehouse.

SapBusinessWarehouseExecutionMode.BasXml

16.03.2021 • 2 minutes to read

Сведения

Параметр "bXML flattening mode" (Режим выравнивания bXML) для выполнения многомерного выражения в SAP Business Warehouse.

SapBusinessWarehouseExecutionMode.BasXmlGzip

16.03.2021 • 2 minutes to read

Сведения

Параметр "Gzip compressed bXML flattening mode" (Режим выравнивания сжатого bXML Gzip) для выполнения многомерного выражения в SAP Business Warehouse. Рекомендуется при низкой задержке или большом количестве запросов.

SapHana.Database

16.03.2021 • 2 minutes to read

Синтаксис

```
SapHana.Database(**server** as text, optional **options** as nullable record) as table
```

Сведения

Возвращает таблицу многомерных пакетов из базы данных SAP HANA `server`. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `Query`: собственный запрос SQL для извлечения данных. Если такой запрос генерирует несколько результирующих наборов, возвращается только первый из них.
- `Distribution`: SapHanaDistribution, который задает значение свойства "Distribution" в строке подключения. Маршрутизация инструкций — это способ определения правильного узла сервера распределенной системы перед выполнением инструкции. Значение по умолчанию — SapHanaDistribution.All.

SapHanaDistribution.All

16.03.2021 • 2 minutes to read

Сведения

Параметр распределения All для SAP HANA.

SapHanaDistribution.Connection

16.03.2021 • 2 minutes to read

Сведения

Параметр распределения "Connection" для SAP HANA.

SapHanaDistribution.Off

16.03.2021 • 2 minutes to read

Сведения

Параметр распределения "Off" для SAP HANA.

SapHanaDistribution.Statement

16.03.2021 • 2 minutes to read

Сведения

Параметр распределения "Statement" для SAP HANA.

SapHanaRangeOperator.Equals

16.03.2021 • 2 minutes to read

Сведения

Оператор диапазона "Equals" (равно) для входных параметров SAP HANA.

SapHanaRangeOperator.GreaterThan

16.03.2021 • 2 minutes to read

Сведения

Оператор диапазона "Greater than" (больше чем) для входных параметров SAP HANA.

SapHanaRangeOperator.GreaterThanOrEquals

16.03.2021 • 2 minutes to read

Сведения

Оператор диапазона "Greater than or equals" (больше или равно) для входных параметров SAP HANA.

SapHanaRangeOperator.LessThan

16.03.2021 • 2 minutes to read

Сведения

Оператор диапазона "Less than" (меньше чем) для входных параметров SAP HANA.

SapHanaRangeOperator.LessThanOrEquals

16.03.2021 • 2 minutes to read

Сведения

Оператор диапазона "Less than or equals" (меньше или равно) для входных параметров SAP HANA.

SapHanaRangeOperator.NotEquals

16.03.2021 • 2 minutes to read

Сведения

Оператор диапазона "Not equals" (не равно) для входных параметров SAP HANA.

SharePoint.Contents

16.03.2021 • 2 minutes to read

Синтаксис

```
SharePoint.Contents(url as text, optional options as nullable record) as table
```

Сведения

Возвращает таблицу, содержащую строку для каждой папки и документа, найденного на указанном сайте SharePoint `url`. Каждая строка содержит свойства папки или файла и ссылку на их содержимое. Можно указать `options` для управления следующими параметрами:

- `ApiVersion`: число (14 или 15) или текстовое значение "Auto", указывающие версию API SharePoint, используемую для этого сайта. Если не указано, используется API версии 14. Если указан параметр Auto, версия сервера будет автоматически обнаружена, если это возможно, в противном случае будет установлена версия по умолчанию — 14. Для сайтов SharePoint, отличных от английского, требуется версия не ниже 15.

SharePoint.Files

16.03.2021 • 2 minutes to read

Синтаксис

```
SharePoint.Files(url as text, optional options as nullable record) as table
```

Сведения

Возвращает таблицу, содержащую строку для каждого документа, найденного в указанном списке SharePoint `url`, и вложенные папки. Каждая строка содержит свойства папки или файла и ссылку на их содержимое. Можно указать `options` для управления следующими параметрами:

- `ApiVersion`: число (14 или 15) или текстовое значение "Auto", указывающие версию API SharePoint, используемую для этого сайта. Если не указано, используется API версии 14. Если указан параметр Auto, версия сервера будет автоматически обнаружена, если это возможно, в противном случае будет установлена версия по умолчанию — 14. Для сайтов SharePoint, отличных от английского, требуется версия не ниже 15.

SharePoint.Tables

16.03.2021 • 2 minutes to read

Синтаксис

```
SharePoint.Tables(url as text, optional options as nullable record) as table
```

Сведения

Возвращает таблицу, содержащую строку для каждого элемента списка, найденного в указанном списке SharePoint `url`. Каждая строка содержит свойства списка. Можно указать `options` для управления следующими параметрами:

- `ApiVersion`: число (14 или 15) или текстовое значение "Auto", указывающие версию API SharePoint, используемую для этого сайта. Если не указано, используется API версии 14. Если указан параметр Auto, версия сервера будет автоматически обнаружена, если это возможно, в противном случае будет установлена версия по умолчанию — 14. Для сайтов SharePoint, отличных от английского, требуется версия не ниже 15.
- `Implementation`
- `ViewMode`

Soda.Feed

16.03.2021 • 2 minutes to read

Синтаксис

```
Soda.Feed(url as text) as table
```

Сведения

Возвращает таблицу с содержимым по указанному URL-адресу `url`, отформатированным в соответствии с SODA 2.0 API. URL-адрес должен указывать на допустимый источник, совместимый с SODA, который заканчивается расширением CSV.

Sql.Database

16.03.2021 • 2 minutes to read

Синтаксис

```
Sql.Database(server as text, database as text, optional options as nullable record) as table
```

О программе

Возвращает таблицу таблиц, представлений и хранимых функций SQL из базы данных SQL Server `database` на сервере `server`. Вместе с сервером можно дополнительно указать порт после двоеточия или точки. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `Query` — собственный запрос SQL для извлечения данных. Если такой запрос генерирует несколько результирующих наборов, возвращается только первый из них.
- `CreateNavigationProperties` — логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — true.
- `NavigationPropertyNameGenerator` — функция, которая используется для создания имен свойств навигации.
- `MaxDegreeOfParallelism` — число, которое задает значение предложения запроса "maxdop" в созданном запросе SQL.
- `CommandTimeout` — длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `ConnectionTimeout` — длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `HierarchicalNavigation` — логическое значение (true или false), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — false.
- `MultiSubnetFailover` — логическое значение (true или false), которое задает значение свойства "MultiSubnetFailover" в строке подключения. Значение по умолчанию — false.
- `UnsafeTypeConversions` — логическое значение (true или false), которое определяет, необходимо ли пытаться выполнить преобразование типов, которое может завершиться сбоем и привести к сбою всего запроса. Не рекомендуется для общего использования.
- `ContextInfo` — Двоичное значение, используемое для задания CONTEXT_INFO перед выполнением каждой команды.
- `OmitSRID` — логическое значение (true или false), которое определяет, необходимо ли опустить SRID при создании Well-Known Text из геометрических и географических типов.

Параметр записи указывается, например, как [option1 = value1, option2 = value2...] или [Query = "select ..."].

Sql.Databases

16.03.2021 • 2 minutes to read

Синтаксис

```
Sql.Databases(server as text, optional options as nullable record) as table
```

О программе

Возвращает таблицу баз данных на указанном сервере SQL Server, `server`. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `CreateNavigationProperties` — логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — true.
- `NavigationPropertyNameGenerator` — функция, которая используется для создания имен свойств навигации.
- `MaxDegreeOfParallelism` — число, которое задает значение предложения запроса "maxdop" в созданном запросе SQL.
- `CommandTimeout` — длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `ConnectionTimeout` — длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `HierarchicalNavigation` — логическое значение (true или false), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — false.
- `MultiSubnetFailover` — логическое значение (true или false), которое задает значение свойства "MultiSubnetFailover" в строке подключения. Значение по умолчанию — false.
- `UnsafeTypeConversions` — логическое значение (true или false), которое определяет, необходимо ли пытаться выполнить преобразование типов, которое может завершиться сбоем и привести к сбою всего запроса. Не рекомендуется для общего использования.
- `ContextInfo` — Двоичное значение, используемое для задания CONTEXT_INFO перед выполнением каждой команды.
- `OmitSRID` — логическое значение (true или false), которое определяет, необходимо ли опустить SRID при создании Well-Known Text из геометрических и географических типов.

Параметр записи указывается, например, как [параметр1 = значение1, параметр2 = значение2...].

Не поддерживает настройку запуска SQL-запроса на сервере. Вместо этого для выполнения SQL-запроса следует использовать `Sql.Database`.

Sybase.Database

16.03.2021 • 2 minutes to read

Синтаксис

```
Sybase.Database(server as text, database as text, optional options as nullable record) as table
```

Сведения

Возвращает список таблиц и представлений SQL, доступных в базе данных Sybase на сервере `server` в экземпляре базы данных `database`. Порт может дополнительно указываться сервером после двоеточия. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `CreateNavigationProperties`: логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — true.
- `NavigationPropertyNameGenerator`: функция, которая используется для создания имен свойств навигации.
- `Query`: собственный запрос SQL для извлечения данных. Если такой запрос генерирует несколько результирующих наборов, возвращается только первый из них.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `HierarchicalNavigation`: логическое значение (true или false), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — false.

Параметр записи указывается, например, как [option1 = value1, option2 = value2...] или [Query = "select ..."].

Teradata.Database

16.03.2021 • 2 minutes to read

Синтаксис

```
Teradata.Database(server as text, optional options as nullable record) as table
```

Сведения

Возвращает таблицу таблиц и представлений SQL из базы данных Teradata на сервере `server`. Порт может дополнительно указываться сервером после двоеточия. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `CreateNavigationProperties`: логическое значение (true или false), которое указывает, следует ли создавать свойства навигации в возвращаемых значениях. Значение по умолчанию — true.
- `NavigationPropertyNameGenerator`: функция, которая используется для создания имен свойств навигации.
- `Query`: собственный запрос SQL для извлечения данных. Если такой запрос генерирует несколько результирующих наборов, возвращается только первый из них.
- `CommandTimeout`: длительность, определяющая время выполнения запроса на стороне сервера до его отмены. Значение по умолчанию — 10 минут.
- `ConnectionTimeout`: длительность, определяющая время ожидания до отмены попытки подключения к серверу. Значение по умолчанию зависит от драйвера.
- `HierarchicalNavigation`: логическое значение (true или false), которое указывает, следует ли просматривать таблицы, сгруппированные по именам схем. Значение по умолчанию — false.

Параметр записи указывается, например, как [option1 = value1, option2 = value2...] или [Query = "select ..."].

WebAction.Request

16.03.2021 • 2 minutes to read

Синтаксис

```
WebAction.Request(method as text, url as text, optional options as nullable record) as action
```

О программе

Создает действие, которое при выполнении возвратит результаты выполнения запроса `method`, выполняемого по адресу `url` с использованием HTTP, в виде двоичного значения. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `Query` — позволяет программно добавить параметры запроса в URL-адрес, не беспокоясь об экранировании.
- `ApiKeyName` — если целевой сайт имеет понятие ключа API, этот параметр можно применить для указания имени (не значения) параметра ключа, который нужно использовать в URL-адресе. Фактическое значение ключа указывается в учетных данных.
- `Headers` — указание этого значения в виде записи позволяет предоставить дополнительные заголовки для HTTP-запроса.
- `Timeout` — указание этого значения в качестве длительности приведет к изменению времени ожидания для HTTP-запроса. Значение по умолчанию — 100 секунд.
- `ExcludedFromCacheKey` — если указать это значение в виде списка, эти ключи заголовков HTTP будут исключены из вычисления для кэширования данных.
- `IsRetry` — если указать это логическое значение как `true`, все существующие ответы в кэше будут игнорироваться при извлечении данных.
- `ManualStatusHandling` — указание этого значения в качестве списка предотвратит встроенную обработку HTTP-запросов, у которых ответ имеет один из этих кодов состояния.
- `RelativePath` — указание этого значения в качестве текста добавляет его к базовому URL-адресу перед выполнением запроса.
- `Content` — указание этого значения приведет к тому, что его содержимое станет телом HTTP-запроса.

Web.BrowserContents

16.03.2021 • 2 minutes to read

Синтаксис

```
Web.BrowserContents(url as text, optional options as nullable record) as text
```

Сведения

Возвращает HTML для указанного `url` в веб-браузере. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `WaitFor` — задает условие ожидания перед загрузкой HTML-кода, а также ожидания загрузки страницы (что всегда выполняется). Может быть записью, содержащей поля времени ожидания и (или) селектора. Если указано только время ожидания, функция будет ожидать указанное время до загрузки HTML. Если указаны и селектор, и время ожидания, и время ожидания истекло до того, как на странице появляется селектор, возникнет ошибка. Если селектор указан без времени ожидания, то применяется время ожидания по умолчанию, равное 30 секундам.

Пример 1

Возвращает HTML-код для <https://microsoft.com>.

```
Web.BrowserContents("https://microsoft.com")
```

```
"<!DOCTYPE html><html xmlns=..."
```

Пример 2

Возвращает HTML-код для <https://microsoft.com> после ожидания появления селектора CSS.

```
Web.BrowserContents("https://microsoft.com", [WaitFor = [Selector = "div.ready"]])
```

```
"<!DOCTYPE html><html xmlns=..."
```

Пример 3

Возвращает HTML-код для <https://microsoft.com> после ожидания в течение десяти секунд.

```
Web.BrowserContents("https://microsoft.com", [WaitFor = [Timeout = #duration(0,0,0,10)]])
```

```
"<!DOCTYPE html><html xmlns=..."
```

Пример 4

Возвращает HTML-код для <https://microsoft.com> после ожидания появления селектора CSS не более десяти секунд.

```
Web.BrowserContents("https://microsoft.com", [WaitFor = [Selector = "div.ready", Timeout =  
#duration(0,0,0,10)]])
```

```
"<!DOCTYPE html><html xmlns=..."
```

Web.Contents

16.03.2021 • 2 minutes to read

Синтаксис

```
Web.Contents(url as text, optional options as nullable record) as binary
```

О программе

Возвращает содержимое, скачанное с адреса `url` в двоичном виде. Для указания дополнительных свойств может быть предоставлен необязательный параметр записи `options`. Запись может содержать следующие поля:

- `Query` — позволяет программно добавить параметры запроса в URL-адрес, не беспокоясь об экранировании.
- `ApiKeyName` — если целевой сайт имеет понятие ключа API, этот параметр можно применить для указания имени (не значения) параметра ключа, который нужно использовать в URL-адресе. Фактическое значение ключа указывается в учетных данных.
- `Headers` — указание этого значения в виде записи позволяет предоставить дополнительные заголовки для HTTP-запроса.
- `Timeout` — указание этого значения в качестве длительности приведет к изменению времени ожидания для HTTP-запроса. Значение по умолчанию — 100 секунд.
- `ExcludedFromCacheKey` — если указать это значение в виде списка, эти ключи заголовков HTTP будут исключены из вычисления для кэширования данных.
- `IsRetry` — если указать это логическое значение как `true`, все существующие ответы в кэше будут игнорироваться при извлечении данных.
- `ManualStatusHandling` — указание этого значения в качестве списка предотвратит встроенную обработку HTTP-запросов, у которых ответ имеет один из этих кодов состояния.
- `RelativePath` — указание этого значения в качестве текста добавляет его к базовому URL-адресу перед выполнением запроса.
- `Content` — указание этого значения изменяет веб-запрос с GET на POST, используя значение параметра в качестве содержимого POST.

Заголовки HTTP-ответа доступны в виде метаданных в двоичном результате. За пределами контекста пользовательского соединителя данных доступен только заголовок Content-Type.

Web.Page

16.03.2021 • 2 minutes to read

Синтаксис

```
Web.Page(html as any) as table
```

Сведения

Возвращает содержимое документа HTML, разбитого на составные структуры, а также представление полного документа и его текста после удаления тегов.

WebMethod.Delete

16.03.2021 • 2 minutes to read

Сведения

Указывает метод DELETE для HTTP.

WebMethod.Get

16.03.2021 • 2 minutes to read

Сведения

Указывает метод GET для HTTP.

WebMethod.Head

16.03.2021 • 2 minutes to read

Сведения

Указывает метод HEAD для HTTP.

WebMethod.Patch

16.03.2021 • 2 minutes to read

Сведения

Указывает метод PATCH для HTTP.

WebMethod.Post

16.03.2021 • 2 minutes to read

Сведения

Указывает метод POST для HTTP.

WebMethod.Put

16.03.2021 • 2 minutes to read

Сведения

Указывает метод PUT для HTTP.

Xml.Document

16.03.2021 • 2 minutes to read

Синтаксис

```
Xml.Document(contents as any, optional encoding as nullable number) as table
```

Сведения

Возвращает содержимое XML-документа в виде иерархической таблицы.

Xml.Tables

16.03.2021 • 2 minutes to read

Синтаксис

```
Xml.Tables(contents as any, optional options as nullable record, optional encoding as nullable number) as table
```

Сведения

Возвращает содержимое XML-документа как вложенную коллекцию преобразованных в плоскую структуру таблиц.

Двоичные функции

16.03.2021 • 6 minutes to read

Эти функции позволяют создавать двоичные данные и управлять ими.

Двоичные форматы

Чтение чисел

ФУНКЦИЯ	ОПИСАНИЕ
BinaryFormat.7BitEncodedSignedInteger	Двоичный формат, который считывает 64-разрядное целое число со знаком, зашифрованное с использованием 7-разрядной кодировки с переменной длиной.
BinaryFormat.7BitEncodedUnsignedInteger	Двоичный формат, который считывает 64-разрядное целое число без знака, зашифрованное с использованием 7-разрядной кодировки с переменной длиной.
BinaryFormat.Binary	Возвращает двоичный формат, который считывает двоичное значение.
BinaryFormat.Byte	Двоичный формат, который считывает 8-разрядное целое число без знака.
BinaryFormat.Choice	Возвращает двоичный формат, который выбирает следующий двоичный формат в зависимости от значения, которое уже было считано.
BinaryFormat.Decimal	Двоичный формат, который считывает 16-байтовое десятичное значение .NET.
BinaryFormat.Double	Двоичный формат, который считывает 8-байтовое значение двойной точности с плавающей запятой в формате IEEE.
BinaryFormat.Group	Возвращает двоичный формат, который считывает группу элементов. Каждому значению элемента предшествует уникальное значение ключа. Результатом является список значений элементов.
BinaryFormat.Length	Возвращает двоичный формат, ограничивающий объем данных, который может быть считан. Both BinaryFormat.List и BinaryFormat.Binary можно использовать для чтения до конца данных. BinaryFormat.Length можно использовать для ограничения числа считанных байтов.
BinaryFormat.List	Возвращает двоичный формат, который считывает последовательность элементов и возвращает список.

ФУНКЦИЯ	ОПИСАНИЕ
BinaryFormat.Null	Двоичный формат, который считывает ноль байт и возвращает NULL.
BinaryFormat.Record	Возвращает двоичный формат, который считывает запись. Каждое поле в записи может иметь разный двоичный формат.
BinaryFormat.SignedInteger16	Двоичный формат, который считывает 16-разрядное целое число со знаком.
BinaryFormat.SignedInteger32	Двоичный формат, который считывает 32-разрядное целое число со знаком.
BinaryFormat.SignedInteger64	Двоичный формат, который считывает 64-разрядное целое число со знаком.
BinaryFormat.Single	Двоичный формат, который считывает 4-байтовое значение одиночной точности с плавающей запятой в формате IEEE.
BinaryFormat.Text	Возвращает двоичный формат, который считывает текстовое значение. Необязательное значение кодирования указывает кодировку текста.
BinaryFormat.Transform	Возвращает двоичный формат, который преобразует значения, считанные другим двоичным форматом.
BinaryFormat.UnsignedInteger16	Двоичный формат, который считывает 16-разрядное целое число без знака.
BinaryFormat.UnsignedInteger32	Двоичный формат, который считывает 32-разрядное целое число без знака.
BinaryFormat.UnsignedInteger64	Двоичный формат, который считывает 64-разрядное целое число без знака.
УПРАВЛЕНИЕ ПОРЯДКОМ БАЙТОВ	ОПИСАНИЕ
BinaryFormat.ByteOrder	Возвращает двоичный формат с порядком следования байтов, указанным функцией.
Table.PartitionValues	Возвращает сведения о том, как секционирована таблица.

Двоичная

ФУНКЦИЯ	ОПИСАНИЕ
Binary.Buffer	Помещает двоичное значение в буфер в памяти. Результат вызова — стабильное двоичное значение (то есть с детерминированной длиной и порядком байтов).

ФУНКЦИЯ	ОПИСАНИЕ
Binary.Combine	Объединяет список двоичных значений в одно двоичное значение.
Binary.Compress	Сжимает двоичное значение с помощью заданного типа сжатия.
Binary.Decompress	Распаковывает двоичное значение с применением указанного типа сжатия.
Binary.From	Возвращает двоичное значение из указанного значения.
Binary.FromList	Преобразует список чисел в двоичное значение.
Binary.FromText	Декодирует данные из текстового формата в двоичный.
Binary.InferContentType	Возвращает запись с полем Content.Type, которое содержит выводимый MIME-тип.
Binary.Length	Возвращает длину двоичных значений.
Binary.Range	Возвращает подмножество двоичного значения, начинающееся со смещения.
Binary.ToList	Преобразует двоичное значение в список чисел
Binary.ToText	Кодирует двоичные данные в текстовом формате.
BinaryEncoding.Base64	Константа, которая используется как тип кодировки, если требуется применять кодировку base-64.
BinaryEncoding.Hex	Константа, которая используется как тип кодировки, если требуется применять шестнадцатеричную кодировку.
BinaryOccurrence.Optional	Ожидается, что элемент встретится во входных данных не более одного раза.
BinaryOccurrence.Repeating	Ожидается, что элемент встретится во входных данных произвольное число раз.
BinaryOccurrence.Required	Ожидается, что элемент встретится во входных данных один раз.
ByteOrder.BigEndian	Возможное значение параметра <code>byteOrder</code> в <code>BinaryFormat.ByteOrder</code> . Самый значащий байт отображается первым при обратном порядке байтов.
ByteOrder.LittleEndian	Возможное значение параметра <code>byteOrder</code> в <code>BinaryFormat.ByteOrder</code> . Наименее значащий байт отображается первым при прямом порядке байтов.
Compression.Brotli	Сжатые данные имеют формат Brotli.

ФУНКЦИЯ	ОПИСАНИЕ
Compression.Deflate	Сжатые данные имеют формат Deflate.
Compression.GZip	Сжатые данные имеют формат GZip.
Compression.LZ4	Сжатые данные имеют формат LZ4.
Compression.None	Данные не сжаты.
Compression.Snappy	Сжатые данные имеют формат Snappy.
Compression.Zstandard	Сжатые данные имеют формат Zstandard.
Occurrence.Optional	Ожидается, что элемент встретится во входных данных не более одного раза.
Occurrence.Repeating	Ожидается, что элемент встретится во входных данных произвольное число раз.
Occurrence.Required	Ожидается, что элемент встретится во входных данных один раз.
#binary	Создает двоичное значение из чисел или текста.

Binary.Buffer

16.03.2021 • 2 minutes to read

Синтаксис

```
Binary.Buffer(binary as nullable binary) as nullable binary
```

Сведения

Помещает двоичное значение в буфер в памяти. Результат вызова — стабильное двоичное значение (то есть с детерминированной длиной и порядком байтов).

Пример 1

Создание стабильной версии двоичного значения.

```
Binary.Buffer(Binary.FromList({0..10}))
```

```
#binary({0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10})
```

Binary.Combine

16.03.2021 • 2 minutes to read

Синтаксис

```
Binary.Combine(binaries as list) as binary
```

Сведения

Объединяет список двоичных значений в одно двоичное значение.

Binary.Compress

16.03.2021 • 2 minutes to read

Синтаксис

```
Binary.Compress(binary as nullable binary, compressionType as number) as nullable binary
```

Сведения

Сжимает двоичное значение с помощью заданного типа сжатия. Результатом этого вызова является сжатая копия входных данных. Типы сжатия включают:

- `Compression.GZip`
- `Compression.Deflate`

Пример 1

Сжатие двоичного значения.

```
Binary.Compress(Binary.FromList(List.Repeat({10}, 1000)), Compression.Deflate)
```

```
#binary({227, 226, 26, 5, 163, 96, 20, 12, 119, 0, 0})
```

Binary.Decompress

16.03.2021 • 2 minutes to read

Синтаксис

```
Binary.Decompress(binary as nullable binary, compressionType as number) as nullable binary
```

Сведения

Распаковывает двоичное значение с применением указанного типа сжатия. Результатом этого вызова является распакованная копия входных данных. Типы сжатия включают:

- `Compression.GZip`
- `Compression.Deflate`

Пример 1

Распаковка двоичного значения.

```
Binary.Decompress(#binary({115, 103, 200, 7, 194, 20, 134, 36, 134, 74, 134, 84, 6, 0}),  
Compression.Deflate)
```

```
#binary({71, 0, 111, 0, 111, 0, 100, 0, 98, 0, 121, 0, 101, 0})
```


Синтаксис

```
Binary.From(value as any, optional encoding as nullable number) as nullable binary
```

Сведения

Возвращает значение `binary` из заданного параметра `value`. Если заданный параметр `value` имеет значение `null`, `Binary.From` возвращает `null`. Если заданный параметр `value` имеет значение `binary`, возвращается `value`. Значения следующих типов можно преобразовать в значение `binary`:

- `text`: значение `binary` из текстового представления. Дополнительные сведения см. в разделе `Binary.FromText`.

Если `value` имеет любой другой тип, возвращается ошибка.

Пример 1

Получение значения `binary` для `"1011"`.

```
Binary.From("1011")
```

```
Binary.FromText("1011", BinaryEncoding.Base64)
```

Binary.FromList

16.03.2021 • 2 minutes to read

Синтаксис

```
Binary.FromList(list as list) as binary
```

Сведения

Преобразует список чисел в двоичное значение.

Binary.FromText

16.03.2021 • 2 minutes to read

Синтаксис

```
Binary.FromText(text as nullable text, optional encoding as nullable number) as nullable binary
```

Сведения

Возвращает результат преобразования текстового значения `text` в двоичное значение (список `number`).

Можно указать `encoding`, чтобы обозначить кодировку, используемую в текстовом значении. Для

`encoding` можно использовать следующие значения `BinaryEncoding`.

- `BinaryEncoding.Base64`: кодировка Base 64
- `BinaryEncoding.Hex`: шестнадцатеричная кодировка

Пример 1

Декодирование `"1011"` в двоичный формат.

```
Binary.FromText("1011")
```

```
Binary.FromText("1011", BinaryEncoding.Base64)
```

Пример 2

Декодирование `"1011"` в двоичное значение с шестнадцатеричной кодировкой.

```
Binary.FromText("1011", BinaryEncoding.Hex)
```

```
Binary.FromText("EBE=", BinaryEncoding.Base64)
```

Binary.InferContentType

16.03.2021 • 2 minutes to read

Синтаксис

```
Binary.InferContentType(source as binary) as record
```

Сведения

Возвращает запись с полем Content.Type, которое содержит выводимый MIME-тип. Если выводимый тип контента — text/*, и обнаружена кодирующая кодовая страница, то дополнительно возвращается поле Content.Encoding с кодировкой потока. Если выводимый тип содержимого — text/csv, а формат использует разделители, дополнительно возвращается поле Csv.PotentialDelimiter, содержащее таблицу для анализа возможных разделителей. Если выводимый тип содержимого — text/csv, а формат использует фиксированную ширину, дополнительно возвращается поле Csv.PotentialPositions, содержащее список для анализа возможных положений столбцов с фиксированной шириной.

Binary.Length

16.03.2021 • 2 minutes to read

Синтаксис

```
Binary.Length(binary as nullable binary) as nullable number
```

Сведения

Возвращает число символов.

Binary.Range

16.03.2021 • 2 minutes to read

Синтаксис

```
Binary.Range(binary as binary, offset as number, optional count as nullable number) as binary
```

Сведения

Возвращает подмножество двоичного значения, начинающееся со смещения `binary`. Необязательный параметр `offset` устанавливает максимальную длину подмножества.

Пример 1

Возвращает подмножество двоичного значения, начинающееся со смещения 6.

```
Binary.Range(#binary({0..10}), 6)
```

```
#binary({6, 7, 8, 9, 10})
```

Пример 2

Возвращает подмножество двоичного значения длиной 2, начинающееся со смещения 6.

```
Binary.Range(#binary({0..10}), 6, 2)
```

```
#binary({6, 7})
```

Binary.ToList

16.03.2021 • 2 minutes to read

Синтаксис

```
Binary.ToList(binary as binary) as list
```

Сведения

Преобразует двоичное значение в список чисел.

Binary.ToText

16.03.2021 • 2 minutes to read

Синтаксис

```
Binary.ToText(binary as nullable binary, optional encoding as nullable number) as nullable text
```

Сведения

Возвращает результат преобразования двоичного списка чисел `binary` в текстовое значение. При необходимости можно указать `encoding`, чтобы обозначить кодировку, используемую в полученном текстовом значении. Для `encoding` можно использовать следующие значения `BinaryEncoding`.

- `BinaryEncoding.Base64`: кодировка Base 64
- `BinaryEncoding.Hex`: шестнадцатеричная кодировка

BinaryEncoding.Base64

16.03.2021 • 2 minutes to read

Сведения

Константа, которая используется как тип кодировки, если требуется применять кодировку base-64.

BinaryEncoding.Hex

16.03.2021 • 2 minutes to read

Сведения

Константа, которая используется как тип кодировки, если требуется применять шестнадцатеричную кодировку.

BinaryFormat.7BitEncodedSignedInteger

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.7BitEncodedSignedInteger(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 64-разрядное целое число со знаком, зашифрованное с использованием 7-разрядной кодировки с переменной длиной.

BinaryFormat.7BitEncodedUnsignedInteger

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.7BitEncodedUnsignedInteger(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 64-разрядное целое число без знака, зашифрованное с использованием 7-разрядной кодировки с переменной длиной.

BinaryFormat.Binary

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.Binary(optional length as any) as function
```

Сведения

Возвращает двоичный формат, который считывает двоичное значение. Если указан параметр `length`, двоичное значение будет содержать это количество байтов. Если параметр `length` не указан, двоичное значение будет содержать оставшиеся байты. `length` можно указать как число или как двоичный формат длины, которая предшествует двоичным данным.

BinaryFormat.Byte

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.Byte(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 8-разрядное целое число без знака.

BinaryFormat.ByteOrder

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.ByteOrder(binaryFormat as function, byteOrder as number) as function
```

Сведения

Возвращает двоичный формат с порядком байтов, указанным `binaryFormat`. По умолчанию используется порядок байтов `ByteOrder.BigEndian`.

BinaryFormat.Choice

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.Choice(binaryFormat as function, chooseFunction as function, optional type as nullable type, optional combineFunction as nullable function) as function
```

Сведения

Возвращает двоичный формат, который выбирает следующий двоичный формат в зависимости от значения, которое уже было считано. Значение двоичного формата, создаваемое этой функцией, работает поэтапно:

- Двоичный формат, заданный параметром `binaryFormat`, используется для считывания значения.
- Значение передается в функцию `choice`, указанную параметром `chooseFunction`.
- Функция `choice` проверяет значение и возвращает второй двоичный формат.
- Второй двоичный формат используется для считывания второго значения.
- Если указана функция `combine`, то первое и второе значения передаются функции `combine`, а результирующее значение возвращается.
- Если функция `combine` не указана, возвращается второе значение.
- Возвращается второе значение.

Необязательный параметр `type` указывает тип двоичного формата, который будет возвращен функцией `choice`. Можно указать `type any`, `type list` или `type binary`. Если параметр `type` не указан, используется `type any`. Если используется `type list` или `type binary`, система может возвращать потоковое значение `binary` или `list` вместо буферизированного, что может уменьшить объем памяти, необходимый для чтения формата.

Пример 1

Считывание списка байтов, где число элементов определяется первым байтом.

```
let
    binaryData = #binary({2, 3, 4, 5}),
    listFormat = BinaryFormat.Choice(
        BinaryFormat.Byte,
        (length) => BinaryFormat.List(BinaryFormat.Byte, length)
    )
in
    listFormat(binaryData)
```

3

4

Пример 2

Считывание списка байтов, где число элементов определяется первым байтом, а первый считанный байт сохраняется.

```
let
  binaryData = #binary({2, 3, 4, 5}),
  listFormat = BinaryFormat.Choice(
    BinaryFormat.Byte,
    (length) => BinaryFormat.Record([
      length = length,
      list = BinaryFormat.List(BinaryFormat.Byte, length)
    ])
  )
in
  listFormat(binaryData)
```

LENGTH	2
LIST	[Список]

Пример 3

Считывание списка байтов, где число элементов определяется первым байтом с использованием списка потоков.

```
let
  binaryData = #binary({2, 3, 4, 5}),
  listFormat = BinaryFormat.Choice(
    BinaryFormat.Byte,
    (length) => BinaryFormat.List(BinaryFormat.Byte, length),
    type list
  )
in
  listFormat(binaryData)
```

3
4

BinaryFormat.Decimal

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.Decimal(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 16-байтовое десятичное значение .NET.

BinaryFormat.Double

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.Double(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 8-байтовое значение двойной точности с плавающей запятой в формате IEEE.

Синтаксис

```
BinaryFormat.Group(binaryFormat as function, group as list, optional extra as nullable function, optional lastKey as any) as function
```

Сведения

Он принимает перечисленные ниже параметры.

- Параметр `binaryFormat` задает двоичный формат значения ключа.
- Параметр `group` предоставляет сведения о группе известных элементов.
- Необязательный параметр `extra` можно использовать для указания функции, которая будет возвращать значение двоичного формата для значения после любого непредвиденного ключа. Если параметр `extra` не указан, то при наличии непредвиденных значений ключа возникнет ошибка.

Параметр `group` задает список определений элементов. Каждое определение элемента — это список, содержащий 3–5 значений, как показано ниже:

- Значение ключа. Значение ключа, соответствующее элементу. Оно должно быть уникальным в пределах набора элементов.
- Формат элемента. Двоичный формат, соответствующий значению элемента. Это позволяет каждому элементу иметь отличный формат.
- Вхождение элемента. Значение `BinaryOccurrence.Type` указывает ожидаемое количество вхождений элемента в группе. Обязательные элементы, которые отсутствуют, вызывают ошибку. Обязательные или необязательные дублирующиеся элементы обрабатываются как непредвиденные значения ключа.
- Значение элемента по умолчанию (необязательно). Если значение элемента по умолчанию отображается в списке определений элементов и не равно `NULL`, оно будет использоваться вместо значения по умолчанию. Значение по умолчанию для повторяющихся или необязательных элементов равно `NULL`, а значение по умолчанию для повторяющихся значений — пустой список `{ }`.
- Преобразование значения элемента (необязательно). Если функция преобразования значения элемента содержится в списке определений элементов и не имеет значение `NULL`, она будет вызвана для преобразования значения элемента перед его возвратом. Функция преобразования вызывается только в том случае, если элемент встречается во входных данных (она никогда не будет вызываться со значением по умолчанию).

Пример 1

В следующем примере предполагается, что у нас есть значение ключа, представляющее собой один байт, с четырьмя ожидаемыми элементами в группе, каждый из которых имеет байт данных после ключа. Элементы отображаются во входных данных следующим образом:

- Ключ 1 является обязательным и отображается со значением 11.
- Ключ 2 повторяется, отображается дважды со значением 22 и приводит к значению `{22, 22}`.
- Ключ 3 является необязательным, не отображается и приводит к значению `NULL`.
- Ключ 4 повторяется, но не отображается и приводит к значению `{ }`.

- Ключ 5 не входит в группу, но отображается один раз со значением 55. Дополнительная функция вызывается со значением ключа 5 и возвращают формат, соответствующий этому значению (BinaryFormat.Byte). Значение 55 считывается и удаляется.

```
let
  b = #binary({
    1, 11,
    2, 22,
    2, 22,
    5, 55,
    1, 11
  }),
  f = BinaryFormat.Group(
    BinaryFormat.Byte,
    {
      {1, BinaryFormat.Byte, BinaryOccurrence.Required},
      {2, BinaryFormat.Byte, BinaryOccurrence.Repeating},
      {3, BinaryFormat.Byte, BinaryOccurrence.Optional},
      {4, BinaryFormat.Byte, BinaryOccurrence.Repeating}
    },
    (extra) => BinaryFormat.Byte
  )
in
  f(b)
```

11

[Список]

[Список]

Пример 2

В следующем примере показано преобразование значения элемента и значение элемента по умолчанию. Повторяющийся элемент с ключом 1 суммирует список значений, считанных с помощью List.Sum. Необязательный элемент с ключом 2 имеет значение по умолчанию 123 вместо NULL.

```
let
  b = #binary({
    1, 101,
    1, 102
  }),
  f = BinaryFormat.Group(
    BinaryFormat.Byte,
    {
      {1, BinaryFormat.Byte, BinaryOccurrence.Repeating,
        0, (list) => List.Sum(list)},
      {2, BinaryFormat.Byte, BinaryOccurrence.Optional, 123}
    }
  )
in
  f(b)
```

203

123

BinaryFormat.Length

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.Length(binaryFormat as function, length as any) as function
```

Сведения

Возвращает двоичный формат, ограничивающий объем данных, который может быть считан. Для чтения до конца данных можно использовать как `BinaryFormat.List`, так `BinaryFormat.Binary`. `BinaryFormat.Length` можно использовать для ограничения числа считанных байтов. Параметр `binaryFormat` задает двоичный формат для ограничения. Параметр `length` указывает число байтов для чтения. Параметр `length` может быть либо числовым значением, либо значением двоичного формата, указывающим формат значения длины, которое предшествует считываемому значению.

Пример 1

Ограничение числа считанных байтов до 2 при чтении списка байтов.

```
let
    binaryData = #binary({1, 2, 3}),
    listFormat = BinaryFormat.Length(
        BinaryFormat.List(BinaryFormat.Byte),
        2
    )
in
    listFormat(binaryData)
```

1

2

Пример 2

Ограничение числа считанных байтов при чтении списка байтов до байтового значения, предшествующего списку.

```
let
    binaryData = #binary({1, 2, 3}),
    listFormat = BinaryFormat.Length(
        BinaryFormat.List(BinaryFormat.Byte),
        BinaryFormat.Byte
    )
in
    listFormat(binaryData)
```

2

BinaryFormat.List

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.List(binaryFormat as function, optional countOrCondition as any) as function
```

Сведения

Возвращает двоичный формат, который считывает последовательность элементов и возвращает `list`. Параметр `binaryFormat` задает двоичный формат каждого элемента. Число считанных элементов можно определить тремя способами:

- Если `countOrCondition` не указан, то двоичный формат будет считываться до тех пор, пока больше не останется элементов.
- Если `countOrCondition` является числом, то двоичный формат считает это количество элементов.
- Если `countOrCondition` является функцией, то эта функция будет вызываться для каждого считанного элемента. Функция возвращает `true`, чтобы продолжить, и `false`, чтобы прекратить чтение элементов. Последний элемент включен в список.
- Если `countOrCondition` является двоичным форматом, то ожидается, что число элементов предшествует списку, а для чтения счетчика используется указанный формат.

Пример 1

Считывание байтов до конца данных.

```
let
  binaryData = #binary({1, 2, 3}),
  listFormat = BinaryFormat.List(BinaryFormat.Byte)
in
  listFormat(binaryData)
```

1

2

3

Пример 2

Считывание 2 байтов.

```
let
  binaryData = #binary({1, 2, 3}),
  listFormat = BinaryFormat.List(BinaryFormat.Byte, 2)
in
  listFormat(binaryData)
```

1
2

Пример 3

Считывание байтов, пока байтовое значение меньше 2.

```
let
  binaryData = #binary({1, 2, 3}),
  listFormat = BinaryFormat.List(BinaryFormat.Byte, (x) => x < 2)
in
  listFormat(binaryData)
```

1
2

BinaryFormat.Null

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.Null(binary as binary) as any
```

Сведения

Двоичный формат, который считывает ноль байт и возвращает NULL.

BinaryFormat.Record

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.Record(record as record) as function
```

Сведения

Возвращает двоичный формат, который считывает запись. Параметр `record` задает формат записи. Каждое поле в записи может иметь разный двоичный формат. Если поле содержит значение, которое не является значением двоичного формата, то данные для этого поля не считываются, а значение поля помещается в результат.

Пример 1

Считывание записи, содержащей одно 16-разрядное целое число и одно 32-разрядное целое число.

```
let
    binaryData = #binary({
        0x00, 0x01,
        0x00, 0x00, 0x00, 0x02
    }),
    recordFormat = BinaryFormat.Record([
        A = BinaryFormat.UnsignedInteger16,
        B = BinaryFormat.UnsignedInteger32
    ])
in
    recordFormat(binaryData)
```

ОБЪЕКТ	1
B	2

BinaryFormat.SignedInteger16

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.SignedInteger16(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 16-разрядное целое число со знаком.

BinaryFormat.SignedInteger32

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.SignedInteger32(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 32-разрядное целое число со знаком.

BinaryFormat.SignedInteger64

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.SignedInteger64(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 64-разрядное целое число со знаком.

BinaryFormat.Single

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.Single(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 4-байтовое значение одиночной точности с плавающей запятой в формате IEEE.

BinaryFormat.Text

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.Text(length as any, optional encoding as nullable number) as function
```

Сведения

Возвращает двоичный формат, который считывает текстовое значение. `length` указывает число байтов для расшифровки или двоичный формат длины, предшествующей тексту. Необязательное значение `encoding` указывает кодировку текста. Если параметр `encoding` не указан, то кодировка определяется по меткам порядка байтов Юникода. Если метки порядка байтов отсутствуют, то используется

```
TextEncoding.Utf8 .
```

Пример 1

Расшифровка двух байтов в виде текста ASCII.

```
let
    binaryData = #binary({65, 66, 67}),
    textFormat = BinaryFormat.Text(2, TextEncoding.Ascii)
in
    textFormat(binaryData)
```

```
"AB"
```

Пример 2

Расшифровка текста ASCII, где длина текста в байтах указывается перед текстом в виде байта.

```
let
    binaryData = #binary({2, 65, 66}),
    textFormat = BinaryFormat.Text(
        BinaryFormat.Byte,
        TextEncoding.Ascii
    )
in
    textFormat(binaryData)
```

```
"AB"
```

BinaryFormat.Transform

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.Transform(binaryFormat as function, function as function) as function
```

Сведения

Возвращает двоичный формат, который преобразует значения, считанные другим двоичным форматом.

Параметр `binaryFormat` указывает двоичный формат, который будет использоваться для считывания значений. `function` вызывается со считанным значением и возвращает преобразованное значение.

Пример 1

Считывает байт и добавляет к нему единицу.

```
let
  binaryData = #binary({1}),
  transformFormat = BinaryFormat.Transform(
    BinaryFormat.Byte,
    (x) => x + 1
  )
in
  transformFormat(binaryData)
```


BinaryFormat.UnsignedInteger16

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.UnsignedInteger16(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 16-разрядное целое число без знака.

BinaryFormat.UnsignedInteger32

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.UnsignedInteger32(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 32-разрядное целое число без знака.

BinaryFormat.UnsignedInteger64

16.03.2021 • 2 minutes to read

Синтаксис

```
BinaryFormat.UnsignedInteger64(binary as binary) as any
```

Сведения

Двоичный формат, который считывает 64-разрядное целое число без знака.

BinaryOccurrence.Optional

16.03.2021 • 2 minutes to read

Сведения

Ожидается, что элемент встретится во входных данных не более одного раза.

BinaryOccurrence.Repeating

16.03.2021 • 2 minutes to read

Сведения

Ожидается, что элемент встретится во входных данных произвольное число раз.

BinaryOccurrence.Required

16.03.2021 • 2 minutes to read

Сведения

Ожидается, что элемент встретится во входных данных один раз.

ByteOrder.BigEndian

16.03.2021 • 2 minutes to read

Сведения

Возможное значение параметра `byteOrder` в `BinaryFormat.ByteOrder`. Самый значащий байт отображается первым при обратном порядке байтов.

ByteOrder.LittleEndian

16.03.2021 • 2 minutes to read

Сведения

Возможное значение параметра `byteOrder` в `BinaryFormat.ByteOrder`. Наименее значащий байт отображается первым при прямом порядке байтов.

Compression.Brotli

16.03.2021 • 2 minutes to read

Сведения

Сжатые данные имеют формат Brotli.

Compression.Deflate

16.03.2021 • 2 minutes to read

Сведения

Сжатые данные имеют формат Deflate.

Compression.GZip

16.03.2021 • 2 minutes to read

Сведения

Сжатые данные имеют формат GZip.

Compression.LZ4

16.03.2021 • 2 minutes to read

Сведения

Сжатые данные имеют формат LZ4.

Compression.None

16.03.2021 • 2 minutes to read

Сведения

Данные не сжаты.

Compression.Snappy

16.03.2021 • 2 minutes to read

Сведения

Сжатые данные имеют формат Snappy.

Compression.Zstandard

16.03.2021 • 2 minutes to read

Сведения

Сжатые данные имеют формат Zstandard.

Occurrence.Optional

16.03.2021 • 2 minutes to read

Сведения

Ожидается, что элемент встретится во входных данных не более одного раза.

Occurrence.Repeating

16.03.2021 • 2 minutes to read

Сведения

Ожидается, что элемент встретится во входных данных произвольное число раз.

Occurrence.Required

16.03.2021 • 2 minutes to read

Сведения

Ожидается, что элемент встретится во входных данных один раз.

#binary

16.03.2021 • 2 minutes to read

Синтаксис

```
#binary(value as any) as any
```

Сведения

Создает двоичное значение из списка чисел или текстового значения в кодировке Base64.

Пример 1

Создание двоичного значения из списка чисел.

```
#binary({0x30, 0x31, 0x32})
```

```
Text.ToBinary("012")
```

Пример 2

Создание двоичного значения из текстового значения в кодировке Base64.

```
#binary("1011")
```

```
Binary.FromText("1011", BinaryEncoding.Base64)
```

Функции объединения

16.03.2021 • 2 minutes to read

Эти функции используются другими функциями библиотеки, которые объединяют значения. Например, `Table.ToList` и `Table.CombineColumns` применяют функцию объединения к каждой строке таблицы, чтобы получить одно значение для каждой строки.

Функции объединения

ФУНКЦИЯ	ОПИСАНИЕ
<code>Combiner.CombineTextByDelimiter</code>	Возвращает функцию, которая объединяет список текстовых значений в один текст с помощью заданного разделителя.
<code>Combiner.CombineTextByEachDelimiter</code>	Возвращает функцию, которая объединяет список текстовых значений в один текст с помощью каждого разделителя в последовательности.
<code>Combiner.CombineTextByLengths</code>	Возвращает функцию, которая объединяет список текстовых значений в один текст с помощью заданных длин.
<code>Combiner.CombineTextByPositions</code>	Возвращает функцию, которая объединяет список текстовых значений в один текст с использованием заданных положений.
<code>Combiner.CombineTextByRanges</code>	Возвращает функцию, которая объединяет список текстовых значений в один текст с использованием заданных положений и длин.

Combiner.CombineTextByDelimiter

16.03.2021 • 2 minutes to read

Синтаксис

```
Combiner.CombineTextByDelimiter(delimiter as text, optional quoteStyle as nullable number) as function
```

Сведения

Возвращает функцию, которая объединяет список текстовых значений в один текст с помощью заданного разделителя.

Combiner.CombineTextByEachDelimiter

16.03.2021 • 2 minutes to read

Синтаксис

```
Combiner.CombineTextByEachDelimiter(delimiters as list, optional quoteStyle as nullable number)  
as function
```

Сведения

Возвращает функцию, которая объединяет список текстовых значений в один текст с помощью каждого разделителя в последовательности.

Combiner.CombineTextByLengths

16.03.2021 • 2 minutes to read

Синтаксис

```
Combiner.CombineTextByLengths(lengths as list, optional template as nullable text) as function
```

Сведения

Возвращает функцию, которая объединяет список текстовых значений в один текст с помощью заданных длин.

Combiner.CombineTextByPositions

16.03.2021 • 2 minutes to read

Синтаксис

```
Combiner.CombineTextByPositions(positions as list, optional template as nullable text) as function
```

Сведения

Возвращает функцию, которая объединяет список текстовых значений в один текст с использованием заданных положений.

Combiner.CombineTextByRanges

16.03.2021 • 2 minutes to read

Синтаксис

```
Combiner.CombineTextByRanges(ranges as list, optional template as nullable text) as function
```

Сведения

Возвращает функцию, которая объединяет список текстовых значений в один текст с использованием заданных положений и длин.

Функции сравнения

16.03.2021 • 2 minutes to read

Эти функции позволяют проверить равенство объектов и определить их порядок.

Функции сравнения

ФУНКЦИЯ	ОПИСАНИЕ
Comparer.Equals	Возвращает логическое значение на основе проверки двух заданных значений на равенство.
Comparer.FromCulture	Возвращает функцию сравнения с учетом языка и региональных параметров и логическое значение, определяющее чувствительность к регистру для операции сравнения. Значением по умолчанию для ignoreCase является false. Значения языка и региональных стандартов представляют собой общепринятые текстовые представления языковых стандартов, используемые в .NET Framework.
Comparer.Ordinal	Возвращает функцию сравнения, использующую порядковые (Ordinal) правила для сравнения значений.
Comparer.OrdinalIgnoreCase	Возвращает функцию сравнения без учета регистра, использующую порядковые (Ordinal) правила для сравнения указанных значений x и y.
Culture.Current	Возвращает текущие значения языка и региональных параметров в системе.

Comparer.Equals

16.03.2021 • 2 minutes to read

Синтаксис

```
Comparer.Equals(comparer as function, x as any, y as any) as logical
```

Сведения

Возвращает значение `logical` на основе проверки двух заданных значений `x` и `y` на равенство с помощью указанного `comparer`.

`comparer` — это модуль `Comparer`, который используется для управления сравнением. Функции сравнения можно использовать для сравнений, учитывающих или не учитывающих регистр, язык и региональные параметры, а также локаль.

В языке формул доступны следующие встроенные функции сравнения:

- `Comparer.Ordinal`: используется для точного сравнения по порядковому номеру
- `Comparer.OrdinalIgnoreCase`: используется для точного сравнения по порядковому номеру без учета регистра
- `Comparer.FromCulture`: используется для сравнения с учетом языка и региональных параметров

Пример 1

Сравнение "1" и "A" с использованием языка "en-US" для определения того, равны ли значения.

```
Comparer.Equals(Comparer.FromCulture("en-us"), "1", "A")
```

`false`

Comparer.FromCulture

16.03.2021 • 2 minutes to read

Синтаксис

```
Comparer.FromCulture(culture as text, optional ignoreCase as nullable logical) as function
```

Сведения

Возвращает функцию сравнения с учетом `culture` и логическое значение `ignoreCase`, определяющее чувствительность к регистру для операции сравнения. Значением по умолчанию для `ignoreCase` является `false`. Значения языка и региональных стандартов представляют собой общепринятые текстовые представления языковых стандартов, используемые в .NET Framework.

Пример 1

Сравнение "a" и "A" с использованием языка "en-US" для определения того, равны ли значения.

```
Comparer.FromCulture("en-us")("a", "A")
```

-1

Пример 2

Сравнение "a" и "A" с использованием языка "en-US" без учета регистра для определения того, равны ли значения.

```
Comparer.FromCulture("en-us", true)("a", "A")
```

0

Comparer.Ordinal

16.03.2021 • 2 minutes to read

Синтаксис

```
Comparer.Ordinal(x as any, y as any) as number
```

Сведения

Возвращает функцию сравнения, использующую порядковые (Ordinal) правила для сравнения указанных значений `x` и `y`.

Пример 1

Используя порядковые правила, выяснить, эквивалентны ли значения "encyclopædia" и "encyclopaedia". Обратите внимание, что они эквивалентны при использовании `Comparer.FromCulture("en-us")`.

```
Comparer.Equals(Comparer.Ordinal, "encyclopædia", "encyclopaedia")
```

```
false
```

Comparer.OrdinalIgnoreCase

16.03.2021 • 2 minutes to read

Синтаксис

```
Comparer.OrdinalIgnoreCase(x as any, y as any) as number
```

Сведения

Возвращает функцию сравнения без учета регистра, использующую порядковые (Ordinal) правила для сравнения указанных значений `x` и `y`.

Пример

Используя порядковые правила без учета регистра, сравним "Abc" с "abc". Обратите внимание, что "Abc" считается меньше "abc", если использовать `Comparer.Ordinal`.

```
Comparer.OrdinalIgnoreCase("Abc", "abc")
```

```
0
```

Сведения

Возвращает имя текущих языка и региональных параметров для приложения.

Функции данных

16.03.2021 • 10 minutes to read

Такие функции позволяют создать компонент даты для значений Date, DateTime и DateTimeZone, а также управлять им.

Дата

ФУНКЦИЯ	ОПИСАНИЕ
Date.AddDays	Возвращает значение Date/DateTime/DateTimeZone, где день увеличивается с шагом приращения, равным заданному количеству дней. Также обрабатывает приращение месяца и года в значении соответствующим образом.
Date.AddMonths	Возвращает значение DateTime, где месяц увеличивается с шагом приращения в n месяцев.
Date.AddQuarters	Возвращает значение Date/DateTime/DateTimeZone с шагом приращения, равным заданному количеству кварталов. Каждый квартал определяется как три месяца. Также обрабатывает приращение года в значении соответствующим образом.
Date.AddWeeks	Возвращает значение Date/DateTime/DateTimeZone с шагом приращения, равным заданному количеству недель. Каждая неделя определяется как семь дней. Также обрабатывает приращение месяца и года в значении соответствующим образом.
Date.AddYears	Возвращает значение DateTime, где год увеличивается с шагом приращения в n лет.
Date.Day	Возвращает день для значения DateTime.
Date.DayOfWeek	Возвращает число (от 0 до 6), соответствующее дню недели предоставленного значения.
Date.DayOfWeekName	Возвращает название дня недели.
Date.DayOfYear	Возвращает число, представляющее день года из значения DateTime.
Date.DaysInMonth	Возвращает число дней в месяце из значения DateTime.
Date.EndOfDay	Возвращает значение DateTime для конца дня.
Date.EndOfMonth	Возвращает значение DateTime для конца месяца.

ФУНКЦИЯ	ОПИСАНИЕ
Date.EndOfQuarter	Возвращает значение Date/DateTime/DateTimeZone, представляющее конец квартала. Части даты и времени сбрасываются до конечных значений квартала. Сведения о часовом поясе сохраняются.
Date.EndOfWeek	Возвращает значение DateTime для конца недели.
Date.EndOfYear	Возвращает значение DateTime для конца года.
Date.From	Возвращает значение даты для заданного значения.
Date.FromText	Возвращает значение даты из набора форматов даты и значения языка и региональных параметров.
Date.IsInCurrentDay	Указывает, наступает ли данное значение даты и времени <code>dateTime</code> в течение текущего дня, по расчету на основе текущей системной даты и времени.
Date.IsInCurrentMonth	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение текущего месяца, по расчету на основе текущей системной даты и времени.
Date.IsInCurrentQuarter	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение текущего квартала, по расчету на основе текущей системной даты и времени.
Date.IsInCurrentWeek	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение текущей недели, по расчету на основе текущей системной даты и времени.
Date.IsInCurrentYear	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение текущего года, по расчету на основе текущей системной даты и времени.
Date.IsInNextDay	Указывает, наступает ли данное значение datetime dateTime в течение следующего дня, по расчету на основе текущей системной даты и времени.
Date.IsInNextMonth	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение следующего месяца, по расчету на основе текущей системной даты и времени.
Date.IsInNextNDays	Указывает, наступает ли данное значение datetime dateTime в течение следующего количества дней, по расчету на основе текущей системной даты и времени.

ФУНКЦИЯ	ОПИСАНИЕ
Date.IsInNextNMonths	Указывает, наступает ли данное значение datetime dateTime в течение следующего количества месяцев, по расчету на основе текущей системной даты и времени.
Date.IsInNextNQuarters	Указывает, наступает ли данное значение datetime dateTime в течение следующего количества кварталов, по расчету на основе текущей системной даты и времени.
Date.IsInNextNWeeks	Указывает, наступает ли данное значение datetime dateTime в течение следующего количества недель, по расчету на основе текущей системной даты и времени.
Date.IsInNextNYears	Указывает, наступает ли данное значение datetime dateTime в течение следующего количества лет, по расчету на основе текущей системной даты и времени.
Date.IsInNextQuarter	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение следующего квартала, по расчету на основе текущей системной даты и времени.
Date.IsInNextWeek	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение следующей недели, по расчету на основе текущей системной даты и времени.
Date.IsInNextYear	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение следующего года, по расчету на основе текущей системной даты и времени.
Date.IsInPreviousDay	Указывает, наступает ли данное значение datetime dateTime в течение предыдущего дня, по расчету на основе текущей системной даты и времени.
Date.IsInPreviousMonth	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение предыдущего месяца, по расчету на основе текущей системной даты и времени.
Date.IsInPreviousNDays	Указывает, наступает ли данное значение datetime dateTime в течение предыдущего количества дней, по расчету на основе текущей системной даты и времени.
Date.IsInPreviousNMonths	Указывает, наступает ли данное значение datetime dateTime в течение предыдущего количества месяцев, по расчету на основе текущей системной даты и времени.

ФУНКЦИЯ	ОПИСАНИЕ
Date.IsInPreviousNQuarters	Указывает, наступает ли данное значение datetime dateTime в течение предыдущего количества кварталов, по расчету на основе текущей системной даты и времени.
Date.IsInPreviousNWeeks	Указывает, наступает ли данное значение datetime dateTime в течение предыдущего количества недель, по расчету на основе текущей системной даты и времени.
Date.IsInPreviousNYears	Указывает, наступает ли данное значение datetime dateTime в течение предыдущего количества лет, по расчету на основе текущей системной даты и времени.
Date.IsInPreviousQuarter	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение предыдущего квартала, по расчету на основе текущей системной даты и времени.
Date.IsInPreviousWeek	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение предыдущей недели, по расчету на основе текущей системной даты и времени.
Date.IsInPreviousYear	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в течение предыдущего года, по расчету на основе текущей системной даты и времени.
Date.IsInYearToDate	Возвращает логическое значение, которое указывает, наступает ли данное значение Date/DateTime/DateTimeZone в период, начинающийся с 1 января текущего года и заканчивающийся текущим днем, по расчету на основе текущей системной даты и времени.
Date.IsLeapYear	Возвращает логическое значение, указывающее, является ли год в значении DateTime високосным годом.
Date.Month	Возвращает месяц из значения DateTime.
Date.MonthName	Возвращает название компонента месяца.
Date.QuarterOfYear	Возвращает число от 1 до 4, обозначающее квартал года из значения DateTime.
Date.StartOfDay	Возвращает значение DateTime, обозначающее начало дня.
Date.StartOfMonth	Возвращает значение DateTime, обозначающее начало месяца.

ФУНКЦИЯ	ОПИСАНИЕ
Date.StartOfQuarter	Возвращает значение DateTime, обозначающее начало квартала.
Date.StartOfWeek	Возвращает значение DateTime, обозначающее начало недели.
Date.StartOfYear	Возвращает значение DateTime, обозначающее начало года.
Date.ToRecord	Возвращает запись, содержащую части значения даты.
Date.ToText	Возвращает текстовое значение из значения даты.
Date.WeekOfMonth	Возвращает число недель в текущем месяце.
Date.WeekOfYear	Возвращает число недель в текущем году.
Date.Year	Возвращает год из значения DateTime.
#date	Создает значение даты из года, месяца и дня.

ЗНАЧЕНИЯ ПАРАМЕТРОВ	ОПИСАНИЕ
Day.Sunday	Представляет воскресенье.
Day.Monday	Представляет понедельник.
Day.Tuesday	Представляет вторник.
Day.Wednesday	Представляет среду.
Day.Thursday	Представляет четверг.
Day.Friday	Представляет пятницу.
Day.Saturday	Представляет субботу.

Date.AddDays

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.AddDays(dateTime as any, numberOfDays as number) as any
```

Сведения

Возвращает результат `date`, `datetime` или `datetimezone` добавления дней (`numberOfDays`) к значению `dateTime` типа `datetime`.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, к которому добавляются дни.
- `numberOfDays`: число добавляемых дней.

Пример 1

Добавление 5 дней к значению `date`, `datetime` или `datetimezone`, представляющему дату 14.05.2011.

```
Date.AddDays(#date(2011, 5, 14), 5)
```

```
#date(2011, 5, 19)
```

Date.AddMonths

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.AddMonths(dateTime as any, numberOfMonths as number) as any
```

Сведения

Возвращает результат `date`, `datetime` или `datetimezone` добавления месяцев (`numberOfMonths`) к значению `dateTime` типа `datetime`.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, к которому добавляются месяцы.
- `numberOfMonths`: число добавляемых месяцев.

Пример 1

Добавление 5 месяцев к значению `date`, `datetime` или `datetimezone`, представляющему дату 14.05.2011.

```
Date.AddMonths(#date(2011, 5, 14), 5)
```

```
#date(2011, 10, 14)
```

Пример 2

Добавление 18 месяцев к значению `date`, `datetime` или `datetimezone`, представляющему дату 14.05.2011 и время 08:15:22.

```
Date.AddMonths(#datetime(2011, 5, 14, 8, 15, 22), 18)
```

```
#datetime(2012, 11, 14, 8, 15, 22)
```

Date.AddQuarters

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.AddQuarters(dateTime as any, numberOfQuarters as number) as any
```

Сведения

Возвращает результат `date`, `datetime` или `datetimezone` добавления кварталов (`numberOfQuarters`) к значению `dateTime` типа `datetime`.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, к которому добавляются кварталы.
- `numberOfQuarters`: число добавляемых кварталов.

Пример 1

Добавление 1 квартала к значению `date`, `datetime` или `datetimezone`, представляющему дату 14.05.2011.

```
Date.AddQuarters(#date(2011, 5, 14), 1)
```

```
#date(2011, 8, 14)
```

Date.AddWeeks

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.AddWeeks(dateTime as any, numberOfWeeks as number) as any
```

Сведения

Возвращает результат `date`, `datetime` или `datetimezone` добавления недель (`numberOfWeeks`) к значению `dateTime` типа `datetime`.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, к которому добавляются недели.
- `numberOfWeeks`: число добавляемых недель.

Пример 1

Добавление 2 недель к значению `date`, `datetime` или `datetimezone`, представляющему дату 14.05.2011.

```
Date.AddWeeks(#date(2011, 5, 14), 2)
```

```
#date(2011, 5, 28)
```


Date.AddYears

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.AddYears(dateTime as any, numberOfYears as number) as any
```

Сведения

Возвращает результат `date`, `datetime` или `datetimezone` добавления `numberOfYears` к значению `dateTime` типа `datetime`.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, к которому добавляются годы.
- `numberOfYears`: количество добавляемых лет.

Пример 1

Добавление 4 лет к значению `date`, `datetime` или `datetimezone`, представляющему дату 14.05.2011.

```
Date.AddYears(#date(2011, 5, 14), 4)
```

```
#date(2015, 5, 14)
```

Пример 2

Добавление 10 лет к значению `date`, `datetime` или `datetimezone`, представляющему дату 14.05.2011 и время 08:15:22.

```
Date.AddYears(#datetime(2011, 5, 14, 8, 15, 22), 10)
```

```
#datetime(2021, 5, 14, 8, 15, 22)
```

Date.Day

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.Day(dateTime as any) as nullable number
```

Сведения

Возвращает компонент дня значения `date`, `datetime` или `datetimezone`.

- `dateTime`: Значение `date`, `datetime` или `datetimezone`, из которого извлекается компонент дня.

Пример 1

Получение компонента дня значения `date`, `datetime` или `datetimezone`, представляющего дату 14.05.2011 и время 17:00:00.

```
Date.Day(#datetime(2011, 5, 14, 17, 0, 0))
```

Date.DayOfWeek

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.DayOfWeek(dateTime as any, optional firstDayOfWeek as nullable number) as nullable number
```

Сведения

Возвращает число (от 0 до 6), соответствующее дню недели предоставленного значения `dateTime`.

- `dateTime`: Значение `date`, `datetime` или `datetimezone`.
- `firstDayOfWeek`: значение `Day`, указывающее день, который должен считаться первым днем недели.
Допустимые значения: `Day.Sunday`, `Day.Monday`, `Day.Tuesday`, `Day.Wednesday`, `Day.Thursday`, `Day.Friday` или `Day.Saturday`. Если параметр не указан, используется значение по умолчанию, зависящее от языка и региональных параметров.

Пример 1

Получение дня недели, на который выпадает понедельник, 21 февраля 2011 г., если первым днем недели считается воскресенье.

```
Date.DayOfWeek(#date(2011, 02, 21), Day.Sunday)`
```

1

Пример 2

Получение дня недели, на который выпадает понедельник, 21 февраля 2011 г., если первым днем недели считается понедельник.

```
Date.DayOfWeek(#date(2011, 02, 21), Day.Monday)
```

0

Date.DayOfWeekName

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.DayOfWeekName(date as any, optional culture as nullable text)
```

Сведения

Возвращает название дня недели для указанного значения `date`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение названия дня недели.

```
Date.DayOfWeekName(#date(2011, 12, 31), "en-US")
```

```
"Saturday"
```

Date.DayOfYear

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.DayOfYear(dateTime as any) as nullable number
```

Сведения

Возвращает число, представляющее день года в предоставленном значении `date`, `datetime` или `datetimezone`: `dateTime`.

Пример 1

Номер дня для 1 марта 2011 г. (`#date(2011, 03, 01)`).

```
Date.DayOfYear(#date(2011, 03, 01))
```

Date.DaysInMonth

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.DaysInMonth(dateTime as any) as nullable number
```

Сведения

Возвращает число дней в месяце в значении `date`, `datetime` или `datetimezone``dateTime`.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, для которого возвращается число дней в месяце.

Пример 1

Число дней в декабре, представленное как `#date(2011, 12, 01)`.

```
Date.DaysInMonth(#date(2011, 12, 01))
```

Date.EndOfDay

16.03.2021 • 2 minutes to read

```
Date.EndOfDay(dateTime as any) as any
```

Сведения

Возвращает значение `date`, `datetime` или `datetimezone`, обозначающее конец дня в `dateTime`. Сведения о часовом поясе сохраняются.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, из которого вычисляется конец дня.

Пример 1

Получение конца дня для 14.05.2011 17:00:00.

```
Date.EndOfDay(#datetime(2011, 5, 14, 17, 0, 0))
```

```
#datetime(2011, 5, 14, 23, 59, 59.9999999)
```

Пример 2

Получение конца дня для 17.05.2011 17:00:00-19:00.

```
Date.EndOfDay(#datetimezone(2011, 5, 17, 5, 0, 0, -7, 0))
```

```
#datetimezone(2011, 5, 17, 23, 59, 59.9999999, -7, 0)
```

Date.EndOfMonth

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.EndOfMonth(dateTime as any) as any
```

Сведения

Возвращает последний день месяца в `dateTime`.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, из которого вычисляется конец месяца.

Пример 1

Возвращение конца месяца для 14.05.2011.

```
Date.EndOfMonth(#date(2011, 5, 14))
```

```
#date(2011, 5, 31)
```

Пример 2

Возвращение конца месяца для 17.05.2011 17:00:00-19:00.

```
Date.EndOfMonth(#datetimezone(2011, 5, 17, 5, 0, 0, -7, 0))
```

```
#datetimezone(2011, 5, 31, 23, 59, 59.9999999, -7, 0)
```


Date.EndOfQuarter

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.EndOfQuarter(dateTime as any) as any
```

Сведения

Возвращает значение `date`, `datetime` или `datetimezone`, обозначающее конец квартала в `dateTime`.

Сведения о часовом поясе сохраняются.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, из которого вычисляется конец квартала.

Пример 1

Поиск конца квартала для 10 октября 2011 г., 8:00 (`#datetime(2011, 10, 10, 8, 0, 0)`).

```
Date.EndOfQuarter(#datetime(2011, 10, 10, 8, 0, 0))
```

```
#datetime(2011, 12, 31, 23, 59, 59.9999999)
```

Date.EndOfWeek

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.EndOfWeek(dateTime as any, optional firstDayOfWeek as nullable number) as any
```

Сведения

Возвращает последний день недели в предоставленных значениях `date`, `datetime` или `datetimezone` `dateTime`. Эта функция принимает необязательный `Day`, `firstDayOfWeek`, чтобы задать первый день недели для этого относительного вычисления. По умолчанию используется значение `Day.Sunday`.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, из которого вычисляется последний день недели
- `firstDayOfWeek`: *[необязательно]* Значение `Day.Type`, представляющее первый день недели.
Возможные значения: `Day.Sunday`, `Day.Monday`, `Day.Tuesday`, `Day.Wednesday`, `Day.Thursday`, `Day.Friday` и `Day.Saturday`. По умолчанию используется значение `Day.Sunday`.

Пример 1

Возвращение конца недели для 14.05.2011.

```
Date.EndOfWeek(#date(2011, 5, 14))
```

```
#date(2011, 5, 14)
```

Пример 2

Возвращение конца недели для 17.05.2011 17:00:00-19:00, если воскресенье — первый день недели.

```
Date.EndOfWeek(#datetimezone(2011, 5, 17, 5, 0, 0, -7, 0), Day.Sunday)
```

```
#datetimezone(2011, 5, 21, 23, 59, 59.9999999, -7, 0)
```

Date.EndOfYear

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.EndOfYear(dateTime as any) as any
```

Сведения

Возвращает значение, представляющее конец года в `dateTime`, включая доли секунды. Сведения о часовом поясе сохраняются.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, из которого вычисляется конец года.

Пример 1

Получение конца года для 14.05.2011 17:00:00.

```
Date.EndOfYear(#datetime(2011, 5, 14, 17, 0, 0))
```

```
#datetime(2011, 12, 31, 23, 59, 59.9999999)
```

Пример 2

Получение конца часа для 17.05.2011 17:00:00–19:00.

```
Date.EndOfYear(#datetimezone(2011, 5, 17, 5, 0, 0, -7, 0))
```

```
#datetimezone(2011, 12, 31, 23, 59, 59.9999999, -7, 0)
```

Date.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.From(value as any, optional culture as nullable text) as nullable date
```

Сведения

Возвращает значение `date` из заданного параметра `value`. Можно также указать необязательный параметр `culture` (например, en-US). Если заданный параметр `value` имеет значение `null`, `Date.From` возвращает `null`. Если заданный параметр `value` имеет значение `date`, возвращается `value`. Значения следующих типов можно преобразовать в значение `date`:

- `text`: значение `date` из текстового представления. Дополнительные сведения см. в разделе `Date.FromText`.
- `datetime`: компонент даты `value`.
- `datetimezone`: компонент даты локального эквивалента `datetime` `value`.
- `number`: компонент даты эквивалента `datetime` OLE Automation, выраженный `value`.

Если `value` имеет любой другой тип, возвращается ошибка.

Пример 1

Преобразование `43910` в значение `date`.

```
Date.From(43910)
```

```
#date(2020, 3, 20)
```

Пример 2

Преобразование `#datetime(1899, 12, 30, 06, 45, 12)` в значение `date`.

```
Date.From(#datetime(1899, 12, 30, 06, 45, 12))
```

```
#date(1899, 12, 30)
```

Date.FromText

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.FromText(text as nullable text, optional culture as nullable text) as nullable date
```

Сведения

Создает значение `date` из текстового представления `text` согласно стандарту формата ISO 8601. Можно также указать необязательный параметр `culture` (например, en-US).

- `Date.FromText("2010-02-19")` // дата, гггг-ММ-дд

Пример 1

Преобразование `"December 31, 2010"` в значение даты.

```
Date.FromText("2010-12-31")
```

```
#date(2010, 12, 31)
```

Пример 2

Преобразование `"December 31, 2010"` в значение даты с другим форматом.

```
Date.FromText("2010, 12, 31")
```

```
#date(2010, 12, 31)
```

Пример 3

Преобразование `"December, 2010"` в значение даты.

```
Date.FromText("2010, 12")
```

```
#date(2010, 12, 1)
```

Пример 4

Преобразование `"2010"` в значение даты.

```
Date.FromText("2010")
```

```
#date(2010, 1, 1)
```

Date.IsInCurrentDay

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInCurrentDay(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение даты и времени `dateTime` в течение текущего дня, по расчету на основе текущей системной даты и времени.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример

Определение того, приходится ли текущее системное время на текущий день.

```
Date.IsInCurrentDay(DateTime.FixedLocalNow())
```

```
true
```

Date.IsInCurrentMonth

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInCurrentMonth(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение datetime `dateTime` в течение текущего месяца, по расчету на основе текущей системной даты и времени.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определить, приходится ли текущее системное время на текущий месяц.

```
Date.IsInCurrentMonth(DateTime.FixedLocalNow())
```

```
true
```

Date.IsInCurrentQuarter

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInCurrentQuarter(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение текущего квартала, по расчету на основе текущей системной даты и времени.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли текущее системное время на текущий квартал.

```
Date.IsInCurrentQuarter(DateTime.FixedLocalNow())
```

```
true
```


Date.IsInCurrentWeek

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInCurrentWeek(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение даты и времени `dateTime` в течение текущей недели, по расчету на основе текущей системной даты и времени.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли текущее системное время на текущую неделю.

```
Date.IsInCurrentWeek(DateTime.FixedLocalNow())
```

```
true
```

Date.IsInCurrentYear

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInCurrentYear(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение текущего года, по расчету на основе текущей системной даты и времени.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли текущее системное время на текущий год.

```
Date.IsInCurrentYear(DateTime.FixedLocalNow())
```

```
true
```

Date.IsInNextDay

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInNextDay(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение даты и времени `dateTime` в течение следующего дня, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий день.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли день, следующий после текущего системного времени, на следующий день.

```
Date.IsInNextDay(Date.AddDays(DateTime.FixedLocalNow(), 1))
```

```
true
```

Date.IsInNextMonth

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInNextMonth(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующего месяца, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий месяц.

- `dateTime` : значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли месяц, идущий после текущего системного времени, на следующий месяц.

```
Date.IsInNextMonth(Date.AddMonths(DateTime.FixedLocalNow(), 1))
```

```
true
```

Date.IsInNextNDays

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInNextNDays(dateTime as any, days as number) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующего количества дней, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий день.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.
- `days`: число дней.

Пример 1

Определение того, приходится ли день, идущий после текущего системного времени, на следующие два дня.

```
Date.IsInNextNDays(Date.AddDays(DateTime.FixedLocalNow(), 1), 2)
```

```
true
```

Date.IsInNextNMonths

16.03.2021 • 2 minutes to read

```
Date.IsInNextNMonths(dateTime as any, months as number) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующего количества месяцев, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий месяц.

- `dateTime` : значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.
- `months` : число месяцев.

Пример 1

Определение того, приходится ли месяц, идущий после текущего системного времени, на следующие два месяца.

```
Date.IsInNextNMonths(Date.AddMonths(DateTime.FixedLocalNow(), 1), 2)
```

```
true
```

Date.IsInNextNQuarters

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInNextNQuarters(dateTime as any, quarters as number) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующего количества кварталов, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий квартал.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.
- `quarters`: число кварталов.

Пример 1

Определение того, приходится ли квартал, идущий после текущего системного времени, на следующие два квартала.

```
Date.IsInNextNQuarters(Date.AddQuarters(DateTime.FixedLocalNow(), 1), 2)
```

```
true
```

Date.IsInNextNWeeks

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInNextNWeeks(dateTime as any, weeks as number) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующего количества недель, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую неделю.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.
- `weeks`: количество недель.

Пример 1

Определение того, приходится ли неделя, идущая после текущего системного времени, на следующие две недели.

```
Date.IsInNextNWeeks(Date.AddDays(DateTime.FixedLocalNow(), 7), 2)
```

```
true
```


Date.IsInNextNYears

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInNextNYears(dateTime as any, years as number) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующего количества лет, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий год.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.
- `years`: количество лет.

Пример 1

Определение того, приходится ли год, идущий после текущего системного времени, на следующие два года.

```
Date.IsInNextNYears(Date.AddYears(DateTime.FixedLocalNow(), 1), 2)
```

```
true
```

Date.IsInNextQuarter

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInNextQuarter(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующего квартала, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий квартал.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

####

Пример 1 Определение того, приходится ли квартал, идущий после текущего системного времени, на следующий квартал.

```
Date.IsInNextQuarter(Date.AddQuarters(DateTime.FixedLocalNow(), 1))
```

```
true
```

Date.IsInNextWeek

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInNextWeek(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующей недели, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую неделю.

- `dateTime` : значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли неделя, идущая после текущего системного времени, на следующую неделю.

```
Date.IsInNextWeek(Date.AddDays(DateTime.FixedLocalNow(), 7))
```

```
true
```

Date.IsInNextYear

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInNextYear(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующего года, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий год.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли год, идущий после текущего системного времени, на следующий год.

```
Date.IsInNextYear(Date.AddYears(DateTime.FixedLocalNow(), 1))
```

```
true
```

Date.IsInPreviousDay

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInPreviousDay(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение предыдущего дня, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий день.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли день, идущий до текущего системного времени, на предыдущий день.

```
Date.IsInPreviousDay(Date.AddDays(DateTime.FixedLocalNow(), -1))
```

```
true
```

Date.IsInPreviousMonth

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInPreviousMonth(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение предыдущего месяца, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий месяц.

- `dateTime` : значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли месяц, идущий до текущего системного времени, на предыдущий месяц.

```
Date.IsInPreviousMonth(Date.AddMonths(DateTime.FixedLocalNow(), -1))
```

```
true
```

Date.IsInPreviousNDays

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInPreviousNDays(dateTime as any, days as number) as nullable logical
```

Сведения

Указывает, приходится ли заданное значение `dateTime` типа `datetime` на определенное число предыдущих дней, исходя из текущих даты и времени в системе. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий день.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.
- `days`: число дней.

Пример 1

Определение того, приходится ли день, идущий до текущего системного времени, на предыдущие два дня.

```
Date.IsInPreviousNDays(Date.AddDays(DateTime.FixedLocalNow(), -1), 2)
```

```
true
```

Date.IsInPreviousNMonths

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInPreviousNMonths(dateTime as any, months as number) as nullable logical
```

Сведения

Указывает, приходится ли заданное значение `dateTime` типа `datetime` на определенное число предыдущих месяцев, исходя из текущих даты и времени в системе. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий месяц.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.
- `months`: число месяцев.

Пример 1

Определение того, находится ли месяц до текущего системного времени в пределах предыдущих двух месяцев.

```
Date.IsInPreviousNMonths(Date.AddMonths(DateTime.FixedLocalNow(), -1), 2)
```

```
true
```


Date.IsInPreviousNQuarters

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInPreviousNQuarters(dateTime as any, quarters as number) as nullable logical
```

Сведения

Указывает, приходится ли заданное значение `dateTime` типа `datetime` на определенное число предыдущих кварталов, исходя из текущих даты и времени в системе. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий квартал.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.
- `quarters`: число кварталов.

Пример 1

Определение того, приходится ли квартал, идущий до текущего системного времени, на предыдущие два квартала.

```
Date.IsInPreviousNQuarters(Date.AddQuarters(DateTime.FixedLocalNow(), -1), 2)
```

```
true
```

Date.IsInPreviousNWeeks

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInPreviousNWeeks(dateTime as any, weeks as number) as nullable logical
```

Сведения

Указывает, приходится ли заданное значение `dateTime` типа `datetime` на определенное число предыдущих недель, исходя из текущих даты и времени в системе. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую неделю.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.
- `weeks`: количество недель.

Пример 1

Определение того, приходится ли неделя до текущего системного времени на предыдущие две недели.

```
Date.IsInPreviousNWeeks(Date.AddDays(DateTime.FixedLocalNow(), -7), 2)
```

```
true
```

Date.IsInPreviousNYears

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInPreviousNYears(dateTime as any, years as number) as nullable logical
```

Сведения

Указывает, приходится ли заданное значение `dateTime` типа `datetime` на определенное число предыдущих лет, исходя из текущих даты и времени в системе. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий год.

- `dateTime` : значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.
- `years` : количество лет.

Пример 1

Определение того, приходится ли год, идущий до текущего системного времени, на предыдущие два года.

```
Date.IsInPreviousNYears(Date.AddYears(DateTime.FixedLocalNow(), -1), 2)
```

```
true
```

Date.IsInPreviousQuarter

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInPreviousQuarter(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение datetime `dateTime` в течение предыдущего квартала, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение false при передаче значения, которое наступает в текущий квартал.

- `dateTime` : значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли квартал, идущий до текущего системного времени, на предыдущий квартал.

```
Date.IsInPreviousQuarter(Date.AddQuarters(DateTime.FixedLocalNow(), -1))
```

```
true
```

Date.IsInPreviousWeek

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInPreviousWeek(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение предыдущей недели, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую неделю.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли неделя, идущая до текущего системного времени, на предыдущую неделю.

```
Date.IsInPreviousWeek(Date.AddDays(DateTime.FixedLocalNow(), -7))
```

```
true
```

Date.IsInPreviousYear

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInPreviousYear(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение datetime `dateTime` в течение предыдущего года, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение false при передаче значения, которое наступает в текущий год.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли год, идущий до текущего системного времени, на предыдущий год.

```
Date.IsInPreviousYear(Date.AddYears(DateTime.FixedLocalNow(), -1))
```

```
true
```

Date.IsInYearToDate

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsInYearToDate(dateTime as any) as nullable logical
```

Сведения

Указывает, приходится ли данное значение `dateTime` типа `datetime` на текущий год и наступает ли оно в течение текущего дня или после него, по расчету на основе текущей системной даты и времени.

- `dateTime`: значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, относится ли текущее системное время к периоду, прошедшему с начала года.

```
Date.IsInYearToDate(DateTime.FixedLocalNow())
```

```
true
```

Date.IsLeapYear

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.IsLeapYear(dateTime as any) as nullable logical
```

Сведения

Указывает, попадает ли заданное значение datetime `dateTime` в високосный год.

- `dateTime` : значение `date`, `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, является ли 2012 год, представленный `#date(2012, 01, 01)`, високосным.

```
Date.IsLeapYear(#date(2012, 01, 01))
```

```
true
```


Date.Month

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.Month(dateTime as any) as nullable number
```

Сведения

Возвращает компонент месяца заданного значения `datetime` — `dateTime`.

Пример 1

Поиск компонента месяца в `#datetime(2011, 12, 31, 9, 15, 36)`.

```
Date.Month(#datetime(2011, 12, 31, 9, 15, 36))
```

Date.MonthName

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.MonthName(date as any, optional culture as nullable text) as nullable text
```

Сведения

Возвращает название компонента месяца для указанного значения `date`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример

Получение названия месяца.

```
Date.MonthName(#datetime(2011, 12, 31, 5, 0, 0), "en-US")
```

```
"December"
```

Date.QuarterOfYear

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.QuarterOfYear(dateTime as any) as nullable number
```

Сведения

Возвращает число от 1 до 4, указывающее квартал года, на который приходится дата `dateTime`. Значение `dateTime` может иметь тип `date`, `datetime` или `datetimezone`.

Пример 1

Поиск квартала года для даты в `#date(2011, 12, 31)`.

```
Date.QuarterOfYear(#date(2011, 12, 31))
```

Date.StartOfDay

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.StartOfDay(dateTime as any) as any
```

Сведения

Возвращает первое значение дня `dateTime`. `dateTime` должно быть значением `date`, `datetime` или `datetimezone`.

Пример 1

Поиск начала дня для 10 октября 2011 г., 8:00 (`#datetime(2011, 10, 10, 8, 0, 0)`).

```
Date.StartOfDay(#datetime(2011, 10, 10, 8, 0, 0))
```

```
#datetime(2011, 10, 10, 0, 0, 0)
```

Date.StartOfMonth

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.StartOfMonth(dateTime as any) as any
```

Сведения

Возвращает первое значение месяца с учетом типа `date` или `datetime`.

Пример 1

Поиск начала месяца для 10 октября 2011 г., 8:10:32 (`#datetime(2011, 10, 10, 8, 10, 32)`).

```
Date.StartOfMonth(#datetime(2011, 10, 10, 8, 10, 32))
```

```
#datetime(2011, 10, 1, 0, 0, 0)
```

Date.StartOfQuarter

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.StartOfQuarter(dateTime as any) as any
```

Сведения

Возвращает первое значение квартала < `dateTime` . `dateTime` должно быть значением `date` , `datetime` или `datetimezone` .

Пример 1

Поиск начала квартала для 10 октября 2011 г., 8:00 (`#datetime(2011, 10, 10, 8, 0, 0)`).

```
Date.StartOfQuarter(#datetime(2011, 10, 10, 8, 0, 0))
```

```
#datetime(2011, 10, 1, 0, 0, 0)
```

Date.StartOfWeek

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.StartOfWeek(dateTime as any, optional firstDayOfWeek as nullable number) as any
```

Сведения

Возвращает первое значение недели с учетом значения `date` , `datetime` или `datetimezone` .

Пример 1

Поиск начала недели для 10 октября 2011 г., 8:10:32 (`#datetime(2011, 10, 10, 8, 10, 32)`).

```
Date.StartOfWeek(#datetime(2011, 10, 10, 8, 10, 32))
```

```
#datetime(2011, 10, 9, 0, 0, 0)
```

Date.StartOfYear

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.StartOfYear(dateTime as any) as any
```

Сведения

Возвращает первое значение года с учетом значения `date`, `datetime` или `datetimezone`.

Пример 1

Поиск начала года для 10 октября 2011 г., 8:10:32 (`#datetime(2011, 10, 10, 8, 10, 32)`).

```
Date.StartOfYear(#datetime(2011, 10, 10, 8, 10, 32))
```

```
#datetime(2011, 1, 1, 0, 0, 0)
```


Date.ToRecord

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.ToRecord(date as date) as record
```

Сведения

Возвращает запись, содержащую части заданного значения даты, `date`.

- `date`: значение `date`, для которого вычисляется запись его частей.

Пример 1

Преобразование значения `#date(2011, 12, 31)` в запись, содержащую части значения даты.

```
Date.ToRecord(#date(2011, 12, 31))
```

ГОД	2011
МЕСЯЦ	12
ДЕНЬ	31

Date.ToText

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.ToText(date as nullable date, optional format as nullable text, optional culture as nullable text) as nullable text
```

О программе

Возвращает текстовое представление `date`. Для форматирования текста можно указать необязательный параметр `format`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение текстового представления `#date` (2010, 12, 31).

```
Date.ToText(#date(2010, 12, 31))
```

```
"12/31/2010"
```

Пример 2

Получение текстового представления `#date(2010, 12, 31)` с возможностью форматирования.

```
Date.ToText(#date(2010, 12, 31), "yyyy/MM/dd")
```

```
"2010/12/31"
```

Date.WeekOfMonth

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.WeekOfMonth(dateTime as any, optional firstDayOfWeek as nullable number) as nullable number
```

Сведения

Возвращает число от 1 до 5, указывающее неделю месяца, на которую приходится дата `dateTime`.

- `dateTime`: Значение `datetime`, для которого определяется неделя месяца.

Пример 1

Определение того, на какую неделю марта приходится 15 марта 2011 г. (`#date(2011, 03, 15)`).

```
Date.WeekOfMonth(#date(2011, 03, 15))
```

Date.WeekOfYear

16.03.2021 • 2 minutes to read

Синтаксис

```
Date.WeekOfYear(dateTime as any, optional firstDayOfWeek as nullable number) as nullable number
```

Сведения

Возвращает число от 1 до 54, указывающее неделю в году, на которую приходится эта дата `dateTime`.

- `dateTime`: значение `datetime`, для которого определяется неделя года.
- `firstDayOfWeek`: необязательное значение `Day.Type`, указывающее день, который считается началом новой недели (например, `Day.Sunday`). Если параметр не указан, используется значение по умолчанию, зависящее от языка и региональных параметров.

Пример 1

Определение того, на какую неделю в году приходится 27 марта 2011 г. (`#date(2011, 03, 27)`).

```
Date.WeekOfYear(#date(2011, 03, 27))
```

14

Пример 2

Определение того, на какую неделю в году приходится 27 марта 2011 г. (`#date(2011, 03, 27)`), если понедельник считается первым днем недели.

```
Date.WeekOfYear(#date(2011, 03, 27), Day.Monday)
```

13

Date.Year

16.03.2021 • 2 minutes to read

```
Date.Year(dateTime as any) as nullable number
```

Сведения

Возвращает компонент года заданного значения `datetime`, `dateTime`.

Пример 1

Поиск компонента года в значении `#datetime(2011, 12, 31, 9, 15, 36)`.

```
Date.Year(#datetime(2011, 12, 31, 9, 15, 36))
```

```
2011
```

Day.Friday

16.03.2021 • 2 minutes to read

Сведения

Возвращает 6, число, обозначающее пятницу.

Day.Monday

16.03.2021 • 2 minutes to read

Сведения

Возвращает 2, число, обозначающее понедельник.

Day.Saturday

16.03.2021 • 2 minutes to read

Сведения

Возвращает 7, число, обозначающее субботу.

Day.Sunday

16.03.2021 • 2 minutes to read

Сведения

Возвращает 1, число, обозначающее воскресенье.

Day.Thursday

16.03.2021 • 2 minutes to read

Сведения

Возвращает 5, число, обозначающее четверг.

Day.Tuesday

16.03.2021 • 2 minutes to read

Сведения

Возвращает 3, число, обозначающее вторник.

Day.Wednesday

16.03.2021 • 2 minutes to read

Сведения

Возвращает 4, число, обозначающее среду.

#date

16.03.2021 • 2 minutes to read

Синтаксис

```
#date(year as number, month as number, day as number) as date
```

Сведения

Создает значение даты из года `year`, месяца `month` и дня `day`. Вызывает ошибку, если не соблюдаются следующие условия:

- $1 \leq \text{год} \leq 9999$
- $1 \leq \text{месяц} \leq 12$
- $1 \leq \text{день} \leq 31$

Функции для работы с датами и временем

16.03.2021 • 4 minutes to read

Эти функции позволяют создавать значения параметров `datetime` и `datetimezone` и управлять ими.

Дата и время

ФУНКЦИЯ	ОПИСАНИЕ
DateTime.AddZone	Добавляет значение <code>timezonehours</code> в качестве смещения для входного значения <code>datetime</code> и возвращает новое значение <code>datetimezone</code> .
DateTime.Date	Возвращает часть даты из значения <code>DateTime</code> .
DateTime.FixedLocalNow	Возвращает значение <code>DateTime</code> , равное текущей дате и времени в системе.
DateTime.From	Возвращает значение <code>datetime</code> из значения.
DateTime.FromFileTime	Возвращает значение <code>DateTime</code> из указанного числа.
DateTime.FromText	Возвращает значение <code>DateTime</code> из набора форматов даты и значения языка и региональных параметров.
DateTime.IsInCurrentHour	Указывает, наступает ли данное значение <code>datetime</code> в течение текущего часа, по расчету на основе текущей системной даты и времени.
DateTime.IsInCurrentMinute	Указывает, наступает ли данное значение <code>datetime</code> в течение текущей минуты, по расчету на основе текущей системной даты и времени.
DateTime.IsInCurrentSecond	Указывает, наступает ли данное значение <code>datetime</code> в течение текущей секунды, по расчету на основе текущей системной даты и времени.
DateTime.IsInNextHour	Указывает, наступает ли данное значение <code>datetime</code> в течение следующего часа, по расчету на основе текущей системной даты и времени.
DateTime.IsInNextMinute	Указывает, наступает ли данное значение <code>datetime</code> в течение следующей минуты, по расчету на основе текущей системной даты и времени.
DateTime.IsInNextNHours	Указывает, наступает ли данное значение <code>datetime</code> в течение следующего количества часов, по расчету на основе текущей системной даты и времени.
DateTime.IsInNextNMinutes	Указывает, наступает ли данное значение <code>datetime</code> в течение следующего количества минут, по расчету на основе текущей системной даты и времени.

ФУНКЦИЯ	ОПИСАНИЕ
DateTime.IsInNextNSeconds	Указывает, наступает ли данное значение datetime в течение следующего количества секунд, по расчету на основе текущей системной даты и времени.
DateTime.IsInNextSecond	Указывает, наступает ли данное значение datetime в течение следующей секунды, по расчету на основе текущей системной даты и времени.
DateTime.IsInPreviousHour	Указывает, наступает ли данное значение datetime в течение предыдущего часа, по расчету на основе текущей системной даты и времени.
DateTime.IsInPreviousMinute	Указывает, наступает ли данное значение datetime в течение предыдущей минуты, по расчету на основе текущей системной даты и времени.
DateTime.IsInPreviousNHours	Указывает, приходится ли заданное значение datetime на определенное число предыдущих часов, исходя из текущих даты и времени в системе.
DateTime.IsInPreviousNMinutes	Указывает, приходится ли заданное значение datetime на определенное число предыдущих минут, исходя из текущих даты и времени в системе.
DateTime.IsInPreviousNSeconds	Указывает, приходится ли заданное значение datetime на определенное число предыдущих секунд, исходя из текущих даты и времени в системе.
DateTime.IsInPreviousSecond	Указывает, наступает ли данное значение datetime в течение предыдущей секунды, по расчету на основе текущей системной даты и времени.
DateTime.LocalNow	Возвращает значение datetime, равное текущей дате и времени в системе.
DateTime.Time	Возвращает часть времени из значения DateTime.
DateTime.ToRecord	Возвращает запись, содержащую части значения DateTime.
DateTime.ToText	Возвращает текстовое значение из значения DateTime.
#datetime	Создает значение datetime из года, месяца, дня, часов, минут и секунд.

DateTime.AddZone

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.AddZone(dateTime as nullable datetime, timezoneHours as number, optional  
timezoneMinutes as nullable number) as nullable datetimetimezone
```

Сведения

Задаёт сведения о часовом поясе для значения даты и времени `dateTime`. В сведения о часовом поясе входит `timezoneHours` и `timezoneMinutes` (второе необязательно).

Пример 1

Задание в поле сведений о часовом поясе для `#datetime(2010, 12, 31, 11, 56, 02)` значение "7 часов 30 минут".

```
DateTime.AddZone(#datetime(2010, 12, 31, 11, 56, 02), 7, 30)
```

```
#datetimetimezone(2010, 12, 31, 11, 56, 2, 7, 30)
```


DateTime.Date

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.Date(dateTime as any) as nullable date
```

Сведения

Возвращает компонент даты `dateTime`, заданное значение `date`, `datetime` или `datetimezone`.

Пример 1

Поиск значение даты `#datetime (2010, 12, 31, 11, 56, 02)`.

```
DateTime.Date(#datetime(2010, 12, 31, 11, 56, 02))
```

```
#date(2010, 12, 31)
```

DateTime.FixedLocalNow

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.FixedLocalNow() as datetime
```

Сведения

Возвращает значение `datetime`, равное текущей дате и времени в системе. Это значение зафиксировано и не меняется при последующих вызовах в отличие от `DateTime.LocalNow`, которое может возвращать разные значения по мере выполнения выражения.

DateTime.From

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.From(value as any, optional culture as nullable text) as nullable datetime
```

Сведения

Возвращает значение `datetime` из заданного параметра `value`. Можно также указать необязательный параметр `culture` (например, en-US). Если заданный параметр `value` имеет значение `null`, `DateTime.From` возвращает `null`. Если заданный параметр `value` имеет значение `datetime`, возвращается `value`. Значения следующих типов можно преобразовать в значение `datetime`:

- `text`: значение `datetime` из текстового представления. Дополнительные сведения см. в разделе `DateTime.FromText`.
- `date`: `datetime` с `value` в качестве компонента даты и `12:00:00 AM` в качестве компонента времени.
- `datetimezone`: `datetime`, локальный эквивалент `value`.
- `time`: `datetime` с эквивалентом даты OLE-автоматизации, `0` в качестве компонента даты и `value` в качестве компонента времени.
- `number`: `datetime` эквивалент даты OLE-автоматизации, выраженный `value`.

Если `value` имеет любой другой тип, возвращается ошибка.

Пример 1

Преобразование `#time(06, 45, 12)` в значение `datetime`.

```
DateTime.From(#time(06, 45, 12))
```

```
#datetime(1899, 12, 30, 06, 45, 12)
```

Пример 2

Преобразование `#date(1975, 4, 4)` в значение `datetime`.

```
DateTime.From(#date(1975, 4, 4))
```

```
#datetime(1975, 4, 4, 0, 0, 0)
```

DateTime.FromFileTime

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.FromFileTime(fileTime as nullable number) as nullable datetime
```

Сведения

Создает значение `datetime` из значения `fileTime` и преобразует его в соответствии с местным часовым поясом. Значение `filetime` — время файла Windows, представляющее количество 100-наносекундных интервалов, прошедших с 00:00 1 января 1601 г. нашей эры в формате UTC.

Пример 1

Преобразование `129876402529842245` в значение `datetime`.

```
DateTime.FromFileTime(129876402529842245)
```

```
#datetime(2012, 7, 24, 14, 50, 52.9842245)
```

DateTime.FromText

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.FromText(text as nullable text, optional culture as nullable text) as nullable datetime
```

Сведения

Создает значение `datetime` из текстового представления `text` согласно стандарту формата ISO 8601.

Можно также указать необязательный параметр `culture` (например, en-US).

- `DateTime.FromText("2010-12-31T01:30:00")` // гggг-ММ-ддТчч:мм:сс

Пример 1

Преобразование "2010-12-31T01:30:25" в значение datetime.

```
DateTime.FromText("2010-12-31T01:30:25")
```

```
#datetime(2010, 12, 31, 1, 30, 25)
```

Пример 2

Преобразование "2010-12-31T01:30" в значение datetime.

```
DateTime.FromText("2010-12-31T01:30")
```

```
#datetime(2010, 12, 31, 1, 30, 0)
```

Пример 3

Преобразование "20101231T013025" в значение datetime.

```
DateTime.FromText("20101231T013025")
```

```
#datetime(2010, 12, 31, 1, 30, 25)
```

Пример 4

Преобразование "20101231T01:30:25" в значение datetime.

```
DateTime.FromText("20101231T01:30:25")
```

```
#datetime(2010, 12, 31, 1, 30, 25)
```

Пример 5

Преобразование "20101231T01:30:25.121212" в значение datetime.

```
DateTime.FromText("20101231T01:30:25.121212")
```

```
#datetime(2010, 12, 31, 1, 30, 25.121212)
```

DateTime.IsInCurrentHour

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInCurrentHour(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение текущего часа, по расчету на основе текущей системной даты и времени.

- `dateTime`: значение `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли текущее системное время на текущий час.

```
DateTime.IsInCurrentHour(DateTime.FixedLocalNow())
```

```
true
```

DateTime.IsInCurrentMinute

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInCurrentMinute(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение текущей минуты, по расчету на основе текущей системной даты и времени.

- `dateTime`: значение `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли текущее системное время на текущую минуту.

```
DateTime.IsInCurrentMinute(DateTime.FixedLocalNow())
```

```
true
```


DateTime.IsInCurrentSecond

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInCurrentSecond(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение даты и времени `dateTime` в течение текущей секунды, по расчету на основе текущей системной даты и времени.

- `dateTime`: значение `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, находится ли текущее системное время в текущей секунде.

```
DateTime.IsInCurrentSecond(DateTime.FixedLocalNow())
```

```
true
```

DateTime.IsInNextHour

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInNextHour(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующего часа, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий час.

- `dateTime` : значение `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли час, идущий после текущего системного времени, на следующий час.

```
DateTime.IsInNextHour(DateTime.FixedLocalNow() + #duration(0, 1, 0, 0))
```

```
true
```

DateTime.IsInNextMinute

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInNextMinute(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `datetime` `dateTime` в течение следующей минуты, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую минуту.

- `dateTime` : значение `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, находится ли минута, следующая за текущим системным временем, в следующей минуте.

```
DateTime.IsInNextMinute(DateTime.FixedLocalNow() + #duration(0, 0, 1, 0))
```

```
true
```

DateTime.IsInNextNHours

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInNextNHours(dateTime as any, hours as number) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующего количества часов, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий час.

- `dateTime` : значение `datetime` или `datetimezone`, которое необходимо вычислить.
- `hours` : количество часов.

Пример 1

Определение того, приходится ли час, идущий после текущего системного времени, на следующие два часа.

```
DateTime.IsInNextNHours(DateTime.FixedLocalNow() + #duration(0, 2, 0, 0), 2)
```

```
true
```

DateTime.IsInNextNMinutes

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInNextNMinutes(dateTime as any, minutes as number) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `datetime` `dateTime` в течение следующего количества минут, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую минуту.

- `dateTime`: значение `datetime` или `datetimezone`, которое необходимо вычислить.
- `minutes`: количество минут.

Пример 1

Определяет, находится ли минута, следующая за текущим системным временем, в следующих двух минутах.

```
DateTime.IsInNextNMinutes(DateTime.FixedLocalNow() + #duration(0, 0, 2, 0), 2)
```

```
true
```

DateTime.IsInNextNSeconds

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInNextNSeconds(dateTime as any, seconds as number) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующего количества секунд, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую секунду.

- `dateTime` : значение `datetime` или `datetimezone`, которое необходимо вычислить.
- `seconds` : количество секунд.

Пример 1

Определение того, приходится ли секунда, идущая после текущего системного времени, на следующие две секунды.

```
DateTime.IsInNextNSeconds(DateTime.FixedLocalNow() + #duration(0, 0, 0, 2), 2)
```

```
true
```

DateTime.IsInNextSecond

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInNextSecond(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение следующей секунды, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую секунду.

- `dateTime` : значение `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли секунда, идущая после текущего системного времени, на следующую секунду.

```
DateTime.IsInNextSecond(DateTime.FixedLocalNow() + #duration(0, 0, 0, 1))
```

```
true
```

DateTime.IsInPreviousHour

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInPreviousHour(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение предыдущего часа, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий час.

- `dateTime`: значение `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли час, идущий до текущего системного времени, на предыдущий час.

```
DateTime.IsInPreviousHour(DateTime.FixedLocalNow() - #duration(0, 1, 0, 0))
```

```
true
```


DateTime.IsInPreviousMinute

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInPreviousMinute(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение предыдущей минуты, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую минуту.

- `dateTime` : значение `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли минута, идущая до текущего системного времени, на предыдущую минуту.

```
DateTime.IsInPreviousMinute(DateTime.FixedLocalNow() - #duration(0, 0, 1, 0))
```

```
true
```

DateTime.IsInPreviousNHours

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInPreviousNHours(dateTime as any, hours as number) as nullable logical
```

Сведения

Указывает, приходится ли заданное значение `dateTime` типа `datetime` на определенное число предыдущих часов, исходя из текущих даты и времени в системе. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущий час.

- `dateTime` : значение `datetime` или `datetimezone`, которое необходимо вычислить.
- `hours` : количество часов.

Пример 1

Определяет, находится ли час, предшествующий текущему системному времени, в предыдущих двух часах.

```
DateTime.IsInPreviousNHours(DateTime.FixedLocalNow() - #duration(0, 2, 0, 0), 2)
```

```
true
```

DateTime.IsInPreviousNMinutes

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInPreviousNMinutes(dateTime as any, minutes as number) as nullable logical
```

Сведения

Указывает, приходится ли заданное значение `dateTime` типа `datetime` на определенное число предыдущих минут, исходя из текущих даты и времени в системе. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую минуту.

- `dateTime` : значение `datetime` или `datetimezone`, которое необходимо вычислить.
- `minutes` : количество минут.

Пример 1

Определение того, приходится ли минута, идущая до текущего системного времени, на предыдущие две минуты.

```
DateTime.IsInPreviousNMinutes(DateTime.FixedLocalNow() - #duration(0, 0, 2, 0), 2)
```

```
true
```

DateTime.IsInPreviousNSeconds

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInPreviousNSeconds(dateTime as any, seconds as number) as nullable logical
```

Сведения

Указывает, приходится ли заданное значение `dateTime` типа `datetime` на определенное число предыдущих секунд, исходя из текущих даты и времени в системе. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую секунду.

- `dateTime` : значение `datetime` или `datetimezone`, которое необходимо вычислить.
- `seconds` : количество секунд.

Пример 1

Определение того, приходится ли секунда, предшествующая текущему системному времени, на предыдущие две секунды.

```
DateTime.IsInPreviousNSeconds(DateTime.FixedLocalNow() - #duration(0, 0, 0, 2), 2)
```

```
true
```

DateTime.IsInPreviousSecond

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.IsInPreviousSecond(dateTime as any) as nullable logical
```

Сведения

Указывает, наступает ли данное значение `dateTime` типа `datetime` в течение предыдущей секунды, по расчету на основе текущей системной даты и времени. Обратите внимание, что эта функция будет возвращать значение `false` при передаче значения, которое наступает в текущую секунду.

- `dateTime`: значение `datetime` или `datetimezone`, которое необходимо вычислить.

Пример 1

Определение того, приходится ли секунда, идущая до текущего системного времени, на предыдущую секунду.

```
DateTime.IsInPreviousSecond(DateTime.FixedLocalNow() - #duration(0, 0, 0, 1))
```

```
true
```

DateTime.LocalNow

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.LocalNow() as datetime
```

Сведения

Возвращает значение `datetime`, равное текущей дате и времени в системе.

DateTime.Time

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.Time(dateTime as any) as nullable time
```

Сведения

Возвращает компонент времени заданного значения datetime `dateTime`.

Пример 1

Поиск значения времени #datetime (2010, 12, 31, 11, 56, 02).

```
DateTime.Time(#datetime(2010, 12, 31, 11, 56, 02))
```

```
#time(11, 56, 2)
```

DateTime.ToRecord

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.ToRecord(dateTime as datetime) as record
```

Сведения

Возвращает запись, содержащую части заданного значения datetime, `dateTime`.

- `dateTime`: значение `datetime`, для которого вычисляется запись его частей.

Пример 1

Преобразование значения `#datetime(2011, 12, 31, 11, 56, 2)` в запись, содержащую значения даты и времени.

```
DateTime.ToRecord(#datetime(2011, 12, 31, 11, 56, 2))
```

ГОД	2011
МЕСЯЦ	12
ДЕНЬ	31
ЧАС	11
МИНУТА	56
СЕКУНДА	2

DateTime.ToText

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTime.ToText(dateTime as nullable datetime, optional format as nullable text, optional culture as nullable text) as nullable text
```

О программе

Возвращает текстовое представление `dateTime`. Для форматирования текста можно указать необязательный параметр `format`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение текстового представления `#datetime(2011, 12, 31, 11, 56, 2)`.

```
DateTime.ToText(#datetime(2010, 12, 31, 11, 56, 2))
```

```
"12/31/2010 11:56:02 AM"
```

Пример 2

Получение текстового представления `#datetime(2011, 12, 31, 11, 56, 2)` с возможностью форматирования.

```
DateTime.ToText(#datetime(2010, 12, 31, 11, 56, 2), "yyyy/MM/ddThh:mm:ss")
```

```
"2010/12/31T11:56:02"
```

#datetime

16.03.2021 • 2 minutes to read

Синтаксис

```
#datetime(year as number, month as number, day as number, hour as number, minute as number, second as number) as any
```

Сведения

Создает значение datetime из целого числа года `year`, месяца `month`, дня `day`, часа `hour`, минуты `minute` и секунды `second` (дробное значение). Вызывает ошибку, если не соблюдаются следующие условия:

- $1 \leq \text{год} \leq 9999$
- $1 \leq \text{месяц} \leq 12$
- $1 \leq \text{день} \leq 31$
- $0 \leq \text{час} \leq 23$
- $0 \leq \text{минута} \leq 59$
- $0 \leq \text{секунда} \leq 59$

Функции для работы с датами, временем и часовыми поясами

16.03.2021 • 2 minutes to read

Эти функции позволяют создавать значения параметра `datetimezone` и управлять ими.

DateTimeZone

ФУНКЦИЯ	ОПИСАНИЕ
DateTimeZone.FixedLocalNow	Возвращает значение <code>DateTimeZone</code> , равное текущим дате, времени и смещению часового пояса в системе.
DateTimeZone.FixedUtcNow	Возвращает текущую дату и время в формате UTC (часовой пояс GMT).
DateTimeZone.From	Возвращает значение <code>datetimezone</code> из значения.
DateTimeZone.FromFileTime	Возвращает значение <code>DateTimeZone</code> из числового значения.
DateTimeZone.FromText	Возвращает значение <code>DateTimeZone</code> из набора форматов даты и значения языка и региональных параметров.
DateTimeZone.LocalNow	Возвращает значение <code>DateTime</code> , равное текущей дате и времени в системе.
DateTimeZone.RemoveZone	Возвращает значение <code>datetime</code> с удаленными данными часового пояса из входного значения <code>datetimezone</code> .
DateTimeZone.SwitchZone	Изменяет данные о часовом поясе из входного значения <code>DateTimeZone</code> .
DateTimeZone.ToLocal	Возвращает значение <code>DateTime</code> из местного часового пояса.
DateTimeZone.ToRecord	Возвращает запись, содержащую части значения <code>DateTime</code> .
DateTimeZone.ToText	Возвращает текстовое значение из значения <code>DateTime</code> .
DateTimeZone.ToUtc	Возвращает значение <code>DateTime</code> для часового пояса UTC.
DateTimeZone.UtcNow	Возвращает значение <code>DateTime</code> , равное текущей дате и времени в системе, в часовом поясе UTC.
DateTimeZone.ZoneHours	Возвращает значение часов часового пояса из значения <code>DateTime</code> .

ФУНКЦИЯ	ОПИСАНИЕ
<code>DateTimeZone.ZoneMinutes</code>	Возвращает значение минут часового пояса из значения <code>DateTime</code> .
<code>#datetimezone</code>	Создает значение <code>datetimezone</code> из года, месяца, дня, часа, минут, секунд и часов смещения и минут смещения.

DateTimeZone.FixedLocalNow

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.FixedLocalNow() as datetimetype
```

Сведения

Возвращает значение `datetime`, равное текущей дате и времени в системе. Возвращаемое значение содержит сведения о часовом поясе, представляющем местный часовой пояс. Это значение зафиксировано и не меняется при последующих вызовах в отличие от `DateTimeZone.LocalNow`, которое может возвращать разные значения по мере выполнения выражения.

DateTimeZone.FixedUtcNow

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.FixedUtcNow() as datetimetype
```

Сведения

Возвращает текущую дату и время в формате UTC (часовой пояс GMT). Это значение зафиксировано и не меняется при последующих вызовах.

DateTimeZone.From

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.From(value as any, optional culture as nullable text) as nullable datetimezone
```

Сведения

Возвращает значение `datetimezone` из заданного параметра `value`. Можно также указать необязательный параметр `culture` (например, en-US). Если заданный параметр `value` имеет значение `null`, `DateTimeZone.From` возвращает `null`. Если заданный параметр `value` имеет значение `datetimezone`, возвращается `value`. Значения следующих типов можно преобразовать в значение `datetimezone`:

- `text`: значение `datetimezone` из текстового представления. Дополнительные сведения см. в разделе `DateTimeZone.FromText`.
- `date`: `datetimezone` с `value` в качестве компонента даты, `12:00:00 AM` в качестве компонента времени и смещением, соответствующим местному часовому поясу.
- `datetime`: `datetimezone` с `value` в качестве `datetime` и смещением, соответствующим местному часовому поясу.
- `time`: `datetimezone` с эквивалентом даты OLE Automation `0` в качестве компонента даты, `value` в качестве компонента времени и смещением, соответствующим местному часовому поясу.
- `number`: `datetimezone` с эквивалентом `datetime` даты OLE Automation, выраженной `value`, и смещением, соответствующим местному часовому поясу.

Если `value` имеет любой другой тип, возвращается ошибка.

Пример 1

Преобразование `"2020-10-30T01:30:00-08:00"` в значение `datetimezone`.

```
DateTimeZone.From("2020-10-30T01:30:00-08:00")
```

```
#datetimezone(2020, 10, 30, 01, 30, 00, -8, 00)
```

DateTimeZone.FromFileTime

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.FromFileTime(fileTime as nullable number) as nullable datetimetypezone
```

Сведения

Создает значение `datetimetypezone` из значения `fileTime` и преобразует его в соответствии с местным часовым поясом. Значение `filetime` — время файла Windows, представляющее количество 100-наносекундных интервалов, прошедших с 00:00 1 января 1601 г. нашей эры в формате UTC.

Пример 1

Преобразование `129876402529842245` в значение `datetimetypezone`.

```
DateTimeZone.FromFileTime(129876402529842245)
```

```
#datetimetype(2012, 7, 24, 14, 50, 52.9842245, -7, 0)
```


DateTimeZone.FromText

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.FromText(text as nullable text, optional culture as nullable text) as nullable datetimetype
```

Сведения

Создает значение `datetimetype` из текстового представления `text` согласно стандарту формата ISO 8601. Можно также указать необязательный параметр `culture` (например, en-US).

- `DateTimeZone.FromText("2010-12-31T01:30:00-08:00")` // гggг-ММ-ддТчч:мм:ccZ

Пример 1

Преобразование `"2010-12-31T01:30:00-08:00"` в значение `datetimetype`.

```
DateTimeZone.FromText("2010-12-31T01:30:00-08:00")
```

```
#datetimetype(2010, 12, 31, 1, 30, 0, -8, 0)
```

Пример 2

Преобразование `"2010-12-31T01:30:00.121212-08:00"` в значение `datetimetype`.

```
DateTimeZone.FromText("2010-12-31T01:30:00.121212-08:00")
```

```
#datetimetype(2010, 12, 31, 1, 30, 0.121212, -8, 0)
```

Пример 3

Преобразование `"2010-12-31T01:30:00Z"` в значение `datetimetype`.

```
DateTimeZone.FromText("2010-12-31T01:30:00Z")
```

```
#datetimetype(2010, 12, 31, 1, 30, 0, 0, 0)
```

Пример 4

Преобразование `"20101231T013000+0800"` в значение `datetimetype`.

```
DateTimeZone.FromText("20101231T013000+0800")
```

```
#datetimetype(2010, 12, 31, 1, 30, 0, 8, 0)
```

DateTimeZone.LocalNow

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.LocalNow() as datetimetypezone
```

Сведения

Возвращает значение `datetimetypezone`, равное текущей дате и времени в системе. Возвращаемое значение содержит сведения о часовом поясе, представляющем местный часовой пояс.

DateTimeZone.RemoveZone

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.RemoveZone(dateTimeZone as nullable datetimetype) as nullable datetime
```

Сведения

Возвращает значение #datetime из `dateTimeZone` с удаленными данными часового пояса.

Пример 1

Удаление данных часового пояса из значения #datetimezone(2011, 12, 31, 9, 15, 36, -7, 0).

```
DateTimeZone.RemoveZone(#datetimezone(2011, 12, 31, 9, 15, 36, -7, 0))
```

```
#datetime(2011, 12, 31, 9, 15, 36)
```

DateTimeZone.SwitchZone

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.SwitchZone(dateTimeZone as nullable datetimezone, timeZoneHours as number, optional  
timeZoneMinutes as nullable number) as nullable datetimezone
```

Сведения

Изменяет компонент часового пояса в значении datetimezone `dateTimeZone` на новый часовой пояс из `timeZoneHours` и при необходимости `timeZoneMinutes`. Если `dateTimeZone` не содержит компонент часового пояса, возникает исключение.

Пример 1

Изменение компонента часового пояса в `#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30)` на 8 часов.

```
DateTimeZone.SwitchZone(#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30), 8)
```

```
#datetimezone(2010, 12, 31, 12, 26, 2, 8, 0)
```

Пример 2

Изменение компонента часового пояса в `#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30)` на -30 минут.

```
DateTimeZone.SwitchZone(#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30), 0, -30)
```

```
#datetimezone(2010, 12, 31, 3, 56, 2, 0, -30)
```

DateTimeZone.ToLocal

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.ToLocal(dateTimeZone as nullable datetimezone) as nullable datetimezone
```

Сведения

Изменяет часовой пояс в значении datetimezone `dateTimeZone` на местный часовой пояс. Если `dateTimeZone` не содержит компонента часового пояса, будет добавлен местный часовой пояс.

Пример 1

Изменение сведений о часовом поясе для #datetimezone (2010, 12, 31, 11, 56, 02, 7, 30) на местный часовой пояс (по тихоокеанскому времени).

```
DateTimeZone.ToLocal(#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30))
```

```
#datetimezone(2010, 12, 31, 12, 26, 2, -8, 0)
```

DateTimeZone.ToRecord

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.ToRecord(dateTimeZone as datetimezone) as record
```

Сведения

Возвращает запись, содержащую части заданного значения datetimezone, **dateTimeZone**.

- **dateTimeZone**: значение **datetimezone**, для которого вычисляется запись его частей.

Пример 1

Преобразование значения **#datetimezone(2011, 12, 31, 11, 56, 2, 8, 0)** в запись, содержащую значения даты, времени и часового пояса.

```
DateTimeZone.ToRecord(#datetimezone(2011, 12, 31, 11, 56, 2, 8, 0))
```

ГОД	2011
МЕСЯЦ	12
ДЕНЬ	31
ЧАС	11
МИНУТА	56
СЕКУНДА	2
ZONEHOURS	8
ZONEMINUTES	0

DateTimeZone.ToText

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.ToText(dateTimeZone as nullable datetimezone, optional format as nullable text, optional culture as nullable text) as nullable text
```

О программе

Возвращает текстовое представление `dateTimeZone`. Для форматирования текста можно указать необязательный параметр `format`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение текстового представления `#datetimezone(2011, 12, 31, 11, 56, 2, 8, 0)`.

```
DateTimeZone.ToText(#datetimezone(2010, 12, 31, 11, 56, 2, 8, 0))
```

```
"12/31/2010 11:56:02 AM +08:00"
```

Пример 2

Получение текстового представления `#datetimezone(2010, 12, 31, 11, 56, 2, 10, 12)` с возможностью форматирования.

```
DateTimeZone.ToText(#datetimezone(2010, 12, 31, 11, 56, 2, 10, 12), "yyyy/MM/ddThh:mm:sszzz")
```

```
"2010/12/31T11:56:02+10:12"
```

DateTimeZone.ToUtc

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.ToUtc(dateTimeZone as nullable datetimetype) as nullable datetimetype
```

Сведения

Изменяет компонент часового пояса в значении `dateTimeZone` типа `datetime` на часовой пояс в формате UTC или всемирного времени. Если `dateTimeZone` не содержит компонента часового пояса, добавляется часовой пояс в формате UTC.

Пример 1

Изменение часового пояса для `#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30)` на часовой пояс в формате UTC.

```
DateTimeZone.ToUtc(#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30))
```

```
#datetimezone(2010, 12, 31, 4, 26, 2, 0, 0)
```


DateTimeZone.UtcNow

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.UtcNow() as datetimetype
```

Сведения

Возвращает текущую дату и время в формате UTC (часовой пояс GMT).

Пример 1

Получить текущую дату и время в формате UTC.

```
DateTimeZone.UtcNow()
```

```
#datetimetype(2011, 8, 16, 23, 34, 37.745, 0, 0)
```

DateTimeZone.ZoneHours

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.ZoneHours(dateTimeZone as nullable datetimezone) as nullable number
```

Сведения

Изменяет часовой пояс значения.

DateTimeZone.ZoneMinutes

16.03.2021 • 2 minutes to read

Синтаксис

```
DateTimeZone.ZoneMinutes(dateTimeZone as nullable datetimetypezone) as nullable number
```

Сведения

Изменяет часовой пояс значения.

#datetimezone

16.03.2021 • 2 minutes to read

Синтаксис

```
#datetimezone(year as number, month as number, day as number, hour as number, minute as number, second as number, offsetHours as number, offsetMinutes as number) as any
```

Сведения

Создает значение datetimezone из целых чисел года `year`, месяца `month`, дня `day`, часа `hour`, минуты `minute`, секунды `second` (дробное значение), часов смещения `offsetHours` и минут смещения `offsetMinutes`. Вызывает ошибку, если не соблюдаются следующие условия:

- $1 \leq \text{год} \leq 9999$
- $1 \leq \text{месяц} \leq 12$
- $1 \leq \text{день} \leq 31$
- $0 \leq \text{час} \leq 23$
- $0 \leq \text{минута} \leq 59$
- $0 \leq \text{секунда} \leq 59$
- $-14 \leq \text{часы смещения} + \text{минуты смещения}/60 \leq 14$

Функции для работы с длительностью

16.03.2021 • 2 minutes to read

Эти функции позволяют создавать значения параметра duration и управлять ими.

Duration

ФУНКЦИЯ	ОПИСАНИЕ
Duration.Days	Возвращает компонент дня значения длительности.
Duration.From	Возвращает значение длительности для заданного значения.
Duration.FromText	Возвращает значение длительности для заданного текстового значения.
Duration.Hours	Возвращает компонент часов значения длительности.
Duration.Minutes	Возвращает компонент минут значения длительности.
Duration.Seconds	Возвращает компонент секунд значения длительности.
Duration.ToRecord	Возвращает запись с частями значения длительности.
Duration.TotalDays	Возвращает общее количество дней для значения длительности.
Duration.TotalHours	Возвращает общее количество часов для значения длительности.
Duration.TotalMinutes	Возвращает общее количество минут для значения длительности.
Duration.TotalSeconds	Возвращает общее количество секунд для значения длительности.
Duration.ToText	Возвращает текстовое значение для заданного значения длительности.
#duration	Создает значение длительности из дней, часов, минут и секунд.

Duration.Days

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.Days(duration as nullable duration) as nullable number
```

Сведения

Возвращает компонент дня заданного значения `duration`, `duration`.

Пример 1

Поиск компонента дня в значении `#duration(5, 4, 3, 2)`.

```
Duration.Days(#duration(5, 4, 3, 2))
```

Duration.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.From(value as any) as nullable duration
```

Сведения

Возвращает значение `duration` из заданного параметра `value`. Если заданный параметр `value` имеет значение `null`, `Duration.From` возвращает `null`. Если заданный параметр `value` имеет значение `duration`, возвращается `value`. Значения следующих типов можно преобразовать в значение `duration`:

- `text`: значение `duration` из текстового формата прошедшего времени (d.h:m:s). Дополнительные сведения см. в разделе `Duration.FromText`.
- `number`: `duration` эквивалентно количеству полных и неполных дней, выраженных `value`.

Если `value` имеет любой другой тип, возвращается ошибка.

Пример 1

Преобразование `2.525` в значение `duration`.

```
Duration.From(2.525)
```

```
#duration(2, 12, 36, 0)
```

Duration.FromText

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.FromText(text as nullable text) as nullable duration
```

Сведения

Возвращает значение длительности из указанного текста `text`. Эта функция принимает следующие форматы:

- (-)чч:мм(:сс(.00))
- (-)ddd(.чч:мм(:сс(.00)))

(все диапазоны задаются с включением границ)

ddd: число дней.

чч: число часов от 0 до 23.

мм: число минут от 0 до 59.

сс: число секунд от 0 до 59.

00: число долей секунды от 0 до 9 999 999.

Пример 1

Преобразование `"2.05:55:20"` в значение `duration`.

```
Duration.FromText("2.05:55:20")
```

```
#duration(2, 5, 55, 20)
```


Duration.Hours

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.Hours(duration as nullable duration) as nullable number
```

Сведения

Возвращает компонент часов заданного значения `duration`, `duration`.

Пример 1

Поиск компонента часов в значении `#duration(5, 4, 3, 2)`.

```
Duration.Hours(#duration(5, 4, 3, 2))
```

Duration.Minutes

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.Minutes(duration as nullable duration) as nullable number
```

Сведения

Возвращает компонент минут заданного значения `duration` — `duration`.

Пример 1

Поиск компонента минут в значении `#duration(5, 4, 3, 2)`.

```
Duration.Minutes(#duration(5, 4, 3, 2))
```

Duration.Seconds

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.Seconds(duration as nullable duration) as nullable number
```

Сведения

Возвращает компонент часов заданных значений `duration` — `duration`.

Пример 1

Поиск компонента секунд в значении `#duration(5, 4, 3, 2)`.

```
Duration.Seconds(#duration(5, 4, 3, 2))
```

Duration.ToRecord

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.ToRecord(duration as duration) as record
```

Сведения

Возвращает запись, содержащую части значения длительности — `duration`.

- `duration`: параметр `duration`, из которого создается запись.

Пример 1

Преобразование `#duration(2, 5, 55, 20)` в запись его частей, включая дни, часы, минуты и секунды, если применимо.

```
Duration.ToRecord(#duration(2, 5, 55, 20))
```

ДНИ	2
ЧАСЫ	5
МИНУТЫ	55
СЕКУНДЫ	20

Duration.TotalDays

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.TotalDays(duration as nullable duration) as nullable number
```

Сведения

Возвращает общее количество дней, охваченных значением `duration`, — `duration`.

Пример 1

Поиск общего количества дней, охваченных `#duration(5, 4, 3, 2)`.

```
Duration.TotalDays(#duration(5, 4, 3, 2))
```

```
5.1687731481481478
```

Duration.TotalHours

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.TotalHours(duration as nullable duration) as nullable number
```

Сведения

Возвращает общее количество часов, охваченных значением `duration`, — `duration`.

Пример 1

Поиск общего количества часов, охваченных `#duration(5, 4, 3, 2)`.

```
Duration.TotalHours(#duration(5, 4, 3, 2))
```

```
124.05055555555555
```

Duration.TotalMinutes

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.TotalMinutes(duration as nullable duration) as nullable number
```

Сведения

Возвращает общее количество минут, охваченных значением `duration`, — `duration`.

Пример 1

Поиск общего количества минут, охваченных `#duration(5, 4, 3, 2)`.

```
Duration.TotalMinutes(#duration(5, 4, 3, 2))
```

```
7443.0333333333338
```

Duration.TotalSeconds

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.TotalSeconds(duration as nullable duration) as nullable number
```

Сведения

Возвращает общее количество секунд, охваченных значением `duration`, — `duration`.

Пример 1

Поиск общего количества секунд, охваченных `#duration(5, 4, 3, 2)`.

```
Duration.TotalSeconds(#duration(5, 4, 3, 2))
```

```
446582
```


Duration.ToText

16.03.2021 • 2 minutes to read

Синтаксис

```
Duration.ToText(duration as nullable duration, optional format as nullable text) as nullable text
```

О программе

Возвращает текстовое представление в формате "день.час: мин:сек" для заданного значения длительности `duration`.

- `duration` — `duration`, из которого вычисляется текстовое представление.
- `format`: *[необязательный]* не рекомендуется, выдаст ошибку, если не равно null.

Пример 1

Преобразование `#duration(2, 5, 55, 20)` в текстовое значение.

```
Duration.ToText(#duration(2, 5, 55, 20))
```

```
"2.05:55:20"
```

#duration

16.03.2021 • 2 minutes to read

Синтаксис

```
#duration(days as number, hours as number, minutes as number, seconds as number) as duration
```

Сведения

Создает значение длительности из числа дней `days`, часов `hours`, минут `minutes` и секунд `seconds`.

Обработка ошибок

16.03.2021 • 2 minutes to read

Эти функции возвращают диагностические трассировки на различных уровнях детализации, а также создают записи об ошибках.

Ошибка

ФУНКЦИЯ	ОПИСАНИЕ
Diagnostics.ActivityId	Возвращает непрозрачный идентификатор для текущего вычисления.
Diagnostics.Trace	Записывает сообщение трассировки, если трассировка включена, и возвращает значение.
Error.Record	Возвращает запись, содержащую поля Reason (Причина), Message (Сообщение) и Detail (Детали), для которых заданы указанные значения. Запись может использоваться для вызова или выдачи ошибки.
TraceLevel.Critical	Возвращает 1, значение для уровня трассировки "Критический".
TraceLevel.Error	Возвращает 2, значение для уровня трассировки "Ошибка".
TraceLevel.Information	Возвращает 4, значение для уровня трассировки "Информация".
TraceLevel.Verbose	Возвращает 5, значение для уровня трассировки "Подробный".
TraceLevel.Warning	Возвращает 3, значение для уровня трассировки "Предупреждение".

Diagnostics.ActivityId

16.03.2021 • 2 minutes to read

Синтаксис

```
Diagnostics.ActivityId() as nullable text
```

Сведения

Возвращает непрозрачный идентификатор для текущего вычисления.

Diagnostics.Trace

16.03.2021 • 2 minutes to read

Синтаксис

```
Diagnostics.Trace(traceLevel as number, message as anynonnull, value as any, optional delayed as nullable logical) as any
```

Сведения

Записывает трассировку `message`, если трассировка включена, и возвращает `value`. Необязательный параметр `delayed` указывает, следует ли откладывать оценку `value` до трассировки сообщения.

`traceLevel` может принимать одно из следующих значений:

- `TraceLevel.Critical` - `TraceLevel.Error`
- `TraceLevel.Warning`
- `TraceLevel.Information`
- `TraceLevel.Verbose`

Пример 1

Трассировка сообщения перед вызовом функции `Text.From` и возврат результата.

```
Diagnostics.Trace(TraceLevel.Information, "TextValueFromNumber", () => Text.From(123), true)
```

```
"123"
```

Error.Record

16.03.2021 • 2 minutes to read

Синтаксис

```
Error.Record(reason as text, optional message as nullable text, optional detail as any) as record
```

Сведения

Возвращает ошибочную запись из указанных текстовых значений для причины (reason), сообщения (message) и сведений (detail).

TraceLevel.Critical

16.03.2021 • 2 minutes to read

Сведения

Возвращает 1, значение для уровня трассировки "Критический".

TraceLevel.Error

16.03.2021 • 2 minutes to read

Сведения

Возвращает 2, значение для уровня трассировки "Ошибка".

TraceLevel.Information

16.03.2021 • 2 minutes to read

Сведения

Возвращает 4, значение для уровня трассировки "Информация".

TraceLevel.Verbose

16.03.2021 • 2 minutes to read

Сведения

Возвращает 5, значение для уровня трассировки "Подробный".

TraceLevel.Warning

16.03.2021 • 2 minutes to read

Сведения

Возвращает 3, значение для уровня трассировки "Предупреждение".

Функции выражений

16.03.2021 • 2 minutes to read

Эти функции позволяют создать и оценить код M.

Expression

ФУНКЦИЯ	ОПИСАНИЕ
Expression.Constant	Возвращает представление константного значения в исходном коде M.
Expression.Evaluate	Возвращает результат вычисления выражения M.
Expression.Identifier	Возвращает представление идентификатора в исходном коде M.

Expression.Constant

16.03.2021 • 2 minutes to read

Синтаксис

```
Expression.Constant(value as any) as text
```

Сведения

Возвращает представление константного значения в исходном коде М.

Пример 1

Получение представления числового значения в исходном коде М.

```
Expression.Constant(123)
```

```
"123"
```

Пример 2

Получение представления значения даты в исходном коде М.

```
Expression.Constant(#date(2035, 01, 02))
```

```
"#date(2035, 1, 2)"
```

Пример 3

Получение представления текстового значения в исходном коде М.

```
Expression.Constant("abc")
```

```
"""abc"""
```

Expression.Evaluate

16.03.2021 • 2 minutes to read

Синтаксис

```
Expression.Evaluate(document as text, optional environment as nullable record) as any
```

Сведения

Возвращает результат вычисления выражения `document` языка M с доступными для ссылки идентификаторами, которые определяет `environment`.

Пример 1

Вычисление простой суммы.

```
Expression.Evaluate("1 + 1")
```

2

Пример 2

Вычисление более сложной суммы.

```
Expression.Evaluate("List.Sum({1, 2, 3})", [List.Sum = List.Sum])
```

6

Пример 3

Вычисление объединения текстового значения с идентификатором.

```
Expression.Evaluate(Expression.Constant("abc") & " " & Expression.Identifier("x"), [x = "def"])
```

"abcdef"

Expression.Identifier

16.03.2021 • 2 minutes to read

Синтаксис

```
Expression.Identifier(name as text) as text
```

Сведения

Возвращает представление идентификатора `name` в исходном коде M.

Пример 1

Получить представление идентификатора в исходном коде M.

```
Expression.Identifier("MyIdentifier")
```

```
"MyIdentifier"
```

Пример 2

Получить представление идентификатора, содержащего пробел, в исходном коде M.

```
Expression.Identifier("My Identifier")
```

```
"#" "My Identifier" ""
```

Значения функции

16.03.2021 • 2 minutes to read

Эти функции позволяют создать и вызвать другие функции M.

Функция

ФУНКЦИЯ	ОПИСАНИЕ
Function.From	Принимает унарную функцию <code>function</code> и создает новую функцию с типом <code>functionType</code> , которая составляет из ее аргументов список и передает его <code>function</code> .
Function.Invoke	Вызывает заданную функцию, используя указанный параметр, и возвращает результат.
Function.InvokeAfter	Возвращает результат вызова функции спустя период задержки.
Function.IsDataSource	Определяет, считается ли функция источником данных.
Function.ScalarVector	Возвращает скалярную функцию типа <code>scalarFunctionType</code> , которая вызывает <code>vectorFunction</code> с одной строкой аргументов и возвращает единые выходные данные.

Function.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Function.From(functionType as type, function as function) as function
```

Сведения

Принимает унарную функцию `function` и создает новую функцию с типом `functionType`, которая составляет из ее аргументов список и передает его `function`.

Пример 1

Преобразует `List.Sum` в функцию с двумя аргументами, которые складываются друг с другом.

```
Function.From(type function (a as number, b as number) as number, List.Sum)(2, 1)
```

3

Пример 2

Преобразует функцию, принимающую список, в функцию с двумя аргументами.

```
Function.From(type function (a as text, b as text) as text, (list) => list{0} & list{1})("2", "1")
```

"21"

Function.Invoke

16.03.2021 • 2 minutes to read

Синтаксис

```
Function.Invoke(function as function, args as list) as any
```

Сведения

Вызывает данную функцию, используя указанный список аргументов, и возвращает результат.

Пример 1

Вызывает Record.FieldNames с одним аргументом [A=1,B=2]

```
Function.Invoke(Record.FieldNames, {[A = 1, B = 2]})
```

Объект
B

Function.InvokeAfter

16.03.2021 • 2 minutes to read

Синтаксис

```
Function.InvokeAfter(function as function, delay as duration) as any
```

Сведения

Возвращает результат вызова `function` спустя период времени `delay`.

Function.IsDataSource

16.03.2021 • 2 minutes to read

Синтаксис

```
Function.IsDataSource(function as function) as logical
```

Сведения

Определяет, считается ли `function` источником данных.

Function.ScalarVector

16.03.2021 • 2 minutes to read

Синтаксис

```
Function.ScalarVector(scalarFunctionType as type, vectorFunction as function) as function
```

Сведения

Возвращает скалярную функцию типа `scalarFunctionType`, которая вызывает `vectorFunction` с одной строкой аргументов и возвращает единые выходные данные. Кроме того, если скалярная функция многократно применяется для каждой строки таблицы входных данных, например `Table.AddColumn`, `vectorFunction` будет применяться один раз для всех входных данных.

`vectorFunction` получит таблицу, столбцы которой совпадают по имени и положению параметров `scalarFunctionType`. Каждая строка этой таблицы содержит аргументы для одного вызова скалярной функции со столбцами, соответствующими параметрам `scalarFunctionType`.

`vectorFunction` должен возвращать список, имеющий ту же длину, что и входная таблица, элемент которой в каждой позиции должен иметь тот же результат, что и вычисление скалярной функции для входной строки той же позиции.

Ожидается, что входная таблица будет передаваться в поток, поэтому ожидается, что `vectorFunction` будет выполнять потоковую передачу выходных данных по мере поступления входных данных, работая только с одним блоком входных данных за раз. В частности, `vectorFunction` не должен перечислять входную таблицу более одного раза.

Функции для работы со строками

16.03.2021 • 2 minutes to read

Эти функции позволяют преобразовать списки текстовых значений в двоичное значение или в одну текстовую строку.

Строки

ФУНКЦИЯ	ОПИСАНИЕ
Lines.FromBinary	Преобразует двоичное значение в список текстовых значений, разделенных разрывами строк.
Lines.FromText	Преобразует текстовое значение в список текстовых значений, разделенных разрывами строк.
Lines.ToBinary	Преобразует список текстовых значений в двоичное значение с использованием указанной кодировки и разделителя lineSeparator. Указанный lineSeparator добавляется к каждой строке. Если этот параметр не указан, используются символы возврата каретки и перевода строки.
Lines.ToText	Преобразует список текста в один текст. Заданный lineSeparator добавляется в каждую строку. Если этот параметр не указан, используются символы возврата каретки и перевода строки.

Lines.FromBinary

16.03.2021 • 2 minutes to read

Синтаксис

```
Lines.FromBinary(binary as binary, optional quoteStyle as nullable number, optional  
includeLineSeparators as nullable logical, optional encoding as nullable number) as list
```

Сведения

Преобразует двоичное значение в список текстовых значений, разделенных разрывами строк. Если указан стиль кавычек, разрывы строк могут отображаться между кавычками. Если значение `includeLineSeparators` равно `true`, то символы разрыва строк включаются в текст.

Lines.FromText

16.03.2021 • 2 minutes to read

Синтаксис

```
Lines.FromText(text as text, optional quoteStyle as nullable number, optional  
includeLineSeparators as nullable logical) as list
```

Сведения

Преобразует текстовое значение в список текстовых значений, разделенных разрывами строк. Если значение `includeLineSeparators` равно `true`, то символы разрыва строк включаются в текст.

- `QuoteStyle.None:` (по умолчанию): обработка символов кавычек не требуется.
- `QuoteStyle.Csv:` : обработка символов кавычек как для формата CSV. Символ двойной кавычки используется для разграничения таких областей. Пара символов двойной кавычки используется для представления одного символа двойной кавычки в пределах такой области.

Lines.ToBinary

16.03.2021 • 2 minutes to read

Синтаксис

```
Lines.ToBinary(lines as list, optional lineSeparator as nullable text, optional encoding as nullable number, optional includeByteOrderMark as nullable logical) as binary
```

Сведения

Преобразует список текстовых значений в двоичное значение с использованием указанной кодировки и разделителя `lineSeparator`. Указанный `lineSeparator` добавляется к каждой строке. Если этот параметр не указан, используются символы возврата каретки и перевода строки.

Lines.ToText

16.03.2021 • 2 minutes to read

Синтаксис

```
Lines.ToText(lines as list, optional lineSeparator as nullable text) as text
```

Сведения

Преобразует список текста в один текст. Заданный `lineSeparator` добавляется в каждую строку. Если этот параметр не указан, используются символы возврата каретки и перевода строки.

Функции для работы со списками

16.03.2021 • 15 minutes to read

Эти функции позволяют создавать значения параметра list и управлять ими.

Информация

ФУНКЦИЯ	ОПИСАНИЕ
List.Count	Возвращает число элементов в списке.
List.NonNullCount	Возвращает число элементов в списке, за исключением значений NULL
List.IsEmpty	Возвращает значение, указывающее, является ли список пустым.

Выбор

ФУНКЦИЯ	ОПИСАНИЕ
List.Alternate	Возвращает список с чередующимися элементами из исходного списка, на основе числа, необязательного repeatInterval и необязательного смещения.
List.Buffer	Помещает в буфер список в памяти. Результат вызова — стабильный список (то есть с детерминированным числом и порядком элементов).
List.Distinct	Фильтрует список вниз, удаляя повторяющиеся элементы. Для управления сравнением на равенство можно указать необязательное значение критерия равенства. Выбирается первое значение из каждой группы равенства.
List.FindText	Выполняет поиск текстового значения в списке значений, включая поля записей.
List.First	Возвращает первое значение списка или указанное значение по умолчанию, если список пуст. Возвращает последний элемент в списке или необязательное значение по умолчанию, если список пуст. Если список пуст и значение по умолчанию не указано, функция возвращает .
List.FirstN	Возвращает первый набор элементов в списке, указывая количество возвращаемых элементов или соответствующее условие, предоставляемое countOrCondition.
List.InsertRange	Вставляет элементы из значений по указанному индексу в списке входных данных.

ФУНКЦИЯ	ОПИСАНИЕ
List.IsDistinct	Возвращает значение, указывающее, является ли список уникальным.
List.Last	Возвращает последний набор элементов в списке, указывая количество возвращаемых элементов или соответствующее условие, предоставляемое countOrCondition.
List.LastN	Возвращает последний набор элементов в списке, указывая количество возвращаемых элементов или соответствующее условие.
List.MatchesAll	Возвращает значение true, если все элементы в списке соответствуют условию.
List.MatchesAny	Возвращает значение true, если любой элемент в списке соответствует условию.
List.Positions	Возвращает список позиций для списка входных данных.
List.Range	Возвращает элементы для подсчета начиная со смещения.
List.Select	Выбирает элементы, соответствующие условию.
List.Single	Возвращает один элемент списка или вызывает Expression.Error, если список содержит более одного элемента.
List.SingleOrDefault	Возвращает один элемент из списка.
List.Skip	Пропускает первый элемент списка. При наличии пустого списка возвращает пустой список. Эта функция принимает необязательный параметр countOrCondition для поддержки пропуска нескольких значений.

Функции преобразования

ФУНКЦИЯ	ОПИСАНИЕ
List.Accumulate	Накапливает результат из списка. Начиная с начального значения эта функция применяет функцию накопителя и возвращает окончательный результат.
List.Combine	Объединяет список списков в один список.
List.ConformToPageReader	Эта функция предназначена только для внутреннего использования.
List.RemoveRange	Возвращает список, удаляющий элементы для подсчета, начиная со смещения. Число по умолчанию — 1.

ФУНКЦИЯ	ОПИСАНИЕ
List.RemoveFirstN	Возвращает список с указанным числом элементов, удаленных из списка, начиная с первого элемента. Число удаленных элементов зависит от необязательного параметра countOrCondition.
List.RemoveItems	Удаляет элементы из списка list1, присутствующие в списке list2, и возвращает новый список.
List.RemoveLastN	Возвращает список с указанным числом элементов, удаленных из списка, начиная с последнего элемента. Число удаленных элементов зависит от необязательного параметра countOrCondition.
List.Repeat	Возвращает список, который повторяет содержимое входного списка указанное число раз.
List.ReplaceRange	Возвращает список, который заменяет указанное количество значений в списке на список replaceWith, начиная с индекса.
List.RemoveMatchingItems	Удаляет все вхождения значений, имеющих в списке.
List.RemoveNulls	Удаляет значения NULL из списка.
List.ReplaceMatchingItems	Заменяет вхождения существующих значений в списке новыми значениями с помощью предоставленного equationCriteria. Старые и новые значения предоставляются параметрами замены. Для управления сравнением на равенство можно указать необязательное значение критерия равенства. Дополнительные сведения об операциях замены и критериях равенства см. в разделе "Значения параметров".
List.ReplaceValue	Ищет в списке значений значение и заменяет каждое его вхождение значением замены.
List.Reverse	Возвращает список с обратным порядком элементов.
List.Split	Разделяет указанный список на список списков с заданным размером страницы.
List.Transform	Выполняет функцию для каждого элемента в списке и возвращает новый список.
List.TransformMany	Возвращает список, элементы которого проецируются из входного списка.

Функции членства

Так как все значения можно проверить на равенство, эти функции могут работать с разнородными списками.

КОМПОНЕНТ	ОПИСАНИЕ
List.AllTrue	Возвращает значение true, если все выражения в списке имеют значение true.
List.AnyTrue	Возвращает значение true, если любое выражение в списке имеет значение true.
List.Contains	Возвращает значение true, если значение находится в списке.
List.ContainsAll	Возвращает значение true, если все элементы в значениях находятся в списке.
List.ContainsAny	Возвращает значение true, если любой элемент в значениях находится в списке.
List.PositionOf	Находит первое вхождение значения в списке и возвращает его расположение.
List.PositionOfAny	Находит первое вхождение значения в значениях и возвращает его расположение.

Операции Set

КОМПОНЕНТ	ОПИСАНИЕ
List.Difference	Возвращает элементы в списке 1, которых нет в списке 2. Поддерживаются дублирующиеся значения.
List.Intersect	Возвращает список из списка списков и объединяет общие элементы в отдельных списках. Поддерживаются дублирующиеся значения.
List.Union	Возвращает список из списка списков и объединяет элементы в отдельных списках. Возвращаемый список содержит все элементы в любых входных списках. Дублирующиеся значения сопоставляются в составе объединения.
List.Zip	Возвращает список списков, объединяя элементы в одной позиции.

Упорядочение

Функции упорядочивания выполняют сравнения. Все сравниваемые значения должны быть сравнимы друг с другом. Это означает, что все они должны иметь одинаковый тип данных (или включать значение NULL, которое всегда имеет наименьшее значение при сравнении). В противном случае возникает Expression.Error.

Сравниваемые типы данных

- Число
- Длительность
- Дата и время

- Текст
- Логические
- NULL

КОМПОНЕНТ	ОПИСАНИЕ
List.Max	Возвращает максимальный элемент в списке или необязательное значение по умолчанию, если список пуст.
List.MaxN	Возвращает максимальные значения в списке. После сортировки строк можно указать необязательные параметры для дальнейшей фильтрации результатов.
List.Median	Возвращает медианный элемент из списка.
List.Min	Возвращает минимальный элемент в списке или необязательное значение по умолчанию, если список пуст.
List.MinN	Возвращает минимальные значения в списке.
List.Sort	Возвращает отсортированный список с использованием критерия сравнения.
List.Percentile	Возвращает выборку процентилей, соответствующих данным вероятностям.
PercentileMode.ExcelExc	При интерполяции значений для List.Percentile нужно будет использовать метод, совместимый с PERCENTILE.EXC в Excel.
PercentileMode.ExcelInc	При интерполяции значений для List.Percentile нужно будет использовать метод, совместимый с PERCENTILE.INC в Excel.
PercentileMode.SqlCont	При интерполяции значений для List.Percentile нужно будет использовать метод, совместимый с PERCENTILE_CONT в SQL Server.
PercentileMode.SqlDisc	При интерполяции значений для List.Percentile нужно будет использовать метод, совместимый с PERCENTILE_DISC в SQL Server.

Средние значения

Эти функции работают с однородными списками чисел, даты и времени и длительности.

КОМПОНЕНТ	ОПИСАНИЕ
List.Average	Возвращает среднее значение из списка в типе данных значений в списке.
List.Mode	Возвращает элемент, который чаще всего появляется в списке.

КОМПОНЕНТ	ОПИСАНИЕ
List.Modes	Возвращает все элементы, которые отображаются с одинаковой максимальной частотой.
List.StandardDeviation	Возвращает стандартное отклонение из списка значений. List.StandardDeviation выполняет оценку на основе выборки. Результатом является число для чисел и длительность для значений даты и времени и длительности.

Сложение

Эти функции работают с однородными списками чисел и длительности.

КОМПОНЕНТ	ОПИСАНИЕ
List.Sum	Возвращает сумму из списка.

Числовые типы

Эти функции работают только с числами.

КОМПОНЕНТ	ОПИСАНИЕ
List.Covariance	Возвращает ковариацию из двух списков в виде числа.
List.Product	Возвращает продукт из списка чисел.

Генераторы

Эти функции создают список значений.

КОМПОНЕНТ	ОПИСАНИЕ
List.Dates	Возвращает список значений даты на основе указанного размера, начиная с начала, и добавляет к каждому значению шаг приращения.
List.DatetimeTimes	Возвращает список значений datetime на основе указанного размера, начиная с начала, и добавляет к каждому значению шаг приращения.
List.DateTimeZones	Возвращает список значений datetimezone на основе указанного размера, начиная с начала, и добавляет к каждому значению шаг приращения.
List.Durations	Возвращает список значений длительности на основе указанного размера, начиная с начала, и добавляет к каждому значению шаг приращения.
List.Generate	Создает список из функции значения, функции условия, следующей функции и необязательной функции для преобразования значений.
List.Numbers	Возвращает список чисел на основе указанного размера, начиная с начального, и добавляет шаг приращения. Шаг приращения по умолчанию равен 1.

КОМПОНЕНТ	ОПИСАНИЕ
List.Random	Возвращает список из указанного числа случайных чисел с необязательным параметром начального значения.
List.Times	Возвращает список значений времени на основе указанного размера, начиная с начала.

Значения параметра

Спецификация вхождения

- Occurrence.First = 0;
- Occurrence.Last = 1;
- Occurrence.All = 2;

Порядок сортировки

- Order.Ascending = 0;
- Order.Descending = 1;

Критерии равенства

Критерий равенства для списка значений можно указать как:

- значение функции, которое является:
 - селектором ключа, определяющим значение в списке для применения критериев равенства;
 - функцией сравнения, которая используется для указания применяемого типа сравнения. Встроенные функции сравнения можно указать. См. раздел о функциях сравнения.
- Значение списка, которое имеет:
 - Ровно два элемента
 - Первый элемент является селектором ключа, как указано выше
 - Второй элемент является функцией сравнения, как указано выше.

Дополнительные сведения и примеры см. в разделе [List.Distinct](#).

Критерии сравнения

Критерий сравнения может быть предоставлен как одно из следующих значений:

- Числовое значение для указания порядка сортировки. Дополнительные сведения см. в главе о порядке сортировки в разделе "Значения параметров".
- Чтобы вычислить ключ, используемый для сортировки, можно использовать функцию с одним аргументом.
- Чтобы выбрать ключ и порядок, критерием сравнения может быть список, содержащий ключ и порядок.
- Чтобы полностью управлять сравнением, можно использовать функцию с двумя аргументами, которая возвращает -1, 0 или 1, учитывая связь между левыми и правыми входными данными. Value.Compare — это метод, который можно использовать для делегирования этой логики.

Дополнительные сведения и примеры см. в разделе [List.Sort](#).

Операции замены

Операции замены задаются значением списка, каждый элемент этого списка должен быть:

- Значением списка, состоящим из двух элементов
- Первый элемент — это старое значение в списке, которое необходимо заменить
- Второй элемент является новым, который должен заменить все вхождения старого значения в списке

List.Accumulate

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Accumulate(list as list, seed as any, accumulator as function) as any
```

Сведения

Накапливает сводное значение из элементов в списке `list` с использованием `accumulator`. Можно задать необязательный параметр начального значения `seed`.

Пример 1

Накапливает сводное значение из элементов в списке {1, 2, 3, 4, 5}, используя выражение ((состояние, текущее значение) => состояние + текущее значение).

```
List.Accumulate({1, 2, 3, 4, 5}, 0, (state, current) => state + current)
```

List.AllTrue

16.03.2021 • 2 minutes to read

Синтаксис

```
List.AllTrue(list as list) as logical
```

Сведения

Возвращает значение true, если все выражения в списке `list` равны true.

Пример 1

Определение того, все ли выражения в списке {true, true, 2 > 0} истинны.

```
List.AllTrue({true, true, 2 > 0})
```

```
true
```

Пример 2

Определение того, все ли выражения в списке {true, true, 2 < 0} истинны.

```
List.AllTrue({true, false, 2 < 0})
```

```
false
```

List.Alternate

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Alternate(list as list, count as number, optional repeatInterval as nullable number, optional offset as nullable number) as list
```

Сведения

Возвращает список, состоящий из всех нечетных элементов смещения в списке. Переключается между извлечением и пропуском значений в списке `list` в зависимости от параметров.

- `count` : указывает число значений, пропущенных каждый раз.
- `repeatInterval` : необязательный интервал повтора для указания количества значений, добавляемых между пропущенными значениями.
- `offset` : необязательный параметр смещения, позволяющий начать пропуск значений с начальным смещением.

Пример 1

Создать список из {1..10}, в котором первое число пропускается.

```
List.Alternate({1..10}, 1)
```

2

3

4

5

6

7

8

9

10

Пример 2

Создать список из {1..10}, в котором пропускается каждое второе значение.

```
List.Alternate({1..10}, 1, 1)
```

2

4

6

8

10

Пример 3

Создать список из {1..10}, который начинается с 1 и пропускает каждое второе значение.

```
List.Alternate({1..10}, 1, 1, 1)
```

1

3

5

7

9

Пример 4

Создать список из {1..10}, который начинается с 1, пропускает одно значение, оставляет два значения и т. д.

```
List.Alternate({1..10}, 1, 2, 1)
```

1

3

4

6

7

9

10

List.AnyTrue

16.03.2021 • 2 minutes to read

Синтаксис

```
List.AnyTrue(list as list) as logical
```

Сведения

Возвращает значение true, если любое выражение в списке `list` истинно.

Пример 1

Определение того, есть ли в списке {true, false, 2 > 0} истинные выражения.

```
List.AnyTrue({true, false, 2>0})
```

true

Пример 2

Определение того, есть ли в списке {2 = 0, false, 2 < 0} истинные выражения.

```
List.AnyTrue({2 = 0, false, 2 < 0})
```

false

List.Average

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Average(list as list, optional precision as nullable number) as any
```

Сведения

Возвращает среднее значение для элементов в списке `list`. Результат указывается с тем же типом данных, что и значения в списке. Работает только с числовыми значениями, значениями даты, времени, datetime, datetimezone и длительности. Если список пуст, возвращается значение NULL.

Пример 1

Найти среднее значение списка чисел `{3, 4, 6}`.

```
List.Average({3, 4, 6})
```

```
4.333333333333333
```

Пример 2

Найти среднее значение из дат: 1 января 2011 г., 2 января 2011 г. и 3 января 2011 г.

```
List.Average({#date(2011, 1, 1), #date(2011, 1, 2), #date(2011, 1, 3)})
```

```
#date(2011, 1, 2)
```


List.Buffer

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Buffer(list as list) as list
```

Сведения

Помещает в буфер список `list` в памяти. Результатом этого вызова является стабильный список.

Пример 1

Создание стабильной копии списка {1..10}.

```
List.Buffer({1..10})
```

1

2

3

4

5

6

7

8

9

10

List.Combine

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Combine(lists as list) as list
```

Сведения

Принимает список списков `lists` и объединяет их в один новый список.

Пример 1

Объединение двух простых списков {1, 2} и {3, 4}.

```
List.Combine({{1, 2}, {3, 4}})
```

1

2

3

4

Пример 2

Объединение двух списков {1, 2} и {3, {4, 5}}, один из которых содержит вложенный список.

```
List.Combine({{1, 2}, {3, {4, 5}}})
```

1

2

3

[Список]

List.ConformToPageReader

16.03.2021 • 2 minutes to read

Синтаксис

```
List.ConformToPageReader(list as list, optional options as nullable record) as table
```

О программе

Эта функция предназначена только для внутреннего использования.

List.Contains

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Contains(list as list, value as any, optional equationCriteria as any) as logical
```

Сведения

Указывает, содержит ли список `list` значение `value`. Возвращает значение `true`, если значение найдено в списке; в противном случае — значение `false`. Для управления проверкой на равенство можно указать необязательное значение критерия равенства — `equationCriteria`.

Пример 1

Определение того, содержит ли список {1, 2, 3, 4, 5} значение 3.

```
List.Contains({1, 2, 3, 4, 5}, 3)
```

```
true
```

Пример 2

Определение того, содержит ли список {1, 2, 3, 4, 5} значение 6.

```
List.Contains({1, 2, 3, 4, 5}, 6)
```

```
false
```

List.ContainsAll

16.03.2021 • 2 minutes to read

Синтаксис

```
List.ContainsAll(list as list, values as list, optional equationCriteria as any) as logical
```

Сведения

Указывает, содержит ли список `list` все значения из другого списка `values`. Возвращает значение `true`, если значение найдено в списке; в противном случае — значение `false`. Для управления проверкой на равенство можно указать необязательное значение критерия равенства — `equationCriteria`.

Пример 1

Определение того, содержит ли список {1, 2, 3, 4, 5} значения 3 и 4.

```
List.ContainsAll({1, 2, 3, 4, 5}, {3, 4})
```

```
true
```

Пример 2

Определение того, содержит ли список {1, 2, 3, 4, 5} значения 5 и 6.

```
List.ContainsAll({1, 2, 3, 4, 5}, {5, 6})
```

```
false
```

List.ContainsAny

16.03.2021 • 2 minutes to read

Синтаксис

```
List.ContainsAny(list as list, values as list, optional equationCriteria as any) as logical
```

Сведения

Указывает, содержит ли список `list` любые значения из другого списка `values`. Возвращает значение `true`, если значение найдено в списке; в противном случае — значение `false`. Для управления проверкой на равенство можно указать необязательное значение критерия равенства — `equationCriteria`.

Пример 1

Определение того, содержит список {1, 2, 3, 4, 5} значение 3 или 9.

```
List.ContainsAny({1, 2, 3, 4, 5}, {3, 9})
```

```
true
```

Пример 2

Определение того, содержит список {1, 2, 3, 4, 5} значение 6 или 7.

```
List.ContainsAny({1, 2, 3, 4, 5}, {6, 7})
```

```
false
```

List.Count

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Count(list as list) as number
```

Сведения

Возвращает число элементов в списке `list`.

Пример 1

Определение количества значений в списке {1, 2, 3}.

```
List.Count({1, 2, 3})
```

3

List.Covariance

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Covariance(numberList1 as list, numberList2 as list) as nullable number
```

Сведения

Возвращает ковариацию между двумя списками `numberList1` и `numberList2`. `numberList1` и `numberList2` должны содержать одинаковое количество значений `number`.

Пример 1

Вычисление ковариации между двумя списками.

```
List.Covariance({1, 2, 3}, {1, 2, 3})
```

```
0.6666666666666667
```


List.Dates

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Dates(start as date, count as number, step as duration) as list
```

Сведения

Возвращает список значений `date` с размером `count` начиная с `start`. Шаг приращения `step` является значением `duration`, которое добавляется к каждому значению.

Пример 1

Создать список из пяти значений, начиная с новогодней ночи (`#date(2011, 12, 31)`), с увеличением значения на один день (`#duration(1, 0, 0, 0)`).

```
List.Dates(#date(2011, 12, 31), 5, #duration(1, 0, 0, 0))
```

31.12.2011 00:00:00

01.01.2012 00:00:00

02.01.2012 00:00:00

03.01.2012 00:00:00

04.01.2012 00:00:00

List.Datetimes

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Datetimes(start as datetime, count as number, step as duration) as list
```

Сведения

Возвращает список значений `datetime` с размером `count` начиная с `start`. Шаг приращения `step` является значением `duration`, которое добавляется к каждому значению.

Пример

Создание списка из 10 значений, начиная за 5 минут до наступления Нового года (`#datetime(2011, 12, 31, 23, 55, 0)`) с шагом приращения 1 минута (`#duration(0, 0, 1, 0)`).

```
List.Datetimes(#datetime(2011, 12, 31, 23, 55, 0), 10, #duration(0, 0, 1, 0))
```

31.12.2011 23:55:00

31.12.2011 23:56:00

31.12.2011 23:57:00

31.12.2011 23:58:00

31.12.2011 23:59:00

01.01.2012 00:00:00

01.01.2012 00:01:00

01.01.2012 00:02:00

01.01.2012 00:03:00

01.01.2012 00:04:00

List.DateTimeZones

16.03.2021 • 2 minutes to read

Синтаксис

```
List.DateTimeZones(start as datetimezone, count as number, step as duration) as list
```

Сведения

Возвращает список значений `datetimezone` с размером `count` начиная с `start`. Шаг приращения `step` является значением `duration`, которое добавляется к каждому значению.

Пример 1

Создать список из 10 значений начиная за 5 минут до наступления Нового года (`#datetimezone(2011, 12, 31, 23, 55, 0, -8, 0)`) с увеличением на 1 минуту (`#duration(0, 0, 1, 0)`).

```
List.DateTimeZones(#datetimezone(2011, 12, 31, 23, 55, 0, -8, 0), 10, #duration(0, 0, 1, 0))
```

31.12.2011 23:55:00-08:00

31.12.2011 23:56:00-08:00

31.12.2011 23:57:00-08:00

31.12.2011 23:58:00-08:00

31.12.2011 23:59:00-08:00

1.1.2012 00:00:00-08:00

1.1.2012 00:01:00-08:00

1.1.2012 00:02:00-08:00

1.1.2012 00:03:00-08:00

1.1.2012 00:04:00-08:00

List.Difference

16.03.2021 • 2 minutes to read

```
List.Difference(list1 as list, list2 as list, optional equationCriteria as any) as list
```

Сведения

Найти элементы в списке `list1`, которых нет в списке `list2`. Поддерживаются дублирующиеся значения. Для управления проверкой на равенство можно указать необязательное значение критерия равенства — `equationCriteria`.

Пример 1

Поиск элементов в списке {1, 2, 3, 4, 5}, которых нет в списке {4, 5, 3}.

```
List.Difference({1, 2, 3, 4, 5}, {4, 5, 3})
```

1

2

Пример 2

Поиск элементов в списке {1, 2}, которых нет в списке {1, 2, 3}.

```
List.Difference({1, 2}, {1, 2, 3})
```

List.Distinct

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Distinct(list as list, optional equationCriteria as any) as list
```

Сведения

Возвращает список, содержащий все значения в списке `list` с удаленными повторениями. Если этот список пуст, то результатом является пустой список.

Пример 1

Удалить повторяющиеся элементы из списка {1, 1, 2, 3, 3, 3}.

```
List.Distinct({1, 1, 2, 3, 3, 3})
```

1

2

3

List.Durations

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Durations(start as duration, count as number, step as duration) as list
```

Сведения

Возвращает список значений `count` `duration` начиная с `start` с шагом приращения `duration` `step`.

Пример

Создание списка из пяти значений, начиная с часа 1, с шагом приращения в один час.

```
List.Durations(#duration(0, 1, 0, 0), 5, #duration(0, 1, 0, 0))
```

01:00:00

02:00:00

03:00:00

04:00:00

05:00:00

List.FindText

16.03.2021 • 2 minutes to read

Синтаксис

```
List.FindText(list as list, text as text) as list
```

Сведения

Возвращает список значений из списка `list`, содержащих значение `text`.

Пример 1

Поиск текстовых значений в списке {"a", "b", "ab"}, совпадающих с "a".

```
List.FindText({"a", "b", "ab"}, "a")
```

a

ab

List.First

16.03.2021 • 2 minutes to read

Синтаксис

```
List.First(list as list, optional defaultValue as any) as any
```

Сведения

Возвращает первый элемент в списке `list` или необязательное значение по умолчанию, `defaultValue`, если список пуст. Если список пуст и значение по умолчанию не указано, функция возвращает `null`.

Пример 1

Поиск первого значения в списке {1, 2, 3}.

```
List.First({1, 2, 3})
```

1

Пример 2

Поиск первого значения в списке {}. Если список пуст, возвращается -1.

```
List.First({}, -1)
```

-1

List.FirstN

16.03.2021 • 2 minutes to read

Синтаксис

```
List.FirstN(list as list, countOrCondition as any) as any
```

Сведения

- Если указано число, возвращается не более чем это число элементов.
- Если указано условие, возвращаются все элементы, которые изначально соответствуют условию. Как только обнаруживается элемент, не соответствующий условию, последующие элементы не рассматриваются.

Пример 1

Нахождение первых значений в списке {3, 4, 5, -1, 7, 8, 2}, превышающих 0.

```
List.FirstN({3, 4, 5, -1, 7, 8, 2}, each _ > 0)
```

3

4

5

List.Generate

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Generate(initial as function, condition as function, next as function, optional selector as nullable function) as list
```

Сведения

Создает список значений с четырьмя функциями, которые создают начальное значение `initial`, проверяют выполнение условия `condition` и в случае успеха выбирают результат и формируют следующее значение `next`. Можно также указать необязательный параметр `selector`.

Пример 1

Создать список, который начинается с 10, остается больше 0 и уменьшается на 1.

```
List.Generate(() => 10, each _ > 0, each _ - 1)
```

10

9

8

7

6

5

4

3

2

1

Пример 2

Создать список записей, содержащих `x` и `y`, где `x` — это значение, а `y` — список. Значение `x` должно быть меньше 10 и представлять число элементов в списке `y`. После создания списка нужно вернуть только значения `x`.

```
List.Generate(  
    () => [x = 1, y = {}],  
    each [x] < 10,  
    each [x = List.Count([y]), y = [y] & {x}],  
    each [x]  
)
```

1

0

1

2

3

4

5

6

7

8

9

List.InsertRange

16.03.2021 • 2 minutes to read

Синтаксис

```
List.InsertRange(list as list, index as number, values as list) as list
```

Сведения

Возвращает новый список, созданный путем вставки значений в `values` в `list` по `index`. Первая позиция в списке находится по индексу 0.

- `list`: целевой список, в который должны быть вставлены значения.
- `index`: индекс целевого списка (`list`), в который должны быть вставлены значения. Первая позиция в списке находится по индексу 0.
- `values`: список значений, вставляемых в `list`.

Пример 1

Вставка списка ({3, 4}) в целевой список ({1, 2, 5}) по индексу 2.

```
List.InsertRange({1, 2, 5}, 2, {3, 4})
```

1

2

3

4

5

Пример 2

Вставка списка с вложенным списком ({1, {1.1, 1.2}}) в целевой список ({2, 3, 4}) по индексу 0.

```
List.InsertRange({2, 3, 4}, 0, {1, {1.1, 1.2}})
```

1

[Список]

2

3

List.Intersect

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Intersect(lists as list, optional equationCriteria as any) as list
```

Сведения

Возвращает пересечение значений списка, содержащихся в списке входных данных `lists`. Можно указать необязательный параметр `equationCriteria`.

Пример 1

Определение пересечения списков {1..5}, {2..6}, {3..7}.

```
List.Intersect({{1..5}, {2..6}, {3..7}})
```

3

4

5

List.IsDistinct

16.03.2021 • 2 minutes to read

Синтаксис

```
List.IsDistinct(list as list, optional equationCriteria as any) as logical
```

Сведения

Возвращает логическое значение, указывающее, имеются ли повторяющиеся значения в списке `list`; `true`, если значения в списке уникальны; `false`, если есть повторяющиеся значения.

Пример 1

Определение того, уникальны ли значения списка {1, 2, 3} (т. е. отсутствуют ли в нем повторяющиеся значения).

```
List.IsDistinct({1, 2, 3})
```

`true`

Пример 2

Определение того, уникальны ли значения списка {1, 2, 3, 3} (т. е. отсутствуют ли в нем повторяющиеся значения).

```
List.IsDistinct({1, 2, 3, 3})
```

`false`

List.IsEmpty

16.03.2021 • 2 minutes to read

Синтаксис

```
List.IsEmpty(list as list) as logical
```

Сведения

Возвращает `true`, если список `list` не содержит значения (длина 0). Если список содержит значения (длина больше 0), возвращает `false`.

Пример 1

Определить, пуст ли список {}.

```
List.IsEmpty({})
```

```
true
```

Пример 2

Определить, пуст ли список {1, 2}.

```
List.IsEmpty({1, 2})
```

```
false
```


List.Last

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Last(list as list, optional defaultValue as any) as any
```

Сведения

Возвращает последний элемент в списке `list` или необязательное значение по умолчанию, `defaultValue`, если список пуст. Если список пуст и значение по умолчанию не указано, функция возвращает `null`.

Пример 1

Поиск последнего значения в списке {1, 2, 3}.

```
List.Last({1, 2, 3})
```

3

Пример 2

Поиск последнего значения в списке {} или -1, если он пуст.

```
List.Last({}, -1)
```

-1

List.LastN

16.03.2021 • 2 minutes to read

Синтаксис

```
List.LastN(list as list, optional countOrCondition as any) as any
```

Сведения

Возвращает последний элемент списка `list`. Если список пуст, возникает исключение. Эта функция принимает необязательный параметр, `countOrCondition`, для поддержки сбора нескольких элементов или фильтрации элементов. `countOrCondition` можно указать тремя способами:

- Если указано число, возвращается не более чем это число элементов.
- Если указано условие, возвращаются все элементы, которые изначально соответствуют условию, начиная с конца списка. Как только обнаруживается элемент, не соответствующий условию, последующие элементы не рассматриваются.
- Если этот параметр имеет значение NULL, возвращается последний элемент в списке.

Пример 1

Поиск последнего значения в списке {3, 4, 5, -1, 7, 8, 2}.

```
List.LastN({3, 4, 5, -1, 7, 8, 2}, 1)
```

2

Пример 2

Поиск последних значений в списке {3, 4, 5, -1, 7, 8, 2}, превышающих 0.

```
List.LastN({3, 4, 5, -1, 7, 8, 2}, each _ > 0)
```

7

8

2

List.MatchesAll

16.03.2021 • 2 minutes to read

Синтаксис

```
List.MatchesAll(list as list, condition as function) as logical
```

Сведения

Возвращает значение `true`, если функции условия `condition` удовлетворяют все значения в списке `list`, в противном случае возвращает значение `false`.

Пример 1

Определить, все ли значения в списке {11, 12, 13} больше 10.

```
List.MatchesAll({11, 12, 13}, each _ > 10)
```

`true`

Пример 2

Определить, все ли значения в списке {1, 2, 3} больше 10.

```
List.MatchesAll({1, 2, 3}, each _ > 10)
```

`false`

List.MatchesAny

16.03.2021 • 2 minutes to read

Синтаксис

```
List.MatchesAny(list as list, condition as function) as logical
```

Сведения

Возвращает значение `true`, если функции условия `condition` удовлетворяет любое значение в списке `list`, в противном случае возвращает значение `false`.

Пример 1

Определить, есть ли в списке {9, 10, 11} хотя бы одно значение больше 10.

```
List.MatchesAny({9, 10, 11}, each _ > 10)
```

`true`

Пример 2

Определить, есть ли в списке {1, 2, 3} хотя бы одно значение больше 10.

```
List.MatchesAny({1, 2, 3}, each _ > 10)
```

`false`

List.Max

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Max(list as list, optional default as any, optional comparisonCriteria as any, optional includeNulls as nullable logical) as any
```

Сведения

Возвращает максимальный элемент в списке `list` или необязательное значение по умолчанию `default`, если список пуст. Для определения способа сравнения элементов в списке можно указать необязательное значение `comparisonCriteria` — `comparisonCriteria`. Если этот параметр имеет значение NULL, используется функция сравнения по умолчанию.

Пример 1

Найти максимальное значение в списке {1, 4, 7, 3, -2, 5}.

```
List.Max({1, 4, 7, 3, -2, 5}, 1)
```

7

Пример 2

Найти максимальное значение в списке {} или вернуть -1, если список пуст.

```
List.Max({}, -1)
```

-1

List.MaxN

16.03.2021 • 2 minutes to read

Синтаксис

```
List.MaxN(list as list, countOrCondition as any, optional comparisonCriteria as any, optional includeNulls as nullable logical) as list
```

Сведения

Возвращает максимальные значения в списке `list`. После сортировки строк можно указать необязательные параметры для дальнейшей фильтрации результатов. Необязательный параметр `countOrCondition` указывает число возвращаемых значений или условие фильтрации. Необязательный параметр `comparisonCriteria` указывает способ сравнения значений в списке.

- `list`: список значений.
- `countOrCondition`: если указано число, возвращается список до `countOrCondition` элементов по возрастанию. Если указано условие, возвращается список элементов, которые изначально соответствуют условию. Как только обнаруживается элемент, не соответствующий условию, последующие элементы не рассматриваются.
- `comparisonCriteria`: *[необязательно]* для определения способа сравнения элементов в списке можно указать необязательное значение `comparisonCriteria`. Если этот параметр имеет значение NULL, используется функция сравнения по умолчанию.

List.Median

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Median(list as list, optional comparisonCriteria as any) as any
```

Сведения

Возвращает медианный элемент из списка `list`. Эта функция возвращает `null`, если список не содержит значения, отличные от `null`. Если в списке четное число элементов, функция возвращает наименьший из двух медианных элементов, за исключением случая, когда список включает только даты, продолжительность, числа или временные метки, — в этом случае возвращается среднее значение двух этих элементов.

Пример 1

Найти медиану списка `{5, 3, 1, 7, 9}`.

```
powerquery-mList.Median({5, 3, 1, 7, 9})
```

5

List.Min

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Min(list as list, optional default as any, optional comparisonCriteria as any, optional includeNulls as nullable logical) as any
```

Сведения

Возвращает минимальный элемент в списке `list` или необязательное значение по умолчанию `default`, если список пуст. Для определения способа сравнения элементов в списке можно указать необязательное значение `comparisonCriteria` — `comparisonCriteria`. Если этот параметр имеет значение NULL, используется функция сравнения по умолчанию.

Пример 1

Найти минимальное значение в списке {1, 4, 7, 3, -2, 5}.

```
List.Min({1, 4, 7, 3, -2, 5})
```

-2

Пример 2

Найти минимум в списке {} или вернуть -1, если список пуст.

```
List.Min({}, -1)
```

-1

List.MinN

16.03.2021 • 2 minutes to read

Синтаксис

```
List.MinN(list as list, countOrCondition as any, optional comparisonCriteria as any, optional includeNulls as nullable logical) as list
```

Сведения

Возвращает минимальные значения в списке `list`. Параметр `countOrCondition` указывает число возвращаемых значений или условие фильтрации. Необязательный параметр `comparisonCriteria` указывает способ сравнения значений в списке.

- `list`: список значений.
- `countOrCondition`: если указано число, возвращается список до `countOrCondition` элементов по возрастанию. Если указано условие, возвращается список элементов, которые изначально соответствуют условию. Как только обнаруживается элемент, не соответствующий условию, последующие элементы не рассматриваются. Если этот параметр имеет значение NULL, возвращается одно наименьшее значение в списке.
- `comparisonCriteria`: *[необязательно]* для определения способа сравнения элементов в списке можно указать необязательное значение `comparisonCriteria`. Если этот параметр имеет значение NULL, используется функция сравнения по умолчанию.

Пример 1

Найти 5 наименьших значений в списке `{3, 4, 5, -1, 7, 8, 2}`.

```
List.MinN({3, 4, 5, -1, 7, 8, 2}, 5)
```

-1

2

3

4

5

List.Mode

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Mode(list as list, optional equationCriteria as any) as any
```

Сведения

Возвращает элемент, который наиболее часто появляется в списке `list`. Если список пуст, возникает исключение. Если несколько элементов встречаются с одинаковой максимальной частотой, выбирается последний из них. Для управления проверкой на равенство можно указать необязательное значение `comparisonCriteria` — `equationCriteria`.

Пример 1

Найти элемент, который появляется чаще всего в списке `{"A", 1, 2, 3, 3, 4, 5}`.

```
List.Mode({"A", 1, 2, 3, 3, 4, 5})
```

3

Пример 2

Найти элемент, который появляется чаще всего в списке `{"A", 1, 2, 3, 3, 4, 5, 5}`.

```
List.Mode({"A", 1, 2, 3, 3, 4, 5, 5})
```

5

List.Modes

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Modes(list as list, optional equationCriteria as any) as list
```

Сведения

Возвращает элемент, который наиболее часто появляется в списке `list`. Если список пуст, возникает исключение. Если несколько элементов встречаются с одинаковой максимальной частотой, выбирается последний из них. Для управления проверкой на равенство можно указать необязательное значение `comparisonCriteria` — `equationCriteria`.

Пример 1

Найти элементы, которые появляются чаще всего в списке `{"A", 1, 2, 3, 3, 4, 5, 5}`.

```
List.Modes({"A", 1, 2, 3, 3, 4, 5, 5})
```

3

5

List.NonNullCount

16.03.2021 • 2 minutes to read

Синтаксис

```
List.NonNullCount(list as list) as number
```

Сведения

Возвращает число элементов, отличных от NULL, в списке `list`.

List.Numbers

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Numbers(start as number, count as number, optional increment as nullable number) as list
```

Сведения

Возвращает список чисел для заданного исходного значения, количества и необязательного значения приращения. Значение приращения по умолчанию равно 1.

- `start` : начальное значение в списке.
- `count` : число создаваемых значений.
- `increment` : *[необязательно]* значение приращения. Если не указано, значения увеличиваются на 1.

Пример 1

Создать список из 10 последовательных чисел начиная с 1.

```
List.Numbers(1, 10)
```

1

2

3

4

5

6

7

8

9

10

Пример 2

Создать список чисел из 10 чисел, начиная с 1, с шагом 2 для каждого последующего числа.

List.Numbers(1, 10, 2)

1

3

5

7

9

11

13

15

17

19

List.Percentile

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Percentile(list as list, percentiles as any, optional options as nullable record) as any
```

Сведения

Возвращает выборку процентилей из списка `list`. Если значение `percentiles` представляет собой число от 0,0 до 1,0, то оно будет рассматриваться как процентиль, а результатом будет одно значение, соответствующее данной вероятности. Если значение `percentiles` представляет собой список чисел со значениями от 0,0 до 1,0, то результатом будет список процентилей, соответствующих входной вероятности.

Параметр `PercentileMode` в `options` позволяет использовать более точный метод интерполяции. Он предназначен для опытных пользователей и не рекомендуется для большинства стандартных случаев. Предопределенные символы `PercentileMode.ExcelInc` и `PercentileMode.ExcelExc` соответствуют методам интерполяции, используемым в функциях Excel `PERCENTILE.INC` и `PERCENTILE.EXC`. По умолчанию действует `PercentileMode.ExcelInc`. Символы `PercentileMode.SqlCont` и `PercentileMode.SqlDisc` действуют соответственно как `PERCENTILE_CONT` и `PERCENTILE_DISC` в SQL Server.

Пример 1

Поиск первого квартиля списка `{5, 3, 1, 7, 9}`.

```
List.Percentile({5, 3, 1, 7, 9}, 0.25)
```

3

Пример 2

Поиск квартилей списка `{5, 3, 1, 7, 9}` с использованием метода интерполяции, соответствующего `PERCENTILE.EXC` в Excel.

```
List.Percentile({5, 3, 1, 7, 9}, {0.25, 0.5, 0.75}, [PercentileMode=PercentileMode.ExcelExc])
```

{2, 5, 8}

List.PositionOf

16.03.2021 • 2 minutes to read

Синтаксис

```
List.PositionOf(list as list, value as any, optional occurrence as nullable number, optional  
equationCriteria as any) as any
```

Сведения

Возвращает смещение, при котором значение `value` отображается в списке `list`. Возвращает `-1`, если значение не отображается. Можно указать необязательный параметр вхождения `occurrence`.

- `occurrence`: максимальное число вхождений, которые можно включить в отчет.

Пример 1

Найти позицию в списке {1, 2, 3}, в которой появляется значение 3.

```
List.PositionOf({1, 2, 3}, 3)
```


List.PositionOfAny

16.03.2021 • 2 minutes to read

Синтаксис

```
List.PositionOfAny(list as list, values as list, optional occurrence as nullable number, optional equationCriteria as any) as any
```

Сведения

Возвращает смещение в списке `list` для первого вхождения значения в списке `values`. Возвращает значение `-1`, если вхождение не найдено. Можно указать необязательный параметр вхождения `occurrence`.

- `occurrence`: максимальное число вхождений, которые можно вернуть.

Пример 1

Найти первую позицию в списке {1, 2, 3}, в которой появляется значение 2 или 3.

```
List.PositionOfAny({1, 2, 3}, {2, 3})
```

List.Positions

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Positions(list as list) as list
```

Сведения

Возвращает список смещений для списка ввода `list`. Если для изменения списка используется `List.Transform`, список позиций может служить для предоставления преобразованию доступа к позиции.

Пример 1

Найти смещения значений в списке {1, 2, 3, 4, null, 5}.

```
List.Positions({1, 2, 3, 4, null, 5})
```

0

1

2

3

4

5

List.Product

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Product(numbersList as list, optional precision as nullable number) as nullable number
```

Сведения

Возвращает произведение чисел, отличных от NULL, в списке `numbersList`. Возвращает значение NULL, если в списке нет значений, отличных от NULL.

Пример 1

Найти произведение чисел в списке `{1, 2, 3, 3, 4, 5, 5}`.

```
List.Product({1, 2, 3, 3, 4, 5, 5})
```

```
1800
```

List.Random

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Random(count as number, optional seed as nullable number) as list
```

Сведения

Возвращает список случайных чисел от 0 до 1 с учетом количества создаваемых значений и необязательного начального значения.

- `count` : Число создаваемых случайных значений.
- `seed` : *[необязательно]* числовое значение, используемое для предоставления начального значения генератору случайных чисел. Если значение не указано, уникальный список случайных чисел создается каждый раз при вызове функции. Если указать начальное значение с числом, каждый вызов функции создает один и тот же список случайных чисел.

Пример 1

Создать список из 3 случайных чисел.

```
List.Random(3)
```

0,992332

0,132334

0,023592

Пример 2

Создать список из 3 случайных чисел с указанным начальным значением.

```
List.Random(3, 2)
```

0,883002

0,245344

0,723212

List.Range

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Range(list as list, offset as number, optional count as nullable number) as list
```

Сведения

Возвращает подмножество списка, начиная со смещения `list`. Необязательный параметр `offset` устанавливает максимальное число элементов в подмножестве.

Пример 1

Нахождение подмножества начиная со смещения 6 в списке чисел от 1 до 10.

```
List.Range({1..10}, 6)
```

7

8

9

10

Пример 2

Нахождение подмножества с количеством элементов 2 начиная со смещения 6 в списке чисел от 1 до 10.

```
List.Range({1..10}, 6, 2)
```

7

8

List.RemoveFirstN

16.03.2021 • 2 minutes to read

Синтаксис

```
List.RemoveFirstN(list as list, optional countOrCondition as any) as list
```

Сведения

Возвращает список, который удаляет первый элемент списка `list`. Если список `list` пуст, возвращается пустой список. Эта функция принимает необязательный параметр `countOrCondition` для поддержки удаления нескольких значений, как указано ниже.

- Если указано число, удалено будет количество элементов до указанного значения.
- Если указано условие, возвращаемый список начинается с первого элемента в `list`, удовлетворяющего условию. Как только обнаруживается элемент, не соответствующий условию, последующие элементы не рассматриваются.
- Если этот параметр имеет значение NULL, наблюдается поведение по умолчанию.

Пример 1

Создание списка из {1, 2, 3, 4, 5} без первых 3 цифр.

```
List.RemoveFirstN({1, 2, 3, 4, 5}, 3)
```

4

5

Пример 2

Создание списка из {5, 4, 2, 6, 1}, который начинается с числа меньше 3.

```
List.RemoveFirstN({5, 4, 2, 6, 1}, each _ > 3)
```

2

6

1

List.RemoveItems

16.03.2021 • 2 minutes to read

Синтаксис

```
List.RemoveItems(list1 as list, list2 as list) as list
```

Сведения

Удаляет все вхождения значений, имеющих в списке `list2`, из списка `list1`. Если значения в `list2` не существуют в `list1`, возвращается исходный список.

Пример 1

Удаление элементов списка {2, 4, 6} из списка {1, 2, 3, 4, 2, 5, 5}.

```
List.RemoveItems({1, 2, 3, 4, 2, 5, 5}, {2, 4, 6})
```

1

3

5

5

List.RemoveLastN

16.03.2021 • 2 minutes to read

Синтаксис

```
List.RemoveLastN(list as list, optional countOrCondition as any) as list
```

Сведения

Возвращает список, который удаляет последние `countOrCondition` элементов в конце списка `list`. Если в `list` меньше `countOrCondition` элементов, возвращается пустой список.

- Если указано число, удалено будет количество элементов до указанного значения.
- Если указано условие, возвращаемый список заканчивается первым элементом из конца `list`, удовлетворяющим условию. Как только обнаруживается элемент, не соответствующий условию, последующие элементы не рассматриваются.
- Если этот параметр имеет значение NULL, удаляется только один элемент.

Пример 1

Создание списка из {1, 2, 3, 4, 5} без последних 3 цифр.

```
List.RemoveLastN({1, 2, 3, 4, 5}, 3)
```

1

2

Пример 2

Создание списка из {5, 4, 2, 6, 4}, который оканчивается на число меньше 3.

```
List.RemoveLastN({5, 4, 2, 6, 4}, each _ > 3)
```

5

4

2

List.RemoveMatchingItems

16.03.2021 • 2 minutes to read

Синтаксис

```
List.RemoveMatchingItems(list1 as list, list2 as list, optional equationCriteria as any) as list
```

Сведения

Удаляет все вхождения значений, имеющих в списке `list2`, из списка `list1`. Если значения в `list2` не существуют в `list1`, возвращается исходный список. Для управления проверкой на равенство можно указать необязательное значение критерия равенства — `equationCriteria`.

Пример 1

Создание списка из {1, 2, 3, 4, 5, 5} без {1, 5}.

```
List.RemoveMatchingItems({1, 2, 3, 4, 5, 5}, {1, 5})
```

2

3

4

List.RemoveNulls

16.03.2021 • 2 minutes to read

Синтаксис

```
List.RemoveNulls(list as list) as list
```

Сведения

Удаляет все вхождения значений NULL из списка `list`. Если в списке нет значений NULL, возвращается исходный список.

Пример 1

Удаление значений NULL из списка {1, 2, 3, null, 4, 5, null, 6}.

```
List.RemoveNulls({1, 2, 3, null, 4, 5, null, 6})
```

1

2

3

4

5

6

List.RemoveRange

16.03.2021 • 2 minutes to read

Синтаксис

```
List.RemoveRange(list as list, index as number, optional count as nullable number) as list
```

Сведения

Удаляет `count` значений из списка `list` начиная с указанной позиции `index`.

Пример 1

Удаление трех значений из списка {1, 2, 3, 4, -6, -2, -1, 5} начиная с индекса 4.

```
List.RemoveRange({1, 2, 3, 4, -6, -2, -1, 5}, 4, 3)
```

1

2

3

4

5

List.Repeat

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Repeat(list as list, count as number) as list
```

Сведения

Возвращает список, `count` раз повторяющий исходный список `list`.

Пример 1

Создание списка, в котором {1, 2} повторяется 3 раза.

```
List.Repeat({1, 2}, 3)
```

1

2

1

2

1

2

List.ReplaceMatchingItems

16.03.2021 • 2 minutes to read

Синтаксис

```
List.ReplaceMatchingItems(list as list, replacements as list, optional equationCriteria as any)
as list
```

Сведения

Выполняет заданные замены в списке `list`. Операция замены `replacements` состоит из списка с двумя значениями — старым и новым, указанными в списке. Для управления проверкой на равенство можно указать необязательное значение критерия равенства — `equationCriteria`.

Пример 1

Создать список из {1, 2, 3, 4, 5}, заменив значение 5 на -5, а значение 1 на -1.

```
List.ReplaceMatchingItems({1, 2, 3, 4, 5}, {{5, -5}, {1, -1}})
```

-1

2

3

4

-5

List.ReplaceRange

16.03.2021 • 2 minutes to read

Синтаксис

```
List.ReplaceRange(list as list, index as number, count as number, replaceWith as list) as list
```

Сведения

Заменяет `count` значений в списке `list` списком `replaceWith` начиная с указанной позиции `index`.

Пример 1

Замена {7, 8, 9} в списке {1, 2, 7, 8, 9, 5} списком {3, 4}.

```
List.ReplaceRange({1, 2, 7, 8, 9, 5}, 2, 3, {3, 4})
```

1

2

3

4

5

List.ReplaceValue

16.03.2021 • 2 minutes to read

Синтаксис

```
List.ReplaceValue(list as list, oldValue as any, newValue as any, replacer as function) as list
```

Сведения

Ищет в списке значений `list` значение `oldValue` и заменяет каждое его вхождение значением замены `newValue`.

Пример 1

Заменить все значения "a" в списке {"a", "B", "a", "a"} значением "A".

```
List.ReplaceValue({"a", "B", "a", "a"}, "a", "A", Replacer.ReplaceText)
```

Объект
B
Объект
Объект

List.Reverse

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Reverse(list as list) as list
```

Сведения

Возвращает список со значениями в списке `list` в обратном порядке.

Пример 1

Создать список из {1..10} в обратном порядке.

```
List.Reverse({1..10})
```

10

9

8

7

6

5

4

3

2

1

List.Select

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Select(list as list, selection as function) as list
```

Сведения

Возвращает список значений из списка `list`, которые удовлетворяют условию выбора `selection`.

Пример 1

Найти в списке {1, -3, 4, 9, -2} значения больше 0.

```
List.Select({1, -3, 4, 9, -2}, each _ > 0)
```

1

4

9

List.Single

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Single(list as list) as any
```

Сведения

Если в списке `list` только один элемент, возвращает этот элемент. Если в списке больше одного элемента или список пуст, функция вызывает исключение.

Пример 1

Нахождение единственного значения в списке {1}.

```
List.Single({1})
```

1

Пример 2

Нахождение единственного значения в списке {1, 2, 3}.

```
List.Single({1, 2, 3})
```

```
[Expression.Error] There were too many elements in the enumeration to complete the operation.
```

List.SingleOrDefault

16.03.2021 • 2 minutes to read

Синтаксис

```
List.SingleOrDefault(list as list, optional default as any) as any
```

Сведения

Если в списке `list` только один элемент, возвращает этот элемент. Если этот список пуст, функция возвращает значение NULL, если не указан необязательный параметр `default`. Если в списке больше одного элемента, функция возвращает ошибку.

Пример 1

Нахождение единственного значения в списке {1}.

```
List.SingleOrDefault({1})
```

1

Пример 2

Нахождение единственного значения в списке {}.

```
List.SingleOrDefault({})
```

null

Пример 3

Нахождение единственного значения в списке {}. Если список пуст, возвращается значение -1.

```
List.SingleOrDefault({}, -1)
```

-1

List.Skip

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Skip(list as list, optional countOrCondition as any) as list
```

Сведения

Возвращает список, который пропускает первый элемент списка `list`. Если список `list` пуст, возвращается пустой список. Эта функция принимает необязательный параметр `countOrCondition` для поддержки пропуска нескольких значений, как указано ниже.

- Если указано число, пропускается количество элементов до указанного значения.
- Если указано условие, возвращаемый список начинается с первого элемента в `list`, удовлетворяющего условию. Как только обнаруживается элемент, не соответствующий условию, последующие элементы не рассматриваются.
- Если этот параметр имеет значение NULL, наблюдается поведение по умолчанию.

Пример 1

Создание списка из {1, 2, 3, 4, 5} без первых 3 цифр.

```
List.Skip({1, 2, 3, 4, 5}, 3)
```

4

5

Пример 2

Создание списка из {5, 4, 2, 6, 1}, который начинается с числа меньше 3.

```
List.Skip({5, 4, 2, 6, 1}, each _ > 3)
```

2

6

1

List.Sort

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Sort(list as list, optional comparisonCriteria as any) as list
```

Сведения

Сортирует список данных `list` по необязательным указанным критериям. Можно указать необязательный параметр `comparisonCriteria` как критерий сравнения. Он может принимать следующие значения:

- Для управления порядком критерий сравнения может быть значением перечисления Order: (`Order.Descending`, `Order.Ascending`).
- Чтобы вычислить ключ, используемый для сортировки, можно использовать функцию с одним аргументом.
- Чтобы выбрать ключ и порядок, критерием сравнения может быть список, содержащий ключ и порядок (`{each 1 / _, Order.Descending}`).
- Чтобы полностью управлять сравнением, можно использовать функцию с двумя аргументами, которая возвращает -1, 0 или 1, учитывая связь между левыми и правыми входными данными. `Value.Compare` — это метод, который можно использовать для делегирования этой логики.

Пример 1

Сортировка списка {2, 3, 1}.

```
List.Sort({2, 3, 1})
```

1

2

3

Пример 2

Сортировка списка {2, 3, 1} в порядке убывания.

```
List.Sort({2, 3, 1}, Order.Descending)
```

3

2

1

Пример 3

Сортировка списка {2, 3, 1} в порядке убывания с помощью метода Value.Compare.

```
List.Sort({2, 3, 1}, (x, y) => Value.Compare(1/x, 1/y))
```

3

2

1

List.Split

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Split(list as list, pageSize as number) as list
```

Сведения

Разделяет `list` на список списков, где первый элемент выходного списка — список с первыми элементами (`pageSize`) из исходного списка, следующий элемент выходного списка — список со следующими элементами (`pageSize`) из исходного списка и т. д.

List.StandardDeviation

16.03.2021 • 2 minutes to read

Синтаксис

```
List.StandardDeviation(numbersList as list) as nullable number
```

Сведения

Возвращает основанную на выборке оценку стандартного отклонения значений в списке `numbersList`. Если `numbersList` — это список чисел, возвращается число. Исключение возникает при пустом списке или списке элементов, не имеющих тип `number`.

Пример 1

Нахождение стандартного отклонения чисел от 1 до 5.

```
List.StandardDeviation({1..5})
```

```
1.5811388300841898
```


List.Sum

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Sum(list as list, optional precision as nullable number) as any
```

Сведения

Возвращает сумму всех значений в списке `list`, отличных от NULL. Возвращает значение NULL, если в списке нет значений, отличных от NULL.

Пример 1

Нахождение суммы чисел в списке `{1, 2, 3}`.

```
List.Sum({1, 2, 3})
```

List.Times

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Times(start as time, count as number, step as duration) as list
```

Сведения

Возвращает список значений `time` с размером `count` начиная с `start`. Шаг приращения `step` является значением `duration`, которое добавляется к каждому значению.

Пример 1

Создание списка из 4 значений, начиная с полудня (`#time(12, 0, 0)`), путем последовательного увеличения времени на один час (`#duration(0, 1, 0, 0)`).

```
List.Times(#time(12, 0, 0), 4, #duration(0, 1, 0, 0))
```

12:00:00

13:00:00

14:00:00

15:00:00

List.Transform

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Transform(list as list, transform as function) as list
```

Сведения

Возвращает новый список значений, применяя функцию преобразования `transform` к списку `list`.

Пример 1

Добавление 1 к каждому значению в списке {1, 2}.

```
List.Transform({1, 2}, each _ + 1)
```

2

3

List.TransformMany

16.03.2021 • 2 minutes to read

Синтаксис

```
List.TransformMany(list as list, collectionTransform as function, resultTransform as function)  
as list
```

Сведения

Возвращает список, элементы которого проецируются из входного списка. Функция `collectionTransform` применяется к каждому элементу, а функция `resultTransform` вызывается для создания результирующего списка. `collectionSelector` имеет сигнатуру (x в любом формате) => ..., где x — это элемент в списке. `resultTransform` проецирует форму результата и имеет сигнатуру (x в любом формате, y в любом формате) => ..., где x — элемент в списке, а y — элемент, полученный путем применения `collectionTransform` к этому элементу.

List.Union

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Union(lists as list, optional equationCriteria as any) as list
```

Сведения

Принимает список списков `lists`, объединяет элементы в отдельных списках и возвращает их в список вывода. Возвращаемый список содержит все элементы в любых входных списках. Эта операция сохраняет традиционную семантику мультимножества, поэтому дублирующиеся значения сопоставляются в составе объединения. Для управления проверкой на равенство можно указать необязательное значение критерия равенства — `equationCriteria`.

Пример 1

Создать объединение списков {1..5}, {2..6}, {3..7}.

```
List.Union({{1..5}, {2..6}, {3..7}})
```

1

2

3

4

5

6

7

List.Zip

16.03.2021 • 2 minutes to read

Синтаксис

```
List.Zip(lists as list) as list
```

Сведения

Принимает список списков `lists` и возвращает список списков, объединяя элементы на одной позиции.

Пример 1

Пакует два простых списка {1, 2} и {3, 4}.

```
List.Zip({1, 2}, {3, 4})
```

[Список]

[Список]

Пример 2

Пакует два простых списка разной длины {1, 2} и {3}.

```
List.Zip({1, 2}, {3})
```

[Список]

[Список]

PercentileMode.ExcelExc

16.03.2021 • 2 minutes to read

Сведения

При интерполяции значений для `List.Percentile` нужно будет использовать метод, совместимый с `PERCENTILE.EXC` в Excel.

PercentileMode.ExcelInc

16.03.2021 • 2 minutes to read

Сведения

При интерполяции значений для `List.Percentile` нужно будет использовать метод, совместимый с `PERCENTILE.INC` в Excel.

PercentileMode.SqlCont

16.03.2021 • 2 minutes to read

Сведения

При интерполяции значений для `List.Percentile` нужно будет использовать метод, совместимый с `PERCENTILE_CONT` в SQL Server.

PercentileMode.SqlDisc

17.03.2021 • 2 minutes to read

Сведения

При интерполяции значений для `List.Percentile` нужно будет использовать метод, совместимый с `PERCENTILE_DISC` в SQL Server.

Логические функции

16.03.2021 • 2 minutes to read

Эти функции позволяют создавать логические значения (true/false) и управлять ими.

Логические

ФУНКЦИЯ	ОПИСАНИЕ
Logical.From	Возвращает логическое значение для значения.
Logical.FromText	Возвращает логическое значение true или false для текстового значения.
Logical.ToText	Возвращает текстовое значение для логического значения.

Logical.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Logical.From(value as any) as nullable logical
```

Сведения

Возвращает значение `logical` из заданного параметра `value`. Если заданный параметр `value` имеет значение `null`, `Logical.From` возвращает `null`. Если заданный параметр `value` имеет значение `logical`, возвращается `value`.

Значения следующих типов можно преобразовать в значение `logical`:

- `text`: значение из текстового значения `logical` — `"true"` либо `"false"`. Дополнительные сведения см. в разделе `Logical.FromText`.
- `number`: `false`, если `value` имеет значение `0`, в противном случае — `true`.

Если `value` имеет любой другой тип, возвращается ошибка.

Пример 1

Преобразование `2` в значение `logical`.

```
Logical.From(2)
```

```
true
```

Logical.FromText

16.03.2021 • 2 minutes to read

Синтаксис

```
Logical.FromText(text as nullable text) as nullable logical
```

Сведения

Создает логическое значение из текстового значения `text` — "true" или "false". Если `text` содержит другую строку, возникает исключение. Текстовое значение `text` не учитывает регистр.

Пример 1

Создание логического значения из текстовой строки "true".

```
Logical.FromText("true")
```

```
true
```

Пример 2

Создание логического значения из текстовой строки "a".

```
Logical.FromText("a")
```

```
[Expression.Error] Could not convert to a logical.
```

Logical.ToText

16.03.2021 • 2 minutes to read

Синтаксис

```
Logical.ToText(logicalValue as nullable logical) as nullable text
```

Сведения

Создает текстовое значение из логического значения `logicalValue` — `true` либо `false`. Если `logicalValue` не является логическим значением, возникает исключение.

Пример 1

Создание текстового значения из логического `true`.

```
Logical.ToText(true)
```

```
"true"
```

Функции для работы с числами

16.03.2021 • 5 minutes to read

Эти функции позволяют создавать числовые значения и управлять ими.

Число

Константы

ФУНКЦИЯ	ОПИСАНИЕ
Number.E	Возвращает 2,7182818284590451, то есть число e с точностью до 16 знаков после запятой.
Number.Epsilon	Возвращает наименьшее возможное число.
Number.NaN	Представляет деление 0/0.
Number.NegativeInfinity	Представляет деление $-1/0$.
Number.PI	Возвращает 3,1415926535897931, то есть число π с точностью до 16 знаков после запятой.
Number.PositiveInfinity	Представляет деление $1/0$.

Сведения

ФУНКЦИЯ	ОПИСАНИЕ
Number.IsEven	Возвращает true, если значение является четным числом.
Number.IsNaN	Возвращает true, если значение является Number.NaN.
Number.IsOdd	Возвращает true, если значение является нечетным числом.

Преобразование и форматирование

ФУНКЦИЯ	ОПИСАНИЕ
Byte.From	Создает 8-разрядное целое число по заданному значению.
Currency.From	Возвращает значение валюты для заданного значения.
Decimal.From	Возвращает значение типа Decimal для заданного значения.
Double.From	Возвращает значение типа Double для заданного значения.

ФУНКЦИЯ	ОПИСАНИЕ
Int8.From	Создает 8-разрядное целое число со знаком по заданному значению.
Int16.From	Создает 16-разрядное целое число по заданному значению.
Int32.From	Создает 32-разрядное целое число по заданному значению.
Int64.From	Создает 64-разрядное целое число по заданному значению.
Number.From	Возвращает числовое значение для заданного значения.
Number.FromText	Возвращает числовое значение для текстового значения.
Number.ToText	Возвращает текстовое значение для числового значения.
Percentage.From	Возвращает значение процента для заданного значения.
Single.From	Возвращает значение типа Single для заданного значения.

Округление

ФУНКЦИЯ	ОПИСАНИЕ
Number.Round	Возвращает число, допускающее значение NULL (n), если значение является целым числом.
Number.RoundAwayFromZero	Возвращает Number.RoundUp(значение) , если значение ≥ 0 , и Number.RoundDown(значение) , если значение < 0 .
Number.RoundDown	Возвращает наибольшее целое число, меньшее или равное числовому значению.
Number.RoundTowardZero	Возвращает Number.RoundDown(x) , если $x \geq 0$, и Number.RoundUp(x) , если $x < 0$.
Number.RoundUp	Возвращает наибольшее целое число, большее или равное числовому значению.

Операции

ФУНКЦИЯ	ОПИСАНИЕ
Number.Abs	Возвращает абсолютное значение числа.
Number.Combinations	Возвращает число комбинаций для заданного числа элементов с необязательным размером комбинации.

ФУНКЦИЯ	ОПИСАНИЕ
Number.Exp	Возвращает число, представляющее результат возведения числа e в степень.
Number.Factorial	Возвращает факториал числа.
Number.IntegerDivide	Делит два числа и возвращает целочисленную часть результата.
Number.Ln	Возвращает натуральный логарифм числа.
Number.Log	Возвращает логарифм числа по заданному основанию.
Number.Log10	Возвращает десятичный логарифм числа.
Number.Mod	Делит два числа и возвращает остаток от деления.
Number.Permutations	Возвращает общее число перестановок для заданного числа элементов с необязательным размером перестановки.
Number.Power	Возвращает число, возведенное в степень.
Number.Sign	Возвращает 1 для положительных чисел, -1 для отрицательных чисел или 0 для нуля.
Number.Sqrt	Возвращает квадратный корень числа.

Случайные

ФУНКЦИЯ	ОПИСАНИЕ
Number.Random	Возвращает случайное дробное число от 0 до 1.
Number.RandomBetween	Возвращает случайное число, лежащее в указанных пределах.

Тригонометрические

ФУНКЦИЯ	ОПИСАНИЕ
Number.Acos	Возвращает арккосинус числа.
Number.Asin	Возвращает арксинус числа.
Number.Atan	Возвращает арктангенс числа.
Number.Atan2	Возвращает арктангенс частного от деления двух чисел.
Number.Cos	Возвращает косинус числа.
Number.Cosh	Возвращает гиперболический косинус числа.

ФУНКЦИЯ	ОПИСАНИЕ
Number.Sin	Возвращает синус числа.
Number.Sinh	Возвращает гиперболический синус числа.
Number.Tan	Возвращает тангенс числа.
Number.Tanh	Возвращает гиперболический тангенс числа.

Байты

ФУНКЦИЯ	ОПИСАНИЕ
Number.BitwiseAnd	Возвращает результат выполнения побитовой операции И с указанными операндами.
Number.BitwiseNot	Возвращает результат выполнения побитовой операции НЕ с указанными операндами.
Number.BitwiseOr	Возвращает результат выполнения побитовой операции ИЛИ с указанными операндами.
Number.BitwiseShiftLeft	Возвращает результат выполнения побитовой операции сдвига влево с указанными операндами.
Number.BitwiseShiftRight	Возвращает результат выполнения побитовой операции сдвига вправо с указанными операндами.
Number.BitwiseXor	Возвращает результат выполнения побитовой операции исключающего ИЛИ с указанными операндами.

ЗНАЧЕНИЯ ПАРАМЕТРОВ	ОПИСАНИЕ
RoundingMode.AwayFromZero	RoundingMode.AwayFromZero
RoundingMode.Down	RoundingMode.Down
RoundingMode.ToEven	RoundingMode.ToEven
RoundingMode.TowardZero	RoundingMode.TowardZero
RoundingMode.Up	RoundingMode.Up

Byte.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Byte.From(value as any, optional culture as nullable text, optional roundingMode as nullable number) as nullable number
```

Сведения

Создает 8-разрядное целое число `number` по заданному `value`. Если заданный параметр `value` имеет значение `null`, `Byte.From` возвращает `null`. Если заданный параметр `value` является `number` в диапазоне 8-разрядного целого числа без дробной части, возвращается `value`. Если он содержит дробную часть, то число округляется с заданным режимом округления. По умолчанию используется режим округления `RoundingMode.ToEven`. Если указанный параметр `value` относится к любому другому типу, используйте `Number.FromText` для преобразования его в значение `number`, после чего применяется предыдущая инструкция по преобразованию значения `number` в 8-разрядное целое число `number`. Доступные режимы округления см. в разделе `Number.Round`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение 8-разрядного целого числа `number` для `"4"`.

```
Byte.From("4")
```

4

Пример 2

Получение 8-разрядного целого числа `number` для `"4.5"` с помощью `RoundingMode.AwayFromZero`.

```
Byte.From("4.5", null, RoundingMode.AwayFromZero)
```

5

=

Currency.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Currency.From(value as any, optional culture as nullable text, optional roundingMode as nullable number) as nullable number
```

Сведения

Возвращает значение `currency` из заданного параметра `value`. Если заданный параметр `value` имеет значение `null`, `Currency.From` возвращает `null`. Если заданный параметр `value` является значением `number` в диапазоне для валюты, дробная часть `value` округляется до 4 десятичных разрядов и возвращается. Если заданный параметр `value` является значением любого другого типа, используйте `Number.FromText` для преобразования его в значение `number`, после чего применяется предыдущая инструкция по преобразованию значения `number` в значение `currency`. Допустимый диапазон для валюты — от `-922,337,203,685,477.5808` до `922,337,203,685,477.5807`. Доступные режимы округления см. в разделе `Number.Round`. Значение по умолчанию — `RoundingMode.ToEven`. Можно также указать необязательный параметр `culture` (например, `en-US`).

Пример 1

Получение значения `currency` для `"1.23455"`.

```
Currency.From("1.23455")
```

1.2346

Пример 2

Получение значения `currency` для `"1.23455"` с использованием `RoundingMode.Down`.

```
Currency.From("1.23455", "en-US", RoundingMode.Down)
```

1.2345

Decimal.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Decimal.From(value as any, optional culture as nullable text) as nullable number
```

Сведения

Возвращает значение `number` типа `Decimal` из данного значения `value`. Если заданный параметр `value` имеет значение `null`, `Decimal.From` возвращает `null`. Если данное значение `value` является значением `number` в диапазоне `Decimal`, возвращается `value`; в противном случае возвращается ошибка. Если данное значение `value` является значением любого другого типа, используйте `Number.FromText` для преобразования его в значение `number`, после чего применяется предыдущая инструкция по преобразованию значения `number` в значение `number` типа `Decimal`. Можно также указать необязательный параметр `culture` (например, `en-US`).

Пример 1

Получение значения `number` типа `Decimal` для `"4.5"`.

```
Decimal.From("4.5")
```

```
4.5
```

Double.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Double.From(value as any, optional culture as nullable text) as nullable number
```

Сведения

Возвращает значение `number` типа `Double` из данного значения `value`. Если заданный параметр `value` имеет значение `null`, `Double.From` возвращает `null`. Если данное значение `value` является значением `number` в диапазоне `Double`, возвращается `value`; в противном случае возвращается ошибка. Если данное значение `value` является значением любого другого типа, используйте `Number.FromText` для преобразования его в значение `number`, после чего применяется предыдущая инструкция по преобразованию значения `number` в значение `number` типа `Double`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение значения `number` типа `Double` для `"4"`.

```
Double.From("4.5")
```

```
4.5
```

Int8.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Int8.From(value as any, optional culture as nullable text, optional roundingMode as nullable number) as nullable number
```

Сведения

Создает 8-разрядное целое число со знаком `number` по заданному `value`. Если заданный параметр `value` имеет значение `null`, `Int8.From` возвращает `null`. Если заданный параметр `value` является `number` в диапазоне 8-разрядного целого числа со знаком без дробной части, возвращается `value`. Если он содержит дробную часть, то число округляется с заданным режимом округления. По умолчанию используется режим округления `RoundingMode.ToEven`. Если заданный `value` относится к любому другому типу, см. раздел `Number.FromText` для преобразования его в значение `number`, а затем применяется предыдущая инструкция по преобразованию значения `number` в 8-разрядное целое число `number`. Доступные режимы округления см. в разделе `Number.Round`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение 8-разрядного целого числа со знаком `number` для `"4"`.

```
Int8.From("4")
```

4

Пример 2

Получение 8-разрядного целого числа со знаком `number` для `"4.5"` с помощью `RoundingMode.AwayFromZero`.

```
Int8.From("4.5", null, RoundingMode.AwayFromZero)
```

5

Int16.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Int16.From(value as any, optional culture as nullable text, optional roundingMode as nullable number) as nullable number
```

Сведения

Создает 16-разрядное целое число `number` по заданному `value`. Если заданный параметр `value` имеет значение `null`, `Int16.From` возвращает `null`. Если заданный параметр `value` является `number` в диапазоне 16-разрядного целого числа без дробной части, возвращается `value`. Если он содержит дробную часть, то число округляется с заданным режимом округления. По умолчанию используется режим округления `RoundingMode.ToEven`. Если заданный `value` относится к любому другому типу, см. раздел `Number.FromText` для преобразования его в значение `number`, а затем применяется предыдущая инструкция по преобразованию значения `number` в 16-разрядное целое число `number`. Доступные режимы округления см. в разделе `Number.Round`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение 16-разрядного целого числа `number` для `"4"`.

```
Int16.From("4")
```

4

Пример 2

Получение 16-разрядного целого числа `number` для `"4.5"` с помощью `RoundingMode.AwayFromZero`.

```
Int16.From("4.5", null, RoundingMode.AwayFromZero)
```

5

Int32.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Int32.From(value as any, optional culture as nullable text, optional roundingMode as nullable number) as nullable number
```

Сведения

Создает 32-разрядное целое число `number` по заданному `value`. Если заданный параметр `value` имеет значение `null`, `Int32.From` возвращает `null`. Если заданный `value` является `number` в диапазоне 32-разрядного целого числа без дробной части, возвращается `value`. Если он содержит дробную часть, то число округляется с заданным режимом округления. По умолчанию используется режим округления `RoundingMode.ToEven`. Если заданный `value` относится к любому другому типу, см. раздел `Number.FromText` для преобразования его в значение `number`, а затем применяется предыдущая инструкция по преобразованию значения `number` в 32-разрядное целое число `number`. Доступные режимы округления см. в разделе `Number.Round`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение 32-разрядного целого числа `number` для `"4"`.

```
Int32.From("4")
```

4

Пример 2

Получение 32-разрядного целого числа `number` для `"4.5"` с помощью `RoundingMode.AwayFromZero`.

```
Int32.From("4.5", null, RoundingMode.AwayFromZero)
```

5

Int64.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Int64.From(value as any, optional culture as nullable text, optional roundingMode as nullable number) as nullable number
```

Сведения

Создает 64-разрядное целое число `number` по заданному `value`. Если заданный параметр `value` имеет значение `null`, `Int64.From` возвращает `null`. Если заданный параметр `value` является `number` в диапазоне 64-разрядного целого числа без дробной части, возвращается `value`. Если он содержит дробную часть, то число округляется с заданным режимом округления. По умолчанию используется режим округления `RoundingMode.ToEven`. Если заданный `value` относится к любому другому типу, см. раздел `Number.FromText` для преобразования его в значение `number`, а затем применяется предыдущая инструкция по преобразованию значения `number` в 64-разрядное целое число `number`. Доступные режимы округления см. в разделе `Number.Round`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение 64-разрядного целого числа `number` для `"4"`.

```
Int64.From("4")
```

4

Пример 2

Получение 64-разрядного целого числа `number` для `"4.5"` с помощью `RoundingMode.AwayFromZero`.

```
Int64.From("4.5", null, RoundingMode.AwayFromZero)
```

5

Number.Abs

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Abs(number as nullable number) as nullable number
```

Сведения

Возвращает абсолютное значение `number`. Если `number` равно NULL, `Number.Abs` возвращает значение NULL.

- `number`: Число `number`, абсолютное значение которого требуется определить.

Пример 1

Абсолютное значение -3.

```
Number.Abs(-3)
```

Number.Acos

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Acos(number as nullable number) as nullable number
```

Сведения

Возвращает арккосинус `number`.

Number.Asin

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Asin(number as nullable number) as nullable number
```

Сведения

Возвращает арксинус `number`.

Number.Atan

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Atan(number as nullable number) as nullable number
```

Сведения

Возвращает арктангенс `number` .

Number.Atan2

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Atan2(y as nullable number, x as nullable number) as nullable number
```

Сведения

Возвращает арктангенс частного от деления двух чисел `y` и `x`. Деление будет осуществляться в виде `y / x`.

Number.BitwiseAnd

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.BitwiseAnd(number1 as nullable number, number2 as nullable number) as nullable number
```

Сведения

Возвращает результат выполнения побитовой операции AND со значениями `number1` и `number2`.

Number.BitwiseNot

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.BitwiseNot(number as any) as any
```

Сведения

Возвращает результат выполнения побитовой операции NOT над `number`.

Number.BitwiseOr

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.BitwiseOr(number1 as nullable number, number2 as nullable number) as nullable number
```

Сведения

Возвращает результат выполнения побитовой операции OR со значениями `number1` и `number2`.

Number.BitwiseShiftLeft

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.BitwiseShiftLeft(number1 as nullable number, number2 as nullable number) as nullable number
```

Сведения

Возвращает результат выполнения побитового сдвига влево для `number1` на указанное число битов `number2`.

Number.BitwiseShiftRight

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.BitwiseShiftRight(number1 as nullable number, number2 as nullable number) as nullable number
```

Сведения

Возвращает результат выполнения побитового сдвига вправо для `number1` на указанное число битов `number2`.

Number.BitwiseXor

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.BitwiseXor(number1 as nullable number, number2 as nullable number) as nullable number
```

Сведения

Возвращает результат выполнения побитовой операции XOR (исключающее ИЛИ) со значениями

`number1` и `number2` .

Number.Combinations

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Combinations(setSize as nullable number, combinationSize as nullable number) as nullable number
```

Сведения

Возвращает число уникальных комбинаций из списка элементов `setSize` с указанным размером комбинации `combinationSize`.

- `setSize`: число элементов в списке.
- `combinationSize`: число элементов в каждой комбинации.

Пример 1

Поиск количества комбинаций из 5 элементов при условии, что комбинация состоит из 3 элементов.

```
Number.Combinations(5, 3)
```

10

Number.Cos

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Cos(number as nullable number) as nullable number
```

Сведения

Возвращает косинус `number`.

Пример 1

Найти косинус угла 0.

```
Number.Cos(0)
```

Number.Cosh

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Cosh(number as nullable number) as nullable number
```

Сведения

Возвращает гиперболический косинус `number`.

Number.E

16.03.2021 • 2 minutes to read

Сведения

Константа, равная 2,7182818284590451, то есть числу e с точностью до 16 десятичных знаков.

Number.Epsilon

16.03.2021 • 2 minutes to read

Сведения

Константа, представляющая наименьшее положительное число, которое может содержать число с плавающей запятой.

Number.Exp

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Exp(number as nullable number) as nullable number
```

Сведения

Возвращает результат возведения e в степень `number` (экспоненциальная функция).

- `number`: число `number`, для которого требуется вычислить экспоненциальную функцию. Если `number` равно NULL, `Number.Exp` возвращает значение NULL.

Пример 1

Возведение e в степень 3.

```
Number.Exp(3)
```

```
20.085536923187668
```

Number.Factorial

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Factorial(number as nullable number) as nullable number
```

Сведения

Возвращает факториал числа `number`.

Пример 1

Найти факториал 10.

```
Number.Factorial(10)
```

```
3628800
```

Number.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.From(value as any, optional culture as nullable text) as nullable number
```

Сведения

Возвращает значение `number` из заданного параметра `value`. Можно также указать необязательный параметр `culture` (например, en-US). Если заданный параметр `value` имеет значение `null`, `Number.From` возвращает `null`. Если заданный параметр `value` имеет значение `number`, возвращается `value`.

Значения следующих типов можно преобразовать в значение `number`:

- `text`: значение `number` из текстового представления. Обрабатываются стандартные текстовые форматы ("15", "3423,10", "5,0E-10"). Дополнительные сведения см. в разделе `Number.FromText`.
- `logical`: 1 для `true`, 0 для `false`.
- `datetime`: число с плавающей запятой двойной точности, содержащее эквивалент даты OLE-автоматизации.
- `datetimezone`: число с плавающей запятой двойной точности, содержащее эквивалент даты OLE-автоматизации для местных даты и времени `value`.
- `date`: число с плавающей запятой двойной точности, содержащее эквивалент даты OLE-автоматизации.
- `time`: выражается в числе неполных дней.
- `duration`: выражается в числе полных и неполных дней.

Если `value` имеет любой другой тип, возвращается ошибка.

Пример 1

Получение значения `number` для `"4"`.

```
powerquery-mNumber.From("4")
```

4

Пример 2

Получение значения `number` для `#datetime(2020, 3, 20, 6, 0, 0)`.

```
Number.From(#datetime(2020, 3, 20, 6, 0, 0))
```

43910.25

Пример 3

Получение значения `number` для `"12.3%"`.

```
Number.From("12.3%")
```

```
0.123
```

Number.FromText

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.FromText(text as nullable text, optional culture as nullable text) as nullable number
```

О программе

Возвращает значение `number` из заданного текстового значения `text`.

- `text` — текстовое представление числового значения. Представление должно быть в общем числовом формате, например "15", "3423,10" или "5,0E-10".
- `culture` — Необязательные параметры языка и региональные параметры, управляющие интерпретацией `text` (например, en-US).

Пример 1

Получение числового значения для `"4"`.

```
Number.FromText("4")
```

4

Пример 2

Получение числового значения для `"5.0e-10"`.

```
Number.FromText("5.0e-10")
```

5E-10

Number.IntegerDivide

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.IntegerDivide(number1 as nullable number, number2 as nullable number, optional precision as nullable number) as nullable number
```

Сведения

Возвращает целочисленную часть результата от деления числа `number1` на число `number2`. Если `number1` или `number2` равны NULL, `Number.IntegerDivide` возвращает значение NULL.

- `number1` : делимое.
- `number2` : делитель.

Пример 1

Деление 6 на 4.

```
Number.IntegerDivide(6, 4)
```

1

Пример 2

Деление 8,3 на 3.

```
Number.IntegerDivide(8.3, 3)
```

2

Number.IsEven

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.IsEven(number as number) as logical
```

Сведения

Указывает, четно ли значение `number`, возвращая `true`, если оно четно, и `false` в противном случае.

Пример 1

Проверка четности числа 625.

```
Number.IsEven(625)
```

```
false
```

Пример 2

Проверка четности числа 82.

```
Number.IsEven(82)
```

```
true
```

Number.IsNaN

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.IsNaN(number as number) as logical
```

Сведения

Указывает, является ли значение нечисловым (NaN). Возвращает `true`, если значение `number` эквивалентно `Number.IsNaN`, в противном случае возвращает `false`.

Пример 1

Проверка того, будет ли получено NaN при делении 0 на 0.

```
Number.IsNaN(0/0)
```

```
true
```

Пример 2

Проверка того, будет ли получено NaN при делении 1 на 0.

```
Number.IsNaN(1/0)
```

```
false
```

Number.IsOdd

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.IsOdd(number as number) as logical
```

Сведения

Указывает, нечетно ли число. Возвращает `true`, если `number` является нечетным числом, в противном случае возвращает `false`.

Пример 1

Проверка того, является ли число 625 нечетным.

```
Number.IsOdd(625)
```

```
true
```

Пример 2

Проверка того, является ли число 82 нечетным.

```
Number.IsOdd(82)
```

```
false
```

Number.Ln

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Ln(number as nullable number) as nullable number
```

Сведения

Возвращает натуральный логарифм числа `number`. Если `number` равно NULL, `Number.Ln` возвращает значение NULL.

####

Пример 1 Получение натурального логарифма числа 15.

```
Number.Ln(15)
```

```
2.70805020110221
```

Number.Log

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Log(number as nullable number, optional base as nullable number) as nullable number
```

Сведения

Возвращает логарифм числа `number` по основанию `base`. Если `base` не указано, применяется значение по умолчанию — `Number.E`. Если `number` равно `NULL`, `Number.Log` возвращает значение `NULL`.

Пример 1

Получение десятичного логарифма для 2.

```
Number.Log(2, 10)
```

```
0.3010299956639812
```

Пример 2

Получение натурального логарифма для 2.

```
Number.Log(2)
```

```
0.69314718055994529
```

Number.Log10

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Log10(number as nullable number) as nullable number
```

Сведения

Возвращает десятичный логарифм числа `number`. Если `number` равно NULL, `Number.Log10` возвращает значение NULL.

Пример 1

Получение десятичного логарифма для 2.

```
Number.Log10(2)
```

```
0.3010299956639812
```

Number.Mod

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Mod(number as nullable number, divisor as nullable number, optional precision as nullable number) as nullable number
```

Сведения

Возвращает остаток от деления целого числа `number` на `divisor`. Если `number` или `divisor` равны NULL, `Number.Mod` возвращает значение NULL.

- `number`: делимое.
- `divisor`: делитель.

Пример 1

Поиск остатка от деления 5 на 3.

```
Number.Mod(5, 3)
```

Number.NaN

16.03.2021 • 2 minutes to read

Сведения

Константа, представляющая 0, деленный на 0.

Number.NegativeInfinity

16.03.2021 • 2 minutes to read

Сведения

Константа, представляющая -1 , деленное на 0 .

Number.Permutations

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Permutations(setSize as nullable number, permutationSize as nullable number) as nullable number
```

Сведения

Возвращает число перестановок, которое можно создать из нескольких элементов `setSize`, с указанным размером перестановки `permutationSize`.

Пример 1

Определение числа перестановок для 5 элементов по 3.

```
Number.Permutations(5, 3)
```

Number.PI

16.03.2021 • 2 minutes to read

Сведения

Константа, равная 3,1415926535897932, то есть числу Пи с точностью до 16 десятичных знаков.

Number.PositiveInfinity

16.03.2021 • 2 minutes to read

Сведения

Константа, представляющая 1, деленное на 0.

Number.Power

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Power(number as nullable number, power as nullable number) as nullable number
```

Сведения

Возвращает результат возведения `number` в степень `power`. Если `number` или `power` равны NULL, `Number.Power` возвращает значение NULL.

- `number` : основание.
- `power` : показатель степени.

Пример 1

Нахождение значения 5 в степени 3 (5 в кубе).

```
Number.Power(5, 3)
```

```
125
```

Number.Random

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Random() as number
```

Сведения

Возвращает случайное число от 0 до 1.

Пример 1

Возвращает случайное число.

```
Number.Random()
```

```
0.919303
```

Number.RandomBetween

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.RandomBetween(bottom as number, top as number) as number
```

Сведения

Возвращает случайное число от `bottom` до `top`.

Пример 1

Получить случайное число между 1 и 5.

```
Number.RandomBetween(1, 5)
```

```
2.546797
```

Number.Round

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Round(number as nullable number, optional digits as nullable number, optional roundingMode as nullable number) as nullable number
```

Сведения

Возвращает результат округления `number` до ближайшего числа. Если `number` равно NULL, `Number.Round` возвращает значение NULL. `number` округляется до ближайшего целого числа, если не указан необязательный параметр `digits`. Если задано `digits`, `number` округляется до `digits` знаков после запятой. Необязательный параметр `roundingMode` указывает направление округления, если округление возможно в обе стороны (возможные значения см. в описании `RoundingMode.Type`).

Пример 1

Округлить 1,234 до ближайшего целого числа.

```
Number.Round(1.234)
```

1

Пример 2

Округлить 1,56 до ближайшего целого числа.

```
Number.Round(1.56)
```

2

Пример 3

Округлить 1,2345 до двух десятичных разрядов.

```
Number.Round(1.2345, 2)
```

1.23

Пример 4

Округлить 1,2345 до трех десятичных разрядов (с округлением в большую сторону).

```
Number.Round(1.2345, 3, RoundingMode.Up)
```

1.235

Пример 5

Округлить 1,2345 до трех десятичных разрядов (с округлением в меньшую сторону).

```
Number.Round(1.2345, 3, RoundingMode.Down)
```

```
1.234
```

Number.RoundAwayFromZero

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.RoundAwayFromZero(number as nullable number, optional digits as nullable number) as nullable number
```

Сведения

Возвращает результат округления `number` в зависимости от знака числа. Эта функция будет округлять положительные числа в большую сторону, а отрицательные — в меньшую. Если задано `digits`, `number` округляется до `digits` знаков после запятой.

Пример 1

Округлить число -1,2 в сторону от нуля.

```
Number.RoundAwayFromZero(-1.2)
```

-2

Пример 2

Округлить число 1,2 в сторону от нуля.

```
Number.RoundAwayFromZero(1.2)
```

2

Пример 3

Округлить число -1,234 от нуля до двух знаков после запятой.

```
Number.RoundAwayFromZero(-1.234, 2)
```

-1.24

Number.RoundDown

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.RoundDown(number as nullable number, optional digits as nullable number) as nullable number
```

Сведения

Возвращает результат округления `number` в меньшую сторону к предыдущему наибольшему целому числу. Если `number` равно NULL, `Number.RoundDown` возвращает значение NULL. Если задано `digits`, `number` округляется до `digits` знаков после запятой.

Пример 1

Округлить 1,234 в меньшую сторону до целого числа.

```
Number.RoundDown(1.234)
```

1

Пример 2

Округлить 1,999 в меньшую сторону до целого числа.

```
Number.RoundDown(1.999)
```

1

Пример 3

Округлить 1,999 до двух десятичных разрядов.

```
Number.RoundDown(1.999, 2)
```

1.99

Number.RoundTowardZero

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.RoundTowardZero(number as nullable number, optional digits as nullable number) as nullable number
```

Сведения

Возвращает результат округления `number` в зависимости от знака числа. Эта функция будет округлять положительные числа в меньшую сторону, а отрицательные — в большую. Если задано `digits`, `number` округляется до `digits` знаков после запятой.

Number.RoundUp

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.RoundUp(number as nullable number, optional digits as nullable number) as nullable number
```

Сведения

Возвращает результат округления `number` в меньшую сторону к предыдущему наибольшему целому числу. Если `number` равно NULL, `Number.RoundDown` возвращает значение NULL. Если задано `digits`, `number` округляется до `digits` знаков после запятой.

Пример 1

Округление 1,234 до целого числа.

```
Number.RoundUp(1.234)
```

2

Пример 2

Округление 1,999 до целого числа.

```
Number.RoundUp(1.999)
```

2

Пример 3

Округление числа 1,234 до двух знаков после запятой.

```
Number.RoundUp(1.234, 2)
```

1.24

Number.Sign

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Sign(number as nullable number) as nullable number
```

Сведения

Возвращает 1, если `number` является положительным числом; значение `-1`, если отрицательным, и 0, если оно равно нулю. Если `number` равно NULL, `Number.Sign` возвращает значение NULL.

Пример 1

Определение знака числа 182.

```
Number.Sign(182)
```

1

Пример 2

Определение знака числа `-182`.

```
Number.Sign(-182)
```

-1

Пример 3

Определение знака числа 0.

```
Number.Sign(0)
```

0

Number.Sin

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Sin(number as nullable number) as nullable number
```

Сведения

Возвращает синус `number`.

Пример 1

Найти синус угла 0.

```
Number.Sin(0)
```

```
0
```

Number.Sinh

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Sinh(number as nullable number) as nullable number
```

Сведения

Возвращает гиперболический синус `number`.

Number.Sqrt

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Sqrt(number as nullable number) as nullable number
```

Сведения

Возвращает квадратный корень числа `number`. Если `number` равно NULL, `Number.Sqrt` возвращает значение NULL. Если это отрицательное значение, возвращается значение `Number.NaN` (не число).

Пример 1

Нахождение квадратного корня числа 625.

```
Number.Sqrt(625)
```

```
25
```

Пример 2

Нахождение квадратного корня числа 85.

```
Number.Sqrt(85)
```

```
9.2195444572928871
```

Number.Tan

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Tan(number as nullable number) as nullable number
```

Сведения

Возвращает тангенс `number`.

Пример 1

Найти тангенс угла 1.

```
Number.Tan(1)
```

```
1.5574077246549023
```

Number.Tanh

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.Tanh(number as nullable number) as nullable number
```

Сведения

Возвращает гиперболический тангенс `number`.

Number.ToText

16.03.2021 • 2 minutes to read

Синтаксис

```
Number.ToText(number as nullable number, optional format as nullable text, optional culture as nullable text) as nullable text
```

О программе

Преобразует числовое значение `number` в текстовое в соответствии с форматом, указанным в `format`. В качестве формата используется одиночный символ, к которому может добавляться указатель точности числа. Для `format` можно использовать перечисленные ниже коды символов.

- "D" или "d": (десятичное) результат представляется в виде целого числа. Указатель точности определяет количество цифр в выходных данных.
- "E" или "e": (Exponential): экспоненциальная запись. Указатель точности определяет максимальное количество десятичных разрядов (по умолчанию 6).
- "F" или "f": (с фиксированной запятой) целая и десятичная дробная части.
- "G" или "g": (общее) наиболее компактная форма из формы с фиксированной запятой и экспоненциальной.
- "N" или "n": (число) целая и десятичная дробная части с разделителями разрядов и десятичным разделителем.
- "P" или "p": (проценты) число, умноженное на 100 и отображаемое со знаком процента.
- "R" или "r": (для обратного преобразования) текстовое значение, которое может быть преобразовано обратно в идентичное число. Указатель точности не учитывается.
- "X" или "x": (шестнадцатеричное) шестнадцатеричное текстовое значение.

Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Представление числа в виде текста без указания формата.

```
Number.ToText(4)
```

```
"4"
```

Пример 2

Представление числа в виде текста в экспоненциальном формате.

```
Number.ToText(4, "e")
```

```
"4.000000e+000"
```

Пример 3

Форматирование числа в виде текста в десятичном формате с ограниченной точностью.

```
Number.ToText(-0.1234, "P1")
```

```
"-12.3 %"
```

Percentage.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Percentage.From(value as any, optional culture as nullable text) as nullable number
```

Сведения

Возвращает значение `percentage` из заданного параметра `value`. Если заданный параметр `value` имеет значение `null`, `Percentage.From` возвращает `null`. Если значение заданного `value` — это `text` с символом процента в конце, то будет возвращено преобразованное десятичное число. В противном случае используйте `Number.From` для преобразования в значение `number`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение значения `percentage` для `"12.3%"`.

```
Percentage.From("12.3%")
```

```
0.123
```

RoundingMode.AwayFromZero

16.03.2021 • 2 minutes to read

Сведения

RoundingMode.AwayFromZero

RoundingMode.Down

16.03.2021 • 2 minutes to read

Сведения

RoundingMode.Down

RoundingMode.ToEven

16.03.2021 • 2 minutes to read

Сведения

RoundingMode.ToEven

RoundingMode.TowardZero

16.03.2021 • 2 minutes to read

Сведения

RoundingMode.TowardZero

RoundingMode.Up

16.03.2021 • 2 minutes to read

Сведения

RoundingMode.Up

Single.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Single.From(value as any, optional culture as nullable text) as nullable number
```

Сведения

Возвращает значение `number` типа `Single` из данного значения `value`. Если заданный параметр `value` имеет значение `null`, `Single.From` возвращает `null`. Если данное значение `value` является значением `number` в диапазоне `Single`, возвращается `value`; в противном случае возвращается ошибка. Если данное значение `value` является значением любого другого типа, используйте `Number.FromText` для преобразования его в значение `number`, после чего применяется предыдущая инструкция по преобразованию значения `number` в значение `number` типа `Single`. Можно также указать необязательный параметр `culture` (например, `en-US`).

Пример 1

Получение значения `number` типа `Single` для `"1.5"`.

```
Single.From("1.5")
```

```
1.5
```

Функции для работы с записями

16.03.2021 • 5 minutes to read

Эти функции позволяют создавать значения записей и управлять ими.

Запись

Информация

ФУНКЦИЯ	ОПИСАНИЕ
Record.FieldCount	Возвращает число полей в записи.
Record.HasFields	Возвращает значение true, если в записи содержатся имена одного или нескольких полей.

Преобразования

ФУНКЦИЯ	ОПИСАНИЕ
Geography.FromWellKnownText	Преобразует текст, представляющий географическое значение в формате WKT, в структурированную запись.
Geography.ToWellKnownText	Преобразует значение структурированной географической точки в представление формата Well-Known Text (WKT).
GeographyPoint.From	Создает запись, представляющую географическую точку по ее компонентам.
Geometry.FromWellKnownText	Преобразует текст, представляющий геометрическое значение в формате Well-Known Text (WKT), в структурированную запись.
Geometry.ToWellKnownText	Преобразует значение структурированной геометрической точки в представление формата Well-Known Text (WKT).
GeometryPoint.From	Создает запись, представляющую геометрическую точку по ее компонентам.
Record.AddField	Добавляет поле, используя имя поля и значение.
Record.Combine	Объединяет записи в списке.
Record.RemoveFields	Возвращает новую запись с измененным порядком заданных полей относительно друг друга. Остальные поля, которые не указаны, остаются в исходном положении.

ФУНКЦИЯ	ОПИСАНИЕ
Record.RenameFields	Возвращает новую запись, в которой заданные поля переименованы. Исходный порядок полученных полей при этом сохраняется. Эта функция поддерживает перестановку и связывание имен полей. Тем не менее все целевые имена и имена оставшихся полей должны составлять уникальный набор. В противном случае возвращается ошибка.
Record.ReorderFields	Возвращает новую запись с измененным порядком полей относительно друг друга. Остальные поля, которые не указаны, остаются в исходном положении. Необходимо указать несколько полей.
Record.TransformFields	Преобразует поля с помощью transformOperations. Дополнительные сведения о значениях, поддерживаемых transformOperations, см. в разделе "Значения параметров".

Выбранное

ФУНКЦИЯ	ОПИСАНИЕ
Record.Field	Возвращает значение заданного поля. С помощью этой функции можно динамически создавать синтаксис поиска полей для заданной записи. В этом случае получается динамическая версия синтаксиса записи [поле].
Record.FieldNames	Возвращает список имен полей в том порядке, в котором они представлены в записи.
Record.FieldOrDefault	Возвращает значение поля в записи или значение по умолчанию, если поле не существует.
Record.FieldValues	Возвращает список значений полей в том порядке, в котором они представлены в записи.
Record.SelectFields	Возвращает новую запись, которая содержит поля, выбранные из входной записи. При этом сохраняется исходный порядок полей.

Сериализация

КОМПОНЕНТ	ОПИСАНИЕ
Record.FromList	Возвращает запись для данного списка значений полей и набора полей.
Record.FromTable	Возвращает запись из таблицы записей, которая содержит имена полей и значения.
Record.ToList	Возвращает список значений, содержащий значения полей из входной записи.

КОМПОНЕНТ	ОПИСАНИЕ
Record.ToTable	Возвращает таблицу записей, которая содержит имена полей и значения из входной записи.

Значения параметров

Следующие определения типа используются для описания значений параметров для представленных выше функций работы с записями.

ОПРЕДЕЛЕНИЕ ТИПА	ОПИСАНИЕ
Параметр MissingField	MissingField.Error = 0; MissingField.Ignore = 1; MissingField.UseNull = 2;
Операции преобразования	Операции преобразования могут задаваться с использованием любого из следующих значений: Значение, представляющее собой список из двух элементов, первый из которых определяет имя поля, а второй — функцию преобразования, которая применяется к этому полю для получения нового значения. Список преобразований определяется с помощью значения списка, а также каждого элемента, который представляет собой значение списка из двух элементов, как описывается выше. Примеры см. в описании Record.TransformFields.
Операции переименования	Операции переименования для записи могут задаваться любым из описываемых ниже способов: Отдельная операция переименования, которая представляет собой список из двух элементов, содержащий старое и новое имя. Примеры см. в описании Record.RenameFields.

Geography.FromWellKnownText

16.03.2021 • 2 minutes to read

Синтаксис

```
Geography.FromWellKnownText(input as nullable text) as nullable record
```

О программе

Преобразует текст, представляющий географическое значение в формате WKT, в структурированную запись. WKT — стандартный формат, определяемый организацией Open Geospatial Consortium (OGC), который является стандартным форматом сериализации, используемым базами данных, в том числе SQL Server.

Geography.ToWellKnownText

16.03.2021 • 2 minutes to read

Синтаксис

```
Geography.ToWellKnownText(input as nullable record, optional omitSRID as nullable logical) as nullable text
```

О программе

Преобразует значение структурированной географической точки в представление формата Well-Known Text (WKT), определяемого организацией Open Geospatial Consortium (OGC), который используется для сериализации во многих базах данных, включая SQL Server.

GeographyPoint.From

16.03.2021 • 2 minutes to read

Синтаксис

```
GeographyPoint.From(longitude as number, latitude as number, optional z as nullable number,  
optional m as nullable number, optional srid as nullable number) as record
```

О программе

Создает запись, представляющую географическую точку из составных частей, например долготы, широты и, если она есть, высоты над уровнем моря (Z) и меры (M). Необязательный идентификатор пространственной ссылки (SRID) можно предоставить, если он отличается от значения по умолчанию (4326).

Geometry.FromWellKnownText

16.03.2021 • 2 minutes to read

Синтаксис

```
Geometry.FromWellKnownText(input as nullable text) as nullable record
```

О программе

Преобразует текст, представляющий геометрическое значение в формате Well-Known Text (WKT), в структурированную запись. WKT — стандартный формат, определяемый организацией Open Geospatial Consortium (OGC), который является стандартным форматом сериализации, используемым базами данных, в том числе SQL Server.

Geometry.ToWellKnownText

16.03.2021 • 2 minutes to read

Синтаксис

```
Geometry.ToWellKnownText(input as nullable record, optional omitsRID as nullable logical) as nullable text
```

О программе

Преобразует значение структурированной геометрической точки в представление формата Well-Known Text (WKT), определяемого организацией Open Geospatial Consortium (OGC), который используется для сериализации во многих базах данных, включая SQL Server.

GeometryPoint.From

16.03.2021 • 2 minutes to read

Синтаксис

```
GeometryPoint.From(x as number, y as number, optional z as nullable number, optional m as nullable number, optional srid as nullable number) as record
```

О программе

Создает запись, представляющую геометрическую точку из составляющих ее частей, например координату X, координату Y и, при наличии, координаты Z и меру (M). Необязательный идентификатор пространственной ссылки (SRID) можно предоставить, если он отличается от значения по умолчанию (0).

MissingField.Error

16.03.2021 • 2 minutes to read

Сведения

Необязательный параметр в записях и табличных функциях, указывающий, что отсутствующие поля должны приводить к ошибке. (Это значение параметра по умолчанию.)

MissingField.Ignore

16.03.2021 • 2 minutes to read

Сведения

Необязательный параметр в записях и табличных функциях, указывающий, что отсутствующие поля должны пропускаться.

MissingField.UseNull

16.03.2021 • 2 minutes to read

Сведения

Необязательный параметр в записях и табличных функциях, указывающий, что отсутствующие поля должны быть включены со значениями NULL.

Record.AddField

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.AddField(record as record, fieldName as text, value as any, optional delayed as nullable logical) as record
```

Сведения

Добавляет поле к записи `record` для заданного имени поля `fieldName` и значения `value`.

Пример 1

Добавление поля Address к записи.

```
Record.AddField([CustomerID = 1, Name = "Bob", Phone = "123-4567"], "Address", "123 Main St.")
```

CUSTOMERID	1
NAME	Bob
НОМЕР ТЕЛЕФОНА	123-4567
АДРЕС	123 Main St.

Record.Combine

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.Combine(records as list) as record
```

Сведения

Объединяет записи в данном `records`. Если `records` содержит значения, отличные от записей, возвращается ошибка.

Пример 1

Создание комбинированной записи из записей.

```
Record.Combine({  
    [CustomerID = 1, Name = "Bob"],  
    [Phone = "123-4567"]  
})
```

CUSTOMERID	1
NAME	Bob
НОМЕР ТЕЛЕФОНА	123-4567

Record.Field

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.Field(record as record, field as text) as any
```

Сведения

Возвращает значение указанного `field` в `record`. Если поле не найдено, возникает исключение.

Пример 1

Нахождение значения поля CustomerID в записи.

```
Record.Field([CustomerID = 1, Name = "Bob", Phone = "123-4567"], "CustomerID")
```

Record.FieldCount

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.FieldCount(record as record) as number
```

Сведения

Возвращает число полей в записи `record`.

Пример 1

Нахождение числа полей в записи.

```
Record.FieldCount([CustomerID = 1, Name = "Bob"])
```

Record.FieldNames

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.FieldNames(record as record) as list
```

Сведения

Возвращает имена полей в записи `record` в виде текста.

Пример 1

Нахождение имен полей в записи.

```
Record.FieldNames([OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0])
```

OrderID
CustomerID
Элемент
Цена

Record.FieldOrDefault

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.FieldOrDefault(record as nullable record, field as text, optional defaultValue as any) as any
```

Сведения

Возвращает значение указанного поля `field` в записи `record`. Если поле не найдено, возвращается необязательное значение `defaultValue`.

Пример 1

Нахождение значения поля Phone в записи или возврат значения NULL, если оно не существует.

```
Record.FieldOrDefault([CustomerID = 1, Name = "Bob"], "Phone")
```

```
null
```

Пример 2

Нахождение значения поля Phone в записи или возврат значения по умолчанию, если оно не существует.

```
Record.FieldOrDefault([CustomerID = 1, Name = "Bob"], "Phone", "123-4567")
```

```
"123-4567"
```

Record.FieldValues

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.FieldValues(record as record) as list
```

Сведения

Возвращает список значений полей в записи `record`.

Пример 1

Нахождение значений полей в записи.

```
Record.FieldValues([CustomerID = 1, Name = "Bob", Phone = "123-4567"])
```

1

Владимир

123-4567

Record.FromList

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.FromList(list as list, fields as any) as record
```

Сведения

Возвращает запись для данного списка `list` значений полей и набора полей. `fields` можно задать либо как список текстовых значений, либо как тип "запись". Если поля неуникальны, выдается ошибка.

Пример 1

Создание записи из списка значений полей и списка имен полей.

```
Record.FromList({1, "Bob", "123-4567"}, {"CustomerID", "Name", "Phone"})
```

CUSTOMERID	1
NAME	Bob
НОМЕР ТЕЛЕФОНА	123-4567

Пример 2

Создание записи из списка значений полей и записи.

```
Record.FromList({1, "Bob", "123-4567"}, type [CustomerID = number, Name = text, Phone = number])
```

CUSTOMERID	1
NAME	Bob
НОМЕР ТЕЛЕФОНА	123-4567

Record.FromTable

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.FromTable(table as table) as record
```

Сведения

Возвращает запись из таблицы записей `table`, которая содержит имена полей и имена значений

`{[Name = name, Value = value]}`. Если имена полей неуникальны, вызывает исключение.

Пример 1

Создание записи из таблицы формы Table.FromRecords({[Name = "CustomerID", Value = 1], [Name = "Name", Value = "Bob"], [Name = "Phone", Value = "123-4567"]}).

```
Record.FromTable(  
    Table.FromRecords({  
        [Name = "CustomerID", Value = 1],  
        [Name = "Name", Value = "Bob"],  
        [Name = "Phone", Value = "123-4567"]  
    })  
)
```

CUSTOMERID	1
NAME	Bob
НОМЕР ТЕЛЕФОНА	123-4567

Record.HasFields

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.HasFields(record as record, fields as any) as logical
```

Сведения

Указывает, содержит ли запись `record` поля, заданные в `fields`, возвращая логическое значение (true или false). Значения нескольких полей можно указать с помощью списка.

Пример 1

Проверка, содержится ли в записи поле CustomerID.

```
Record.HasFields([CustomerID = 1, Name = "Bob", Phone = "123-4567"], "CustomerID")
```

```
true
```

Пример 2

Проверка, содержатся ли в записи поля CustomerID и Address.

```
Record.HasFields([CustomerID = 1, Name = "Bob", Phone = "123-4567"], {"CustomerID", "Address"})
```

```
false
```

Record.RemoveFields

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.RemoveFields(record as record, fields as any, optional missingField as nullable number)
as record
```

Сведения

Возвращает запись, которая удаляет все поля, указанные в списке `fields`, из входных данных `record`.
Если указанное поле не существует, возникает исключение.

Пример 1

Удаление поля Price из записи.

```
Record.RemoveFields([CustomerID = 1, Item = "Fishing rod", Price = 18.00], "Price")
```

CUSTOMERID	1
ЭЛЕМЕНТ	Удочка

Пример 2

Удаление поля Price и Item из записи.

```
Record.RemoveFields([CustomerID = 1, Item = "Fishing rod", Price = 18.00], {"Price", "Item"})
```

CUSTOMERID	1
------------	---

Record.RenameFields

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.RenameFields(record as record, renames as list, optional missingField as nullable number)
as record
```

Сведения

Возвращает запись после присвоения полям во входных данных `record` новых имен, указанных в списке `renames`. Для нескольких переименований можно использовать вложенный список ({ {старое1, новое1}, {старое2, новое2} }).

Пример 1

Переименование поля UnitPrice в Price в записи.

```
Record.RenameFields(
    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", UnitPrice = 100.0],
    {"UnitPrice", "Price"}
)
```

ORDERID	1
CUSTOMERID	1
ЭЛЕМЕНТ	Удочка
ЦЕНА	100

Пример 2

Переименование поля UnitPrice в Price, а поле OrderNum в OrderID в записи.

```
Record.RenameFields(
    [OrderNum = 1, CustomerID = 1, Item = "Fishing rod", UnitPrice = 100.0],
    {
        {"UnitPrice", "Price"},
        {"OrderNum", "OrderID"}
    }
)
```

ORDERID	1
CUSTOMERID	1
ЭЛЕМЕНТ	Удочка

ЦЕНА	100
------	-----

Record.ReorderFields

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.ReorderFields(record as record, fieldOrder as list, optional missingField as nullable number) as record
```

Сведения

Возвращает запись после изменения порядка полей в `record` согласно порядку, указанному в списке `fieldOrder`. Значения полей сохраняются, а поля, не указанные в `fieldOrder`, остаются в исходном положении.

Пример 1

Изменение порядка некоторых полей в записи.

```
Record.ReorderFields(  
    [CustomerID = 1, OrderID = 1, Item = "Fishing rod", Price = 100.0],  
    {"OrderID", "CustomerID"}  
)
```

ORDERID	1
CUSTOMERID	1
ЭЛЕМЕНТ	Удочка
ЦЕНА	100

Record.SelectFields

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.SelectFields(record as record, fields as any, optional missingField as nullable number)
as record
```

Сведения

Возвращает запись, которая включает только поля, указанные в списке `fields`, из входных данных `record`.

Пример 1

Выбрать поля Item и Price в записи.

```
Record.SelectFields(
    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],
    {"Item", "Price"}
)
```

ЭЛЕМЕНТ	Удочка
ЦЕНА	100

Record.ToList

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.ToList(record as record) as list
```

Сведения

Возвращает список значений, содержащих значения полей из входных данных `record`.

Пример 1

Извлечь значения полей из записи.

```
Record.ToList([A = 1, B = 2, C = 3])
```

1

2

3

Record.ToTable

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.ToTable(record as record) as table
```

Сведения

Возвращает таблицу, содержащую столбцы `Name` и `Value` со строкой для каждого поля в `record`.

Пример 1

Возврат таблицы из записи.

```
Record.ToTable([OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0])
```

NAME	ЗНАЧЕНИЕ
OrderID	1
CustomerID	1
Элемент	Удочка
Цена	100

Record.TransformFields

16.03.2021 • 2 minutes to read

Синтаксис

```
Record.TransformFields(record as record, transformOperations as list, optional missingField as nullable number) as record
```

Сведения

Возвращает запись после применения преобразований, указанных в списке `transformOperations`, в `record`. Одновременно может быть преобразовано одно поле или несколько.

В случае одного преобразуемого поля предполагается, что `transformOperations` — список с двумя элементами. Первый элемент в `transformOperations` указывает имя поля, а второй элемент в `transformOperations` указывает функцию, используемую для преобразования. Например:

```
{"Quantity", Number.FromText}
```

В случае преобразования нескольких полей предполагается, что `transformOperations` — список списков, где каждый внутренний список представляет собой пару из имени поля и операции преобразования. Например:

```
{{"Quantity", Number.FromText}, {"UnitPrice", Number.FromText}}
```

Пример 1

Преобразовать поле Price в число.

```
Record.TransformFields(  
    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = "100.0"],  
    {"Price", Number.FromText}  
)
```

ORDERID	1
CUSTOMERID	1
ЭЛЕМЕНТ	Удочка
ЦЕНА	100

Пример 2

Преобразовать поля OrderID и Price в числа.

```
Record.TransformFields(  
    [OrderID = "1", CustomerID = 1, Item = "Fishing rod", Price = "100.0"],  
    {"OrderID", Number.FromText}, {"Price", Number.FromText}  
)
```

ORDERID	1
CUSTOMERID	1
ЭЛЕМЕНТ	Удочка
ЦЕНА	100

Функции замены

16.03.2021 • 2 minutes to read

Эти функции используются другими функциями в библиотеке для замены заданного значения.

Функции замены

ФУНКЦИЯ	ОПИСАНИЕ
Replacer.ReplaceText	Эта функция предоставляется для функции List.ReplaceValue или Table.ReplaceValue и обеспечивает замену текстовых значений в значениях списка и таблицы соответственно.
Replacer.ReplaceValue	Эта функция предоставляется для функции List.ReplaceValue или Table.ReplaceValue и обеспечивает замену значений в значениях списка и таблицы соответственно.

Replacer.ReplaceText

16.03.2021 • 2 minutes to read

Синтаксис

```
Replacer.ReplaceText(text as nullable text, old as text, new as text) as nullable text
```

Сведения

Заменяет текст `old` в исходном тексте `text` текстом `new`. Эту функцию замены можно использовать в `List.ReplaceValue` и `Table.ReplaceValue`.

Пример 1

Заменить текст "hE" на "He" в строке "hEllo world".

```
Replacer.ReplaceText("hEllo world", "hE", "He")
```

```
"Hello world"
```

Replacer.ReplaceValue

16.03.2021 • 2 minutes to read

Синтаксис

```
Replacer.ReplaceValue(value as any, old as any, new as any) as any
```

Сведения

Заменяет значение `old` в исходном значении `value` значением `new`. Эту функцию замены можно использовать в `List.ReplaceValue` и `Table.ReplaceValue`.

Пример 1

Заменить значение 11 значением 10.

```
Replacer.ReplaceValue(11, 11, 10)
```

10

Функции разделения

16.03.2021 • 2 minutes to read

Эти функции позволяют разделить текст.

Разделитель

ФУНКЦИЯ	ОПИСАНИЕ
Splitter.SplitByNothing	Возвращает функцию, которая не разбивает текст, возвращая свой аргумент как единый список элементов.
Splitter.SplitTextByCharacterTransition	Возвращает функцию, которая разделяет текст на текстовый список в соответствии с переходом от одного типа символов к другому.
Splitter.SplitTextByAnyDelimiter	Возвращает функцию, которая разделяет текст, используя любой поддерживаемый разделитель.
Splitter.SplitTextByDelimiter	Возвращает функцию, которая будет разбивать текст в соответствии с разделителем.
Splitter.SplitTextByEachDelimiter	Возвращает функцию, которая разбивает текст каждым разделителем по очереди.
Splitter.SplitTextByLengths	Возвращает функцию, которая разбивает текст в соответствии с указанными длинами.
Splitter.SplitTextByPositions	Возвращает функцию, которая разбивает текст в соответствии с указанными позициями.
Splitter.SplitTextByRanges	Возвращает функцию, которая разбивает текст в соответствии с указанными диапазонами.
Splitter.SplitTextByRepeatedLengths	Возвращает функцию, которая разбивает текст на текстовый список в цикле, после каждого отрезка указанной длины.
Splitter.SplitTextByWhitespace	Возвращает функцию, которая разбивает текст в соответствии с пробелами.

ЗНАЧЕНИЯ ПАРАМЕТРОВ	ОПИСАНИЕ
QuoteStyle.Csv	Символы кавычек указывают начало строки в кавычках. Вложенные кавычки обозначаются двумя символами кавычек.
QuoteStyle.None	Выбор символов кавычек не имеет значения.

QuoteStyle.Csv

16.03.2021 • 2 minutes to read

Сведения

Символы кавычек указывают начало строки в кавычках. Вложенные кавычки обозначаются двумя символами кавычек.

QuoteStyle.None

16.03.2021 • 2 minutes to read

Сведения

Выбор символов кавычек не имеет значения.

Splitter.SplitByNothing

16.03.2021 • 2 minutes to read

Синтаксис

```
Splitter.SplitByNothing() as function
```

Сведения

Возвращает функцию, которая не разбивает текст, возвращая свой аргумент как единый список элементов.

Splitter.SplitTextByAnyDelimiter

16.03.2021 • 2 minutes to read

Синтаксис

```
Splitter.SplitTextByAnyDelimiter(delimiters as list, optional quoteStyle as nullable number,  
optional startAtEnd as nullable logical) as function
```

Сведения

Возвращает функцию, которая разбивает текст на текстовый список по всем указанным разделителям.

Splitter.SplitTextByCharacterTransition

16.03.2021 • 2 minutes to read

Синтаксис

```
Splitter.SplitTextByCharacterTransition(before as anynonnull, after as anynonnull) as function
```

Сведения

Возвращает функцию, которая разделяет текст на текстовый список в соответствии с переходом от одного типа символов к другому. Параметры `before` и `after` могут быть либо списком символов, либо функцией, которая принимает символ и возвращает значение true/false.

Splitter.SplitTextByDelimiter

16.03.2021 • 2 minutes to read

Синтаксис

```
Splitter.SplitTextByDelimiter(delimiter as text, optional quoteStyle as nullable number) as function
```

Сведения

Возвращает функцию, которая разбивает текст на текстовый список в соответствии с указанным разделителем.

Splitter.SplitTextByEachDelimiter

16.03.2021 • 2 minutes to read

Синтаксис

```
Splitter.SplitTextByEachDelimiter(delimiters as list, optional quoteStyle as nullable number,  
optional startAtEnd as nullable logical) as function
```

Сведения

Возвращает функцию, которая разбивает текст на текстовый список по всем заданным разделителям в последовательности.

Splitter.SplitTextByLengths

16.03.2021 • 2 minutes to read

Синтаксис

```
Splitter.SplitTextByLengths(lengths as list, optional startAtEnd as nullable logical) as function
```

Сведения

Возвращает функцию, которая разбивает текст на текстовый список по каждой указанной длине.

Splitter.SplitTextByPositions

16.03.2021 • 2 minutes to read

Синтаксис

```
Splitter.SplitTextByPositions(positions as list, optional startAtEnd as nullable logical) as function
```

Сведения

Возвращает функцию, которая разбивает текст на текстовый список по всем указанным позициям.

Splitter.SplitTextByRanges

16.03.2021 • 2 minutes to read

Синтаксис

```
Splitter.SplitTextByRanges(ranges as list, optional startAtEnd as nullable logical) as function
```

Сведения

Возвращает функцию, которая разбивает текст на текстовый список по заданным значениям смещения и длины.

Splitter.SplitTextByRepeatedLengths

16.03.2021 • 2 minutes to read

Синтаксис

```
Splitter.SplitTextByRepeatedLengths(length as number, optional startAtEnd as nullable logical)  
as function
```

Сведения

Возвращает функцию, которая разбивает текст на текстовый список в цикле, после каждого отрезка указанной длины.

Splitter.SplitTextByWhitespace

16.03.2021 • 2 minutes to read

Синтаксис

```
Splitter.SplitTextByWhitespace(optional quoteStyle as nullable number) as function
```

Сведения

Возвращает функцию, которая разбивает текст на текстовый список по пробелам.

Табличные функции

16.03.2021 • 25 minutes to read

Эти функции позволяют создавать табличные значения и управлять ими.

Конструирование таблиц

ФУНКЦИЯ	ОПИСАНИЕ
ItemExpression.From	Возвращает AST для тела функции.
ItemExpression.Item	Узел AST, представляющий элемент в выражении элемента.
RowExpression.Column	Возвращает AST, представляющий доступ к столбцу в выражении строки.
RowExpression.From	Возвращает AST для тела функции.
RowExpression.Row	Узел AST, представляющий строку в выражении строки.
Table.FromColumns	Возвращает таблицу из списка, содержащего вложенные списки с именами и значениями столбцов.
Table.FromList	Преобразует список в таблицу путем применения заданной функции разбиения к каждому элементу в списке.
Table.FromRecords	Возвращает таблицу из списка записей.
Table.FromRows	Создает таблицу из списка, в которой каждый элемент списка является списком, содержащим значения столбца для одной строки.
Table.FromValue	Возвращает таблицу со столбцом, содержащим указанное значение или список значений.
Table.FuzzyGroup	На основе нечеткого соответствия группирует строки таблицы со значениями строк указанного столбца.
Table.FuzzyJoin	Соединяет строки из двух таблиц, обладающие нечетким соответствием по заданным ключам.
Table.FuzzyNestedJoin	Выполняет нечеткое соединение таблиц по указанным столбцам и выводит результат соединения в новом столбце.
Table.Split	Разделяет указанную таблицу на список таблиц с заданным размером страницы.

ФУНКЦИЯ	ОПИСАНИЕ
<code>Table.View</code>	Создает или расширяет таблицу, используя определенные пользователем обработки операций запросов и действий.
<code>Table.ViewFunction</code>	Создает функцию, которая может быть перехвачена обработчиком, определенным в представлении (через <code>Table.View</code>).

Преобразования

ФУНКЦИЯ	ОПИСАНИЕ
<code>Table.ToColumns</code>	Возвращает список вложенных списков, каждый из которых представляет столбец значений во входной таблице.
<code>Table.ToList</code>	Возвращает таблицу в список путем применения заданной функции объединения к каждой строке значений в таблице.
<code>Table.ToRecords</code>	Возвращает список записей из входной таблицы.
<code>Table.ToRows</code>	Возвращает вложенный список значений из входной таблицы.

Информация

ФУНКЦИЯ	ОПИСАНИЕ
<code>Table.ColumnCount</code>	Возвращает количество столбцов в таблице.
<code>Table.IsEmpty</code>	Возвращает значение true, если таблица не содержит строк.
<code>Table.Profile</code>	Возвращает профиль столбцов таблицы.
<code>Table.RowCount</code>	Возвращает количество строк в таблице.
<code>Table.Schema</code>	Возвращает таблицу, содержащую описание столбцов (например, схему) указанной таблицы.
<code>Tables.GetRelationships</code>	Возвращает связи между набором таблиц.

Операции со строками

ФУНКЦИЯ	ОПИСАНИЕ
<code>Table.AlternateRows</code>	Возвращает таблицу, содержащую шаблон чередования строк из таблицы.

ФУНКЦИЯ	ОПИСАНИЕ
Table.Combine	Возвращает таблицу, полученную в результате объединения списка таблиц. Все таблицы должны иметь одинаковую структуру типов строк.
Table.FindText	Возвращает таблицу, содержащую только строки с указанным текстом в одной из ячеек или их части.
Table.First	Возвращает первую строку таблицы.
Table.FirstN	Возвращает первые строки таблицы в зависимости от параметра countOrCondition.
Table.FirstValue	Возвращает первый столбец первой строки таблицы или указанное значение по умолчанию.
Table.FromPartitions	Возвращает таблицу, которая является результатом объединения набора секционированных таблиц в новые столбцы. При необходимости можно указать тип столбца. Значение по умолчанию — any.
Table.InsertRows	Возвращает таблицу со списком строк, вставленных в таблицу по индексу. Каждая вставляемая строка должна соответствовать типу строки таблицы.
Table.Last	Возвращает последнюю строку таблицы.
Table.LastN	Возвращает последние строки таблицы в зависимости от параметра countOrCondition.
Table.MatchesAllRows	Возвращает значение true, если все строки в таблице соответствуют условию.
Table.MatchesAnyRows	Возвращает значение true, если любые строки в таблице соответствуют условию.
Table.Partition	Разделяет таблицу на список групп таблиц на основе значения столбца каждой строки и хэш-функции. Хэш-функция применяется к значению столбца строки для получения хэш-значения для строки. Остаток от деления хэш-значений определяет, в какой из возвращаемых таблиц будет помещена строка.
Table.Range	Возвращает указанное количество строк из таблицы, начиная со смещения.
Table.RemoveFirstN	Возвращает таблицу с указанным числом строк, удаленным из таблицы, начиная с первой строки. Число удаленных строк зависит от необязательного параметра countOrCondition.
Table.RemoveLastN	Возвращает таблицу с указанным числом строк, удаленным из таблицы, начиная с последней строки. Число удаленных строк зависит от необязательного параметра countOrCondition.

ФУНКЦИЯ	ОПИСАНИЕ
Table.RemoveRows	Возвращает таблицу с указанным числом строк, удаленным из таблицы, начиная со смещения.
Table.RemoveRowsWithErrors	Возвращает таблицу со всеми строками, удаленными из таблицы, которые содержат ошибку хотя бы в одной ячейке строки.
Table.Repeat	Возвращает таблицу, содержащую строки таблицы, повторяющиеся указанное количество раз.
Table.ReplaceRows	Возвращает таблицу, в которой указанное количество строк, начинающееся со смещения, заменяется предоставленными строками.
Table.ReverseRows	Возвращает таблицу со строками в обратном порядке.
Table.SelectRows	Возвращает таблицу, содержащую только те строки, которые соответствуют условию.
Table.SelectRowsWithErrors	Возвращает таблицу только со строками таблицы, которые содержат ошибку хотя бы в одной ячейке строки.
Table.SingleRow	Возвращает одну строку из таблицы.
Table.Skip	Возвращает таблицу, которая не содержит первую строку или строки таблицы.

Операции со столбцами

ФУНКЦИЯ	ОПИСАНИЕ
Table.Column	Возвращает значения столбца в таблице.
Table.ColumnNames	Возвращает имя столбцов в таблице.
Table.ColumnsOfType	Возвращает список с именами столбцов, соответствующих указанному типу.
Table.DemoteHeaders	Понижает уровень строки заголовка до первой строки таблицы.
Table.DuplicateColumn	Дублирует столбец с указанным именем. Значения и тип копируются из исходного столбца.
Table.HasColumns	Возвращает значение true, если таблица содержит указанный столбец или столбцы.

ФУНКЦИЯ	ОПИСАНИЕ
Table.Pivot	При наличии таблицы и столбца атрибутов, содержащих pivotValues, создает новые столбцы для каждого из значений сведения, и назначает им значения из valueColumn. Можно предоставить необязательную функцию aggregationFunction для обработки нескольких вхождений одного и того же значения ключа в столбце атрибутов.
Table.PrefixColumns	Возвращает таблицу, где для всех столбцов задано текстовое значение в качестве префикса.
Table.PromoteHeaders	Продвигает первую строку таблицы в заголовок или имена столбцов.
Table.RemoveColumns	Возвращает таблицу, не имеющую определенного столбца или столбцов.
Table.ReorderColumns	Возвращает таблицу с конкретными столбцами в заданном порядке относительно друг друга.
Table.RenameColumns	Возвращает таблицу со столбцами, переименованными, как указано.
Table.SelectColumns	Возвращает таблицу, содержащую только определенные столбцы.
Table.TransformColumnNames	Преобразует имена столбцов с помощью предоставленной функции.
Table.Unpivot	При наличии списка столбцов таблицы преобразует эти столбцы в пары "атрибут-значение".
Table.UnpivotOtherColumns	Преобразует все столбцы, отличные от заданного набора, в пары "атрибут-значение", объединенные с остальными значениями в каждой строке.

Параметры

ЗНАЧЕНИЯ ПАРАМЕТРОВ	ОПИСАНИЕ
JoinKind.Inner	Возможное значение необязательного параметра <code>JoinKind</code> в <code>Table.Join</code> . Таблица, полученная в результате внутреннего соединения, содержит по одной строке для каждой пары строк из указанных таблиц, которые были определены как совпадающие на основе указанных ключевых столбцов.
JoinKind.LeftOuter	Возможное значение необязательного параметра <code>JoinKind</code> в <code>Table.Join</code> . Левое внешнее соединение обеспечивает отображение в результате всех строк первой таблицы.

ЗНАЧЕНИЯ ПАРАМЕТРОВ	ОПИСАНИЕ
<code>JoinKind.RightOuter</code>	Возможное значение необязательного параметра <code>JoinKind</code> в <code>Table.Join</code> . Правое внешнее соединение обеспечивает отображение в результате всех строк второй таблицы.
<code>JoinKind.FullOuter</code>	Возможное значение необязательного параметра <code>JoinKind</code> в <code>Table.Join</code> . Полное внешнее соединение обеспечивает отображение в результате всех строк обеих таблиц. Строки, для которых не найдено совпадений в другой таблице, соединяются со строкой по умолчанию, содержащей значения NULL для всех ее столбцов.
<code>JoinKind.LeftAnti</code>	Возможное значение необязательного параметра <code>JoinKind</code> в <code>Table.Join</code> . Левое антисоединение возвращает все строки первой таблицы, для которых отсутствует совпадение во второй таблице.
<code>JoinKind.RightAnti</code>	Возможное значение необязательного параметра <code>JoinKind</code> в <code>Table.Join</code> . Правое антисоединение возвращает все строки второй таблицы, для которых отсутствует совпадение в первой таблице.
<code>MissingField.Error</code>	Необязательный параметр в записях и табличных функциях, указывающий, что отсутствующие поля должны приводить к ошибке. (Это значение параметра по умолчанию.)
<code>MissingField.Ignore</code>	Необязательный параметр в записях и табличных функциях, указывающий, что отсутствующие поля должны пропускаться.
<code>MissingField.UseNull</code>	Необязательный параметр в записях и табличных функциях, указывающий, что отсутствующие поля должны быть включены со значениями NULL.
<code>GroupKind.Global</code>	<code>GroupKind.Global</code>
<code>GroupKind.Local</code>	<code>GroupKind.Local</code>
<code>ExtraValues.List</code>	Если функция разделения возвращает больше столбцов, чем таблица ожидает, они будут собраны в список.
<code>ExtraValues.Ignore</code>	Если функция разделения возвращает больше столбцов, чем таблица ожидает, они будут пропускаться.
<code>ExtraValues.Error</code>	Если функция разделения возвращает больше столбцов, чем таблица ожидает, возникает ошибка.
<code>JoinAlgorithm.Dynamic</code>	<code>JoinAlgorithm.Dynamic</code>
<code>JoinAlgorithm.PairwiseHash</code>	<code>JoinAlgorithm.PairwiseHash</code>
<code>JoinAlgorithm.SortMerge</code>	<code>JoinAlgorithm.SortMerge</code>

ЗНАЧЕНИЯ ПАРАМЕТРОВ	ОПИСАНИЕ
<code>JoinAlgorithm.LeftHash</code>	<code>JoinAlgorithm.LeftHash</code>
<code>JoinAlgorithm.RightHash</code>	<code>JoinAlgorithm.RightHash</code>
<code>JoinAlgorithm.LeftIndex</code>	<code>JoinAlgorithm.LeftIndex</code>
<code>JoinAlgorithm.RightIndex</code>	<code>JoinAlgorithm.RightIndex</code>
<code>JoinSide.Left</code>	Указывает левую таблицу соединения.
<code>JoinSide.Right</code>	Указывает правую таблицу соединения.

Преобразование

Параметры для параметров группы

- `GroupKind.Global = 0;`
- `GroupKind.Local = 1;`

Параметры для типов соединения

- `JoinKind.Inner = 0;`
- `JoinKind.LeftOuter = 1;`
- `JoinKind.RightOuter = 2;`
- `JoinKind.FullOuter = 3;`
- `JoinKind.LeftAnti = 4;`
- `JoinKind.RightAnti = 5;`

Алгоритм соединения

Следующие значения `JoinAlgorithm` можно указать для `Table.Join`

`JoinAlgorithm.Dynamic` 0,

`JoinAlgorithm.PairwiseHash` 1,

`JoinAlgorithm.SortMerge` 2,

`JoinAlgorithm.LeftHash` 3,

`JoinAlgorithm.RightHash` 4,

```
JoinAlgorithm.LeftIndex      5,
```

```
JoinAlgorithm.RightIndex     6,
```

ЗНАЧЕНИЯ ПАРАМЕТРОВ	ОПИСАНИЕ
JoinSide.Left	Указывает левую таблицу соединения.
JoinSide.Right	Указывает правую таблицу соединения.

Пример данных

В примерах этого раздела используются следующие таблицы.

Таблица Customers

```
Customers = Table.FromRecords({

    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],

    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],

    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],

    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]

})
```

Таблица Orders

```
Orders = Table.FromRecords({

    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],

    [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],

    [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0],

    [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200.0],

    [OrderID = 5, CustomerID = 3, Item = "Band-aids", Price = 2.0],

    [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20.0],

    [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],

    [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100.0],

    [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]

})
```

ФУНКЦИЯ	ОПИСАНИЕ
Table.AddColumn	Добавляет столбец с именем newColumnName в таблицу.

ФУНКЦИЯ	ОПИСАНИЕ
Table.AddFuzzyClusterColumn	Добавляет новый столбец с репрезентативными значениями, полученными нечетким группированием значений указанного столбца в таблице.
Table.AddIndexColumn	Возвращает таблицу с новым столбцом с указанным именем, которое для каждой строки содержит индекс строки в таблице.
Table.AddJoinColumn	Выполняет вложенное соединение table1 и table2 из конкретных столбцов и создает результат соединения в виде столбца newColumnName для каждой строки таблицы table1.
Table.AddKey	Добавление ключа в таблицу.
Table.AggregateTableColumn	Объединяет таблицы, вложенные в конкретный столбец, в несколько столбцов, содержащих объединенные значения для этих таблиц.
Table.CombineColumns	Table.CombineColumns объединяет столбцы с помощью функции объединения для создания нового столбца. Table.CombineColumns является противоположностью Table.SplitColumns.
Table.CombineColumnsToRecord	Объединяет указанные столбцы в новый столбец записей со значениями, в котором каждая запись содержит имена и значения полей, соответствующие именам и значениям объединяемых столбцов.
Table.ConformToPageReader	Эта функция предназначена только для внутреннего использования.
Table.ExpandListColumn	Получив исходный столбец списков в таблице, создает копию строки для каждого значения в списке.
Table.ExpandRecordColumn	Развертывает столбец записей в столбцы с каждым из значений.
Table.ExpandTableColumn	Развертывает столбец записей или столбец таблиц в несколько столбцов в содержащей их таблице.
Table.FillDown	Заменяет значения NULL в указанном столбце или столбцах таблицы последними значениями, отличными от NULL, в столбце.
Table.FillUp	Возвращает таблицу из указанной таблицы, где значение следующей ячейки распространяется на ячейки со значением NULL выше в указанном столбце.
Table.FilterWithDataTable	
Table.Group	Группирует строки таблицы по значениям ключевых столбцов для каждой строки.

ФУНКЦИЯ	ОПИСАНИЕ
Table.Join	Объединяет строки таблицы table1 со строками table2 на основе равенства значений ключевых столбцов, выбранных в table1, key1 и table2, key2.
Table.Keys	Возвращает список имен ключевых столбцов из таблицы.
Table.NestedJoin	Соединяет строки таблиц на основе равенства ключей. Результаты вводятся в новый столбец.
Table.ReplaceErrorValues	Заменяет значения ошибок в указанных столбцах соответствующими указанными значениями.
Table.ReplaceKeys	Возвращает новую таблицу с новыми сведениями о ключе, заданными в аргументе keys.
Table.ReplaceRelationshipIdentity	
Table.ReplaceValue	Заменяет значение oldValue на newValue в конкретных столбцах таблицы с помощью предоставленной функции замены, например text.Replace или Value.Replace.
Table.SplitColumn	Возвращает новый набор столбцов из одного столбца, применяя функцию разделения к каждому значению.
Table.TransformColumns	Преобразует столбцы из таблицы с помощью функции.
Table.TransformColumnTypes	Преобразует типы столбцов из таблицы с помощью типа.
Table.TransformRows	Преобразует строки из таблицы с помощью функции преобразования.
Table.Transpose	Возвращает таблицу со столбцами, преобразованными в строки, и строки, преобразованные в столбцы, из входной таблицы.

Членство

Параметры для проверок членства

Спецификация вхождения

`Occurrence.First` = 0

`Occurrence.Last` = 1

`Occurrence.All` = 2

КОМПОНЕНТ	ОПИСАНИЕ
Table.Contains	Определяет, появляется ли запись в виде строки в таблице.
Table.ContainsAll	Определяет, появляются ли все указанные записи в виде строк в таблице.
Table.ContainsAny	Определяет, появляются ли какие-либо указанные записи в виде строк в таблице.
Table.Distinct	Удаляет повторяющиеся строки из таблицы, обеспечивая уникальность всех оставшихся строк.
Table.IsDistinct	Определяет, содержит ли таблица только уникальные строки.
Table.PositionOf	Определяет позицию или позиции строки в таблице.
Table.PositionOfAny	Определяет позицию или позиции любой из указанных строк в таблице.
Table.RemoveMatchingRows	Удаляет все вхождения строк из таблицы.
Table.ReplaceMatchingRows	Заменяет определенные строки из таблицы новыми строками.

Упорядочение

Демонстрационные данные

В примерах этого раздела используются следующие таблицы.

Таблица Employees

```

Employees = Table.FromRecords(

    {[Name="Bill",   Level=7,   Salary=100000]},

    [Name="Barb",   Level=8,   Salary=150000]},

    [Name="Andrew", Level=6,   Salary=85000]},

    [Name="Nikki",  Level=5,   Salary=75000]},

    [Name="Margo",  Level=3,   Salary=45000]},

    [Name="Jeff",   Level=10,  Salary=200000]}},

type table [

    Name = text,

    Level = number,

    Salary = number

])

```

КОМПОНЕНТ	ОПИСАНИЕ
Table.Max	Возвращает самую большую строку или строки из таблицы с помощью <code>comparisonCriteria</code> .
Table.MaxN	Возвращает из таблицы количество N самых длинных строк. После сортировки строк необходимо указать параметр <code>countOrCondition</code> для дальнейшей фильтрации результатов.
Table.Min	Возвращает наименьшую строку или строки из таблицы с помощью <code>comparisonCriteria</code> .
Table.MinN	Возвращает количество N наименьших строк в заданной таблице. После сортировки строк необходимо указать параметр <code>countOrCondition</code> для дальнейшей фильтрации результатов.
Table.Sort	Сортирует строки в таблице с помощью <code>comparisonCriteria</code> или порядка по умолчанию, если он не указан.

Другое

КОМПОНЕНТ	ОПИСАНИЕ
Table.Buffer	Помещает таблицу в буфер памяти, изолируя ее от внешних изменений во время оценки.

Значения параметров

Именованние выходных столбцов

Этот параметр представляет собой список текстовых значений, указывающих имена столбцов результирующей таблицы. Этот параметр обычно используется в функциях построения таблиц, таких как `Table.FromRows` и `Table.FromList`.

Критерии сравнения

Критерий сравнения может быть предоставлен как одно из следующих значений:

- Числовое значение для указания порядка сортировки. См. раздел о порядке сортировки в разделе о значениях параметров.
- Чтобы вычислить ключ, используемый для сортировки, можно использовать функцию с одним аргументом.
- Чтобы выбрать ключ и порядок, критерием сравнения может быть список, содержащий ключ и порядок.
- Чтобы полностью управлять сравнением, можно использовать функцию с двумя аргументами, которая возвращает `-1`, `0` или `1`, учитывая связь между левыми и правыми входными данными. `Value.Compare` — это метод, который можно использовать для делегирования этой логики.

Примеры см. в описании [Table.Sort](#).

Критерий количества или условия

Этот критерий обычно используется при упорядочивании или в операциях со строками. Он определяет

количество строк, возвращаемых в таблице, и может принимать две формы — число или условие:

- Число указывает, сколько значений следует вернуть, а также соответствующую функцию.
- Если указано условие, возвращается строка, содержащая значения, которые изначально соответствуют условию. Если значение не соответствует условию, дальнейшие значения не учитываются.

См. [Table.FirstN](#) или [Table.MaxN](#).

Обработка дополнительных значений

Используется для указания того, как функция должна управлять дополнительными значениями в строке. Этот параметр указывается в виде числа, которое сопоставляется с параметрами ниже.

```
ExtraValues.List = 0  
  
ExtraValues.Error = 1  
  
ExtraValues.Ignore = 2
```

Дополнительные сведения см. в разделе [Table.FromList](#).

Обработка отсутствующих столбцов

Используется для указания того, как функция должна работать с отсутствующими столбцами. Этот параметр указывается в виде числа, которое сопоставляется с параметрами ниже.

```
MissingField.Error = 0;  
  
MissingField.Ignore = 1;  
  
MissingField.UseNull = 2;
```

Используется в преобразовании или операциях со столбцами. Примеры см. в разделе [Table.TransformColumns](#).

Порядок сортировки

Используется для указания того, как должны быть отсортированы результаты. Этот параметр указывается в виде числа, которое сопоставляется с параметрами ниже.

```
Order.Ascending = 0  
  
Order.Descending = 1
```

Критерии равенства

Критерий равенства для таблиц можно указать как:

- значение функции, которое является:
 - селектором ключа, определяющим столбец в таблице для применения условий равенства;
 - функцией сравнения, которая используется для указания применяемого типа сравнения. Встроенные функции сравнения можно указать. См. раздел о функциях сравнения.
- Список столбцов в таблице для применения условий равенства

Примеры см. в описании для [Table.Distinct](#).

ExtraValues.Error

16.03.2021 • 2 minutes to read

Сведения

Если функция разделения возвращает больше столбцов, чем таблица ожидает, возникает ошибка.

ExtraValues.Ignore

16.03.2021 • 2 minutes to read

Сведения

Если функция разделения возвращает больше столбцов, чем таблица ожидает, они будут пропускаться.

ExtraValues.List

16.03.2021 • 2 minutes to read

Сведения

Если функция разделения возвращает больше столбцов, чем таблица ожидает, они будут собраны в список.

GroupKind.Global

16.03.2021 • 2 minutes to read

Сведения

Синтаксис

GroupKind.Global

GroupKind.Local

16.03.2021 • 2 minutes to read

Синтаксис

```
GroupKind.Local
```

Сведения

GroupKind.Local

ItemExpression.From

16.03.2021 • 2 minutes to read

Синтаксис

```
ItemExpression.From(function as function) as record
```

Сведения

Возвращает AST для тела функции `function`, нормализованного в *выражение элемента*.

- В качестве функции необходимо использовать лямбда-выражение с 1 аргументом.
- Все ссылки на параметр функции заменяются на `ItemExpression.Item`.
- AST будет упрощен и будет содержать только узлы следующих видов:
 - `Constant`
 - `Invocation`
 - `Unary`
 - `Binary`
 - `If`
 - `FieldAccess`
 - `NotImplemented`

Если не удастся вернуть выражение элемента AST для тела функции `function`, возникает ошибка.

Пример 1

Возвращает AST для тела функции `each _ <> null`.

```
ItemExpression.From(each _ <> null)
```

KIND	Двоичные данные
ОПЕРАТОР	NotEquals
ЛЕВЫЙ	[Запись]
ПРАВЫЙ	[Запись]

ItemExpression.Item

16.03.2021 • 2 minutes to read

Сведения

Узел AST, представляющий элемент в выражении элемента.

JoinAlgorithm.Dynamic

16.03.2021 • 2 minutes to read

Сведения

JoinAlgorithm.Dynamic

JoinAlgorithm.LeftHash

16.03.2021 • 2 minutes to read

Сведения

JoinAlgorithm.LeftHash

JoinAlgorithm.LeftIndex

16.03.2021 • 2 minutes to read

Сведения

JoinAlgorithm.LeftIndex

JoinAlgorithm.PairwiseHash

16.03.2021 • 2 minutes to read

Сведения

JoinAlgorithm.PairwiseHash

JoinAlgorithm.RightHash

16.03.2021 • 2 minutes to read

Сведения

JoinAlgorithm.RightHash

JoinAlgorithm.RightIndex

16.03.2021 • 2 minutes to read

Сведения

JoinAlgorithm.RightIndex

JoinAlgorithm.SortMerge

16.03.2021 • 2 minutes to read

Сведения

JoinAlgorithm.SortMerge

JoinKind.FullOuter

16.03.2021 • 2 minutes to read

Сведения

Возможное значение необязательного параметра `JoinKind` в `Table.Join`. Полное внешнее соединение обеспечивает отображение в результате всех строк обеих таблиц. Строки, для которых не найдено совпадений в другой таблице, соединяются со строкой по умолчанию, содержащей значения NULL для всех ее столбцов.

JoinKind.Inner

16.03.2021 • 2 minutes to read

Сведения

Возможное значение необязательного параметра `JoinKind` в `Table.Join`. Таблица, полученная в результате внутреннего соединения, содержит по одной строке для каждой пары строк из указанных таблиц, которые были определены как совпадающие на основе указанных ключевых столбцов.

JoinKind.LeftAnti

16.03.2021 • 2 minutes to read

Сведения

Возможное значение необязательного параметра `JoinKind` в `Table.Join`. Левое антисоединение возвращает все строки первой таблицы, для которых отсутствует совпадение во второй таблице.

JoinKind.LeftOuter

16.03.2021 • 2 minutes to read

Сведения

Возможное значение необязательного параметра `JoinKind` в `Table.Join`. Левое внешнее соединение обеспечивает отображение в результате всех строк первой таблицы.

JoinKind.RightAnti

16.03.2021 • 2 minutes to read

Сведения

Возможное значение необязательного параметра `JoinKind` в `Table.Join`. Правое антисоединение возвращает все строки второй таблицы, для которых отсутствует совпадение в первой таблице.

JoinKind.RightOuter

16.03.2021 • 2 minutes to read

Сведения

Возможное значение необязательного параметра `JoinKind` в `Table.Join`. Правое внешнее соединение обеспечивает отображение в результате всех строк второй таблицы.

JoinSide.Left

16.03.2021 • 2 minutes to read

Сведения

Указывает левую таблицу соединения.

JoinSide.Right

16.03.2021 • 2 minutes to read

Сведения

Указывает правую таблицу соединения.

Occurrence.All

16.03.2021 • 2 minutes to read

Сведения

Возвращается список позиций всех вхождений найденных значений.

Occurrence.First

16.03.2021 • 2 minutes to read

Сведения

Возвращается позиция первого вхождения найденного значения.

Occurrence.Last

16.03.2021 • 2 minutes to read

Сведения

Возвращается позиция последнего вхождения найденного значения.

Order.Ascending

16.03.2021 • 2 minutes to read

Сведения

Тип функции, сортирующий список по возрастанию.

Order.Descending

16.03.2021 • 2 minutes to read

Сведения

Тип функции, сортирующий список по убыванию.

RowExpression.Column

16.03.2021 • 2 minutes to read

Синтаксис

```
RowExpression.Column(columnName as text) as record
```

Сведения

Возвращает AST, представляющий доступ к столбцу columnName строки в выражении строки.

Пример 1

Создает AST, представляющий доступ к столбцу CustomerName.

```
RowExpression.Column("CustomerName")
```

KIND	FieldAccess
EXPRESSION	[Запись]
MEMBERNAME	CustomerName

RowExpression.From

16.03.2021 • 2 minutes to read

Синтаксис

```
RowExpression.From(function as function) as record
```

Сведения

Возвращает AST для тела функции `function`, нормализованного в *строковое выражение*.

- В качестве функции необходимо использовать лямбда-выражение с 1 аргументом.
- Все ссылки на параметр функции заменяются на `RowExpression.Row`.
- Все ссылки на столбцы заменяются на `RowExpression.Column(columnName)`.
- AST будет упрощен и будет содержать только узлы следующих видов:
 - Constant
 - Invocation
 - Unary
 - Binary
 - If
 - FieldAccess
 - NotImplemented

Если не удастся вернуть строковое выражение AST для тела функции `function`, возникает ошибка.

Пример 1

Возвращает AST для тела функции `[CustomerID] = "ALFKI"`

```
RowExpression.From(each [CustomerName] = "ALFKI")
```

KIND	Двоичные данные
ОПЕРАТОР	Равно
ЛЕВЫЙ	[Запись]
ПРАВЫЙ	[Запись]

RowExpression.Row

16.03.2021 • 2 minutes to read

Сведения

Узел AST, представляющий строку в выражении строки.

Table.AddColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.AddColumn(table as table, newColumnName as text, columnGenerator as function, optional columnType as nullable type) as table
```

Сведения

Добавляет столбец с именем `newColumnName` в таблицу `table`. Значения для этого столбца вычисляются с помощью заданной функции выбора `columnGenerator`, при этом каждая строка берется в качестве входных данных.

Пример 1

Добавление столбца с именем TotalPrice в таблицу, где каждое значение является суммой значений из столбцов [Price] и [Shipping].

```
Table.AddColumn(  
    Table.FromRecords({  
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0, Shipping = 10.00],  
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0, Shipping = 15.00],  
        [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0, Shipping = 10.00]  
    }  
),  
    "TotalPrice",  
    each [Price] + [Shipping]  
)
```

ORDERID	CUSTOMERID	ЭЛЕМЕНТ	ЦЕНА	ОТПРАВКА	ОБЩАЯ СТОИМОСТЬ
1	1	Удочка	100	10	110
2	1	Черви, 1 фунт	5	15	20
3	2	Рыболовная сеть	25	10	35

Table.AddFuzzyClusterColumn

16.03.2021 • 3 minutes to read

Синтаксис

```
Table.AddFuzzyClusterColumn(table as table, columnName as text, newColumnName as text, optional options as nullable record) as table
```

О программе

Добавляет новый столбец `newColumnName` в `table` с репрезентативными значениями `columnName`. Репрезентативные значения получаются путем нечеткого сопоставления значений в `columnName` для каждой строки.

Для указания способа сравнения ключевых столбцов можно включить необязательный набор `options`. Возможны следующие значения.

- `Culture` — позволяет группировать записи на основе правил конкретного языка и региональных параметров. Это может быть любой допустимый язык и региональные параметры. Например, параметр языка и региональных параметров "ja-JP" объединяет записи, основанные на японском языке. Значение по умолчанию — "", которое объединяет элементы на инвариантном языке и региональных параметрах английского языка.
- `IgnoreCase` — логическое значение (true/false), разрешающее объединение ключей без учета регистра. Например, если true, то "Виноград" объединяется с "виноград". Значение по умолчанию — true.
- `IgnoreSpace` — логическое значение (true/false), которое позволяет объединять текстовые части для поиска групп. Например, если true, то "Вино град" объединяется с "Виноград". Значение по умолчанию — true.
- `SimilarityColumnName` — имя столбца, которое показывает сходство между входным значением и репрезентативным значением для этих входных данных. Значение по умолчанию — NULL. В этом случае новый столбец для сходства не будет добавлен.
- `Threshold` — Число от 0,00 до 1,00, указывающее показатель сходства, в котором будут сгруппированы два значения. Например, элемент "Виноград" группируется с элементом "Винорад" (пропущена буква "г"), только если для этого порога задано значение меньше 0,90. Пороговое значение 1,00 означает точное совпадение для объединения. Значение по умолчанию — 0,80.
- `TransformationTable` — Таблица, позволяющая объединить записи на основе пользовательских сопоставлений значений. Она должна содержать столбцы "От" и "До". Например, элемент "Виноград" объединяется с элементом "Изюм", если в таблице преобразования есть столбец "Из", содержащий "Виноград", и столбец "До", содержащий "Изюм". Обратите внимание, что преобразование будет применено ко всем вхождениям текста в таблице преобразования. В приведенной выше таблице преобразования элемент "Виноград сладкий" будет также объединяться с элементом "Изюм сладкий".

Пример 1

Поиск репрезентативных значений для местоположения сотрудников.


```

Table.AddFuzzyClusterColumn(
    Table.FromRecords(
        {
            [EmployeeID = 1, Location = "Seattle"],
            [EmployeeID = 2, Location = "seattl"],
            [EmployeeID = 3, Location = "Vancouver"],
            [EmployeeID = 4, Location = "Seatle"],
            [EmployeeID = 5, Location = "vancouver"],
            [EmployeeID = 6, Location = "Seattle"],
            [EmployeeID = 7, Location = "Vancouver"]
        },
        type table [EmployeeID = nullable number, Location = nullable text]
    ),
    "Location",
    "Location_Cleaned",
    [IgnoreCase = true, IgnoreSpace = true]
)

```

```

Table.FromRecords(
    {
        [EmployeeID = 1, Location = "Seattle", Location_Cleaned = "Seattle"],
        [EmployeeID = 2, Location = "seattl", Location_Cleaned = "Seattle"],
        [EmployeeID = 3, Location = "Vancouver", Location_Cleaned = "Vancouver"],
        [EmployeeID = 4, Location = "Seatle", Location_Cleaned = "Seattle"],
        [EmployeeID = 5, Location = "vancouver", Location_Cleaned = "Vancouver"],
        [EmployeeID = 6, Location = "Seattle", Location_Cleaned = "Seattle"],
        [EmployeeID = 7, Location = "Vancouver", Location_Cleaned = "Vancouver"]
    },
    type table [EmployeeID = nullable number, Location = nullable text, Location_Cleaned = nullable text]
)

```

Table.AddIndexColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.AddIndexColumn(table as table, newColumnName as text, optional initialValue as nullable number, optional increment as nullable number, optional columnType as nullable type) as table
```

Сведения

Добавляет столбец `newColumnName` в `table` с явно указанными значениями позиции. Необязательное значение, `initialValue`, начальное значение индекса. Необязательное значение, `increment`, указывает шаг приращения для каждого значения индекса.

Пример 1

Добавление столбца индекса с именем Index в таблицу.

```
Table.AddIndexColumn(  
    Table.FromRecords(  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    ),  
    "Index"  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА	ИНДЕКС
1	Владимир	123-4567	0
2	Артем	987-6543	1
3	Виталий	543-7890	2
4	Ringo	232-1550	3

Пример 2

Добавление в таблицу столбца индекса с именем index с начальным значением 10 и шагом приращения 5.

```
Table.AddIndexColumn(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    "Index",  
    10,  
    5  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА	ИНДЕКС
1	Владимир	123-4567	10
2	Артем	987-6543	15
3	Виталий	543-7890	20
4	Ringo	232-1550	25

Table.AddJoinColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.AddJoinColumn(table1 as table, key1 as any, table2 as function, key2 as any, newColumnName as text) as table
```

Сведения

Объединяет строки таблицы `table1` со строками `table2` на основе равенства значений ключевых столбцов, выбранных в `key1` (для `table1`) и `key2` (для `table2`). Результаты записываются в столбец с именем `newColumnName`. Эта функция ведет себя аналогично `Table.Join` с `JoinKind LeftOuter`, за исключением того, что результаты соединения представлены во вложенном формате, а не в виде плоской структуры.

Пример 1

Добавление столбца соединения для `{{[saleID = 1, item = "Shirt"], [saleID = 2, item = "Hat"]}}` с именем `"price/stock"` из таблицы `{{[saleID = 1, price = 20], [saleID = 2, price = 10]}}`, соединенной по `[saleID]`.

```
Table.AddJoinColumn(
    Table.FromRecords({
        [saleID = 1, item = "Shirt"],
        [saleID = 2, item = "Hat"]
    }),
    "saleID",
    () => Table.FromRecords({
        [saleID = 1, price = 20, stock = 1234],
        [saleID = 2, price = 10, stock = 5643]
    }),
    "saleID",
    "price"
)
```

SALEID	ITEM	PRICE
1	Рубашка	[Таблица]
2	Шапка	[Таблица]

Table.AddKey

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.AddKey(table as table, columns as list, isPrimary as logical) as table
```

Сведения

Добавить ключ в `table`, где `columns` — это подмножество имен столбцов таблицы `table`, которое определяет ключ, а `isPrimary` указывает, является ли ключ первичным.

Пример 1

Добавление в `{[Id = 1, Name = "Hello There"], [Id = 2, Name = "Good Bye"]}` ключа, состоящего из `"Id"`, и назначение его первичным.

```
let
    tableType = type table [Id = Int32.Type, Name = text],
    table = Table.FromRecords({
        [Id = 1, Name = "Hello There"],
        [Id = 2, Name = "Good Bye"]
    }),
    resultTable = Table.AddKey(table, {"Id"}, true)
in
    resultTable
```

ИДЕНТИФИКАТОР	ИМЯ
1	Привет
2	До свидания

Table.AggregateTableColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.AggregateTableColumn(table as table, column as text, aggregations as list) as table
```

Сведения

Объединяет таблицы в `table` [`column`] в несколько столбцов, содержащих объединенные значения для таблиц. `aggregations` используется для указания столбцов, содержащих таблицы для статистической обработки, статистические функции, которые применяются к таблицам для создания их значений, и имена агрегированных столбцов, которые требуется создать.

Пример 1

Статистическое вычисление столбцов таблицы в [`t`] в таблице

{[`t` = {[`a`=1, `b`=2, `c`=3], [`a`=2,`b`=4,`c`=6]}, `b` = 2]} в сумму [`t.a`], минимум и максимум [`t.b`], а также количество значений в [`t.a`].

```
Table.AggregateTableColumn(
    Table.FromRecords(
        {
            [
                t = Table.FromRecords({
                    [a = 1, b = 2, c = 3],
                    [a = 2, b = 4, c = 6]
                }),
                b = 2
            ]
        },
        type table [t = table [a = number, b = number, c = number], b = number]
    ),
    "t",
    {
        {"a", List.Sum, "sum of t.a"},
        {"b", List.Min, "min of t.b"},
        {"b", List.Max, "max of t.b"},
        {"a", List.Count, "count of t.a"}
    }
)
```

СУММА Т.А	МИНИМУМ Т.В	МАКСИМУМ Т.В	ЧИСЛО Т.А	В
3	2	4	2	2

Table.AlternateRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.AlternateRows(table as table, offset as number, skip as number, take as number) as table
```

Сведения

Сохраняет исходное смещение, затем попеременно принимает и пропускает следующие строки.

- `table` — входная таблица.
- `offset` : число строк, которые необходимо разместить перед началом итераций.
- `skip` : число строк, удаляемых в каждой итерации.
- `take` : число строк, сохраняемых в каждой итерации.

Пример 1

Получение таблицы из таблицы, в которой, начиная с первой строки, пропускается одно значение, а затем сохраняется одно значение.

```
Table.AlternateRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    }  
),  
    1,  
    1,  
    1  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567
3	Виталий	543-7890

Table.Buffer

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Buffer(table as table) as table
```

Сведения

Помещает таблицу в буфер памяти, изолируя ее от внешних изменений во время оценки.

Table.Column

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Column(table as table, column as text) as list
```

Сведения

Возвращает столбец данных, указанный с помощью `column`, из таблицы `table` в виде списка.

Пример 1

Возврат значений из столбца [Name] таблицы.

```
Table.Column(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
    "Name"  
)
```

Bob
Артем
Виталий
Ringo

Table.ColumnCount

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ColumnCount(table as table) as number
```

Сведения

Возвращает количество столбцов в таблице `table`.

Пример 1

Определение числа столбцов в таблице.

```
Table.ColumnCount(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    })  
)
```

Table.ColumnNames

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ColumnNames(table as table) as list
```

Сведения

Возвращает имена столбцов в таблице `table` в виде текстового списка.

Пример 1

Нахождение имен столбцов таблицы.

```
Table.ColumnNames(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    })  
)
```

CustomerID
Имя
Номер телефона

Table.ColumnsOfType

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ColumnsOfType(table as table, listOfTypes as list) as list
```

Сведения

Возвращает список с именами столбцов таблицы `table`, соответствующих указанным в `listOfTypes` типам.

Пример 1

Возврат имен столбцов типа `Number.Type` из таблицы.

```
Table.ColumnsOfType(  
    Table.FromRecords(  
        {[a = 1, b = "hello"]},  
        type table[a = Number.Type, b = Text.Type]  
    ),  
    {type number}  
)
```

a

Table.Combine

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Combine(tables as list, optional columns as any) as table
```

Сведения

Возвращает таблицу, полученную в результате объединения списка таблиц `tables`. Результирующая таблица будет иметь тип структуры строк, определяемой `columns` или объединением типов входных данных, если `columns` не указан.

Пример 1

Объединение трех таблиц.

```
Table.Combine({
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),
    Table.FromRecords({[CustomerID = 2, Name = "Jim", Phone = "987-6543"]}),
    Table.FromRecords({[CustomerID = 3, Name = "Paul", Phone = "543-7890"]})
})
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567
2	Артем	987-6543
3	Виталий	543-7890

Пример 2

Объединение трех таблиц с разными структурами.

```
Table.Combine({
    Table.FromRecords({[Name = "Bob", Phone = "123-4567"]}),
    Table.FromRecords({[Fax = "987-6543", Phone = "838-7171"]}),
    Table.FromRecords({[Cell = "543-7890"]})
})
```

ИМЯ	НОМЕР ТЕЛЕФОНА	ФАКС	CELL (ЯЧЕЙКА)
Владимир	123-4567		
	838-7171	987-6543	
			543-7890

Пример 3

Объединение двух таблиц и проекция на заданный тип.

```
Table.Combine(  
    {  
        Table.FromRecords({[Name = "Bob", Phone = "123-4567"]}),  
        Table.FromRecords({[Fax = "987-6543", Phone = "838-7171"]}),  
        Table.FromRecords({[Cell = "543-7890"]})  
    },  
    {"CustomerID", "Name"}  
)
```

CUSTOMERID	ИМЯ
	Bob

Table.CombineColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.CombineColumns(table as table, sourceColumns as list, combiner as function, column as text) as table
```

Сведения

Объединяет указанные столбцы в новый столбец с помощью заданной функции объединения.

Table.CombineColumnsToRecord

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.CombineColumnsToRecord(table as table, newColumnName as text, sourceColumns as list,  
optional options as nullable record) as table
```

О программе

Объединяет указанные столбцы `table` в новый столбец записей `newColumnName` со значениями, в котором каждая запись содержит имена и значения полей, соответствующие именам и значениям объединяемых столбцов. Если для `options` указана запись, могут быть предоставлены следующие параметры:

- `DisplayNameColumn`. При указании в качестве текста означает, что данное имя столбца должно рассматриваться как отображаемое имя записи. Это не обязательно должен быть один из столбцов в самой записи.
- `TypeName`. При указании в качестве текста предоставляет имя логического типа для результирующей записи, которое можно использовать во время загрузки данных для управления поведением среды загрузки.

Table.ConformToPageReader

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ConformToPageReader(table as table, shapingFunction as function) as table
```

О программе

Эта функция предназначена только для внутреннего использования.

Table.Contains

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Contains(table as table, row as record, optional equationCriteria as any) as logical
```

Сведения

Указывает, появляется ли указанная запись `row` в виде строки в `table`. Для управления сравнением строк таблицы можно указать необязательный параметр `equationCriteria`.

Пример 1

Определение, содержит ли таблица строку.

```
Table.Contains(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    [Name = "Bob"]  
)
```

true

Пример 2

Определение, содержит ли таблица строку.

```
Table.Contains(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    [Name = "Ted"]  
)
```

false

Пример 3

Определение, содержит ли таблица строку, путем сравнения только столбца [Name].

```
Table.Contains(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    [CustomerID = 4, Name = "Bob"],  
    "Name"  
)
```

true

Table.ContainsAll

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ContainsAll(table as table, rows as list, optional equationCriteria as any) as logical
```

Сведения

Указывает, появляются ли все указанные записи в списке записей `rows` в виде строки в таблице `table`. Для управления сравнением строк таблицы можно указать необязательный параметр `equationCriteria`.

Пример 1

Определение того, содержит ли таблица все строки, путем сравнения только столбца [CustomerID].

```
Table.ContainsAll(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }},  
    {  
        [CustomerID = 1, Name = "Bill"],  
        [CustomerID = 2, Name = "Fred"]  
    },  
    "CustomerID"  
)
```

true

Пример 2

Определение того, содержит ли таблица все строки.

```
Table.ContainsAll(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }},  
    {  
        [CustomerID = 1, Name = "Bill"],  
        [CustomerID = 2, Name = "Fred"]  
    }  
)
```

false

Table.ContainsAny

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ContainsAny(table as table, rows as list, optional equationCriteria as any) as logical
```

Сведения

Указывает, появляется ли любая из указанных записей в списке записей `rows` в виде строки в таблице `table`. Для управления сравнением строк таблицы можно указать необязательный параметр `equationCriteria`.

Пример 1

Определение, содержит ли таблица `{{[a = 1, b = 2], [a = 3, b = 4]}}` строки `[a = 1, b = 2]` или `[a = 3, b = 5]`.

```
Table.ContainsAny(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = 3, b = 4]  
    }),  
    {  
        [a = 1, b = 2],  
        [a = 3, b = 5]  
    }  
)
```

true

Пример 2

Определение, содержит ли таблица `{{[a = 1, b = 2], [a = 3, b = 4]}}` строки `[a = 1, b = 3]` или `[a = 3, b = 5]`.

```
Table.ContainsAny(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = 3, b = 4]  
    }),  
    {  
        [a = 1, b = 3],  
        [a = 3, b = 5]  
    }  
)
```

false

Пример 3

Определение, содержит ли таблица `(Table.FromRecords({[a = 1, b = 2], [a = 3, b = 4]}))` строки `[a = 1, b = 3]` или `[a = 3, b = 5]`, путем сравнения только столбца `[a]`.

```
Table.ContainsAny(  
  Table.FromRecords({  
    [a = 1, b = 2],  
    [a = 3, b = 4]  
  }),  
  {  
    [a = 1, b = 3],  
    [a = 3, b = 5]  
  },  
  "a"  
)
```

`true`

Table.DemoteHeaders

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.DemoteHeaders(table as table) as table
```

Сведения

Переносит заголовки столбцов (то есть имена столбцов) в первую строку значений. Имена столбцов по умолчанию: Column1, Column2 и т. д.

Пример 1

Понижение уровня первой строки значений в таблице.

```
Table.DemoteHeaders(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"]  
    })  
)
```

COLUMN1	COLUMN2	СТОЛБЕЦ3
CustomerID	Имя	Номер телефона
1	Владимир	123-4567
2	Артем	987-6543

Table.Distinct

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Distinct(table as table, optional equationCriteria as any) as table
```

Сведения

Удаляет повторяющиеся строки из таблицы `table`. Необязательный параметр, `equationCriteria`, указывает, какие столбцы таблицы проверяются на дублирование. Если `equationCriteria` не указан, проверяются все столбцы.

Пример 1

Удаление повторяющихся строк из таблицы.

```
Table.Distinct(  
    Table.FromRecords({  
        [a = "A", b = "a"],  
        [a = "B", b = "b"],  
        [a = "A", b = "a"]  
    })  
)
```

A	B
Объект	a
B	b

Пример 2

Удаление повторяющихся строк из столбца [b] в таблице

```
(([a = "A", b = "a"], [a = "B", b = "a"], [a = "A", b = "b"])).
```

```
Table.Distinct(  
    Table.FromRecords({  
        [a = "A", b = "a"],  
        [a = "B", b = "a"],  
        [a = "A", b = "b"]  
    }),  
    "b"  
)
```

A	B
Объект	a

Объект	b
--------	---

Table.DuplicateColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.DuplicateColumn(table as table, columnName as text, newColumnName as text, optional  
columnType as nullable type) as table
```

Сведения

Дублирует столбец с именем `columnName` в таблицу `table`. Значения и тип столбца `newColumnName` копируются из столбца `columnName`.

Пример

Дублирование столбца "a" в столбец "copied column" в таблице `([a = 1, b = 2], [a = 3, b = 4])`.

```
Table.DuplicateColumn(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = 3, b = 4]  
    }},  
    "a",  
    "copied column"  
)
```

A	B	COPIED COLUMN
1	2	1
3	4	3

Table.ExpandListColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ExpandListColumn(table as table, column as text) as table
```

Сведения

При наличии `table`, где `column` является списком значений, разделяет список на строку для каждого значения. Значения в других столбцах дублируются в каждой создаваемой новой строке.

Пример 1

Разделение столбца списка [Name] в таблице.

```
Table.ExpandListColumn(  
    Table.FromRecords({[Name = {"Bob", "Jim", "Paul"}, Discount = .15]}),  
    "Name"  
)
```

NAME	СКИДКА
Bob	0,15
Артем	0,15
Виталий	0,15

Table.ExpandRecordColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ExpandRecordColumn(table as table, column as text, fieldNames as list, optional  
newColumnNames as nullable list) as table
```

Сведения

При наличии `column` записей во входных данных `table` создает таблицу со столбцом для каждого поля в записи. При необходимости можно указать `newColumnNames` для обеспечения уникальности имен столбцов в новой таблице.

- `table`: исходная таблица со столбцом записей для развертывания.
- `column`: столбец, который необходимо развернуть.
- `fieldNames`: список полей, которые необходимо развернуть в столбцы таблицы.
- `newColumnNames`: список имен столбцов для присвоения новым столбцам. Новые имена столбцов не могут дублировать столбцы в новой таблице.

Пример 1

Развертывание столбца [a] в таблице `{{[a = [aa = 1, bb = 2, cc = 3], b = 2]}}` в 3 столбца: "aa", "bb" и "cc".

```
Table.ExpandRecordColumn(  
    Table.FromRecords({  
        [  
            a = [aa = 1, bb = 2, cc = 3],  
            b = 2  
        ]  
    }},  
    "a",  
    {"aa", "bb", "cc"})  
)
```

AA	BB	CC	B
1	2	3	2

Table.ExpandTableColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ExpandTableColumn(table as table, column as text, columnNames as list, optional newColumnNames as nullable list) as table
```

Сведения

Разворачивает таблицы в `table` [`column`] на несколько строк и столбцов. `columnNames` используется для выбора столбцов, которые нужно развернуть из внутренней таблицы. Укажите `newColumnNames`, чтобы избежать конфликтов между существующими столбцами и новыми столбцами.

Пример 1

Разверните столбцы таблицы в `[a]` в таблице `{{[t = {[a=1, b=2, c=3], [a=2,b=4,c=6]}, b = 2]}}` в 3 столбца `[t.a]`, `[t.b]` и `[t.c]`.

```
Table.ExpandTableColumn(
    Table.FromRecords({
        [
            t = Table.FromRecords({
                [a = 1, b = 2, c = 3],
                [a = 2, b = 4, c = 6]
            }),
            b = 2
        ]
    }),
    "t",
    {"a", "b", "c"},
    {"t.a", "t.b", "t.c"}
)
```

T.A	T.B	T.C	B
1	2	3	2
2	4	6	2

Table.FillDown

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FillDown(table as table, columns as list) as table
```

Сведения

Возвращает таблицу из указанного `table`, где значение предыдущей ячейки распространяется на ячейки со значением NULL ниже в указанном `columns`.

Пример 1

Получение таблицы со значениями NULL в столбце [Place], заполненном значениями, расположенными над ними в таблице.

```
Table.FillDown(  
    Table.FromRecords({  
        [Place = 1, Name = "Bob"],  
        [Place = null, Name = "John"],  
        [Place = 2, Name = "Brad"],  
        [Place = 3, Name = "Mark"],  
        [Place = null, Name = "Tom"],  
        [Place = null, Name = "Adam"]  
    }  
),  
    {"Place"}  
)
```

МЕСТО	ИМЯ
1	Владимир
1	Джон
2	Brad
3	Метка
3	Tom
3	Адам

Table.FillUp

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FillUp(table as table, columns as list) as table
```

Сведения

Возвращает таблицу из указанного `table`, где значение следующей ячейки распространяется на ячейки со значением NULL выше в указанном `columns`.

Пример 1

Получение таблицы со значениями NULL в столбце [Column2], заполненном значениями, расположенными под ними в таблице.

```
Table.FillUp(  
    Table.FromRecords({  
        [Column1 = 1, Column2 = 2],  
        [Column1 = 3, Column2 = null],  
        [Column1 = 5, Column2 = 3]  
    }  
),  
{"Column2"}  
)
```

COLUMN1	COLUMN2
1	2
3	3
5	3

Table.FilterWithDataTable

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FilterWithDataTable(**table** as table, **dataTableIdentifier** as text) as any
```

Сведения

Table.FilterWithDataTable

Table.FindText

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FindText(table as table, text as text) as table
```

Сведения

Возвращает строки в таблице `table`, которая содержит текст `text`. Если текст не найден, возвращается пустая таблица.

Пример 1

Нахождение в таблице строк, содержащих "Bob".

```
Table.FindText(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
    "Bob"  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567

Table.First

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.First(table as table, optional default as any) as any
```

Сведения

Возвращает первую строку из `table` или необязательное значение по умолчанию `default`, если таблица пуста.

Пример 1

Нахождение первой строки таблицы.

```
Table.First(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    })  
)
```

CUSTOMERID	1
NAME	Bob
НОМЕР ТЕЛЕФОНА	123-4567

Пример 2

Нахождение первой строки таблицы `{{}}` или возврат `[a = 0, b = 0]`, если она пуста.

```
Table.First(Table.FromRecords({}), [a = 0, b = 0])
```

A	0
B	0

Table.FirstN

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FirstN(table as table, countOrCondition as any) as table
```

Сведения

Возвращает первые строки таблицы `table` в зависимости от значения `countOrCondition`:

- Если `countOrCondition` является числом, то будет возвращено это число строк (начиная с верхней).
- Если `countOrCondition` является условием, будут возвращаться строки, соответствующие условию, до строки, которая не соответствует условию.

Пример 1

Нахождение первых двух строк таблицы.

```
Table.FirstN(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]
    }),
    2
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567
2	Артем	987-6543

Пример 2

Нахождение первых строк, в которых `[a] > 0`, в таблице.

```
Table.FirstN(
    Table.FromRecords({
        [a = 1, b = 2],
        [a = 3, b = 4],
        [a = -5, b = -6]
    }),
    each [a] > 0
)
```

A	B
1	2

Table.FirstValue

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FirstValue(table as table, optional default as any) as any
```

Сведения

Возвращает первый столбец первой строки таблицы `table` или указанное значение по умолчанию.

Table.FromColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FromColumns(lists as list, optional columns as any) as table
```

Сведения

Создает таблицу типа `columns` из списка `lists`, содержащего вложенные списки с именами и значениями столбцов. Если в некоторых столбцах больше значений, то отсутствующие значения будут заполнены значением по умолчанию NULL, если столбцы допускают значение NULL.

Пример 1

Возврат таблицы из списка имен клиентов в список. Каждое значение в элементе списка клиентов преобразуется в значение строки, и каждый список становится столбцом.

```
Table.FromColumns({
    {1, "Bob", "123-4567"},
    {2, "Jim", "987-6543"},
    {3, "Paul", "543-7890"}
})
```

COLUMN1	COLUMN2	СТОЛБЕЦ3
1	2	3
Bob	Артем	Виталий
123-4567	987-6543	543-7890

Пример 2

Создание таблицы из заданного списка столбцов и списка имен столбцов.

```
Table.FromColumns(
    {
        {1, "Bob", "123-4567"},
        {2, "Jim", "987-6543"},
        {3, "Paul", "543-7890"}
    },
    {"CustomerID", "Name", "Phone"}
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	2	3

Bob	Артем	Виталий
123-4567	987-6543	543-7890

Пример 3

Создание таблицы с другим числом столбцов на строку. Значение отсутствующей строки равно NULL.

```
Table.FromColumns(  
    {  
        {1, 2, 3},  
        {4, 5},  
        {6, 7, 8, 9}  
    },  
    {"column1", "column2", "column3"}  
)
```

COLUMN1	COLUMN2	COLUMN3
1	4	6
2	5	7
3		8
		9

Table.FromList

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FromList(list as list, optional splitter as nullable function, optional columns as any, optional default as any, optional extraValues as nullable number) as table
```

Сведения

Преобразует список `list` в таблицу путем применения заданной функции разбиения `splitter` к каждому элементу в списке. По умолчанию предполагается, что список представляет собой список текстовых значений, разделенных запятыми. Необязательный `columns` может иметь следующие значения: число столбцов, список имен столбцов или тип таблицы. При необходимости также можно указать `default` и `extraValues`.

Пример 1

Создание таблицы из списка со столбцом, имеющим имя Letters, с применением разделителя по умолчанию.

```
Table.FromList({"a", "b", "c", "d"}, null, {"Letters"})
```

БУКВЫ

a

b

c

d

Пример 2

Создание таблицы из списка с применением разделителя Record.FieldValues со столбцами CustomerID и Name в результирующей таблице.

```
Table.FromList(
{
    [CustomerID = 1, Name = "Bob"],
    [CustomerID = 2, Name = "Jim"]
},
Record.FieldValues,
{"CustomerID", "Name"}
)
```

CUSTOMERID

ИМЯ

1	Владимир
2	Артем

Table.FromPartitions

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FromPartitions(partitionColumn as text, partitions as list, optional partitionColumnType as nullable type) as table
```

Сведения

Возвращает таблицу, которая является результатом объединения набора секционированных таблиц `partitions`. `partitionColumn` — это имя добавляемого столбца. Тип столбца по умолчанию — `any`, но может быть задан `partitionColumnType`.

Пример 1

Поиск типа элемента в списке `{number}`.

```
Table.FromPartitions(
    "Year",
    {
        {
            1994,
            Table.FromPartitions(
                "Month",
                {
                    {
                        "Jan",
                        Table.FromPartitions(
                            "Day",
                            {
                                {1, #table({"Foo"}, {"Bar"})}},
                                {2, #table({"Foo"}, {"Bar"})}}
                            }
                        )
                    },
                    {
                        "Feb",
                        Table.FromPartitions(
                            "Day",
                            {
                                {3, #table({"Foo"}, {"Bar"})}},
                                {4, #table({"Foo"}, {"Bar"})}}
                            }
                        )
                    }
                }
            )
        }
    }
)
```

foo	день	месяц	год
Линейчатая	1	Январь	1994

Линейчатая	2	Январь	1994
Линейчатая	3	Февраль	1994
Линейчатая	4	Февраль	1994

Table.FromRecords

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FromRecords(records as list, optional columns as any, optional missingField as nullable number) as table
```

Сведения

Преобразует список записей `records` в таблицу.

Пример 1

Создание таблицы из записей с помощью имен полей записей в качестве имен столбцов.

```
Table.FromRecords({
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"]
})
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567
2	Артем	987-6543
3	Виталий	543-7890

Пример 2

Создание таблицы из записей с введенными столбцами и выбор числовых столбцов.

```
Table.ColumnsOfType(
    Table.FromRecords(
        {[CustomerID = 1, Name = "Bob"]},
        type table[CustomerID = Number.Type, Name = Text.Type]
    ),
    {type number}
)
```

CustomerID

Table.FromRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FromRows(rows as list, optional columns as any) as table
```

Сведения

Создает таблицу из списка `rows`, в которой каждый элемент списка является внутренним списком, содержащим значения столбца для одной строки. Для `columns` можно указать необязательный список имен столбцов, тип таблицы или число столбцов.

Пример 1

Получение таблицы со столбцом [CustomerID] со значениями {1, 2}, столбцом [Name] со значениями {"Bob", "Jim"} и столбцом [Phone] со значениями {"123-4567", "987-6543"}.

```
Table.FromRows(  
    {  
        {1, "Bob", "123-4567"},  
        {2, "Jim", "987-6543"}  
    },  
    {"CustomerID", "Name", "Phone"}  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567
2	Артем	987-6543

Пример 2

Получение таблицы со столбцом [CustomerID] со значениями {1, 2}, столбцом [Name] со значениями {"Bob", "Jim"} и столбцом [Phone] со значениями {"123-4567", "987-6543"}, где [CustomerID] имеет числовой тип, а [Name] и [Phone] имеют текстовый тип.

```
Table.FromRows(  
    {  
        {1, "Bob", "123-4567"},  
        {2, "Jim", "987-6543"}  
    },  
    type table [CustomerID = number, Name = text, Phone = text]  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567

2	Арте́м	987-6543
---	--------	----------

Table.FromValue

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.FromValue(value as any, optional options as nullable record) as table
```

Сведения

Создает таблицу со столбцом, содержащим указанное значение или список значений, `value`. Для управления следующими параметрами можно указать необязательный параметр записи `options`:

- `DefaultColumnName`: имя столбца, используемого при создании таблицы из списка или скалярного значения.

Пример 1

Создание таблицы из значения 1.

```
Table.FromValue(1)
```

ЗНАЧЕНИЕ
1

Пример 2

Создание таблицы из списка.

```
Table.FromValue({1, "Bob", "123-4567"})
```

ЗНАЧЕНИЕ
1
Владимир
123-4567

Пример 3

Создание из значения 1 таблицы с настраиваемым именем столбца.

```
Table.FromValue(1, [DefaultColumnName = "MyValue"])
```

MYVALUE

Table.FuzzyGroup

16.03.2021 • 3 minutes to read

Синтаксис

```
Table.FuzzyGroup(table as table, key as any, aggregatedColumns as list, optional options as nullable record) as table
```

Сведения

На основе нечеткого соответствия группирует строки таблицы `table` со значениями строк указанного столбца `key`. Для каждой группы создается запись, содержащая ключевые столбцы (и их значения) вместе со всеми агрегированными столбцами, заданными `aggregatedColumns`. Эта функция не может гарантировать возврат строк в фиксированном порядке.

Для указания способа сравнения ключевых столбцов можно включить необязательный набор `options`. Возможны следующие значения.

- `Culture` — позволяет группировать записи на основе правил конкретного языка и региональных параметров. Это может быть любой допустимый язык и региональные параметры. Например, параметр языка и региональных параметров "ja-JP" объединяет записи, основанные на японском языке. Значение по умолчанию — "", которое объединяет элементы на инвариантном языке и региональных параметрах английского языка.
- `IgnoreCase` — логическое значение (true/false), разрешающее объединение ключей без учета регистра. Например, если true, то "Виноград" объединяется с "виноград". Значение по умолчанию — true.
- `IgnoreSpace` — логическое значение (true/false), которое позволяет объединять текстовые части для поиска групп. Например, если true, то "Вино град" объединяется с "Виноград". Значение по умолчанию — true.
- `SimilarityColumnName` — имя столбца, которое показывает сходство между входным значением и репрезентативным значением для этих входных данных. Значение по умолчанию — NULL. В этом случае новый столбец для сходства не будет добавлен.
- `Threshold` — Число от 0,00 до 1,00, указывающее показатель сходства, в котором будут сгруппированы два значения. Например, элемент "Виноград" группируется с элементом "Винорад" (пропущена буква "г"), только если для этого порога задано значение меньше 0,90. Пороговое значение 1,00 означает точное совпадение для объединения. Значение по умолчанию — 0,80.
- `TransformationTable` — Таблица, позволяющая объединить записи на основе пользовательских сопоставлений значений. Она должна содержать столбцы "От" и "До". Например, элемент "Виноград" объединяется с элементом "Изюм", если в таблице преобразования есть столбец "Из", содержащий "Виноград", и столбец "До", содержащий "Изюм". Обратите внимание, что преобразование будет применено ко всем вхождениям текста в таблице преобразования. В приведенной выше таблице преобразования элемент "Виноград сладкий" будет также объединяться с элементом "Изюм сладкий".

Пример

Группирование таблицы с добавлением агрегатного столбца [Count], который содержит число сотрудников в каждом расположении (`each Table.RowCount(_)`).

```

Table.FuzzyGroup(
    Table.FromRecords(
        {
            [EmployeeID = 1, Location = "Seattle"],
            [EmployeeID = 2, Location = "seattl"],
            [EmployeeID = 3, Location = "Vancouver"],
            [EmployeeID = 4, Location = "Seatle"],
            [EmployeeID = 5, Location = "vancouver"],
            [EmployeeID = 6, Location = "Seattle"],
            [EmployeeID = 7, Location = "Vancouver"]
        },
        type table [EmployeeID = nullable number, Location = nullable text]
    ),
    "Location",
    {"Count", each Table.RowCount(_)},
    [IgnoreCase = true, IgnoreSpace = true]
)

```

МЕСТОПОЛОЖЕНИЕ	COUNT
Сиэтл	4
Ванкувер	3

Table.FuzzyJoin

16.03.2021 • 4 minutes to read

Синтаксис

```
Table.FuzzyJoin(table1 as table, key1 as any, table2 as table, key2 as any, optional joinKind as nullable number, optional joinOptions as nullable record) as table
```

Сведения

Объединяет строки таблицы `table1` со строками `table2` на основе нечеткого соответствия значений ключевых столбцов, выбранных в `key1` (для `table1`) и `key2` (для `table2`).

Нечеткое соответствие — это сравнение, основанное на сходстве, а не на равенстве текста.

По умолчанию выполняется внутреннее соединение, однако для указания типа соединения может быть включен необязательный `joinKind`. Доступные параметры:

- `JoinKind.Inner`
- `JoinKind.LeftOuter`
- `JoinKind.RightOuter`
- `JoinKind.FullOuter`
- `JoinKind.LeftAnti`
- `JoinKind.RightAnti`

Для указания способа сравнения ключевых столбцов можно включить необязательный набор `joinOptions`. Доступные параметры:

- `ConcurrentRequests` — Число от 1 до 8, указывающее количество параллельных потоков, используемых для нечеткого соответствия. Значение по умолчанию — 1.
- `Culture` — Позволяет сопоставлять записи на основе правил конкретного языка и региональных параметров. Это может быть любой допустимый язык и региональные параметры. Например, параметр языка и региональных параметров "ja-JP" соответствует записям, основанным на японском языке. Значение по умолчанию — "", которое соответствует на инвариантном языке и региональных параметрах английского языка.
- `IgnoreCase` — Логическое значение (true/false), разрешающее сопоставление ключей без учета регистра. Например, если true, то "Виноград" сопоставляется с "виноград". Значение по умолчанию — true.
- `IgnoreSpace` — Логическое значение (true/false), которое позволяет объединять текстовые части для поиска совпадений. Например, если true, то "Вино град" сопоставляется с "виноград". Значение по умолчанию — true.
- `NumberOfMatches` — целое число, которое задает максимальное количество совпадающих строк, которые могут быть возвращены для каждой входной строки. Например, значение 1 будет возвращать не более одной совпадающей строки для каждой входной строки. Если этот параметр не указан, возвращаются все совпадающие строки.
- `SimilarityColumnName` — имя столбца, которое показывает сходство между входным значением и репрезентативным значением для этих входных данных. Значение по умолчанию — NULL. В этом случае новый столбец для сходства не будет добавлен.

- `Threshold` — Число от 0,00 до 1,00, указывающее показатель сходства, в котором будут сопоставлены два значения. Например, элемент "Виноград" группируется с элементом "Винорад" (пропущена буква "г"), только если для этого порога задано значение меньше 0,90. Пороговое значение 1,00 означает точное совпадение. Значение по умолчанию — 0,80.
- `TransformationTable` — Таблица, позволяющая сопоставлять записи на основе пользовательских сопоставлений значений. Она должна содержать столбцы "От" и "До". Например, элемент "Виноград" соответствует элементу "Изюм", если в таблице преобразования есть столбец "Из", содержащий "Виноград", и столбец "В", содержащий "Изюм". Обратите внимание, что преобразование будет применено ко всем вхождениям текста в таблице преобразования. В приведенной выше таблице преобразования элемент "Виноград сладкий" будет также сопоставляться с элементом "Изюм сладкий".

Пример

Левое внутреннее нечеткое соединение двух таблиц на основе [FirstName]

```
Table.FuzzyJoin(
    Table.FromRecords(
        {
            [CustomerID = 1, FirstName1 = "Bob", Phone = "555-1234"],
            [CustomerID = 2, FirstName1 = "Robert", Phone = "555-4567"]
        },
        type table [CustomerID = nullable number, FirstName1 = nullable text, Phone = nullable text]
    ),
    {"FirstName1"},
    Table.FromRecords(
        {
            [CustomerStateID = 1, FirstName2 = "Bob", State = "TX"],
            [CustomerStateID = 2, FirstName2 = "bOB", State = "CA"]
        },
        type table [CustomerStateID = nullable number, FirstName2 = nullable text, State = nullable text]
    ),
    {"FirstName2"},
    JoinKind.LeftOuter,
    [IgnoreCase = true, IgnoreSpace = false]
)
```

CUSTOMERID	FIRSTNAME1	ТЕЛЕФОН	CUSTOMERSTATEID	FIRSTNAME2	СОСТОЯНИЕ
1	Владимир	555-1234	1	Владимир	TX
1	Владимир	555-1234	2	bOB	Целостности и доступности
2	Robert	555-4567			

Table.FuzzyNestedJoin

16.03.2021 • 4 minutes to read

Синтаксис

```
Table.FuzzyNestedJoin(table1 as table, key1 as any, table2 as table, key2 as any, newColumnName as text, optional joinKind as nullable number, optional joinOptions as nullable record) as table
```

Сведения

Объединяет строки таблицы `table1` со строками `table2` на основе нечеткого соответствия значений ключевых столбцов, выбранных в `key1` (для `table1`) и `key2` (для `table2`). Результаты возвращаются в новом столбце с именем `newColumnName`.

Нечеткое соответствие — это сравнение, основанное на сходстве, а не на равенстве текста.

Необязательное значение `joinKind` указывает вид выполняемого соединения. По умолчанию выполняется левое внешнее соединение, если `joinKind` не указано. Доступные параметры:

- `JoinKind.Inner`
- `JoinKind.LeftOuter`
- `JoinKind.RightOuter`
- `JoinKind.FullOuter`
- `JoinKind.LeftAnti`
- `JoinKind.RightAnti`

Для указания способа сравнения ключевых столбцов можно включить необязательный набор `joinOptions`. Доступные параметры:

- `ConcurrentRequests` — Число от 1 до 8, указывающее количество параллельных потоков, используемых для нечеткого соответствия. Значение по умолчанию — 1.
- `Culture` — Позволяет сопоставлять записи на основе правил конкретного языка и региональных параметров. Это может быть любой допустимый язык и региональные параметры. Например, параметр языка и региональных параметров "ja-JP" соответствует записям, основанным на японском языке. Значение по умолчанию — "", которое соответствует на инвариантном языке и региональных параметрах английского языка.
- `IgnoreCase` — Логическое значение (true/false), разрешающее сопоставление ключей без учета регистра. Например, если true, то "Виноград" сопоставляется с "виноград". Значение по умолчанию — true.
- `IgnoreSpace` — Логическое значение (true/false), которое позволяет объединять текстовые части для поиска совпадений. Например, если true, то "Вино град" сопоставляется с "виноград". Значение по умолчанию — true.
- `NumberOfMatches` — целое число, которое задает максимальное количество совпадающих строк, которые могут быть возвращены для каждой входной строки. Например, значение 1 будет возвращать не более одной совпадающей строки для каждой входной строки. Если этот параметр не указан, возвращаются все совпадающие строки.
- `SimilarityColumnName` — имя столбца, которое показывает сходство между входным значением и репрезентативным значением для этих входных данных. Значение по умолчанию — NULL. В этом

случае новый столбец для сходства не будет добавлен.

- `Threshold` — Число от 0,00 до 1,00, указывающее показатель сходства, в котором будут сопоставлены два значения. Например, элемент "Виноград" группируется с элементом "Винорад" (пропущена буква "г"), только если для этого порога задано значение меньше 0,90. Пороговое значение 1,00 означает точное совпадение. Значение по умолчанию — 0,80.
- `TransformationTable` — Таблица, позволяющая сопоставлять записи на основе пользовательских сопоставлений значений. Она должна содержать столбцы "От" и "До". Например, элемент "Виноград" соответствует элементу "Изюм", если в таблице преобразования есть столбец "Из", содержащий "Виноград", и столбец "В", содержащий "Изюм". Обратите внимание, что преобразование будет применено ко всем вхождениям текста в таблице преобразования. В приведенной выше таблице преобразования элемент "Виноград сладкий" будет также сопоставляться с элементом "Изюм сладкий".

Пример

Левое внутреннее нечеткое соединение двух таблиц на основе [FirstName]

```
Table.FuzzyNestedJoin(  
    Table.FromRecords(  
        {  
            [CustomerID = 1, FirstName1 = "Bob", Phone = "555-1234"],  
            [CustomerID = 2, FirstName1 = "Robert", Phone = "555-4567"]  
        },  
        type table [CustomerID = nullable number, FirstName1 = nullable text, Phone = nullable text]  
    ),  
    {"FirstName1"},  
    Table.FromRecords(  
        {  
            [CustomerStateID = 1, FirstName2 = "Bob", State = "TX"],  
            [CustomerStateID = 2, FirstName2 = "bOB", State = "CA"]  
        },  
        type table [CustomerStateID = nullable number, FirstName2 = nullable text, State = nullable text]  
    ),  
    {"FirstName2"},  
    "NestedTable",  
    JoinKind.LeftOuter,  
    [IgnoreCase = true, IgnoreSpace = false]  
)
```

CUSTOMERID	FIRSTNAME1	ТЕЛЕФОН	NESTEDTABLE
1	Владимир	555-1234	[Таблица]
2	Robert	555-4567	[Таблица]

Table.Group

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Group(table as table, key as any, aggregatedColumns as list, optional groupKind as nullable number, optional comparer as nullable function) as table
```

Сведения

Группирует строки `table` по значениям в указанном столбце `key` для каждой строки. Для каждой группы создается запись, содержащая ключевые столбцы (и их значения) вместе со всеми агрегированными столбцами, заданными `aggregatedColumns`. Примечание. Если функция сравнения соответствует нескольким ключам, могут возвращаться разные ключи. Эта функция не может гарантировать возврат строк в фиксированном порядке. При необходимости также можно указать `groupKind` и `comparer`.

Пример 1

Группирование таблицы путем добавления агрегированного столбца [total] с суммой цен (each List.Sum([price])).

```
Table.Group(
    Table.FromRecords({
        [CustomerID = 1, price = 20],
        [CustomerID = 2, price = 10],
        [CustomerID = 2, price = 20],
        [CustomerID = 1, price = 10],
        [CustomerID = 3, price = 20],
        [CustomerID = 3, price = 5]
    }),
    "CustomerID",
    {"total", each List.Sum([price])}
)
```

CUSTOMERID	TOTAL
1	30
2	30
3	25

Table.HasColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.HasColumns(table as table, columns as any) as logical
```

Сведения

Указывает, содержит ли `table` указанные столбцы `columns`. Возвращает `true`, если таблица содержит столбцы, в противном случае — `false`.

Пример 1

Определение, содержит ли таблица столбец [Name].

```
TTable.HasColumns(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
    "Name"  
)
```

true

Пример 2

Определение, содержит ли таблица столбцы [Name] и [PhoneNumber].

```
Table.HasColumns(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
    {"Name", "PhoneNumber"}  
)
```

false

Table.InsertRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.InsertRows(table as table, offset as number, rows as list) as table
```

Сведения

Возвращает таблицу со списком строк `rows`, вставленных в `table` в заданной позиции `offset`. Каждый столбец строки для вставки должен согласовываться по типу со столбцами таблицы.

Пример 1

Вставка строки в таблицу в позиции 1.

```
Table.InsertRows(  
    Table.FromRecords(  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"]  
    ),  
    1,  
    {[CustomerID = 3, Name = "Paul", Phone = "543-7890"]}  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567
3	Виталий	543-7890
2	Артем	987-6543

Пример 2

Вставка двух строк в таблицу в позиции 1.

```
Table.InsertRows(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    1,  
    {  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    }  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567

2	Артём	987-6543
3	Виталий	543-7890

Table.IsDistinct

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.IsDistinct(table as table, optional comparisonCriteria as any) as logical
```

Сведения

Указывает, содержит ли `table` только уникальные строки (без повторений). Возвращает `true`, если строки различаются, в противном случае — `false`. Необязательный параметр, `comparisonCriteria`, указывает, какие столбцы таблицы проверяются на дублирование. Если `comparisonCriteria` не указан, проверяются все столбцы.

Пример 1

Определение того, является ли таблица уникальной.

```
Table.IsDistinct(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    })  
)
```

true

Пример 2

Определение того, уникальны ли все столбцы в таблице.

```
Table.IsDistinct(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 5, Name = "Bob", Phone = "232-1550"]  
    }),  
    "Name"  
)
```

false

Table.IsEmpty

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.IsEmpty(table as table) as logical
```

Сведения

Указывает, содержит ли `table` хотя бы одну строку. Возвращает `true`, если строки отсутствуют (то есть таблица пуста), и `false` в остальных случаях.

Пример 1

Определение того, является ли таблица пустой.

```
Table.IsEmpty(  
    Table.FromRecords(  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    )  
)
```

false

Пример 2

Определение того, является ли таблица `{}` пустой.

```
Table.IsEmpty(Table.FromRecords({}))
```

true

Table.Join

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Join(table1 as table, key1 as any, table2 as table, key2 as any, optional joinKind as nullable number, optional joinAlgorithm as nullable number, optional keyEqualityComparers as nullable list) as table
```

Сведения

Объединяет строки таблицы `table1` со строками `table2` на основе равенства значений ключевых столбцов, выбранных в `key1` (для `table1`) и `key2` (для `table2`).

По умолчанию выполняется внутреннее соединение, однако для указания типа соединения может быть включен необязательный `joinKind`. Доступные параметры:

- `JoinKind.Inner`
- `JoinKind.LeftOuter`
- `JoinKind.RightOuter`
- `JoinKind.FullOuter`
- `JoinKind.LeftAnti`
- `JoinKind.RightAnti`

Для указания способа сравнения ключевых столбцов можно включить необязательный набор `keyEqualityComparers`. Сейчас эта функция предназначена только для внутреннего использования.

Пример 1

Внутреннее соединение двух таблиц по полю `[CustomerID]`

```
Table.Join(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
    }),
    "CustomerID",
    Table.FromRecords({
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
        [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0],
        [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200.0],
        [OrderID = 5, CustomerID = 3, Item = "Band-aids", Price = 2.0],
        [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20.0],
        [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25]
    }),
    "CustomerID"
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА	ORDERID	ЭЛЕМЕНТ	ЦЕНА
1	Владимир	123-4567	1	Удочка	100
1	Владимир	123-4567	2	Черви, 1 фунт	5
2	Артем	987-6543	3	Рыболовная сеть	25
3	Виталий	543-7890	4	Электроудочка	200
3	Виталий	543-7890	5	Лейкопластыри	2
1	Владимир	123-4567	6	Коробка для снастей	20

Table.Keys

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Keys(table as table) as list
```

Сведения

Table.Keys

Table.Last

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Last(table as table, optional default as any) as any
```

Сведения

Возвращает последнюю строку из `table` или необязательное значение по умолчанию `default`, если таблица пуста.

Пример 1

Нахождение последней строки таблицы.

```
Table.Last(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    })  
)
```

CUSTOMERID	3
ИМЯ	Виталий
НОМЕР ТЕЛЕФОНА	543-7890

Пример 2

Нахождение последней строки таблицы `({})` или возврат `[a = 0, b = 0]`, если она пуста.

```
Table.Last(Table.FromRecords({}), [a = 0, b = 0])
```

A	0
B	0

Table.LastN

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.LastN(table as table, countOrCondition as any) as table
```

Сведения

Возвращает последние строки таблицы `table` в зависимости от значения `countOrCondition`:

- Если `countOrCondition` является числом, то будет возвращено соответствующее количество строк, начиная с позиции (последняя — `countOrCondition`).
- Если `countOrCondition` является условием, то строки возвращаются в порядке возрастания до тех пор, пока не будет встречена строка, не соответствующая условию.

Пример 1

Нахождение последних двух строк таблицы.

```
Table.LastN(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]
    }),
    2
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
2	Артем	987-6543
3	Виталий	543-7890

Пример 2

Нахождение последних строк, в которых `[a] > 0`, в таблице.

```
Table.LastN(
    Table.FromRecords({
        [a = -1, b = -2],
        [a = 3, b = 4],
        [a = 5, b = 6]
    }),
    each _ [a] > 0
)
```

A	B
---	---

3	4
5	6

Table.MatchesAllRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.MatchesAllRows(table as table, condition as function) as logical
```

Сведения

Указывает, все ли строки в `table` соответствуют заданному `condition`. Возвращает `true`, если все строки соответствуют условию, `false` — в противном случае.

Пример 1

Определить, все ли значения строк в столбце [a] в таблице являются четными.

```
Table.MatchesAllRows(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 6, b = 8]  
    }),  
    each Number.Mod([a], 2) = 0  
)
```

true

Пример 2

Определить, все ли значения строк равны [a = 1, b = 2], в таблице `{{[a = 1, b = 2], [a = 3, b = 4]}}`.

```
Table.MatchesAllRows(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = -3, b = 4]  
    }),  
    each _ = [a = 1, b = 2]  
)
```

false

Table.MatchesAnyRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.MatchesAnyRows(table as table, condition as function) as logical
```

Сведения

Указывает, есть ли в `table` строка, соответствующая заданному `condition`. Возвращает `true`, если хотя бы одна строка соответствует условию, `false` — в противном случае.

Пример 1

Определить, является ли хотя бы одно из значений строк в столбце [a] в таблице

```
(([a = 2, b = 4], [a = 6, b = 8])) четным.
```

```
Table.MatchesAnyRows(  
    Table.FromRecords({  
        [a = 1, b = 4],  
        [a = 3, b = 8]  
    }  
),  
    each Number.Mod([a], 2) = 0  
)
```

```
false
```

Пример 2

Определить, имеется ли хотя бы одно значение строк [a = 1, b = 2] в таблице

```
(([a = 1, b = 2], [a = 3, b = 4])).
```

```
Table.MatchesAnyRows(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = -3, b = 4]  
    }  
),  
    each _ = [a = 1, b = 2]  
)
```

```
true
```

Table.Max

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Max(table as table, comparisonCriteria as any, optional default as any) as any
```

Сведения

Возвращает наибольшую строку в `table`, исходя из критериев `comparisonCriteria`. Если таблица пуста, то возвращается значение необязательного параметра `default`.

Пример 1

Нахождение строки с наибольшим значением в столбце [a] в таблице `{{[a = 2, b = 4], [a = 6, b = 8]}}`.

```
Table.Max(  
    Table.FromRecords(  
        [a = 2, b = 4],  
        [a = 6, b = 8]  
    ),  
    "a"  
)
```

A	6
B	8

Пример 2

Нахождение строки с наибольшим значением в столбце [a] в таблице `{}`. Если таблица пуста, возвращается значение `-1`.

```
Table.Max(#table({"a"}, {}), "a", -1)
```

Table.MaxN

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.MaxN(table as table, comparisonCriteria as any, countOrCondition as any) as table
```

Сведения

Возвращает наибольшую строку или строки в `table`, исходя из критериев `comparisonCriteria`. Для дальнейшей фильтрации строк после сортировки следует указать параметр `countOrCondition`. Обратите внимание, что алгоритм сортировки не гарантирует фиксированного порядка сортировки в результате. Параметр `countOrCondition` может принимать несколько форм:

- Если указано число, возвращается список до `countOrCondition` элементов по возрастанию.
- Если указано условие, возвращается список элементов, которые изначально соответствуют условию. Как только обнаруживается элемент, не соответствующий условию, последующие элементы не рассматриваются.

Пример 1

Найти строку с наибольшим значением в столбце [a] с условием [a] > 0 в таблице. Помните, что строки сортируются до применения фильтра.

```
Table.MaxN(
    Table.FromRecords({
        [a = 2, b = 4],
        [a = 0, b = 0],
        [a = 6, b = 2]
    }),
    "a",
    each [a] > 0
)
```

A	B
6	2
2	4

Пример 2

Найти строку с наибольшим значением в столбце [a] с условием [b] > 0 в таблице. Помните, что строки сортируются до применения фильтра.

```
Table.MaxN(  
  Table.FromRecords({  
    [a = 2, b = 4],  
    [a = 8, b = 0],  
    [a = 6, b = 2]  
  }),  
  "a",  
  each [b] > 0  
)
```

Table.Min

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Min(table as table, comparisonCriteria as any, optional default as any) as any
```

Сведения

Возвращает наименьшую строку в `table`, исходя из критериев `comparisonCriteria`. Если таблица пуста, то возвращается значение необязательного параметра `default`.

Пример 1

Нахождение строки с наименьшим значением в столбце [a] в таблице.

```
Table.Min(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 6, b = 8]  
    }  
),  
    "a"  
)
```

A	2
B	4

Пример 2

Нахождение строки с наименьшим значением в столбце [a] в таблице. Если таблица пуста, возвращается значение -1.

```
Table.Min(#table({"a"}, {}), "a", -1)
```

-1

Table.MinN

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.MinN(table as table, comparisonCriteria as any, countOrCondition as any) as table
```

Сведения

Возвращает наименьшую строку или строки в `table`, исходя из критериев `comparisonCriteria`. Для дальнейшей фильтрации строк после сортировки следует указать параметр `countOrCondition`. Обратите внимание, что алгоритм сортировки не гарантирует фиксированного порядка сортировки в результате. Параметр `countOrCondition` может принимать несколько форм:

- Если указано число, возвращается список до `countOrCondition` элементов по возрастанию.
- Если указано условие, возвращается список элементов, которые изначально соответствуют условию. Как только обнаруживается элемент, не соответствующий условию, последующие элементы не рассматриваются.

Пример 1

Нахождение строки с наименьшим значением в столбце [a] с условием [a] < 3 в таблице. Помните, что строки сортируются до применения фильтра.

```
Table.MinN(
    Table.FromRecords({
        [a = 2, b = 4],
        [a = 0, b = 0],
        [a = 6, b = 4]
    }),
    "a",
    each [a] < 3
)
```

A	B
0	0
2	4

Пример 2

Нахождение строки с наименьшим значением в столбце [a] с условием [b] < 0 в таблице. Помните, что строки сортируются до применения фильтра.

```
Table.MinN(  
  Table.FromRecords({  
    [a = 2, b = 4],  
    [a = 8, b = 0],  
    [a = 6, b = 2]  
  }),  
  "a",  
  each [b] < 0  
)
```

Table.NestedJoin

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.NestedJoin(table1 as table, key1 as any, table2 as any, key2 as any, newColumnName as text, optional joinKind as nullable number, optional keyEqualityComparers as nullable list) as table
```

Сведения

Объединяет строки таблицы `table1` со строками `table2` на основе равенства значений ключевых столбцов, выбранных в `key1` (для `table1`) и `key2` (для `table2`). Результаты записываются в столбец с именем `newColumnName`.

Необязательное значение `joinKind` указывает вид выполняемого соединения. По умолчанию выполняется левое внешнее соединение, если `joinKind` не указано.

Для указания способа сравнения ключевых столбцов можно включить необязательный набор `keyEqualityComparers`. Сейчас эта функция предназначена только для внутреннего использования.

Table.Partition

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Partition(table as table, column as text, groups as number, hash as function) as list
```

Сведения

Секционирует `table` в список из `groups` таблиц на основании значения `column` и функции `hash`. Функция `hash` применяется к значению строки `column` для получения хэш-значения строки. Остаток от целочисленного деления хэш-значения `groups` определяет, в какую из возвращенных таблиц будет помещена строка.

- `table`: таблица для секционирования.
- `column`: столбец для хэширования, позволяющий определить, в какой из возвращенных таблиц находится строка.
- `groups`: количество таблиц, на которое будет секционирована исходная таблица.
- `hash`: функция, применяемая для получения хэш-значения.

Пример

Секционировать таблицу `(([a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4]))` в две таблицы по столбцу `[a]`, используя значения столбцов в качестве хэш-функции.

```
Table.Partition(  
    Table.FromRecords(  
        [a = 2, b = 4],  
        [a = 1, b = 4],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    )),  
    "a",  
    2,  
    each _  
)
```

```
{  
    Table.FromRecords(  
        [a = 2, b = 4],  
        [a = 2, b = 4]  
    )),  
    Table.FromRecords(  
        [a = 1, b = 4],  
        [a = 1, b = 4]  
    )  
}
```

Table.PartitionValues

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Partition(table as table, column as text, groups as number, hash as function) as list
```

Сведения

Секционирует `table` в список из `groups` таблиц на основании значения `column` и функции `hash`. Функция `hash` применяется к значению строки `column` для получения хэш-значения строки. Остаток от целочисленного деления хэш-значения `groups` определяет, в какую из возвращенных таблиц будет помещена строка.

- `table`: таблица для секционирования.
- `column`: столбец для хэширования, позволяющий определить, в какой из возвращенных таблиц находится строка.
- `groups`: количество таблиц, на которое будет секционирована исходная таблица.
- `hash`: функция, применяемая для получения хэш-значения.

Пример 1

Секционировать таблицу `([{a = 2, b = 4}, {a = 6, b = 8}, {a = 2, b = 4}, {a = 1, b = 4}])` в две таблицы по столбцу `a`, используя значения столбцов в качестве хэш-функции.

```
Table.Partition(Table.FromRecords([a = 2, b = 4], [a = 1, b = 4], [a = 2, b = 4], [a = 1, b = 4])), "a", 2, each _)
```

[Таблица]

[Таблица]

Table.Pivot

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Pivot(table as table, pivotValues as list, attributeColumn as text, valueColumn as text, optional aggregationFunction as nullable function) as table
```

Сведения

Принимая пару столбцов, представляющих пары "атрибут-значение", переставляет данные из столбца атрибутов в заголовки столбцов.

Пример 1

Получить значения "a", "b" и "c" в столбце атрибутов таблицы

```
({ [ key = "x", attribute = "a", value = 1 ], [ key = "x", attribute = "c", value = 3 ], [ key = "y", attribute = "a", value = 2 ], [ key = "y", attribute = "b", value = 4 ] })
```

и свести их в их собственный столбец.

```
Table.Pivot(  
    Table.FromRecords({  
        [key = "x", attribute = "a", value = 1],  
        [key = "x", attribute = "c", value = 3],  
        [key = "y", attribute = "a", value = 2],  
        [key = "y", attribute = "b", value = 4]  
    }  
),  
    {"a", "b", "c"},  
    "attribute",  
    "value"  
)
```

KEY	A	B	C
x	1		3
y	2	4	

Пример 2

Получить значения "a", "b" и "c" в столбце атрибутов таблицы

```
({ [ key = "x", attribute = "a", value = 1 ], [ key = "x", attribute = "c", value = 3 ], [ key = "x", attribute = "c", value = 5 ], [ key = "y", attribute = "a", value = 2 ], [ key = "y", attribute = "b", value = 4 ] })
```

и свести их в их собственный столбец. Атрибут "c" для ключа "x" имеет несколько связанных с ним значений, поэтому используйте функцию List.Max для разрешения конфликта.

```
Table.Pivot(  
    Table.FromRecords({  
        [key = "x", attribute = "a", value = 1],  
        [key = "x", attribute = "c", value = 3],  
        [key = "x", attribute = "c", value = 5],  
        [key = "y", attribute = "a", value = 2],  
        [key = "y", attribute = "b", value = 4]  
    }  
),  
{"a", "b", "c"},  
"attribute",  
"value",  
List.Max  
)
```

KEY	A	B	C
x	1		5
y	2	4	

Table.PositionOf

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.PositionOf(table as table, row as record, optional occurrence as any, optional equationCriteria as any) as any
```

Сведения

Возвращает позицию строки первого вхождения `row` в указанной `table`. Возвращает значение `-1`, если вхождение не найдено.

- `table` — входная таблица.
- `row` — строка в таблице, позиция которой должна быть найдена.
- `occurrence` — *[необязательно]* указывает, какие вхождения строки возвращать.
- `equationCriteria` — *[необязательно]* управляет сравнением строк таблицы.

Пример 1

Найти позицию первого вхождения `[a = 2, b = 4]` в таблице

```
(([a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4])) .
```

```
Table.PositionOf(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 1, b = 4],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    } ),  
    [a = 2, b = 4]  
)
```

0

Пример 2

Найти позицию второго вхождения `[a = 2, b = 4]` в таблице

```
(([a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4])) .
```

```
Table.PositionOf(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 1, b = 4],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    } ),  
    [a = 2, b = 4],  
    1  
)
```


Пример 3

Найти позицию всех вхождений [a = 2, b = 4] в таблице

({[a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4]}).

```
Table.PositionOf(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 1, b = 4],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    }  
),  
    [a = 2, b = 4],  
    Occurrence.All  
)
```

0

2

Table.PositionOfAny

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.PositionOfAny(table as table, rows as list, optional occurrence as nullable number,  
optional equationCriteria as any) as any
```

Сведения

Возвращает позиции строк из `table` от первого вхождения в списке `rows`. Возвращает значение `-1`, если вхождение не найдено.

- `table` — входная таблица.
- `rows` — список строк в таблице для поиска позиций.
- `occurrence` — *[необязательно]* указывает, какие вхождения строки возвращать.
- `equationCriteria` — *[необязательно]* управляет сравнением строк таблицы.

Пример 1

Найти позицию первого вхождения `[a = 2, b = 4]` или `[a = 6, b = 8]` в таблице

```
(([a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4])) .
```

```
Table.PositionOfAny(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 1, b = 4],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    } ),  
    {  
        [a = 2, b = 4],  
        [a = 6, b = 8]  
    }  
)
```

0

Пример 2

Найти позицию всех вхождений `[a = 2, b = 4]` или `[a = 6, b = 8]` в таблице

```
(([a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4])) .
```

```
Table.PositionOfAny(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 6, b = 8],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    }),  
    {  
        [a = 2, b = 4],  
        [a = 6, b = 8]  
    },  
    Occurrence.All  
)
```

0

1

2

Table.PrefixColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.PrefixColumns(table as table, prefix as text) as table
```

Сведения

Возвращает таблицу, где все имена столбцов из предоставленной таблицы `table` имеют в качестве префикса заданный текст (`prefix`), а также точку в формате `prefix.ColumnName`.

Пример 1

Задание префикса MyTable для столбцов в таблице.

```
Table.PrefixColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    "MyTable"  
)
```

MYTABLE.CUSTOMERID	MYTABLE.NAME	MYTABLE.PHONE
1	Владимир	123-4567

Table.Profile

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Profile(table as table, optional additionalAggregates as nullable list) as table
```

Сведения

Возвращает профиль столбцов в `table`.

Для каждого столбца в соответствующих случаях возвращаются следующие сведения:

- minimum
- maximum
- average
- стандартное отклонение
- count
- Число значений NULL
- Число уникальных значений

Table.PromoteHeaders

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.PromoteHeaders(table as table, optional options as nullable record) as table
```

Сведения

Назначает первую строку значений в качестве новых заголовков столбцов (т. е. имен столбцов). По умолчанию заголовками назначаются только текстовые или числовые значения. Допустимые параметры:

PromoteAllScalars : если задано значение `true`, все скалярные значения в первой строке назначаются заголовками с использованием языка и региональных параметров (`Culture`), если они указаны (или текущего языкового стандарта документа). Для значений, которые невозможно преобразовать в текст, будет использоваться имя столбца по умолчанию.

Culture : имя языка и региональных параметров для данных.

Пример 1

Повышение уровня первой строки значений в таблице.

```
Table.PromoteHeaders(
    Table.FromRecords({
        [Column1 = "CustomerID", Column2 = "Name", Column3 = #date(1980, 1, 1)],
        [Column1 = 1, Column2 = "Bob", Column3 = #date(1980, 1, 1)]
    })
)
```

CUSTOMERID	ИМЯ	СТОЛБЕЦ3
1	Владимир	1.1.1980 00:00:00

Пример 2

Назначение всех скалярных значений в первой строке таблицы заголовками.

```
Table.PromoteHeaders(
    Table.FromRecords({
        [Rank = 1, Name = "Name", Date = #date(1980, 1, 1)],
        [Rank = 1, Name = "Bob", Date = #date(1980, 1, 1)]
    },
    [PromoteAllScalars = true, Culture = "en-US"]
)
```

1	NAME	1.1.1980
1	Владимир	1.1.1980 00:00:00

Table.Range

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Range(table as table, offset as number, optional count as nullable number) as table
```

Сведения

Возвращает строки из `table`, начиная с заданного смещения `offset`. Необязательный параметр `count` указывает количество возвращаемых строк. По умолчанию возвращаются все строки после смещения.

Пример 1

Получение всех строк в таблице, начиная со смещения 1.

```
Table.Range(  
    Table.FromRecords(  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    ),  
    1  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
2	Артем	987-6543
3	Виталий	543-7890
4	Ringo	232-1550

Пример 2

Получение одной строки в таблице, начиная со смещения 1.

```
Table.Range(  
    Table.FromRecords(  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    ),  
    1,  
    1  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
2	Артем	987-6543

2	Арте́м	987-6543
---	--------	----------

Table.RemoveColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.RemoveColumns(table as table, columns as any, optional missingField as nullable number) as table
```

Сведения

Удаляет указанные столбцы (`columns`) из предоставленной таблицы (`table`). Если столбец не существует, возникает исключение, кроме случаев, когда с помощью необязательного параметра `missingField` указана альтернатива (например, `MissingField.UseNull` или `MissingField.Ignore`).

Пример 1

Удаление столбца [Phone] из таблицы.

```
Table.RemoveColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    "Phone"
```

CUSTOMERID	ИМЯ
1	Владимир

Пример 2

Удаление столбца [Address] из таблицы. Выдает ошибку, если таблица не существует.

```
Table.RemoveColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    "Address"  
)
```

```
[Expression.Error] The field 'Address' of the record was not found.
```

Table.RemoveFirstN

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.RemoveFirstN(table as table, optional countOrCondition as any) as table
```

Сведения

Возвращает таблицу, которая не содержит первое указанное число строк `countOrCondition` таблицы `table`. Число удаленных строк зависит от необязательного параметра `countOrCondition`.

- Если `countOrCondition` опущено, удаляется только первая строка.
- Если `countOrCondition` является числом, то будет удалено это число строк (начиная с верхней).
- Если `countOrCondition` является условием, строки, соответствующие условию, будут удалены, пока строка не будет соответствовать условию.

Пример 1

Удаление первой строки таблицы.

```
Table.RemoveFirstN(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
    1  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
2	Артем	987-6543
3	Виталий	543-7890
4	Ringo	232-1550

Пример 2

Удаление первых двух строк таблицы.

```

Table.RemoveFirstN(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
    })),
    2
)

```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
3	Виталий	543-7890
4	Ringo	232-1550

Пример 3

Удаление из таблицы первых строк, в которых [CustomerID] <= 2.

```

Table.RemoveFirstN(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"] ,
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"] ,
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
    })),
    each [CustomerID] <= 2
)

```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
3	Виталий	543-7890
4	Ringo	232-1550

Table.RemoveLastN

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.RemoveLastN(table as table, optional countOrCondition as any) as table
```

Сведения

Возвращает таблицу, не содержащую последние строки (**countOrCondition**) в таблице **table** . Число удаленных строк зависит от необязательного параметра **countOrCondition** .

- Если **countOrCondition** опущено, удаляется только последняя строка.
- Если **countOrCondition** является числом, то будет удалено это число строк (начиная с конца).
- Если **countOrCondition** является условием, строки, соответствующие условию, будут удалены, пока строка не будет соответствовать условию.

Пример 1

Удаление последней строки таблицы.

```
Table.RemoveLastN(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
    1  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567
2	Артем	987-6543
3	Виталий	543-7890

Пример 2

Удаление из таблицы последних строк, в которых [CustomerID] > 2.

```
Table.RemoveLastN(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    each [CustomerID] >= 2  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567

Table.RemoveMatchingRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.RemoveMatchingRows(table as table, rows as list, optional equationCriteria as any) as table
```

Сведения

Удаляет все вхождения указанных строк `rows` из таблицы `table`. Для управления сравнением строк таблицы можно указать необязательный параметр `equationCriteria`.

Пример 1

Удалить все строки, где `[a = 1]`, из таблицы `({[a = 1, b = 2], [a = 3, b = 4], [a = 1, b = 6]})`.

```
Table.RemoveMatchingRows(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = 3, b = 4],  
        [a = 1, b = 6]  
    }  
),  
    {[a = 1]},  
    "a"  
)
```

A	B
3	4

Table.RemoveRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.RemoveRows(table as table, offset as number, optional count as nullable number) as table
```

Сведения

Удаляет `count` строк от начала `table`, начиная с указанной `offset`. Если параметр `count` не предоставлен, используется значение по умолчанию 1.

Пример 1

Удаление первой строки из таблицы.

```
Table.RemoveRows(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
    }),
    0
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
2	Артем	987-6543
3	Виталий	543-7890
4	Ringo	232-1550

Пример 2

Удаление из таблицы строки в позиции 1.

```
Table.RemoveRows(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
    }),
    1
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
------------	-----	----------------

1	Владимир	123-4567
3	Виталий	543-7890
4	Ringo	232-1550

Пример 3

Удаление двух строк из таблицы, начиная с позиции 1.

```
Table.RemoveRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    1,  
    2  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567
4	Ringo	232-1550

Table.RemoveRowsWithErrors

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.RemoveRowsWithErrors(table as table, optional columns as nullable list) as table
```

Сведения

Возвращает таблицу со строками, удаленными из входной таблицы и содержащими ошибку по меньшей мере в одной из ячеек. Если указан список столбцов, на наличие ошибок проверяются только ячейки в указанных столбцах.

Пример 1

Удалить ошибочное значение из первой строки.

```
Table.RemoveRowsWithErrors(  
    Table.FromRecords({  
        [Column1 = ...],  
        [Column1 = 2],  
        [Column1 = 3]  
    })  
)
```

COLUMN1
2
3

Table.RenameColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.RenameColumns(table as table, renames as list, optional missingField as nullable number)
as table
```

Сведения

Выполняет заданные переименования для столбцов таблицы `table`. Операция замены `renames` состоит из списка с двумя значениями — старым и новым именем столбца, указанными в списке. Если столбец не существует, возникает исключение, кроме случаев, когда с помощью необязательного параметра `missingField` указана альтернатива (например, `MissingField.UseNull` или `MissingField.Ignore`).

Пример 1

Замена имени столбца CustomerNum на CustomerID в таблице.

```
Table.RenameColumns(
    Table.FromRecords({[CustomerNum = 1, Name = "Bob", Phone = "123-4567"]}),
    {"CustomerNum", "CustomerID"}
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567

Пример 2

Замена имени столбца CustomerNum на CustomerID и имени PhoneNum на Phone в таблице.

```
Table.RenameColumns(
    Table.FromRecords({[CustomerNum = 1, Name = "Bob", PhoneNum = "123-4567"]}),
    {
        {"CustomerNum", "CustomerID"},
        {"PhoneNum", "Phone"}
    }
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567

Пример 3

Замена имени столбца NewCol на NewColumn в таблице и пропуск замены, если столбец не существует.

```
Table.RenameColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    {"NewCol", "NewColumn"},  
    MissingField.Ignore  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567

Table.ReorderColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ReorderColumns(table as table, columnOrder as list, optional missingField as nullable number) as table
```

Сведения

Возвращает таблицу из входных данных `table`, при этом столбцы идут в порядке, указанном в `columnOrder`. Столбцы, не указанные в списке, не будут переупорядочены. Если столбец не существует, возникает исключение, кроме случаев, когда с помощью необязательного параметра `missingField` указана альтернатива (например, `MissingField.UseNull` или `MissingField.Ignore`).

Пример 1

Смена порядка следования столбцов [Phone] и [Name] в таблице.

```
Table.ReorderColumns(  
    Table.FromRecords({[CustomerID = 1, Phone = "123-4567", Name = "Bob"]}),  
    {"Name", "Phone"}  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567

Пример 2

Смена порядка следования столбцов [Phone] и [Address] или использование `MissingField.Ignore` в таблице. Таблица не изменяется, так как столбец [Address] не существует.

```
Table.ReorderColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    {"Phone", "Address"},  
    MissingField.Ignore  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
1	Владимир	123-4567

Table.Repeat

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Repeat(table as table, count as number) as table
```

Сведения

Возвращает таблицу со строками из входных данных `table`, повторенными указанное число раз (`count`).

Пример 1

Повтор строк в таблице дважды.

```
Table.Repeat(  
    Table.FromRecords({  
        [a = 1, b = "hello"],  
        [a = 3, b = "world"]  
    }),  
    2
```

A	B
1	hello
3	мир
1	hello
3	мир

Table.ReplaceErrorValues

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ReplaceErrorValues(table as table, errorReplacement as list) as table
```

Сведения

Заменяет значения ошибок в указанных столбцах `table` новыми значениями в списке `errorReplacement`.
Формат списка имеет вид {{столбец1, значение1}, ...}. Для каждого столбца может быть только одно замещающее значение; если указать столбец несколько раз, возникнет ошибка.

Пример 1

Замена в таблице значения ошибки текстом world.

```
Table.ReplaceErrorValues(  
    Table.FromRows({{1, "hello"}, {3, ...}}, {"A", "B"}),  
    {"B", "world"}  
)
```

ОБЪЕКТ	В
1	hello
3	мир

Пример 2

Замена в таблице значения ошибки в столбце A текстом hello, а в столбце B — текстом world.

```
Table.ReplaceErrorValues(  
    Table.FromRows({{..., ...}, {1, 2}}, {"A", "B"}),  
    {"A", "hello"}, {"B", "world"}  
)
```

ОБЪЕКТ	В
hello	мир
1	2

Table.ReplaceKeys

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ReplaceKeys(table as table, keys as list) as table
```

Сведения

Table.ReplaceKeys

Table.ReplaceMatchingRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ReplaceMatchingRows(table as table, replacements as list, optional equationCriteria as any) as table
```

Сведения

Заменяет все указанные строки в `table` предоставленными строками. Строки, которые необходимо заменить, и сами замены определяются в `replacements` с использованием формата {старое, новое}. Для управления сравнением строк таблицы может быть указан необязательный параметр `equationCriteria`.

Пример 1

Замена в таблице строк [a = 1, b = 2] и [a = 2, b = 3] на [a = -1, b = -2], [a = -2, b = -3].

```
Table.ReplaceMatchingRows(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = 2, b = 3],  
        [a = 3, b = 4],  
        [a = 1, b = 2]  
    }  
),  
{  
    {[a = 1, b = 2], [a = -1, b = -2]},  
    {[a = 2, b = 3], [a = -2, b = -3]}  
}  
)
```

A	B
-1	-2
-2	-3
3	4
-1	-2

Table.ReplaceRelationshipIdentity

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ReplaceRelationshipIdentity(value as any, identity as text) as any
```

Сведения

Table.ReplaceRelationshipIdentity

Table.ReplaceRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ReplaceRows(table as table, offset as number, count as number, rows as list) as table
```

Сведения

Заменяет указанное количество строк `count` во входных данных `table` заданными в `rows` начиная с `offset`. Параметр `rows` представляет собой список записей.

- `table`: таблица, в которой выполняется замена.
- `offset`: количество строк, которые следует пропустить, прежде чем выполнять замену.
- `count`: количество строк для замены.
- `rows`: список записей строк, вставляемых в `table` в расположении, заданном `offset`.

Пример 1

Замена 3 строк начиная с позиции 1.

```
Table.ReplaceRows(  
    Table.FromRecords({  
        [Column1 = 1],  
        [Column1 = 2],  
        [Column1 = 3],  
        [Column1 = 4],  
        [Column1 = 5]  
    }  
),  
1,  
3,  
{[Column1 = 6], [Column1 = 7]}  
)
```

COLUMN1
1
6
7
5

Table.ReplaceValue

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ReplaceValue(table as table, oldValue as any, newValue as any, replacer as function, columnsToSearch as list) as table
```

Сведения

Заменяет `oldValue` на `newValue` в указанных столбцах таблицы `table`.

Пример 1

Замена в таблице текста goodbye текстом world.

```
Table.ReplaceValue(
    Table.FromRecords({
        [a = 1, b = "hello"],
        [a = 3, b = "goodbye"]
    }),
    "goodbye",
    "world",
    Replacer.ReplaceText,
    {"b"}
)
```

A	B
1	hello
3	мир

Пример 2

Замена в таблице текста ur текстом or.

```
Table.ReplaceValue(
    Table.FromRecords({
        [a = 1, b = "hello"],
        [a = 3, b = "wurld"]
    }),
    "ur",
    "or",
    Replacer.ReplaceText,
    {"b"}
)
```

A	B
1	hello

Table.Reverse

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Reverse(text as nullable text) as nullable text
```

Сведения

Обращает порядок указанного `text`.

Пример 1

Обратный порядок текста "123".

```
Text.Reverse("123")
```

```
"321"
```

Table.ReverseRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ReverseRows(table as table) as table
```

Сведения

Возвращает таблицу со строками из входных данных `table` в обратном порядке.

Пример 1

Изменение порядка строк в таблице на противоположный.

```
Table.ReverseRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    })  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
4	Ringo	232-1550
3	Виталий	543-7890
2	Артем	987-6543
1	Владимир	123-4567

Table.RowCount

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.RowCount(table as table) as number
```

Сведения

Возвращает количество строк в `table`.

Пример 1

Нахождение числа строк в таблице.

```
Table.RowCount(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    })  
)
```

Table.Schema

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Schema(table as table) as table
```

Сведения

Возвращает таблицу, описывающую столбцы объекта `table`.

Каждая строка в этой таблице описывает свойства столбца объекта `table`:

Имя столбца	Описание
<code>Name</code>	Имя столбца.
<code>Position</code>	Отсчитываемый от нуля порядковый номер столбца в объекте <code>table</code> .
<code>TypeName</code>	Имя типа столбца.
<code>Kind</code>	Вид типа столбца.
<code>IsNullable</code>	Может ли столбец содержать значения <code>null</code> .
<code>NumericPrecisionBase</code>	Основание системы счисления (например, base-2, base-10) для полей <code>NumericPrecision</code> и <code>NumericScale</code> .
<code>NumericPrecision</code>	Точность числового столбца по основанию, заданному параметром <code>NumericPrecisionBase</code> . Это максимальное число цифр, которое может быть представлено значением этого типа (включая дробные разряды).
<code>NumericScale</code>	Масштаб числового столбца по основанию, заданному параметром <code>NumericPrecisionBase</code> . Это число цифр в дробной части значения этого типа. Значение <code>0</code> указывает на фиксированный масштаб без дробных разрядов. Значение <code>null</code> указывает, что масштаб неизвестен (так как он варьируется или не определен).
<code>DateTimePrecision</code>	Максимальное количество цифр дробной части, поддерживаемое в значении секунд для даты или времени.
<code>MaxLength</code>	Максимальное число символов в столбце <code>text</code> или максимальное число байтов в столбце <code>binary</code> .
<code>IsVariableLength</code>	Указывает, может ли этот столбец иметь разную длину (до <code>MaxLength</code>) или имеет фиксированный размер.

NativeTypeName	Имя типа столбца в системе собственных типов источника (например, <code>nvarchar</code> для SQL Server).
NativeDefaultExpression	Выражение по умолчанию для значения этого столбца на языке собственных выражения источника (например, <code>42</code> или <code>newid()</code> для SQL Server).
Description	Описание столбца.

Table.SelectColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.SelectColumns(table as table, columns as any, optional missingField as nullable number) as table
```

Сведения

Возвращает `table` только с указанными `columns`.

- `table`: Предоставленная таблица.
- `columns`: Список возвращаемых столбцов из таблицы `table`. Столбцы в возвращенной таблице перечислены в порядке, указанном в `columns`.
- `missingField`: *(Необязательно)* Что делать, если столбец не существует. Пример: `MissingField.UseNull` или `MissingField.Ignore`.

Пример 1

Включить только столбец [Name].

```
Table.SelectColumns(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    "Name"  
)
```

NAME
Bob
Артем
Виталий
Ringo

Пример 2

Включить только столбцы [CustomerID] и [Name].

```
Table.SelectColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    {"CustomerID", "Name"}  
)
```

CUSTOMERID	ИМЯ
1	Владимир

Пример 3

Если включенный столбец не существует, результат по умолчанию будет ошибкой.

```
Table.SelectColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    "NewColumn"  
)
```

```
[Expression.Error] The field 'NewColumn' of the record wasn't found.
```

Пример 4

Если включенный столбец не существует, параметр `MissingField.UseNull` создаст столбец с значениями NULL.

```
Table.SelectColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    {"CustomerID", "NewColumn"},  
    MissingField.UseNull  
)
```

CUSTOMERID	NEWCOLUMN
1	

Table.SelectRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.SelectRows(table as table, condition as function) as table
```

Сведения

Возвращает таблицу строк из `table`, согласующуюся с выбором `condition`.

Пример 1

Выбрать строки в таблице со значениями в столбце [CustomerID] больше 2.

```
Table.SelectRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
    each [CustomerID] > 2  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
3	Виталий	543-7890
4	Ringo	232-1550

Пример 2

Выбрать строки таблицы, в которых имена не содержат "B".

```
Table.SelectRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
    each not Text.Contains([Name], "B")  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
2	Артем	987-6543
3	Виталий	543-7890

4	Ringo	232-1550
---	-------	----------

Table.SelectRowsWithErrors

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.SelectRowsWithErrors(table as table, optional columns as nullable list) as table
```

Сведения

Возвращает таблицу, содержащую только те строки входной таблицы, которые содержат ошибку по крайней мере в одной из ячеек. Если указан список столбцов, на наличие ошибок проверяются только ячейки в указанных столбцах.

Пример 1

Выбрать имена клиентов, в строках которых есть ошибки.

```
Table.SelectRowsWithErrors(  
    Table.FromRecords({  
        [CustomerID = ..., Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    })  
)[Name]
```

Bob

Table.SingleRow

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.SingleRow(table as table) as record
```

Сведения

Возвращает единственную строку в одной строке `table`. Если `table` содержит более одной строки, возникает исключение.

Пример 1

Возврат одной строки из таблицы.

```
Table.SingleRow(Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}))
```

CUSTOMERID	1
NAME	Bob
НОМЕР ТЕЛЕФОНА	123-4567

Table.Skip

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Skip(table as table, optional countOrCondition as any) as table
```

Сведения

Возвращает таблицу, которая не содержит первое указанное число строк `countOrCondition` таблицы `table`. Число пропущенных строк зависит от необязательного параметра `countOrCondition`.

- Если параметр `countOrCondition` опущен, пропускается только первая строка.
- Если `countOrCondition` является числом, то будет пропущено это число строк (начиная с верхней).
- Если `countOrCondition` является условием, строки, соответствующие условию, будут пропускаться до строки, которая не соответствует условию.

Пример 1

Пропуск первой строки таблицы.

```
Table.Skip(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
    1  
)
```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
2	Артем	987-6543
3	Виталий	543-7890
4	Ringo	232-1550

Пример 2

Пропуск первых двух строк таблицы.


```

Table.Skip(
  Table.FromRecords({
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
  }),
  2
)

```

CUSTOMERID	ИМЯ	НОМЕР ТЕЛЕФОНА
3	Виталий	543-7890
4	Ringo	232-1550

Пример 3

Пропуск первых строк таблицы, в которых [Price] > 25.

```

Table.Skip(
  Table.FromRecords({
    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],
    [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
    [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0],
    [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200.0],
    [OrderID = 5, CustomerID = 3, Item = "Band-aids", Price = 2.0],
    [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20.0],
    [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],
    [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100.0],
    [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]
  }),
  each [Price] > 25
)

```

ORDERID	CUSTOMERID	ЭЛЕМЕНТ	ЦЕНА
2	1	Черви, 1 фунт	5
3	2	Рыболовная сеть	25
4	3	Электроудочка	200
5	3	Лейкопластыри	2
6	1	Коробка для снастей	20
7	5	Наживка	3.25
8	5	Удочка	100
9	6	Наживка	3.25

Table.Sort

16.03.2021 • 3 minutes to read

Синтаксис

```
Table.Sort(table as table, comparisonCriteria as any) as table
```

Сведения

Сортирует `table`, используя список из одного или нескольких имен столбцов и необязательного параметра `comparisonCriteria` в форме `{ { col1, comparisonCriteria }, {col2} }`.

Пример 1

Сортировка таблицы по столбцу OrderID.

```
Table.Sort(
    Table.FromRecords({
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
        [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0],
        [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200.0],
        [OrderID = 5, CustomerID = 3, Item = "Band-aids", Price = 2.0],
        [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20.0],
        [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],
        [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100.0],
        [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]
    }),
    {"OrderID"}
)
```

ORDERID	CUSTOMERID	ЭЛЕМЕНТ	ЦЕНА
1	1	Удочка	100
2	1	Черви, 1 фунт	5
3	2	Рыболовная сеть	25
4	3	Электроудочка	200
5	3	Лейкопластыри	2
6	1	Коробка для снастей	20
7	5	Наживка	3.25
8	5	Удочка	100
9	6	Наживка	3.25

Пример 2

Сортировка таблицы по столбцу OrderID в порядке убывания.

```
Table.Sort(  
    Table.FromRecords({  
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],  
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],  
        [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0],  
        [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200.0],  
        [OrderID = 5, CustomerID = 3, Item = "Band aids", Price = 2.0],  
        [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20.0],  
        [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],  
        [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100.0],  
        [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]  
    }  
),  
    {"OrderID", Order.Descending}  
)
```

ORDERID	CUSTOMERID	ЭЛЕМЕНТ	ЦЕНА
9	6	Наживка	3.25
8	5	Удочка	100
7	5	Наживка	3.25
6	1	Коробка для снастей	20
5	3	Лейкопластыри	2
4	3	Электроудочка	200
3	2	Рыболовная сеть	25
2	1	Черви, 1 фунт	5
1	1	Удочка	100

Пример 3

Сортировка таблицы по столбцу CustomerID, а затем по столбцу OrderID, при этом по CustomerID — в порядке возрастания.

```

Table.Sort(
    Table.FromRecords({
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
        [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0],
        [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200.0],
        [OrderID = 5, CustomerID = 3, Item = "Band-aids", Price = 2.0],
        [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20.0],
        [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],
        [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100.0],
        [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]
    }),
    {
        {"CustomerID", Order.Ascending},
        "OrderID"
    }
)

```

ORDERID	CUSTOMERID	ЭЛЕМЕНТ	ЦЕНА
1	1	Удочка	100
2	1	Черви, 1 фунт	5
6	1	Коробка для снастей	20
3	2	Рыболовная сеть	25
4	3	Электроудочка	200
5	3	Лейкопластыри	2
7	5	Наживка	3.25
8	5	Удочка	100
9	6	Наживка	3.25

Table.Split

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Split(table as table, pageSize as number) as list
```

Сведения

Разделяет `table` на список таблиц, где первый элемент списка — таблица с первыми строками (`pageSize`) из исходной таблицы, следующий элемент списка — таблица со следующими строками (`pageSize`) из исходной таблицы и т. д.

Пример 1

Разделите таблицу с пятью записями на таблицы с двумя записями в каждой.

```
let
    Customers = Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
        [CustomerID = 4, Name = "Cristina", Phone = "232-1550"],
        [CustomerID = 5, Name = "Anita", Phone = "530-1459"]
    })
in
    Table.Split(Customers, 2)
```

[Таблица]

[Таблица]

[Таблица]

Table.SplitColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.SplitColumn(table as table, sourceColumn as text, splitter as function, optional  
columnNamesOrNumber as any, optional default as any, optional extraColumns as any) as table
```

Сведения

Разделяет указанные столбцы на набор дополнительных столбцов с помощью заданной функции разделения.

Пример 1

Разделение столбца [Name] на два столбца в позиции буквы "i"

```
let  
    Customers = Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Cristina", Phone = "232-1550"]  
    })  
in  
    Table.SplitColumn(Customers, "Name", Splitter.SplitTextByDelimiter("i"), 2
```

CUSTOMERID	NAME.1	NAME.2	НОМЕР ТЕЛЕФОНА
1	Владимир		123-4567
2	J	m	987-6543
3	Виталий		543-7890
4	Cr	st	232-1550

Table.ToColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ToColumns(table as table) as list
```

Сведения

Создает список вложенных списков из таблицы `table`. Каждый элемент списка является внутренним списком, содержащим значения столбца.

Пример

Создание списка значений столбца из таблицы.

```
Table.ToColumns(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"]  
    })  
)
```

[Список]

[Список]

[Список]

Table.ToList

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ToList(table as table, optional combiner as nullable function) as list
```

Сведения

Преобразует таблицу в список путем применения заданной функции объединения к каждой строке значений в таблице.

Пример 1

Объединить текст каждой строки с запятой.

```
Table.ToList(  
    Table.FromRows(  
        {  
            {Number.ToText(1), "Bob", "123-4567"},  
            {Number.ToText(2), "Jim", "987-6543"},  
            {Number.ToText(3), "Paul", "543-7890"}  
        }  
    ),  
    Combiner.CombineTextByDelimiter(",")  
)
```

1,Bob,123-4567
2,Jim,987-6543
3,Paul,543-7890

Table.ToRecords

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ToRecords(table as table) as list
```

Сведения

Преобразует таблицу `table` в список записей.

Пример

Преобразовать таблицу в список записей.

```
Table.ToRecords(  
    Table.FromRows(  
        {  
            {1, "Bob", "123-4567"},  
            {2, "Jim", "987-6543"},  
            {3, "Paul", "543-7890"}  
        },  
        {"CustomerID", "Name", "Phone"}  
    )  
)
```

[Запись]

[Запись]

[Запись]

Table.ToRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ToRows(table as table) as list
```

Сведения

Создает список вложенных списков из таблицы `table`. Каждый элемент списка является внутренним списком, содержащим значения строки.

Пример

Создание списка значений строк из таблицы.

```
Table.ToRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    })  
)
```

[Список]

[Список]

[Список]

Table.TransformColumnNames

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.TransformColumnNames(table as table, nameGenerator as function, optional options as nullable record) as table
```

Сведения

Преобразует имена столбцов с помощью заданной функции `nameGenerator`. Допустимые параметры:

`MaxLength` указывает максимальную длину новых имен столбцов. Если заданная функция дает более длинное имя столбца, оно будет обрезано.

`Comparer` используется для управления сравнением при формировании новых имен столбцов. Функции сравнения можно использовать для сравнений, учитывающих или не учитывающих регистр, язык и региональные параметры, а также локаль.

В языке формул доступны следующие встроенные функции сравнения:

- `Comparer.Ordinal` : используется для точного сравнения по порядковому номеру
- `Comparer.OrdinalIgnoreCase` : используется для точного сравнения по порядковому номеру без учета регистра
- `Comparer.FromCulture` : используется для сравнения с учетом языка и региональных параметров

Пример 1

Удаление символа `#(tab)` из имен столбцов.

```
Table.TransformColumnNames(Table.FromRecords({["Col#(tab)umn" = 1]}), Text.Clean)
```

СТОЛБЕЦ
1

Пример 2

Преобразование имен столбцов для создания имен длиной 6 символов без учета регистра.

```
Table.TransformColumnNames(  
    Table.FromRecords({[ColumnNum = 1, columnnum = 2, columnNUM = 3]}),  
    Text.Clean,  
    [MaxLength = 6, Comparer = Comparer.OrdinalIgnoreCase]  
)
```

СТОЛБЕЦ	COLUMN1	COLUMN2
1	2	3

Table.TransformColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.TransformColumns(table as table, transformOperations as list, optional  
defaultTransformation as nullable function, optional missingField as nullable number) as table
```

Сведения

Возвращает таблицу из входного `table`, применяя операцию преобразования к столбцу, указанному в параметре `transformOperations` (где формат — { имя столбца, преобразование }). Если столбец не существует, возникает исключение, кроме случаев, когда с помощью необязательного параметра `defaultTransformation` указана альтернатива (например, `MissingField.UseNull` или `MissingField.Ignore`).

Пример 1

Преобразовать числовые значения в столбце [A] в числовые значения.

```
Table.TransformColumns(  
    Table.FromRecords({  
        [A = "1", B = 2],  
        [A = "5", B = 10]  
    }  
),  
    {"A", Number.FromText}  
)
```

ОБЪЕКТ	B
1	2
5	10

Пример 2

Преобразовать числовые значения в отсутствующем столбце [X] в текстовые значения, пропуская несуществующие столбцы.

```
Table.TransformColumns(  
    Table.FromRecords({  
        [A = "1", B = 2],  
        [A = "5", B = 10]  
    }  
),  
    {"X", Number.FromText},  
    null,  
    MissingField.Ignore  
)
```

ОБЪЕКТ	B
--------	---

1	2
5	10

Пример 3

Преобразовать числовые значения в отсутствующем столбце [X] в текстовые значения, по умолчанию присваивая значение NULL для несуществующих столбцов.

```
Table.TransformColumns(
    Table.FromRecords({
        [A = "1", B = 2],
        [A = "5", B = 10]
    }),
    {"X", Number.FromText},
    null,
    MissingField.UseNull
)
```

ОБЪЕКТ	B	X
1	2	
5	10	

Пример 4

Преобразовать числовые значения в отсутствующем столбце [X] в текстовые значения, возвращая ошибку для несуществующих столбцов.

```
Table.TransformColumns(
    Table.FromRecords({
        [A = "1", B = 2],
        [A = "5", B = 10]
    }),
    {"X", Number.FromText}
)
```

```
[Expression.Error] The column 'X' of the table wasn't found.
```

Table.TransformColumnTypes

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.TransformColumnTypes(table as table, typeTransformations as list, optional culture as nullable text) as table
```

Сведения

Возвращает таблицу из входных данных `table`, применяя операцию преобразования к столбцам, указанным в параметре `typeTransformations` (с форматом {имя столбца, имя типа}), с использованием языка и региональных параметров, указанных в необязательном параметре `culture` (например, en-US). Если столбец не существует, возникает исключение.

Пример 1

Преобразовать числовые значения в столбце [a] в текстовые значения из таблицы

```
(([a = 1, b = 2], [a = 3, b = 4])).
```

```
Table.TransformColumnTypes(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = 3, b = 4]  
    }},  
    {"a", type text},  
    "en-US"  
)
```

A	B
1	2
3	4

Table.TransformRows

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.TransformRows(table as table, transform as function) as list
```

Сведения

Создает таблицу из `table`, применяя операцию `transform` к строкам. Если указан возвращаемый тип функции `transform`, то результатом будет таблица со строками этого типа. Во всех остальных случаях результатом этой функции будет список с типом элементов согласно возвращаемому типу функции преобразования.

Пример 1

Преобразовать строки в список чисел из таблицы `({[A = 1], [A = 2], [A = 3], [A = 4], [A = 5]})`.

```
Table.TransformRows(  
    Table.FromRecords({  
        [a = 1],  
        [a = 2],  
        [a = 3],  
        [a = 4],  
        [a = 5]  
    }),  
    each [a]  
)
```

1

2

3

4

5

Пример 2

Преобразовать строки в столбце [A] в текстовые значения в столбце [B] из таблицы

```
({[A = 1], [A = 2], [A = 3], [A = 4], [A = 5]})
```

```
Table.TransformRows(  
    Table.FromRecords({  
        [a = 1],  
        [a = 2],  
        [a = 3],  
        [a = 4],  
        [a = 5]  
    }  
),  
    (row) as record => [B = Number.ToText(row[a])]  
)
```

[Запись]

[Запись]

[Запись]

[Запись]

[Запись]

Table.Transpose

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Transpose(table as table, optional columns as any) as table
```

Сведения

Превращает столбцы в строки, а строки в столбцы.

Пример 1

Превратить строки таблицы пар "имя-значение" в столбцы.

```
Table.Transpose(  
    Table.FromRecords({  
        [Name = "Full Name", Value = "Fred"],  
        [Name = "Age", Value = 42],  
        [Name = "Country", Value = "UK"]  
    })  
)
```

COLUMN1	COLUMN2	СТОЛБЕЦ3
Полное имя	возраст;	Country
Fred	42	Великобритания

Table.Unpivot

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.Unpivot(table as table, pivotColumns as list, attributeColumn as text, valueColumn as text) as table
```

Сведения

Преобразует набор столбцов в таблице в пары "атрибут-значение" в сочетании с другими значениями в каждой строке.

Пример 1

Взять столбцы "a", "b" и "c" в таблице

([[key = "x", a = 1, b = null, c = 3], [key = "y", a = 2, b = 4, c = null]]) и преобразовать их в пары "атрибут-значение".

```
Table.Unpivot(  
    Table.FromRecords({  
        [key = "x", a = 1, b = null, c = 3],  
        [key = "y", a = 2, b = 4, c = null]  
    }  
),  
{"a", "b", "c"},  
"attribute",  
"value"  
)
```

KEY	АТРИБУТ	ЗНАЧЕНИЕ
x	a	1
x	c	3
y	a	2
y	b	4

Table.UnpivotOtherColumns

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.UnpivotOtherColumns(table as table, pivotColumns as list, attributeColumn as text,  
valueColumn as text) as table
```

Сведения

Преобразует все столбцы, отличные от заданного набора, в пары "атрибут-значение", объединенные с остальными значениями в каждой строке.

Пример 1

Преобразует все столбцы, отличные от заданного набора, в пары "атрибут-значение", объединенные с остальными значениями в каждой строке.

```
Table.UnpivotOtherColumns(  
    Table.FromRecords({  
        [key = "key1", attribute1 = 1, attribute2 = 2, attribute3 = 3],  
        [key = "key2", attribute1 = 4, attribute2 = 5, attribute3 = 6]  
    }  
),  
{"key"},  
"column1",  
"column2"  
)
```

KEY	COLUMN1	COLUMN2
key1	attribute1	1
key1	attribute2	2
key1	attribute3	3
key2	attribute1	4
key2	attribute2	5
key2	attribute3	6

Table.View

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.View(table as nullable table, handlers as record) as table
```

Сведения

Возвращает представление `table`, в котором функции, указанные в `handlers`, используются вместо заданного по умолчанию порядка выполнения операции, когда операция применяется к представлению. Функции обработчика являются необязательными. Если для операции не указана функция обработчика, к `table` применяется порядок выполнения операции, заданный по умолчанию (кроме случая `GetExpression`).

Функции обработчика должны возвращать значение, семантически эквивалентное результату применения операции к `table` (или результирующее представление в случае `GetExpression`).

Если функция обработчика вызывает ошибку, к представлению применяется порядок выполнения операции, заданный по умолчанию.

Для реализации свертывания к источнику данных можно использовать `Table.View` — преобразование M-запросов в запросы, соответствующие источнику (например, для создания инструкций T-SQL из M-запросов).

Более подробное описание `Table.View` см. в опубликованной документации.

Table.ViewFunction

16.03.2021 • 2 minutes to read

Синтаксис

```
Table.ViewFunction(function as function) as function
```

Сведения

Создает функцию представления на основе `function`, которую можно обрабатывать в представлении, созданном `Table.View`.

Обработчик `OnInvoke`Table.View` можно использовать для определения обработчика для функции представления.

Как и в случае с обработчиками для встроенных операций, если обработчик `OnInvoke` не указан или не обрабатывает функцию представления либо обработчик вызывает ошибку, `function` применяется в поверх представления.

Более подробное описание `Table.View` и настраиваемые функции представления см. в опубликованной документации.

Tables.GetRelationships

16.03.2021 • 2 minutes to read

Синтаксис

```
Tables.GetRelationships(tables as table, optional dataColumn as nullable text) as table
```

Сведения

Получает связи между набором таблиц. Предполагается, что набор `tables` должен иметь структуру, аналогичную таблице переходов. Столбец, указанный с помощью `dataColumn`, содержит фактические таблицы данных.

#table

16.03.2021 • 2 minutes to read

Синтаксис

```
#table(columns as any, rows as any) as any
```

Сведения

Создает значение таблицы из столбцов `columns` и списка `rows`, где каждый элемент списка является внутренним списком, содержащим значения столбца для одной строки. `columns` может быть списком имен столбцов, типом таблицы, числом столбцов или значением NULL.

Текстовые функции

16.03.2021 • 7 minutes to read

Эти функции позволяют создавать текстовые значения и управлять ими.

Текст

Информация

ФУНКЦИЯ	ОПИСАНИЕ
Text.InferNumberType	Выводит числовой тип с детализацией (Int64.Type, Double.Type и т. д.) для <code>text</code> с использованием <code>culture</code> .
Text.Length	Возвращает количество символов в текстовом значении.

Текстовые сравнения

ФУНКЦИЯ	ОПИСАНИЕ
Character.FromNumber	Возвращает значение, преобразованное в значение символа.
Character.ToNumber	Возвращает символ, преобразованный в числовое значение.
Guid.From	Возвращает значение <code>Guid.Type</code> из заданного параметра <code>value</code> .
Json.FromValue	Создает представление JSON указанного значения.
Text.From	Возвращает текстовое представление числа, даты, времени, значения <code>datetime</code> , значения <code>datetimezone</code> , логического значения, длительности или двоичного значения. Если значение равно NULL, функция <code>Text.From</code> возвращает NULL. Необязательный параметр <code>culture</code> используется для форматирования значения текста в соответствии с заданным атрибутом языка и региональных параметров.
Text.FromBinary	Декодирует данные из двоичного значения в текстовое, используя кодировку.
Text.NewGuid	Возвращает значение GUID в виде текстового значения.
Text.ToBinary	Кодирует текстовое значение в двоичное, используя кодировку.
Text.ToList	Возвращает список символов на основе текстового значения.

ФУНКЦИЯ	ОПИСАНИЕ
Value.FromText	Декодирует значение из текстового представления и интерпретирует его как значение соответствующего типа. Функция Value.FromText принимает текстовое значение и возвращает число, логическое значение, значение NULL, значение даты и времени, значение длительности или текстовое значение. Пустое текстовое значение интерпретируется как значение NULL.

Извлечение

ФУНКЦИЯ	ОПИСАНИЕ
Text.At	Возвращает символ, начиная с нулевого смещения.
Text.Middle	Возвращает подстроку длиной не более заданной.
Text.Range	Возвращает заданное число символов из текстового значения, начиная с нулевого смещения.
Text.Start	Возвращает заданное число символов с начала текстового значения.
ФУНКЦИЯ	ОПИСАНИЕ
Text.End	Возвращает заданное число символов с конца текстового значения.

Изменение

ФУНКЦИЯ	ОПИСАНИЕ
Text.Insert	Возвращает текстовое значение, в которое начиная с нулевого смещения вставляется newValue.
Text.Remove	Удаляет все вхождения символа или списка символов из текстового значения. Параметр removeChars может представлять значение символа или список значений символов.
Text.RemoveRange	Удаляет заданное число символов из текстового значения, начиная с нулевого смещения.
Text.Replace	Заменяет все вхождения подстроки новым текстовым значением.
Text.ReplaceRange	Заменяет заданное число символов в текстовом значении, начиная с нулевого смещения, новым текстовым значением.
Text.Select	Выбирает все вхождения заданного символа или списка символов из входного текстового значения.

Членство

ФУНКЦИЯ	ОПИСАНИЕ
Text.Contains	Возвращает true, если подстрока текстового значения найдена в строке текстового значения. В остальных случаях возвращается false.
Text.EndsWith	Возвращает логическое значение, которое указывает, найдена ли подстрока текстового значения в конце строки.
Text.PositionOf	Возвращает первое вхождение подстроки в строку и его позицию, начиная со смещения startOffset.
Text.PositionOfAny	Возвращает первое вхождение текстового значения в список и его позицию, начиная со смещения startOffset.
Text.StartsWith	Возвращает логическое значение, которое указывает, найдена ли подстрока текстового значения в начале строки.

Преобразования

ФУНКЦИЯ	ОПИСАНИЕ
Text.AfterDelimiter	Возвращает часть текста после заданного разделителя.
Text.BeforeDelimiter	Возвращает часть текста перед заданным разделителем.
Text.BetweenDelimiters	Возвращает часть текста между заданными начальным (startDelimiter) и конечным (endDelimiter) разделителями.
Text.Clean	Возвращает исходное текстовое значение после удаления из него непечатаемых символов.
Text.Combine	Возвращает текстовое значение, которое является результатом объединения всех текстовых значений. Все значения отделяются разделителем.
Text.Lower	Возвращает текстовое значение в нижнем регистре.
Text.PadEnd	Возвращает текстовое значение, дополненное в конце до заданной длины с использованием заданного заполняющего символа.
Text.PadStart	Возвращает текстовое значение, дополненное в начале до заданной длины с использованием заданного заполняющего символа. Если заполняющий символ не задан, используется пробел.
Text.Proper	Возвращает текстовое значение, в котором первая буква каждого слова имеет верхний регистр.
Text.Repeat	Возвращает текстовое значение, состоящее из входного текстового значения, повторенного заданное число раз.

ФУНКЦИЯ	ОПИСАНИЕ
<code>Text.Reverse</code>	Обращает порядок заданного текста.
<code>Text.Split</code>	Возвращает список, который содержит части текстового значения, отделенные заданным текстовым разделителем.
<code>Text.SplitAny</code>	Возвращает список, который содержит части текстового значения, отделенные любыми текстовыми разделителями.
<code>Text.Trim</code>	Удаляет из текста все вхождения <code>trimChars</code> .
<code>Text.TrimEnd</code>	Удаляет с конца исходного текстового значения все вхождения символов, заданных в <code>trimChars</code> .
<code>Text.TrimStart</code>	Удаляет с начала исходного текстового значения все вхождения символов, заданных в <code>trimChars</code> .
<code>Text.Upper</code>	Возвращает текстовое значение в верхнем регистре.

Параметры

ЗНАЧЕНИЯ ПАРАМЕТРОВ	ОПИСАНИЕ
<code>Occurrence.All</code>	Возвращается список позиций всех вхождений найденных значений.
<code>Occurrence.First</code>	Возвращается позиция первого вхождения найденного значения.
<code>Occurrence.Last</code>	Возвращается позиция последнего вхождения найденного значения.
<code>RelativePosition.FromEnd</code>	Указывает, что индексацию следует вести с конца входных данных.
<code>RelativePosition.FromStart</code>	Указывает, что индексацию следует вести с начала входных данных.
<code>TextEncoding.Ascii</code>	Используется для выбора двоичной формы ASCII.
<code>TextEncoding.BigEndianUnicode</code>	Используется для выбора двоичной формы UTF16 с обратным порядком байтов.
<code>TextEncoding.Unicode</code>	Используется для выбора двоичной формы UTF16 с прямым порядком байтов.
<code>TextEncoding.Utf8</code>	Используется для выбора двоичной формы UTF8.
<code>TextEncoding.Utf16</code>	Используется для выбора двоичной формы UTF16 с прямым порядком байтов.
<code>TextEncoding.Windows</code>	Используется для выбора двоичной формы Windows.

Character.FromNumber

16.03.2021 • 2 minutes to read

Синтаксис

```
Character.FromNumber(number as nullable number) as nullable text
```

Сведения

Возвращает символ, эквивалентный значению.

Пример 1

Найти значение символа для числа 9.

```
Character.FromNumber(9)
```

```
"#(tab)"
```

Character.ToNumber

16.03.2021 • 2 minutes to read

Синтаксис

```
Character.ToNumber(character as nullable text) as nullable number
```

Сведения

Возвращает число, эквивалентное символу `character`.

Пример 1

Найти числовое значение для символа "#(tab)", 9.

```
Character.ToNumber("#(tab)")
```

9

Синтаксис

```
Guid.From(value as nullable text) as nullable text
```

Сведения

Возвращает значение `Guid.Type` из заданного параметра `value`. Если заданный параметр `value` имеет значение `null`, `Guid.From` возвращает `null`. Будет выполнена проверка, чтобы убедиться, что заданный `value` имеет допустимый формат. Допустимые форматы указаны в примерах.

Пример 1

Идентификатор GUID может быть представлен в виде 32 шестнадцатеричных цифр, идущих непрерывно.

```
Guid.From("05FE1DADC8C24F3BA4C2D194116B4967")
```

```
"05fe1dad-c8c2-4f3b-a4c2-d194116b4967"
```

Пример 2

Идентификатор GUID может быть представлен в виде 32 шестнадцатеричных цифр, разделенных дефисами на блоки по 8-4-4-4-12.

```
Guid.From("05FE1DAD-C8C2-4F3B-A4C2-D194116B4967")
```

```
"05fe1dad-c8c2-4f3b-a4c2-d194116b4967"
```

Пример 3

Идентификатор GUID может быть представлен в виде 32 шестнадцатеричных цифр, разделенных дефисами и заключенных в фигурные скобки.

```
Guid.From("{05FE1DAD-C8C2-4F3B-A4C2-D194116B4967}")
```

```
"05fe1dad-c8c2-4f3b-a4c2-d194116b4967"
```

Пример 4

Идентификатор GUID может быть представлен в виде 32 шестнадцатеричных цифр, разделенных дефисами и заключенных в круглые скобки.

```
Guid.From("(05FE1DAD-C8C2-4F3B-A4C2-D194116B4967)")
```

```
"05fe1dad-c8c2-4f3b-a4c2-d194116b4967"
```

Json.FromValue

16.03.2021 • 2 minutes to read

Синтаксис

```
Json.FromValue(value as any, optional encoding as nullable number) as binary
```

Сведения

Создает представление JSON указанного значения `value` с кодировкой текста, определенной `encoding`. Если `encoding` опускается, используется UTF8. Значения представляются следующим образом:

- значения NULL, текстовые и логические значения — в виде соответствующих типов JSON;
- числа — как числа в формате JSON, за исключением того, что `#infinity`, `-#infinity` и `#nan` преобразуются в значения NULL;
- списки — в виде массивов JSON;
- записи — в виде объектов JSON;
- таблицы — в виде массивов объектов;
- значения даты, времени, зон и длительности — в виде текста ISO-8601;
- двоичные значения — в виде текста в кодировке Base-64.
- Типы и функции вызывают ошибку.

Пример 1

Преобразование сложного значения в JSON.

```
Text.FromBinary(Json.FromValue([A = {1, true, "3"}, B = #date(2012, 3, 25)]))
```

```
"{"A": [1, true, "3"], "B": "2012-03-25"}"
```

RelativePosition.FromEnd

16.03.2021 • 2 minutes to read

Сведения

Указывает, что индексацию следует вести с конца входных данных.

RelativePosition.FromStart

16.03.2021 • 2 minutes to read

Сведения

Указывает, что индексацию следует вести с начала входных данных.

Text.AfterDelimiter

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.AfterDelimiter(text as nullable text, delimiter as text, optional index as any) as any
```

Сведения

Возвращает часть `text` после `delimiter`. Необязательный числовой параметр `index` указывает, какое вхождение `delimiter` следует рассматривать. Необязательный список `index` указывает, какое вхождение `delimiter` следует рассматривать, а также откуда вести индексацию — с начала или с конца входных данных.

Пример 1

Получить часть строки "111-222-333" после (первого) дефиса.

```
Text.AfterDelimiter("111-222-333", "-")
```

```
"222-333"
```

Пример 2

Получить часть строки "111-222-333" после второго дефиса.

```
Text.AfterDelimiter("111-222-333", "-", 1)
```

```
"333"
```

Пример 3

Получить часть строки "111-222-333" после второго дефиса с конца.

```
Text.AfterDelimiter("111-222-333", "-", {1, RelativePosition.FromEnd})
```

```
"222-333"
```

Text.At

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.At(text as nullable text, index as number) as nullable text
```

Сведения

Возвращает символ в текстовом значении `text` в позиции `index`. Первый символ в тексте находится в позиции 0.

Пример 1

Найти символ в позиции 4 в строке "Hello, World".

```
Text.At("Hello, World", 4)
```

```
"o"
```

Text.BeforeDelimiter

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.BeforeDelimiter(text as nullable text, delimiter as text, optional index as any) as any
```

Сведения

Возвращает часть `text` перед указанным `delimiter`. Необязательный числовой параметр `index` указывает, какое вхождение `delimiter` следует рассматривать. Необязательный список `index` указывает, какое вхождение `delimiter` следует рассматривать, а также откуда вести индексацию — с начала или с конца входных данных.

Пример 1

Получить часть строки "111-222-333" перед (первым) дефисом.

```
Text.BeforeDelimiter("111-222-333", "-")
```

```
"111"
```

Пример 2

Получить часть строки "111-222-333" перед вторым дефисом.

```
Text.BeforeDelimiter("111-222-333", "-", 1)
```

```
"111-222"
```

Пример 3

Получить часть строки "111-222-333" перед вторым дефисом с конца.

```
Text.BeforeDelimiter("111-222-333", "-", {1, RelativePosition.FromEnd})
```

```
"111"
```

Text.BetweenDelimiters

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.BetweenDelimiters(text as nullable text, startDelimiter as text, endDelimiter as text,  
optional startIndex as any, optional endIndex as any) as any
```

Сведения

Возвращает часть `text` между указанными `startDelimiter` и `endDelimiter`. Необязательный числовой параметр `startIndex` указывает, какое вхождение `startDelimiter` следует рассматривать.

Необязательный список `startIndex` указывает, какое вхождение `startDelimiter` следует рассматривать, а также откуда вести индексацию — с начала или с конца входных данных. `endIndex` аналогичен, за исключением того, что индексация выполняется относительно `startIndex`.

Пример 1

Получить часть строки "111 (222) 333 (444)" между (первой) открывающей скобкой и (первой) закрывающей скобкой после нее.

```
Text.BetweenDelimiters("111 (222) 333 (444)", "(", ")")
```

```
"222"
```

Пример 2

Получить часть строки "111 (222) 333 (444)" между второй открывающей скобкой и первой закрывающей скобкой после нее.

```
Text.BetweenDelimiters("111 (222) 333 (444)", "(", ")", 1, 0)
```

```
"444"
```

Пример 3

Получить часть строки "111 (222) 333 (444)" между второй открывающей скобкой с конца и второй закрывающей скобкой после нее.

```
Text.BetweenDelimiters("111 (222) 333 (444)", "(", ")", {1, RelativePosition.FromEnd}, {1,  
RelativePosition.FromStart})
```

```
"222) 333 (444"
```

Text.Clean

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Clean(text as nullable text) as nullable text
```

Сведения

Возвращает текстовое значение с удалением из `text` всех управляющих символов.

Пример 1

Удаление переводов строки и других управляющих символов из текстового значения.

```
Text.Clean("ABC#(lf)D")
```

```
"ABCD"
```

Text.Combine

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Combine(texts as list, optional separator as nullable text) as text
```

Сведения

Возвращает результат объединения списка текстовых значений `texts` в одном текстовом значении. Можно указать дополнительный разделитель `separator`, используемый в итоговом объединенном тексте.

Пример 1

Объединить текстовые значения "Seattle" и "WA".

```
Text.Combine({"Seattle", "WA"})
```

```
"SeattleWA"
```

Пример 2

Объединить текстовые значения "Seattle" и "WA", разделенные запятыми и пробелом, ", ".

```
Text.Combine({"Seattle", "WA"}, ", ")
```

```
"Seattle, WA"
```

Text.Contains

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Contains(text as nullable text, substring as text, optional comparer as nullable function)
as nullable logical
```

Сведения

Определяет, содержит ли текст `text` в себе текст `substring`. Возвращает значение `True`, если текст найден.

`comparer` — это модуль `Comparer`, который используется для управления сравнением. Функции сравнения можно использовать для сравнений, учитывающих или не учитывающих регистр, язык и региональные параметры, а также локаль.

В языке формул доступны следующие встроенные функции сравнения:

- `Comparer.Ordinal`: используется для точного сравнения по порядковому номеру
- `Comparer.OrdinalIgnoreCase`: используется для точного сравнения по порядковому номеру без учета регистра
- `Comparer.FromCulture`: используется для сравнения с учетом языка и региональных параметров

Пример 1

Найти, содержит ли текст "Hello World" подстроку "Hello".

```
Text.Contains("Hello World", "Hello")
```

`true`

Пример 2

Найти, содержит ли текст "Hello World" подстроку "hello".

```
Text.Contains("Hello World", "hello")
```

`false`

Text.End

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.End(text as nullable text, count as number) as nullable text
```

Сведения

Возвращает значение `text`, которое содержит последние `count` символов значения `text`, `text`.

Пример 1

Получить последние 5 символов строки "Hello, World".

```
Text.End("Hello, World", 5)
```

```
"World"
```

Text.EndsWith

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.EndsWith(text as nullable text, substring as text, optional comparer as nullable function) as nullable logical
```

Сведения

Указывает, завершается ли этот текст `text` указанным значением `substring`. Проверка выполняется с учетом регистра.

`comparer` — это модуль `Comparer`, который используется для управления сравнением. Функции сравнения можно использовать для сравнений, учитывающих или не учитывающих регистр, язык и региональные параметры, а также локаль.

В языке формул доступны следующие встроенные функции сравнения:

- `Comparer.Ordinal`: используется для точного сравнения по порядковому номеру
- `Comparer.OrdinalIgnoreCase`: используется для точного сравнения по порядковому номеру без учета регистра
- `Comparer.FromCulture`: используется для сравнения с учетом языка и региональных параметров

Пример 1

Проверить, оканчивается ли "Hello, World" на "world".

```
Text.EndsWith("Hello, World", "world")
```

false

Пример 2

Проверить, оканчивается ли "Hello, World" на "World".

```
Text.EndsWith("Hello, World", "World")
```

true

Text.Format

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Format(formatString as text, arguments as any, optional culture as nullable text) as text
```

Сведения

Возвращает отформатированный текст, который создается путем применения `arguments` из списка или записи к строке форматирования `formatString`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Форматирование списка чисел.

```
Text.Format("#{0}, #{1}, and #{2}.", {17, 7, 22})
```

```
"17, 7, and 22."
```

Пример 2

Форматирование разных типов данных из записи в соответствии с языком и региональными параметрами.

```
Text.Format(
    "The time for the #[distance] km run held in #[city] on #[date] was #[duration].",
    [
        city = "Seattle",
        date = #date(2015, 3, 10),
        duration = #duration(0, 0, 54, 40),
        distance = 10
    ],
    "en-US"
```

```
"The time for the 10 km run held in Seattle on 3/10/2015 was 00:54:40."
```

Text.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.From(value as any, optional culture as nullable text) as nullable text
```

Сведения

Возвращает текстовое представление `value`. Значение `value` может иметь тип `number`, `date`, `time`, `datetime`, `datetimezone`, `logical`, `duration` или `binary`. Если указанное значение равно NULL, `Text.From` возвращает значение NULL. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Создать текстовое значение из числа 3.

```
Text.From(3)
```

```
"3"
```

Text.FromBinary

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.FromBinary(binary as nullable binary, optional encoding as nullable number) as nullable text
```

Сведения

Декодирует данные `binary` из двоичного значения в текстовое, используя тип `encoding`.

Text.InferNumberType

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.InferNumberType(text as text, optional culture as nullable text) as type
```

Сведения

Выводит числовой тип с детализацией (Int64.Type, Double.Type и т. д.) для `text`. Возникает ошибка, если `text` не является числом. Можно также указать необязательный параметр `culture` (например, en-US).

Text.Insert

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Insert(text as nullable text, offset as number, newText as text) as nullable text
```

Сведения

Возвращает результат вставки текстового значения `newText` в текстовое значение `text` в позиции `offset`. Позиции начинаются с номера 0.

Пример 1

Вставить "C" между "B" и "D" в строке "ABD".

```
Text.Insert("ABD", 2, "C")
```

```
"ABCD"
```

Text.Length

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Length(text as nullable text) as nullable number
```

Сведения

Возвращает число символов в тексте `text`.

Пример 1

Определить, сколько символов содержится в тексте "Hello World".

```
Text.Length("Hello World")
```


Text.Lower

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Lower(text as nullable text, optional culture as nullable text) as nullable text
```

Сведения

Возвращает результат преобразования всех символов в `text` в нижний регистр. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение версии строки "AbCd" в нижнем регистре.

```
Text.Lower("AbCd")
```

```
"abcd"
```

Text.Middle

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Middle(text as nullable text, start as number, optional count as nullable number) as nullable text
```

Сведения

Возвращает символы (`count` шт.) или идет до конца `text`; при смещении `start`.

Пример 1

Найти подстроку из текста "Hello World" длиной 5 символов, начиная с индекса 6.

```
Text.Middle("Hello World", 6, 5)
```

```
"World"
```

Пример 2

Найти подстроку в тексте "Hello World", начиная с индекса 6 до конца.

```
Text.Middle("Hello World", 6, 20)
```

```
"World"
```

Text.NewGuid

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.NewGuid() as text
```

Сведения

Возвращает новый случайный глобальный уникальный идентификатор (GUID).

Text.PadEnd

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.PadEnd(text as nullable text, count as number, optional character as nullable text) as nullable text
```

Сведения

Возвращает значение `text`, заполненное до длины `count` путем вставки пробелов в конце текстового значения `text`. Для указания символа, используемого для заполнения, можно использовать необязательный символ `character`. Символом заполнения по умолчанию является пробел.

Пример 1

Заполнить конец текстового значения так, чтобы в значении было 10 символов.

```
Text.PadEnd("Name", 10)
```

```
"Name      "
```

Пример 2

Заполнить конец текстового значения знаком "|" так, чтобы в значении было 10 символов.

```
Text.PadEnd("Name", 10, "|")
```

```
"Name|||||"
```

Text.PadStart

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.PadStart(text as nullable text, count as number, optional character as nullable text) as nullable text
```

Сведения

Возвращает значение `text`, заполненное до длины `count` путем вставки пробелов в начале текстового значения `text`. Для указания символа, используемого для заполнения, можно использовать необязательный символ `character`. Символом заполнения по умолчанию является пробел.

Пример 1

Заполнить начало текстового значения так, чтобы в значении было 10 символов.

```
Text.PadStart("Name", 10)
```

```
"      Name"
```

Пример 2

Заполнить начало текстового значения знаком "|" так, чтобы в значении было 10 символов.

```
Text.PadStart("Name", 10, "|")
```

```
"|||||Name"
```

Синтаксис

```
Text.PositionOf(text as text, substring as text, optional occurrence as nullable number,  
optional comparer as nullable function) as any
```

Сведения

Возвращает позицию указанного вхождения `substring` текстового значения, обнаруженной в `text`. Для указания позиции возвращаемого вхождения можно использовать необязательный параметр `occurrence` (первое вхождение по умолчанию). Возвращает значение `-1`, если `substring` не найден.

`comparer` — это модуль `Comparer`, который используется для управления сравнением. Функции сравнения можно использовать для сравнений, учитывающих или не учитывающих регистр, язык и региональные параметры, а также локаль.

В языке формул доступны следующие встроенные функции сравнения:

- `Comparer.Ordinal`: используется для точного сравнения по порядковому номеру
- `Comparer.OrdinalIgnoreCase`: используется для точного сравнения по порядковому номеру без учета регистра
- `Comparer.FromCulture`: используется для сравнения с учетом языка и региональных параметров

Пример 1

Получить позицию первого вхождения "World" в тексте "Hello, World! Hello, World!".

```
Text.PositionOf("Hello, World! Hello, World!", "World")
```

7

Пример 2

Получить позицию последнего вхождения "World" в "Hello, World! Hello, World!".

```
Text.PositionOf("Hello, World! Hello, World!", "World", Occurrence.Last)
```

21

Text.PositionOfAny

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.PositionOfAny(text as text, characters as list, optional occurrence as nullable number) as any
```

Сведения

Возвращает позицию первого вхождения любого из символов в списке символов `text`, обнаруженного в текстовом значении `characters`. Для указания позиции возвращаемого вхождения можно использовать необязательный параметр `occurrence`.

Пример 1

Найти позицию "W" в тексте "Hello, World!".

```
Text.PositionOfAny("Hello, World!", {"W"})
```

7

Пример 2

Найти позицию "W" или "H" в тексте "Hello, World!".

```
Text.PositionOfAny("Hello, World!", {"H", "W"})
```

0

Синтаксис

```
Text.Proper(text as nullable text, optional culture as nullable text) as nullable text
```

Сведения

Возвращает результат перевода в верхний регистр только первой буквы каждого слова в текстовом значении `text`. Все остальные буквы возвращаются в нижнем регистре. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Использование `Text.Proper` для простого предложения.

```
Text.Proper("the QUICK BrOwN fOx jUmPs oVER tHe LAzy DoG")
```

```
"The Quick Brown Fox Jumps Over The Lazy Dog"
```


Text.Range

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Range(text as nullable text, offset as number, optional count as nullable number) as nullable text
```

Сведения

Возвращает подстроку из текста `text`, найденную по смещению `offset`. Можно использовать необязательный параметр `count`, чтобы указать число возвращаемых символов. Выдает ошибку, если символов недостаточно.

Пример 1

Найти подстроку из текста "Hello World", начиная с индекса 6.

```
Text.Range("Hello World", 6)
```

```
"World"
```

Пример 2

Найти подстроку из текста "Hello World Hello" длиной 5 символов, начиная с индекса 6.

```
Text.Range("Hello World Hello", 6, 5)
```

```
"World"
```

Text.Remove

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Remove(text as nullable text, removeChars as any) as nullable text
```

Сведения

Возвращает копию текстового значения `text`, в которой удалены все символы из `removeChars`.

Пример 1

Удаление символов "," и ";" из текстового значения.

```
Text.Remove("a,b;c", {",",";"})
```

```
"abc"
```

Text.RemoveRange

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.RemoveRange(text as nullable text, offset as number, optional count as nullable number) as nullable text
```

Сведения

Возвращает копию текстового значения `text`, в которой удалены все символы с позиции `offset`. Можно использовать необязательный параметр `count`, чтобы указать число символов, которое необходимо удалить. Значение `count` по умолчанию — 1. Значения позиций начинаются с 0.

Пример 1

Удалить 1 символ из текстового значения "ABEFC" в позиции 2.

```
Text.RemoveRange("ABEFC", 2)
```

```
"ABFC"
```

Пример 2

Удалить 2 символа из текстового значения "ABEFC", начиная с позиции 2.

```
Text.RemoveRange("ABEFC", 2, 2)
```

```
"ABC"
```

Text.Repeat

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Repeat(text as nullable text, count as number) as nullable text
```

Сведения

Возвращает текстовое значение, состоящее из входного текста `text`, повторенного `count` раз.

Пример 1

Повторить текст "a" пять раз.

```
Text.Repeat("a", 5)
```

```
"aaaaa"
```

Пример 2

Повторить текст "helloworld" три раза.

```
Text.Repeat("helloworld.", 3)
```

```
"helloworld.helloworld.helloworld."
```

Text.Replace

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Replace(text as nullable text, old as text, new as text) as nullable text
```

Сведения

Возвращает результат замены всех вхождений текстового значения `old` в текстовом значении `text` текстовым значением `new`. В этой функции учитывается регистр.

Пример 1

Заменить все вхождения "the" в предложении на "a".

```
Text.Replace("the quick brown fox jumps over the lazy dog", "the", "a")
```

```
"a quick brown fox jumps over a lazy dog"
```

Text.ReplaceRange

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.ReplaceRange(text as nullable text, offset as number, count as number, newText as text) as nullable text
```

Сведения

Возвращает результат удаления заданного количества символов `count` из текстового значения `text`, начиная с позиции `offset`, затем вставляет текстовое значение `newText` в ту же позицию в `text`.

Пример 1

Заменить один символ в позиции 2 в текстовом значении "ABGF" новым текстовым значением "CDE".

```
Text.ReplaceRange("ABGF", 2, 1, "CDE")
```

```
"ABCDEF"
```

Text.Reverse

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Reverse(text as nullable text) as nullable text
```

Сведения

Обращает порядок указанного `text`.

Пример 1

Обратный порядок текста "123".

```
Text.Reverse("123")
```

```
"321"
```

Text.Select

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Select(text as nullable text, selectChars as any) as nullable text
```

Сведения

Возвращает копию текстового значения `text`, из которой удалены все символы, которых нет в `selectChars`.

Пример 1

Выберите в текстовом значении все символы в диапазоне от "а" до "я".

```
Text.Select("a,b;c", {"a".."z"})
```

```
"abc"
```


Text.Split

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Split(text as text, separator as text) as list
```

Сведения

Возвращает список текстовых значений, полученных в результате разбиения текстового значения `text` на основе указанного разделителя `separator`.

Пример 1

Создать список из текстового значения "Name|Address|PhoneNumber", разбитого с помощью разделителя "|".

```
Text.Split("Name|Address|PhoneNumber", "|")
```

name
Адрес
PhoneNumber

Text.SplitAny

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.SplitAny(text as text, separators as text) as list
```

Сведения

Возвращает список текстовых значений, полученных в результате разбиения текстового значения `text` на основе любого символа в указанном разделителе `separators`.

Пример 1

Создать список из текстового значения "Jamie|Campbell|Admin|Adventure Works|www.adventure-works.com".

```
Text.SplitAny("Jamie|Campbell|Admin|Adventure Works|www.adventure-works.com", "|")
```

Jamie
Campbell
Административный
Adventure Works
www.adventure-works.com

Text.Start

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Start(text as nullable text, count as number) as nullable text
```

Сведения

Возвращает первые `count` символов из `text` как текстовое значение.

Пример 1

Получение первых 5 символов "Hello, World".

```
Text.Start("Hello, World", 5)
```

```
"Hello"
```

Text.StartsWith

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.StartsWith(text as nullable text, substring as text, optional comparer as nullable function) as nullable logical
```

Сведения

Возвращает значение true, если текстовое значение `text` начинается с текстового значения `substring`.

- `text`: значение `text`, которое требуется найти.
- `substring`: значение `text`, являющееся подстрокой, которую требуется найти в `substring`.
- `comparer`: [необязательно] `Comparer`, используемый для управления сравнением. Например, `Comparer.OrdinalIgnoreCase` может использоваться для выполнения поиска без учета регистра.

`comparer` — это модуль `Comparer`, который используется для управления сравнением. Функции сравнения можно использовать для сравнений, учитывающих или не учитывающих регистр, язык и региональные параметры, а также локаль.

В языке формул доступны следующие встроенные функции сравнения:

- `Comparer.Ordinal`: используется для точного сравнения по порядковому номеру
- `Comparer.OrdinalIgnoreCase`: используется для точного сравнения по порядковому номеру без учета регистра
- `Comparer.FromCulture`: используется для сравнения с учетом языка и региональных параметров

Пример 1

Проверить, начинается ли текст "Hello, World" с "hello".

```
Text.StartsWith("Hello, World", "hello")
```

false

Пример 2

Проверить, начинается ли текст "Hello, World" с "Hello".

```
Text.StartsWith("Hello, World", "Hello")
```

true

Text.ToBinary

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.ToBinary(text as nullable text, optional encoding as nullable number, optional  
includeByteOrderMark as nullable logical) as nullable binary
```

Сведения

Кодирует заданное текстовое значение `text` в двоичное значение с помощью указанного `encoding`.

Text.ToList

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.ToList(text as text) as list
```

Сведения

Возвращает список значений символов из заданного текстового значения `text`.

Пример 1

Создать список значений символов из текста "Hello World".

```
Text.ToList("Hello World")
```

H

l

l

l

o

W

o

r

l

d

Text.Trim

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Trim(text as nullable text, optional trim as any) as nullable text
```

Сведения

Возвращает результат удаления всех начальных и конечных пробелов из текстового значения `text`.

Пример 1

Удаление начальных и конечных пробелов из " a b c d ".

```
Text.Trim("    a b c d    ")
```

```
"a b c d"
```

Text.TrimEnd

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.TrimEnd(text as nullable text, optional trim as any) as nullable text
```

Сведения

Возвращает результат удаления всех конечных пробелов из текстового значения `text`.

Пример 1

Удалить конечные пробелы из " a b c d ".

```
Text.TrimEnd("    a b c d    ")
```

```
"    a b c d"
```


Text.TrimStart

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.TrimStart(text as nullable text, optional trim as any) as nullable text
```

Сведения

Возвращает результат удаления всех начальных пробелов из текстового значения `text`.

Пример 1

Удаление начальных пробелов из " a b c d ".

```
Text.TrimStart("  a b c d  ")
```

```
"a b c d  "
```

Text.Upper

16.03.2021 • 2 minutes to read

Синтаксис

```
Text.Upper(text as nullable text, optional culture as nullable text) as nullable text
```

Сведения

Возвращает результат преобразования всех символов в `text` в верхний регистр. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получить версию строки "aBcD" в верхнем регистре.

```
Text.Upper("aBcD")
```

```
"ABCD"
```

TextEncoding.Ascii

16.03.2021 • 2 minutes to read

Сведения

Используется для выбора двоичной формы ASCII.

TextEncoding.BigEndianUnicode

16.03.2021 • 2 minutes to read

Сведения

Используется для выбора двоичной формы UTF16 с обратным порядком байтов.

TextEncoding.Unicode

16.03.2021 • 2 minutes to read

Сведения

Используется для выбора двоичной формы UTF16 с прямым порядком байтов.

TextEncoding.Utf8

16.03.2021 • 2 minutes to read

Сведения

Используется для выбора двоичной формы UTF8.

TextEncoding.Utf16

16.03.2021 • 2 minutes to read

Сведения

Используется для выбора двоичной формы UTF16 с прямым порядком байтов.

TextEncoding.Windows

16.03.2021 • 2 minutes to read

Сведения

Используется для выбора двоичной формы Windows.

Функции для работы со временем

16.03.2021 • 2 minutes to read

Эти функции позволяют создавать значения параметра time и управлять ими.

Время

ФУНКЦИЯ	ОПИСАНИЕ
Time.EndOfHour	Возвращает значение DateTime из конца часа.
Time.From	Возвращает значение времени из заданного значения.
Time.FromText	Возвращает значение времени из набора форматов даты.
Time.Hour	Возвращает значение часов из значения DateTime.
Time.Minute	Возвращает значение минут из значения DateTime.
Time.Second	Возвращает значение секунд из значения DateTime.
Time.StartOfHour	Возвращает первое значение часов из значения времени.
Time.ToRecord	Возвращает запись, содержащую части значения даты.
Time.ToText	Возвращает текстовое значение из значения времени.
#time	Создает значение времени из часов, минут и секунд.

Time.EndOfHour

16.03.2021 • 2 minutes to read

Синтаксис

```
Time.EndOfHour(dateTime as any) as any
```

Сведения

Возвращает значение `time`, `datetime` или `datetimezone`, представляющее конец часа в `dateTime`, включая доли секунды. Сведения о часовом поясе сохраняются.

- `dateTime`: значение `time`, `datetime` или `datetimezone`, из которого вычисляется конец часа.

Пример 1

Получение конца часа для 14.05.2011 17:00:00.

```
Time.EndOfHour(#datetime(2011, 5, 14, 17, 0, 0))
```

```
#datetime(2011, 5, 14, 17, 59, 59.9999999)
```

Пример 2

Получение конца часа для 17.05.2011 17:00:00–19:00.

```
Time.EndOfHour(#datetimezone(2011, 5, 17, 5, 0, 0, -7, 0))
```

```
#datetimezone(2011, 5, 17, 5, 59, 59.9999999, -7, 0)
```

Time.From

16.03.2021 • 2 minutes to read

Синтаксис

```
Time.From(value as any, optional culture as nullable text) as nullable time
```

Сведения

Возвращает значение `time` из заданного параметра `value`. Можно также указать необязательный параметр `culture` (например, en-US). Если заданный параметр `value` имеет значение `null`, `Time.From` возвращает `null`. Если заданный параметр `value` имеет значение `time`, возвращается `value`. Значения следующих типов можно преобразовать в значение `time`:

- `text`: значение `time` из текстового представления. Дополнительные сведения см. в разделе `Time.FromText`.
- `datetime`: компонент времени `value`.
- `datetimezone`: компонент времени локального эквивалента `datetime` `value`.
- `number`: `time` эквивалентно количеству неполных дней, выраженных `value`. Если значение `value` является отрицательным либо больше или равно 1, возвращается ошибка.

Если `value` имеет любой другой тип, возвращается ошибка.

Пример 1

Преобразование `0.7575` в значение `time`.

```
Time.From(0.7575)
```

```
#time(18, 10, 48)
```

Пример 2

Преобразование `#datetime(1899, 12, 30, 06, 45, 12)` в значение `time`.

```
Time.From(#datetime(1899, 12, 30, 06, 45, 12))
```

```
#time(06, 45, 12)
```

Time.FromText

16.03.2021 • 2 minutes to read

Синтаксис

```
Time.FromText(text as nullable text, optional culture as nullable text) as nullable time
```

Сведения

Создает значение `time` из текстового представления `text` согласно стандарту формата ISO 8601.

Можно также указать необязательный параметр `culture` (например, en-US).

- `Time.FromText("12:34:12")` // Время, чч:мм:сс
- `Time.FromText("12:34:12.1254425")` // чч:мм:мм.ннннннн

Пример 1

Преобразование `"10:12:31am"` в значение времени.

```
Time.FromText("10:12:31am")
```

```
#time(10, 12, 31)
```

Пример 2

Преобразование `"1012"` в значение времени.

```
Time.FromText("1012")
```

```
#time(10, 12, 00)
```

Пример 3

Преобразование `"10"` в значение времени.

```
Time.FromText("10")
```

```
#time(10, 00, 00)
```

Time.Hour

16.03.2021 • 2 minutes to read

Синтаксис

```
Time.Hour(dateTime as any) as nullable number
```

Сведения

Возвращает компонент часов заданных значений `time`, `datetime` или `datetimezone`, — `dateTime`.

Пример 1

Поиск часа в #datetime(2011, 12, 31, 9, 15, 36).

```
Time.Hour(#datetime(2011, 12, 31, 9, 15, 36))
```

Time.Minute

16.03.2021 • 2 minutes to read

Синтаксис

```
Time.Minute(dateTime as any) as nullable number
```

Сведения

Возвращает компонент минуты заданного значения `time`, `datetime` или `datetimezone` — `dateTime`.

Пример 1

Поиск компонента минуты в `#datetime(2011, 12, 31, 9, 15, 36)`.

```
Time.Minute(#datetime(2011, 12, 31, 9, 15, 36))
```

Time.Second

16.03.2021 • 2 minutes to read

Синтаксис

```
Time.Second(dateTime as any) as nullable number`
```

Сведения

Возвращает компонент секунд заданных значений `time`, `datetime` или `datetimezone`, — `dateTime`.

Пример 1

Поиск второго значения в значении datetime.

```
Time.Second(#datetime(2011, 12, 31, 9, 15, 36.5))
```

```
36.5
```

Time.StartOfHour

16.03.2021 • 2 minutes to read

Синтаксис

```
Time.StartOfHour(dateTime as any) as any
```

Сведения

Возвращает первое значение часа с учетом типа `time`, `datetime` или `datetimezone`.

Пример 1

Поиск начала часа для 10 октября 2011 г., 8:10:32 (`#datetime(2011, 10, 10, 8, 10, 32)`).

```
Time.StartOfHour(#datetime(2011, 10, 10, 8, 10, 32))
```

```
#datetime(2011, 10, 10, 8, 0, 0)
```


Time.ToRecord

16.03.2021 • 2 minutes to read

Синтаксис

```
Time.ToRecord(time as time) as record
```

Сведения

Возвращает запись, содержащую части заданного значения времени, `time`.

- `time`: значение `time`, для которого вычисляется запись его частей.

Пример 1

Преобразование значения `#time(11, 56, 2)` в запись, содержащую значения времени.

```
Time.ToRecord(#time(11, 56, 2))
```

ЧАС	11
МИНУТА	56
СЕКУНДА	2

Time.ToText

16.03.2021 • 2 minutes to read

Синтаксис

```
Time.ToText(time as nullable time, optional format as nullable text, optional culture as nullable text) as nullable text
```

О программе

Возвращает текстовое представление `time`. Для форматирования текста можно указать необязательный параметр `format`. Можно также указать необязательный параметр `culture` (например, en-US).

Пример 1

Получение текстового представления `#time(11, 56, 2)`.

```
Time.ToText(#time(11, 56, 2))
```

```
"11:56 AM"
```

Пример 2

Получение текстового представления `#time(11, 56, 2)` с возможностью форматирования.

```
Time.ToText(#time(11, 56, 2), "hh:mm")
```

```
"11:56"
```

#time

16.03.2021 • 2 minutes to read

Синтаксис

```
#time(hour as number, minute as number, second as number) as time
```

Сведения

Создает значение времени из целого числа часа `hour`, минуты `minute` и секунды `second` (дробное значение). Вызывает ошибку, если не соблюдаются следующие условия:

- $0 \leq \text{час} \leq 24$
- $0 \leq \text{минута} \leq 59$
- $0 \leq \text{секунда} \leq 59$
- Если значение часа равно 24, то минута и секунда должны быть равны 0

ФУНКЦИИ ТИПОВ

16.03.2021 • 2 minutes to read

Эти функции позволяют создавать значения типа и управлять ими.

Type

ФУНКЦИЯ	ОПИСАНИЕ
Type.AddTableKey	Добавляет ключ в тип таблицы.
Type.ClosedRecord	Возвращает закрытую версию данного типа записи (или такой же тип, если он уже закрыт).
Type.Facets	Возвращает аспекты типа.
Type.ForFunction	Создает тип функции на основе заданного.
Type.ForRecord	Возвращает тип записи для записи полей.
Type.FunctionParameters	Возвращает запись со значениями полей, в качестве которых указаны имена параметров типа функции, а также их значения, отражающие соответствующие типы.
Type.FunctionRequiredParameters	Возвращает число, обозначающее минимальное количество параметров, необходимое для вызова типа функции.
Type.FunctionReturn	Возвращает тип, возвращенный типом функции.
Type.Is	Type.Is
Type.IsNullable	Возвращает значение true, если тип является допускающим значение NULL. В противном случае возвращается значение false.
Type.IsOpenRecord	Возвращает данные о том, является ли тип записи открытым.
Type.ListItem	Возвращает тип элемента на основании типа списка.
Type.NonNullable	Возвращает тип, не допускающий значение NULL.
Type.OpenRecord	Возвращает открытую версию типа записи (или такой же тип, если запись уже является открытой).
Type.RecordFields	Возвращает запись, описывающую поля типа записи. Каждое поле возвращенного типа записи имеет соответствующее имя и значение в виде записи [Type = type, Opional = logical].

ФУНКЦИЯ	ОПИСАНИЕ
Type.ReplaceFacets	Заменяет аспекты типа.
Type.ReplaceTableKeys	Заменяет ключи в типе таблицы.
Type.TableColumn	Возвращает тип столбца в таблице.
Type.TableKeys	Возвращает ключи из типа таблицы.
Type.TableRow	Возвращает тип строки из типа таблицы.
Type.TableSchema	Возвращает таблицу, содержащую описание столбцов (схему) указанного типа таблицы.
Type.Union	Возвращает объединение списка типов.

Type.AddTableKey

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.AddTableKey(table as type, columns as list, isPrimary as logical) as type
```

Сведения

Добавляет ключ к данному типу таблицы.

Type.ClosedRecord

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.ClosedRecord(type as type) as type
```

Сведения

Возвращает закрытую версию заданного типа `record` `type` (или такой же тип, если он уже закрыт).

Пример 1

Создание закрытой версии `type [ A = number, ... ]`.

```
Type.ClosedRecord(type [A = number, ...])
```

```
type [A = number]
```

Type.Facets

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.Facets(type as type) as record
```

Сведения

Возвращает запись, содержащую аспекты `type`

Type.ForFunction

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.ForFunction(signature as record, min as number) as type
```

Сведения

Создает `function type` из `signature`, запись `ReturnType` и `Parameters`, и `min` — минимальное число аргументов, необходимых для вызова функции.

Пример 1

Создает тип функции, которая принимает числовой параметр X и возвращает число.

```
Type.ForFunction([ReturnType = type number, Parameters = [X = type number]], 1)
```

```
type function (X as number) as number
```

Type.ForRecord

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.ForRecord(fields as record, open as logical) as type
```

Сведения

Возвращает тип, представляющий записи с определенными ограничениями на тип для полей.

Type.FunctionParameters

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.FunctionParameters(type as type) as record
```

Сведения

Возвращает запись со значениями полей, в качестве которых указаны имена параметров `type`, а также их значения, для которых указаны соответствующие типы.

Пример

Поиск типов параметров для функции `(x as number, y as text)`.

```
Type.FunctionParameters(type function (x as number, y as text) as any)
```

x	[Type]
y	[Type]

Type.FunctionRequiredParameters

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.FunctionRequiredParameters(type as type) as number
```

Сведения

Возвращает число, обозначающее минимальное количество параметров, необходимых для вызова входных данных `type` функции.

Пример 1

Поиск количества обязательных параметров для функции `(x as number, optional y as text)`.

```
Type.FunctionRequiredParameters(type function (x as number, optional y as text) as any)
```

Type.FunctionReturn

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.FunctionReturn(type as type) as type
```

Сведения

Возвращает тип, возвращенный функцией `type`.

Пример 1

Найти возвращаемый тип `() as any`.

```
Type.FunctionReturn(type function () as any)
```

```
type any
```

Type.Is

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.Is(type1 as type, type2 as type) as logical
```

Сведения

Type.Is

Type.IsNullable

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.IsNullable(type as type) as logical
```

Сведения

Возвращает `true`, если тип `nullable`, и `false` в остальных случаях.

Пример 1

Определить, допускает ли тип `number` значения NULL.

```
Type.IsNullable(type number)
```

```
false
```

Пример 2

Определить, допускает ли тип `type nullable number` значения NULL.

```
Type.IsNullable(type nullable number)
```

```
true
```

Type.IsOpenRecord

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.IsOpenRecord(type as type) as logical
```

Сведения

Возвращает значение `logical`, указывающее, является ли запись `type` открытой.

Пример 1

Определение того, открыта ли запись `type [ A = number, ... ]`.

```
Type.IsOpenRecord(type [A = number, ...])
```

```
true
```


Type.ListItem

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.ListItem(type as type) as type
```

Сведения

Возвращает тип элемента из списка `type`.

Пример 1

Поиск типа элемента в списке `{number}`.

```
Type.ListItem(type {number})
```

```
type number
```

Type.NonNullable

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.NonNullable(type as type) as type
```

Сведения

Возвращает тип, отличный от `nullable`, из `type`.

Пример 1

Возврат типа, не допускающего значение NULL, для `type nullable number`.

```
Type.NonNullable(type nullable number)
```

```
type number
```

Type.OpenRecord

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.OpenRecord(type as type) as type
```

Сведения

Возвращает открытую версию заданного типа `record` `type` (или такой же тип, если запись уже является открытой).

Пример 1

Создать открытую версию `type [ A = number ]`.

```
Type.OpenRecord(type [A = number])
```

```
type [A = number, ...]
```

Type.RecordFields

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.RecordFields(type as type) as record
```

Сведения

Возвращает запись, описывающую поля записи `type`. У каждого поля возвращаемого типа записи есть соответствующее имя и значение в виде записи `[ Type = type, Optional = logical ]`.

Пример

Поиск имени и значения записи `[ A = number, optional B = any ]`.

```
Type.RecordFields(type [A = number, optional B = any])
```

ОБЪЕКТ	[Запись]
B	[Запись]

Type.ReplaceFacets

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.ReplaceFacets(type as type, facets as record) as type
```

Сведения

Заменяет аспекты `type` на аспекты из записи `facets`.

Type.ReplaceTableKeys

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.ReplaceTableKeys(tableType as type, keys as list) as type
```

Сведения

Возвращает новый тип таблицы, где все ключи заменены указанным списком ключей.

Type.TableColumn

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.TableColumn(tableType as type, column as text) as type
```

Сведения

Возвращает тип столбца `column` в типе таблицы `tableType`.

Type.TableKeys

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.TableKeys(tableType as type) as list
```

Сведения

Возвращает (возможно, пустой) список ключей для данного типа таблицы.

Type.TableRow

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.TableRow(table as type) as type
```

Сведения

Type.TableRow

Type.TableSchema

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.TableSchema(tableType as type) as table
```

Сведения

Возвращает таблицу, описывающую столбцы объекта `tableType`.

Type.Union

16.03.2021 • 2 minutes to read

Синтаксис

```
Type.Union(types as list) as type
```

Сведения

Возвращает объединение типов в `types`.

Функции для работы с URI

16.03.2021 • 2 minutes to read

Эти функции позволяют создать строки запросов URI и управлять ими.

URI

ФУНКЦИЯ	ОПИСАНИЕ
Uri.BuildQueryString	Собирает запись в строку запроса универсального кода ресурса (URI).
Uri.Combine	Возвращает URI на основе сочетания базовой и относительной частей.
Uri.EscapeDataString	Кодирует специальные знаки в соответствии с RFC 3986.
Uri.Parts	Возвращает значение записи с полями, для которых заданы части текстового значения URI.

Uri.BuildQueryString

16.03.2021 • 2 minutes to read

Синтаксис

```
Uri.BuildQueryString(query as record) as text
```

Сведения

Соберите запись `query` в строку запроса универсального кода ресурса, при необходимости добавив escape-символы.

Пример

Закодируйте строку запроса, которая содержит специальные символы.

```
Uri.BuildQueryString([a = "1", b = "+$"])
```

```
"a=1&b=%2B%24"
```

Uri.Combine

16.03.2021 • 2 minutes to read

Синтаксис

```
Uri.Combine(baseUri as text, relativeUri as text) as text
```

Сведения

Возвращает абсолютный URI, представляющие собой сочетание входных данных `baseUri` и `relativeUri`.

Uri.EscapeDataString

16.03.2021 • 2 minutes to read

Синтаксис

```
Uri.EscapeDataString(data as text) as text
```

Сведения

Кодирует специальные символы во входных данных `data` в соответствии с правилами RFC 3986.

Пример

Закодируйте специальные символы в "+money\$".

```
Uri.EscapeDataString("+money$")
```

```
"%2Bmoney%24"
```

Uri.Parts

16.03.2021 • 2 minutes to read

Синтаксис

```
Uri.Parts(absoluteUri as text) as record
```

Сведения

Возвращает (как запись) части введенного `absoluteUri` с такими значениями, как схема, узел, порт, путь, запрос, фрагмент, имя пользователя и пароль.

Пример 1

Найти части абсолютного URI "www.adventure-works.com".

```
Uri.Parts("www.adventure-works.com")
```

СХЕМА	http
УЗЕЛ	www.adventure-works.com
ПОРТ	80
ПУТЬ	/
ЗАПРОС	[Запись]
FRAGMENT	
USERNAME	
ПАРОЛЬ	

Пример 2

Декодировать закодированную строку.

```
let  
    UriUnescapeDataString = (data as text) as text => Uri.Parts("http://contoso?a=" & data)[Query][a]  
in  
    UriUnescapeDataString("%2Bmoney%24")
```

```
" +money$ "
```


Функции для работы со значениями

16.03.2021 • 4 minutes to read

Эти функции позволяют оценивать значения и выполнять операции с ними.

Операции со значениями

ФУНКЦИЯ	ОПИСАНИЕ
Value.Alternates	Выражает альтернативные планы запросов.
Value.Compare	Возвращает значение 1, 0 или -1, если value1 соответственно больше value2, равно ему или меньше этого значения. Также можно указать необязательную функцию сравнения.
Value.Equals	Проверяет равенство двух значений.
Value.Expression	Возвращает объект AST, представляющий выражение значения.
Value.NativeQuery	Вычисляет запрос для целевого объекта.
Value.NullableEquals	Возвращает логическое значение или значение NULL на основе двух значений.
Value.Optimize	Если значение представляет собой запрос, который можно оптимизировать, возвращает оптимизированный запрос. В противном случае возвращает значение.
Value.Type	Возвращает тип заданного значения.

Арифметические операции

КОМПОНЕНТ	ОПИСАНИЕ
Value.Add	Возвращает сумму двух значений.
Value.Divide	Возвращает результат деления первого значения на второе.
Value.Multiply	Возвращает произведение двух значений.
Value.Subtract	Возвращает разность двух значений.

Параметры арифметических операций

КОМПОНЕНТ	ОПИСАНИЕ
-----------	----------

КОМПОНЕНТ	ОПИСАНИЕ
Precision.Double	Необязательный параметр для встроенных арифметических операторов для указания двойной точности.
Precision.Decimal	Необязательный параметр для встроенных арифметических операторов для указания десятичной точности.

Типы параметров

ТИП	ОПИСАНИЕ
Value.As	Функция Value.As соответствует оператору As в языке формул. Выражение "значение as тип" утверждает, что аргумент значения совместим с типом в соответствии с применением оператора Is. В случае несовместимости возвращается ошибка.
Value.Is	Функция Value.Is соответствует оператору Is в языке формул. Выражение "значение is тип" возвращает true, если приписываемый тип значения совместим с типом, и false, если они несовместимы.
Value.ReplaceType	С помощью функции Value.ReplaceType можно приписать тип значению. Функция Value.ReplaceType возвращает либо новое значение приписываемого типа, либо ошибку, если новый тип несовместим с собственным примитивным типом значения. В частности, эта функция возвращает ошибку при попытке приписать абстрактный тип, такой как Any. При замене типа записи новый тип должен содержать то же количество полей. При этом замена старых полей новыми осуществляется по порядку следования, а не по имени. Аналогичным образом, при замене типа таблицы новый тип должен содержать то же количество столбцов, а замена старых столбцов новыми происходит в порядке их следования.

РЕАЛИЗАЦИЯ	ОПИСАНИЕ
DirectQueryCapabilities.From	DirectQueryCapabilities.From
Embedded.Value	Доступ к значению по имени во внедренном гибридном веб-приложении.
Value.Firewall	Value.Firewall
Variable.Value	Variable.Value
SqlExpression.SchemaFrom	SqlExpression.SchemaFrom
SqlExpression.ToExpression	SqlExpression.ToExpression

Метаданные

КОМПОНЕНТ	ОПИСАНИЕ
Value.Metadata	Возвращает запись, содержащую метаданные входных данных.
Value.RemoveMetadata	Удаляет метаданные для значения и возвращает исходное значение.
Value.ReplaceMetadata	Заменяет метаданные для значения предоставленной новой записью метаданных и возвращает исходное значение, к которому присоединены новые метаданные.

Журнал преобразований

ФУНКЦИЯ	ОПИСАНИЕ
Graph.Nodes	Эта функция предназначена только для внутреннего использования.
Value.Lineage	Эта функция предназначена только для внутреннего использования.
Value.Traits	Эта функция предназначена только для внутреннего использования.

DirectQueryCapabilities.From

16.03.2021 • 2 minutes to read

Синтаксис

```
DirectQueryCapabilities.From(value as any) as table
```

Сведения

DirectQueryCapabilities.From

Embedded.Value

16.03.2021 • 2 minutes to read

Синтаксис

```
Embedded.Value(value as any, path as text) as any
```

Сведения

Доступ к значению по имени во внедренном гибридном веб-приложении.

Graph.Nodes

16.03.2021 • 2 minutes to read

Синтаксис

```
Graph.Nodes(graph as record) as list
```

О программе

Эта функция предназначена только для внутреннего использования.

Precision.Decimal

16.03.2021 • 2 minutes to read

Сведения

Необязательный параметр для встроенных арифметических операторов для указания десятичной точности.

Precision.Double

16.03.2021 • 2 minutes to read

Сведения

Необязательный параметр для встроенных арифметических операторов для указания двойной точности.

SqlExpression.SchemaFrom

16.03.2021 • 2 minutes to read

Синтаксис

```
SqlExpression.SchemaFrom(schema as any) as any
```

Сведения

SqlExpression.SchemaFrom

SqlExpression.ToExpression

16.03.2021 • 2 minutes to read

Синтаксис

```
SqlExpression.ToExpression(sql as text, environment as record) as text
```

Сведения

SqlExpression.ToExpression

Value.Add

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Add(value1 as any, value2 as any, optional precision as nullable number) as any
```

Сведения

Возвращает сумму `value1` и `value2`. Можно задать необязательный параметр `precision`, который по умолчанию имеет значение `Precision.Double`.

Value.Alternates

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Alternates(alternates as list) as any
```

Сведения

Выражает альтернативные планы запросов в соответствующем выражении, полученном с помощью

```
Value.Expression(Value.Optimize(...))
```

 . Иное использование не подразумевается.

Value.As

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.As(value as any, type as type) as any
```

Сведения

Value.As

Value.Compare

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Compare(value1 as any, value2 as any, optional precision as nullable number) as number
```

Сведения

Возвращает значение -1 , 0 или 1 , если первое значение соответственно меньше второго, равно ему или больше него.

Value.Divide

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Divide(value1 as any, value2 as any, optional precision as nullable number) as any
```

Сведения

Возвращает результат деления `value1` на `value2`. Можно задать необязательный параметр `precision`, который по умолчанию имеет значение `Precision.Double`.

Value.Equals

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Equals(value1 as any, value2 as any, optional precision as nullable number) as logical
```

Сведения

Возвращает значение true, если значение `value1` равно значению `value2`, в противном случае — false.

Value.Expression

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Expression(value as any) as nullable record
```

О программе

Возвращает объект AST, представляющий выражение значения.

Value.Firewall

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Firewall(key as text) as any
```

Сведения

Value.Firewall

Value.FromText

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.FromText(text as any, optional culture as nullable text) as any
```

Сведения

Декодирует значение из текстового представления `text` и интерпретирует его как значение соответствующего типа. `Value.FromText` принимает текстовое значение и возвращает число, логическое значение, значение NULL, значение даты и времени, значение длительности или текстовое значение. Пустое текстовое значение интерпретируется как значение NULL. Можно также указать необязательный параметр `culture` (например, en-US).

Value.Is

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Is(value as any, type as type) as logical
```

Сведения

Value.Is

Value.Lineage

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Lineage(value as any) as any
```

О программе

Эта функция предназначена только для внутреннего использования.

Value.Metadata

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Metadata(value as any) as any
```

Сведения

Возвращает запись, содержащую метаданные входных данных.

Value.Multiply

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Multiply(value1 as any, value2 as any, optional precision as nullable number) as any
```

Сведения

Возвращает произведение `value1` на `value2`. Можно задать необязательный параметр `precision`, который по умолчанию имеет значение `Precision.Double`.

Value.NativeQuery

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.NativeQuery(target as any, query as text, optional parameters as any, optional options as nullable record) as any
```

Сведения

Вычисляет `query` в `target` с помощью параметров, указанных в `parameters` и в `options`.

Выходные данные запроса определяются `target`.

В `target` предоставляется контекст для операции, описанной `query`.

В `query` описывается запрос, который будет выполнен в `target`. `query` обычно выражается так же, как в `target` (например, в инструкции T-SQL).

Необязательное значение `parameters` может содержать соответствующий список или запись для предоставления значений параметров, ожидаемых `query`.

Необязательная запись `options` может содержать параметры, которые влияют на поведение вычисления `query` в `target`. Эти параметры относятся к `target`.

Value.NullableEquals

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.NullableEquals(value1 as any, value2 as any, optional precision as nullable number) as nullable logical
```

Сведения

Возвращает NULL, если любой из аргументов `value1` и `value2` равен NULL, в противном случае — эквивалент `Value.Equals`.

Value.Optimize

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Optimize(value as any) as any
```

О программе

Если `value` при использовании в `Value.Expression` представляет запрос, который можно оптимизировать, эта функция определяет, что должно возвращаться оптимизированное выражение. В противном случае `value` будет передаваться без изменений.

Value.RemoveMetadata

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.RemoveMetadata(value as any, optional metaValue as any) as any
```

Сведения

Удаляет входные данные метаданных.

Value.ReplaceMetadata

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.ReplaceMetadata(value as any, metaValue as any) as any
```

Сведения

Заменяет метаданные входных данных.

Value.ReplaceType

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.ReplaceType(value as any, type as type) as any
```

Сведения

Value.ReplaceType

Value.Subtract

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Subtract(value1 as any, value2 as any, optional precision as nullable number) as any
```

Сведения

Возвращает разность `value1` и `value2`. Можно задать необязательный параметр `precision`, который по умолчанию имеет значение `Precision.Double`.

Value.Traits

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Traits(value as any) as table
```

О программе

Эта функция предназначена только для внутреннего использования.

Value.Type

16.03.2021 • 2 minutes to read

Синтаксис

```
Value.Type(value as any) as type
```

Сведения

Возвращает тип заданного значения.

Variable.Value

16.03.2021 • 2 minutes to read

Синтаксис

```
Variable.Value(identifier as text) as any
```

Сведения

Variable.Value